

STM32-V6 开发板

二代示波器教程



STM32F429BIT6

武汉安富莱电子有限公司



本文档适用的开发板：

序号	型号配置	说明
1	STM32-V6+4.3寸电阻屏	主板+4.3寸RGB屏（480*320/电阻触摸） 有亚克力保护
2	STM32-V6+4.3寸电容屏	主板+4.3寸RGB屏（480*272/电容触摸） 有亚克力保护
3	STM32-V6+5.0寸电阻屏	主板+5.0寸RGB屏（800*480/电阻触摸） 有亚克力保护
4	STM32-V6+5.0寸电容屏	主板+5.0寸RGB屏（800*480/电容触摸） 有亚克力保护
5	STM32-V6+7.0寸电阻屏	主板+7.0寸RGB屏（800*480/电阻触摸） 有亚克力保护
6	STM32-V6+7.0寸电容屏	主板+7.0寸RGB屏（800*480/电容触摸） 有亚克力保护
7	STM32-V6主板	仅主板 无显示屏，无亚克力保护

(1) 开发板配套的显示模块可以单独购买。

(2) 所有配置中的主板和光盘都是相同的，差异仅在于显示模块和亚克力板。

[点击此处直接购买](#)

销售QQ: 1295744630

销售旺旺: armfly

微信公众号: 安富莱电子

销售电话: 13638617262

邮箱: armfly@qq.com

公司网址: www.armfly.com

技术支持论坛: bbs.armfly.com

淘宝直销: armfly.taobao.com



武汉安富莱电子有限公司

专业开发板、显示模块制造商

承接项目开发（提供生产供货服务）

第1章 示波器基础知识

本章的内容整理自网络，主要讲解示波器的基础知识。如果初学的话非常有必要对这部分知识有一个了解。因为示波器是硬件调试必不可少的设备。

- 1.1 什么是示波器
- 1.2 示波器的发展史
- 1.3 示波器的基础概念
- 1.4 触发系统
- 1.5 李沙育图形
- 1.6 窗函数选择
- 1.7 总结

1.1 什么是示波器

示波器是形象地显示信号幅度随时间变化的波形显示仪器，是一种综合的信号特性测试仪，是电子测量仪器的基本种类。

自然界运行着各种形式的正弦波，比如海浪、地震、声波、爆破、空气中传播的声音，或者身体运转的自然节律。物理世界里，能量、振动粒子和不可见的力无处不在。即使是光（波粒二象物质）也有自己的基频，并因为基频的不同呈现出不同的颜色。通过传感器，这些力可以转变为电信号，以便通过示波器能够进行观察和研究。有了示波器，科学家、工程师、技术人员、教育工作者和他人能够“观察”随时间变化的事件。

示波器是任何设计、制造或是维修电子设备的必备之物。当今世界瞬息万变，工程师们需要最好的工具，快速而精确地解决测量疑难。在工程师看来，面对当今各种测量挑战，示波器自然是满足要求的关键工具。示波器的用途不仅仅局限于电子领域。示波器利用信号变换器，适用于各种各样的物理现象。信号变换器能够响应各种物理激励源，使之转变为电信号，包括声音、机械应力、压力、光、热。麦克风属于信号变换器，它实现把声音转变为电信号。由示波器收集科学数据的例子如图 1 所示。

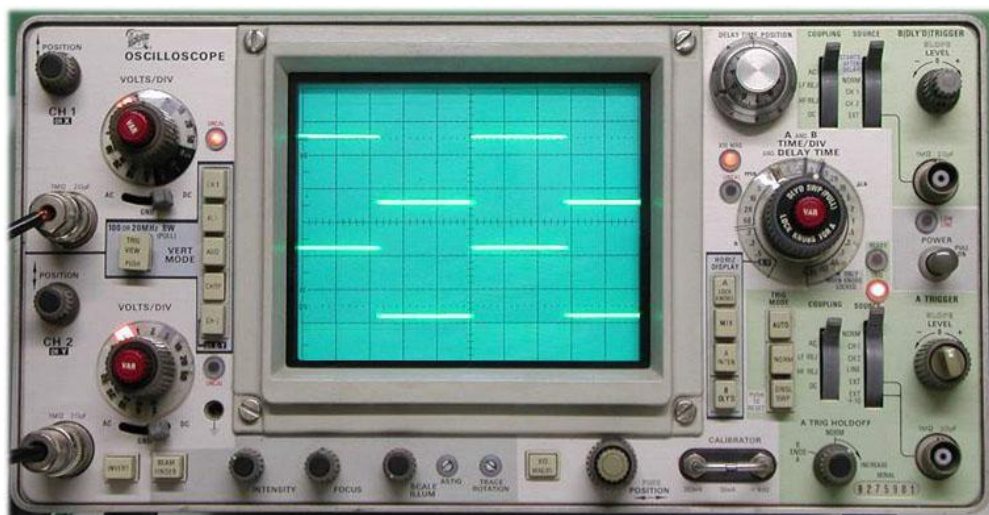
从物理学家到电视维修人员，各种人士都使用示波器。汽车工程师使用示波器来测量发动机的振动。医师使用示波器测量脑电波。描述示波器的用途是没有止境的。

1.2 示波器的发展史

电子设备可以划分为两类：模拟设备和数字设备。模拟设备的电压变化连续，而数字设备处理的是代表电压采样的离散二代码。传统的电唱机是模拟设备，而 CD 播放器是属于数字设备。同样，示波器也能分为模拟和数字类型。模拟和数字示波器都能够胜任大多数的应用。但是，对于一些特定应用，由于两者具备的不同特性，每种类型都有适合和不适合的地方。作进一步划分，数字示波器可以分为数字存储示波器（DSO）、数字荧光示波器（DPO）和采样示波器。

1.2.1 模拟示波器

初期主要是模拟示波器（CRT），始创于二十世纪四十年代，最早应用于雷达和电视的开发泰克成功开发带宽 10MHz 的同步示波器，这是近代示波器的基础。

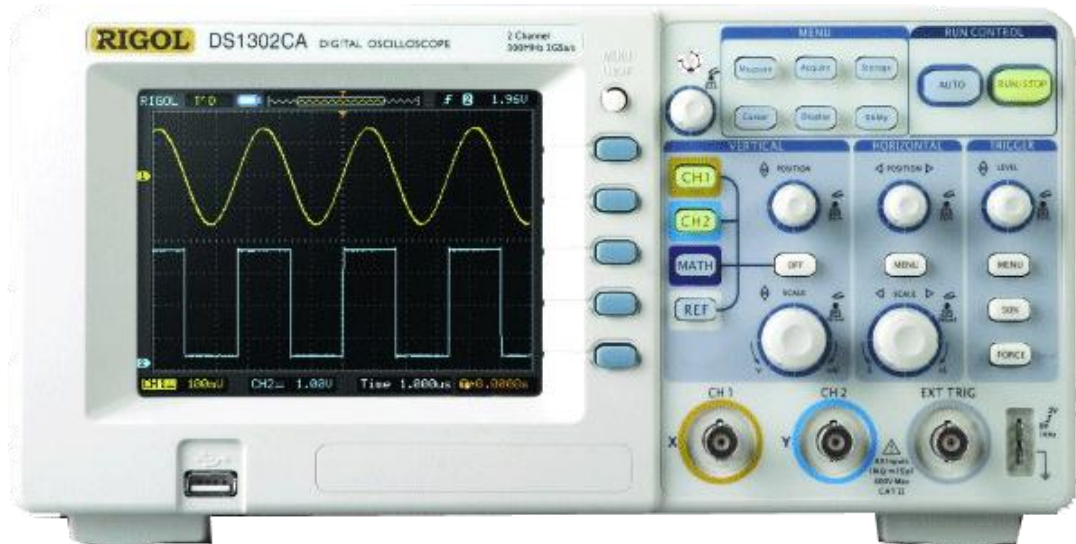


在本质上，模拟示波器工作方式是直接测量信号电压，并通过从左到右穿过示波器屏幕的电子束在垂直方向描绘电压。示波器屏幕通常是阴极射线管（CRT）。电子束投到荧幕的某处，屏幕后面总会有明亮的荧光物质。当电子束水平扫过显示器时，信号的电压是电子束发生上下偏转，跟踪波形直接反映到屏幕上。在屏幕同一位置电子束投射的频度越大，显示得也越亮。CRT 限制着模拟示波器显示的频率范围。在频率非常低的地方，信号呈现出明亮而缓慢移动的点，而很难分辨出波形。在高频处，起局限作用的是 CRT 的写速度。当信号频率超过 CRT 的写速度时，显示出来的过于暗淡，难于观察。模拟示波器的极限频率约为 1GHz。当把示波器探头和电路连接到一起后，电压信号通过探头到达示波器的

垂直系统。图解出模拟示波器是如何显示被测信号。设置垂直标度（对伏特/格进行控制）后，衰减器能够减小信号的电压，而放大器可以增加信号电压。随后，信号直接到达 CRT 的垂直偏转板。电压作用于这些垂直偏转板，引起亮点在屏幕中移动。亮点是由打在 CRT 内部荧光物质上的电子束产生的。正电压引起点向上运动，而负电压引起点向下运动。

1.2.2 数字示波器

中期数字示波器独领风骚（DSO），始创于二十世纪九十年代，数字示波器提高带宽到 1GHz 以上，全面性能超越模拟示波器。

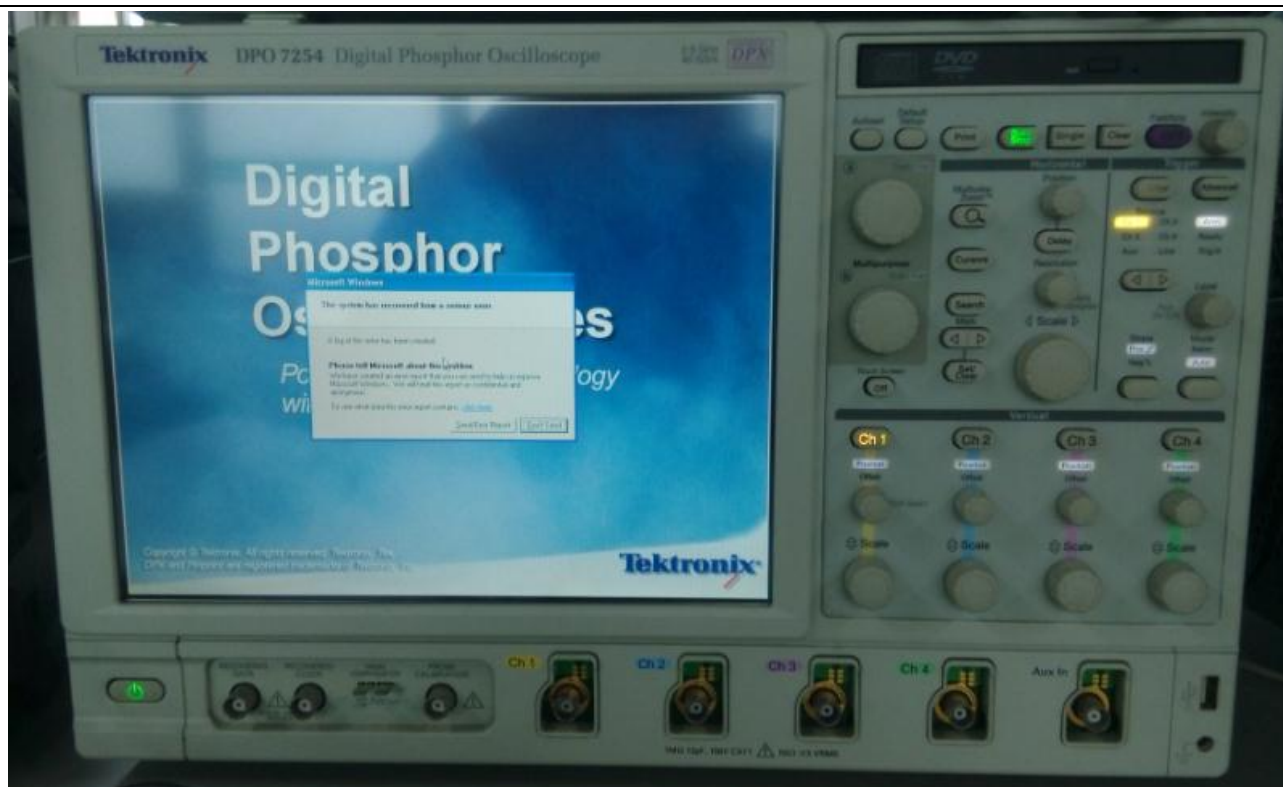


模拟示波器和数字示波器的比较：

波形显示 示波器类别	简单重复信号						复杂动态 重复信号	定时 测量	非 重复 单次 信号	触 发 功 能	预 触 发	示 波 器 带 宽	数 据 处 理
	稳 定 信 号	变 化 信 号	异 常 信 息	缓 变 信 号	随 机 毛 刺	快 沿 信 号							
模拟示波器 CRT显示技术	好	好	差	差	差	差	好	差	不能	边沿	不能	低	不能
DSO示波器 等效采样技术	好	差	差	好	差	好	差	好	差	多种	能	高	能
DSO示波器 实时采样技术	好	好	好	好	好	好	差	好	好	多种	能	高	能

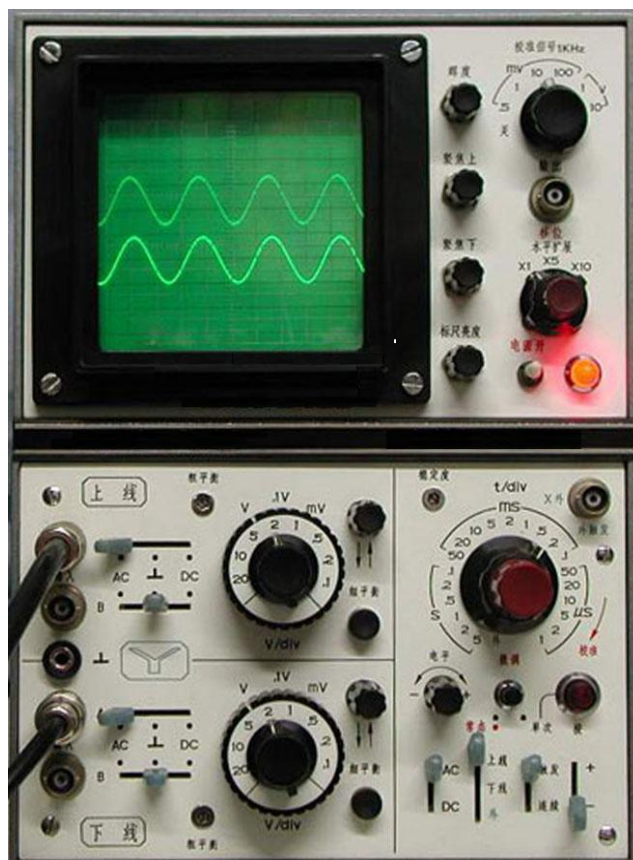
1.2.3 数字荧光示波器（DPO）

数字荧光示波器（DPO）为示波器系列增加了一种新的类型。DPO 的体系结构使之能提供独特的捕获和显示能力，加速重构信号。DSO 使用串行处理的体系结构来捕获、显示和分析信号；相对而言，DPO 为完成这些功能采用的是并行的体系结构，如下图所示(Tek DPO)。DPO 采用 ASIC 硬件构架捕获波形图像，提供高速率的波形采集率，信号的可视化程度很高。它增加了证明数字系统中的瞬态事件的可能性。随后将对该并行处理体系结构进行阐述。



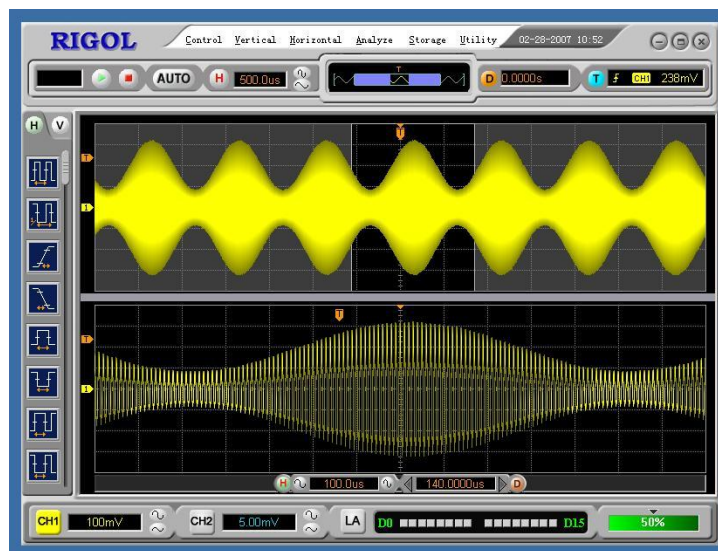
1.2.4 高灵敏度示波器

带宽很低，1MHz 左右，灵敏度很高，可到几十微伏每格，用以测量和显示一般示波器不能观察到的各种微弱的电信号。



1.2.5 虚拟示波器

利用计算机资源做数据处理和显示，体积小巧。



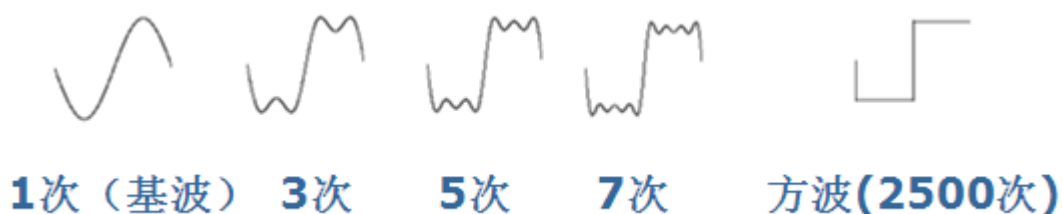
1.3 示波器基本概念

1.3.1 波的组成

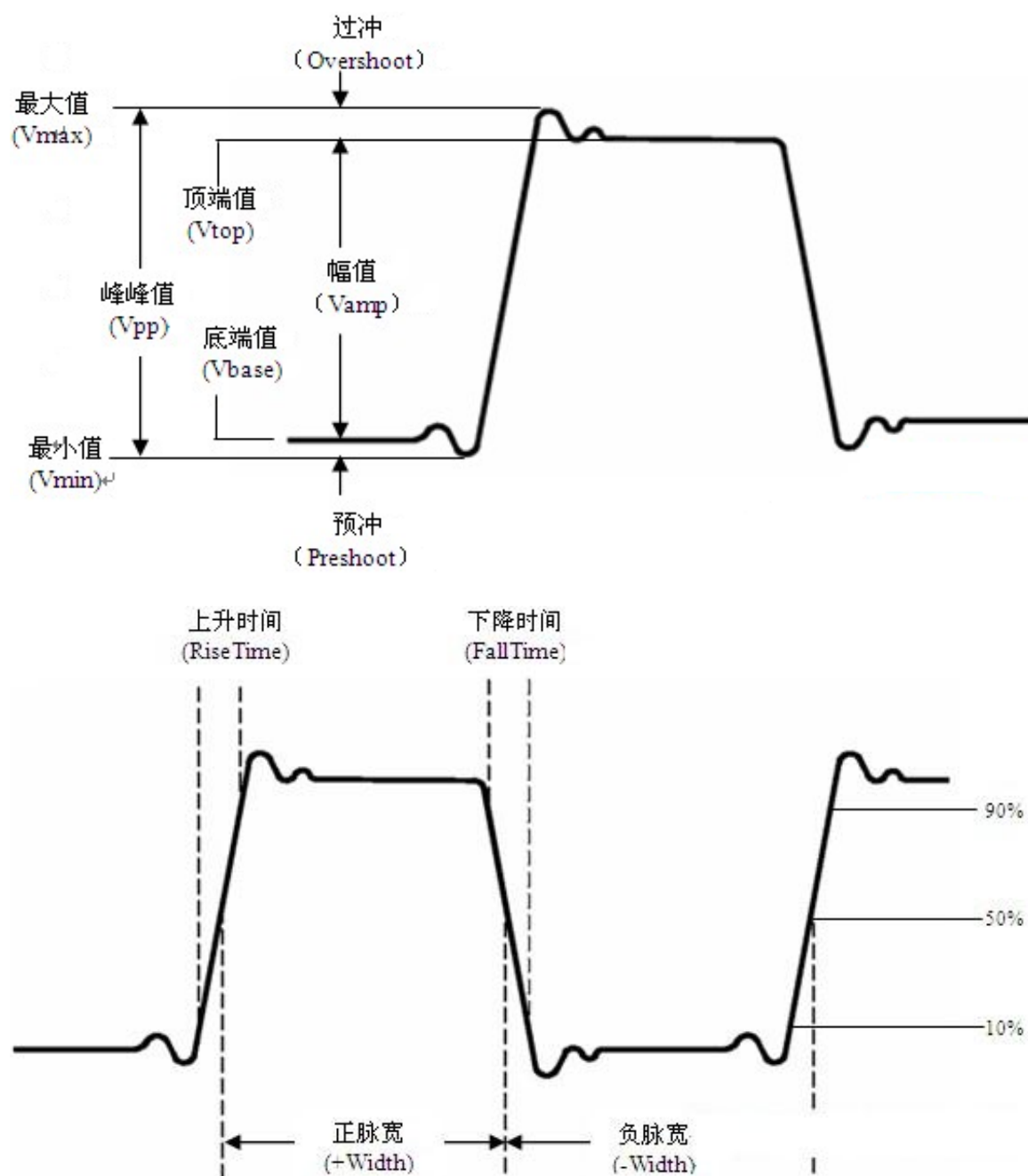
正弦波是波形的基本组成，任何非正弦波都可以看成是基波和无数不同频率的谐波分量组成。对于非正弦波由最小值过渡到最大值的时间越短，所含的谐波分量也就越多，波形所含谐波的频率也越高。

对于脉冲波占空比越小，波形所含谐波就越多，谐波频率分量也越高。

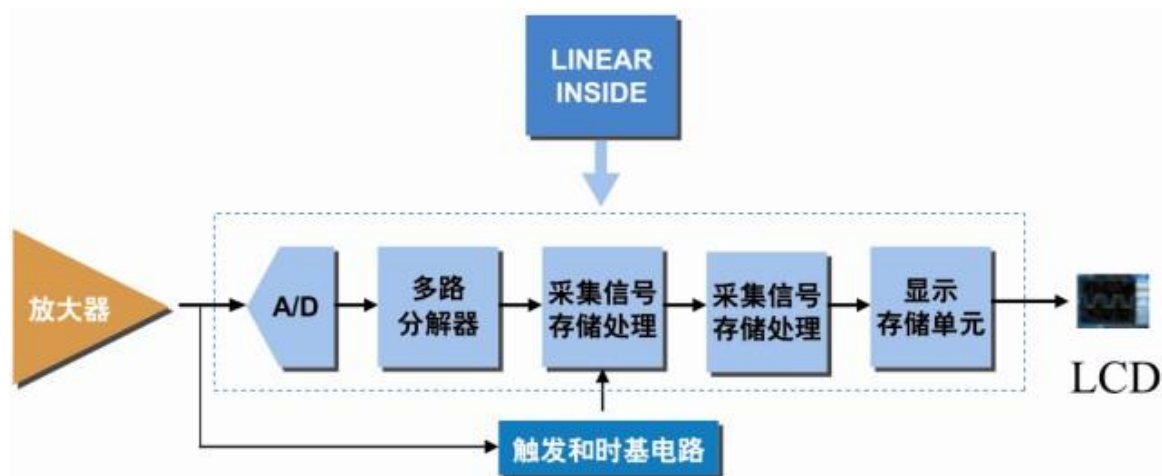
比如：方波是由基波以及 3,5,7,9.....次谐波分量递加而成(这个咱们在介绍 FFT 的时候有讲解)。



1.3.2 波的基本参数



1.3.3 数字示波器串行处理

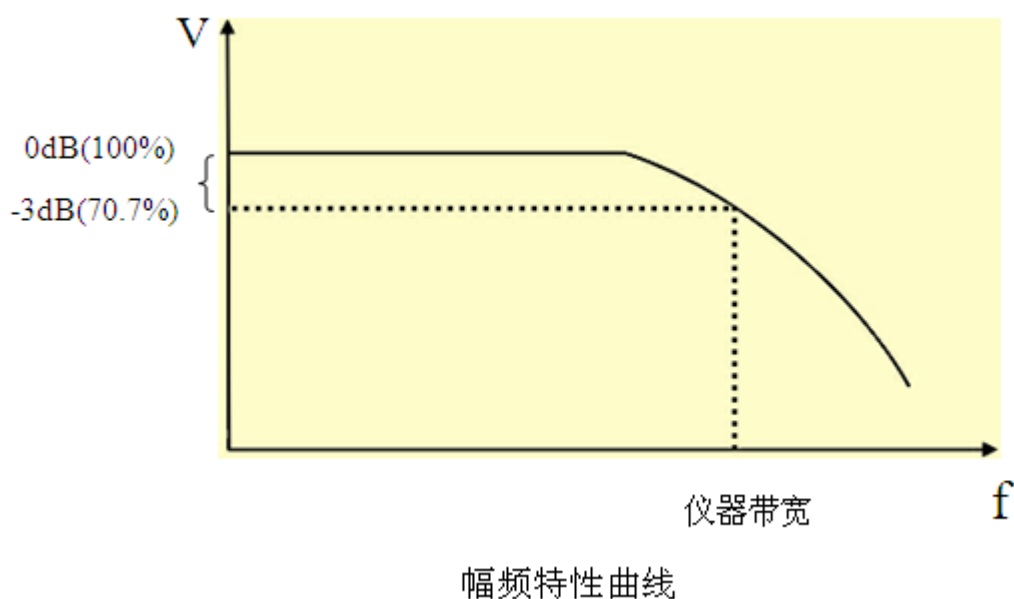


DSO 第一部分（输入）是垂直放大器。在这一阶段，垂直控制系统方便您调整幅度和位置范围。紧接着，在水平系统的模数转换器（ADC）部分，信号实时在离散点采样，采样位置的信号电压转换为数字值，这些数字值称为采样点。该处理过程称为信号数字化。水平系统的采样时钟决定 ADC 采样的频度。该速率称为采样速率，表示为样值每秒（S/s）。来自 ADC 的采样点存储在捕获存储区内，叫做波形点。几个采样点可以组成一个波形点。波形点共同组成一条波形记录。创建一条波形记录的波形点的数量称为记录长度。触发系统决定记录的起始和终止点。DSO 信号通道中包括微处理器，被测信号在显示之前要通过微处理器处理。微处理器处理信号，调整显示运行，管理前面板调节装置，等等。信号通过显存，最后显示到示波器屏幕中。在示波器的能力范围之内，采样点会经过补充处理，显示效果得到增强。可以增加预触发，使在触发点之前也能观察到结果。

目前大多数数字示波器也提供自动参数测量，使测量过程得到简化。DSO 提供高性能处理单脉冲信号和多通道的能力。DSO 是低重复率或者单脉冲、高速、多通道设计应用的完美工具。在数字设计实践中，工程师常常同时检查四路甚至更多的信号，而 DSO 则成为标准的合作伙伴。

1.3.4 带宽

数字示波器带宽也称为模拟带宽，指示波器前端输入放大器的带宽，相当于一个低通滤波。定义为在幅频特性曲线中，随正弦波频率的增加，信号的幅度下降到 3dB(70.7%)，此时的频率点称为示波器的带宽。

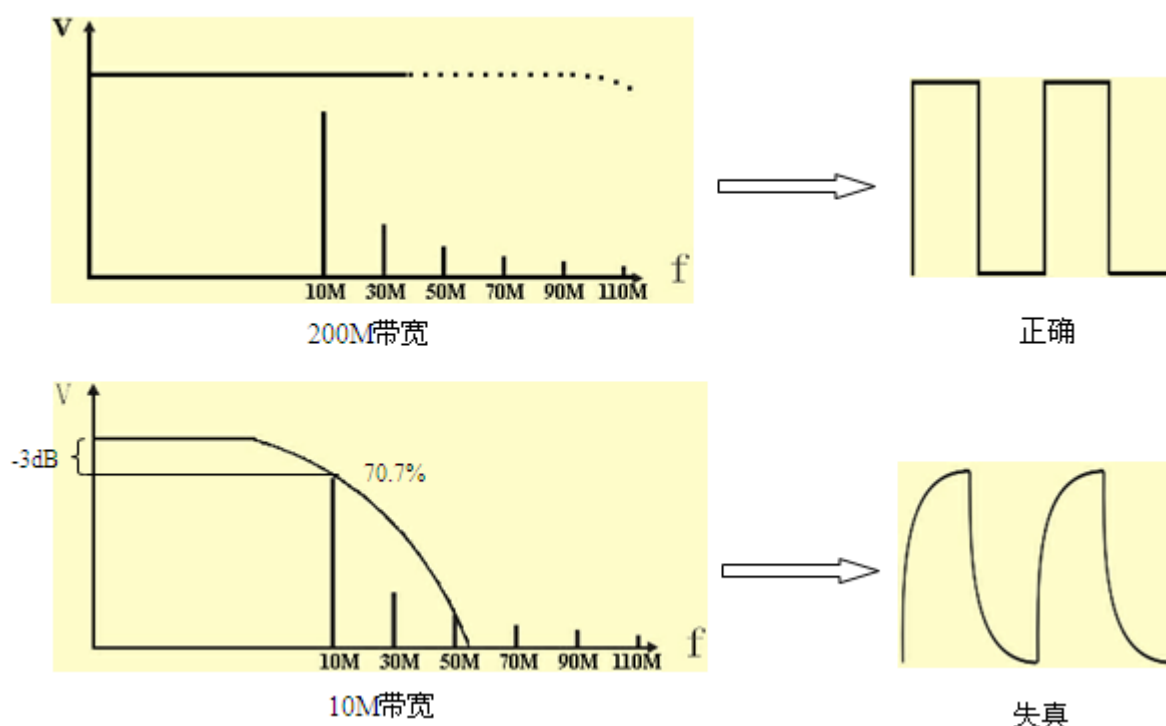


在波形的主要谐波分量中提到过，如果要对波形进行准确测量应该让示波器的带宽大于波形的主要谐波分量。因此对于正弦波可以要求示波器的带宽大于波形的频率，但是对应非正弦波则要求示波器的带宽大于波形的最大主要谐波频率。

对于带宽带来的波形影响具体表现在以下两方面：

1. 由于低带宽导致的主要谐波分量消失，使原本规则的波形呈圆弧状接近正弦波。
2. 低带宽给波形的上升时间和幅度的测量带来较大的误差。

下列图示为一个 10MHz 的方波在 200MHz 带宽和 10MHz 带宽示波器上的显示效果图。



1.3.5 带宽与最高频率准则

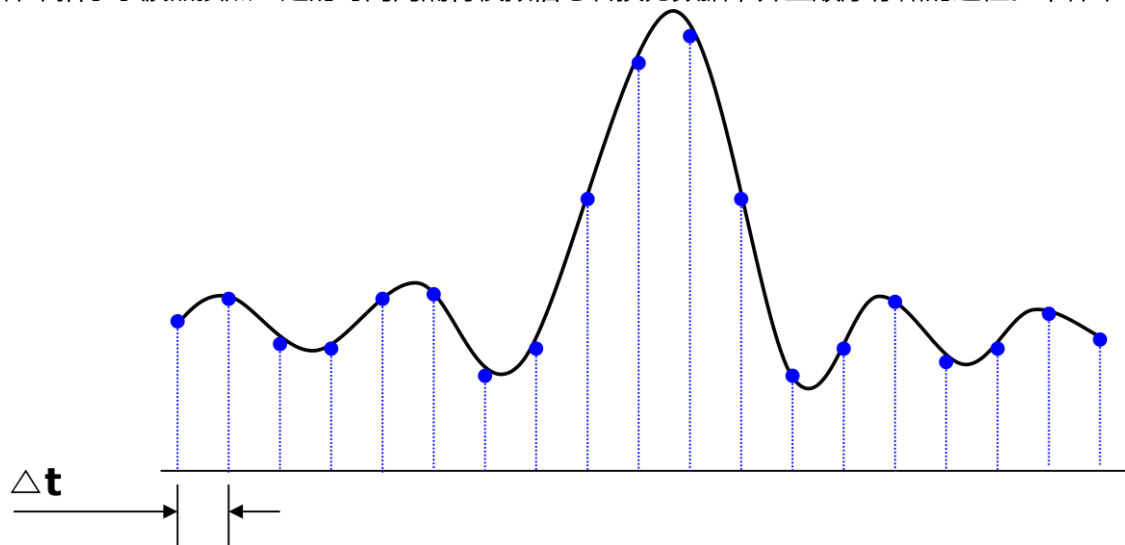
5 倍准则 (The 5 times rule)。

示波器所需带宽 = 被测信号的最高频率成分 $\times 5$ 。

测定示波器带宽的方法：在具体操作中准确表征信号幅度，并运用 5 倍准则。使用五倍准则选定的示波器的测量误差将不会超过 $\pm 2\%$ ，对今天的操作来说已经足够。然而，随着信号速率的增加，这个经验准则将不再适用。记住，带宽越高，再现的信号就越准确。

1.3.6 采样率

采样率指示示波器按照一定的时间间隔将模拟信号转换为数据，并且顺序存储的过程。采样率 $= 1/\Delta t$



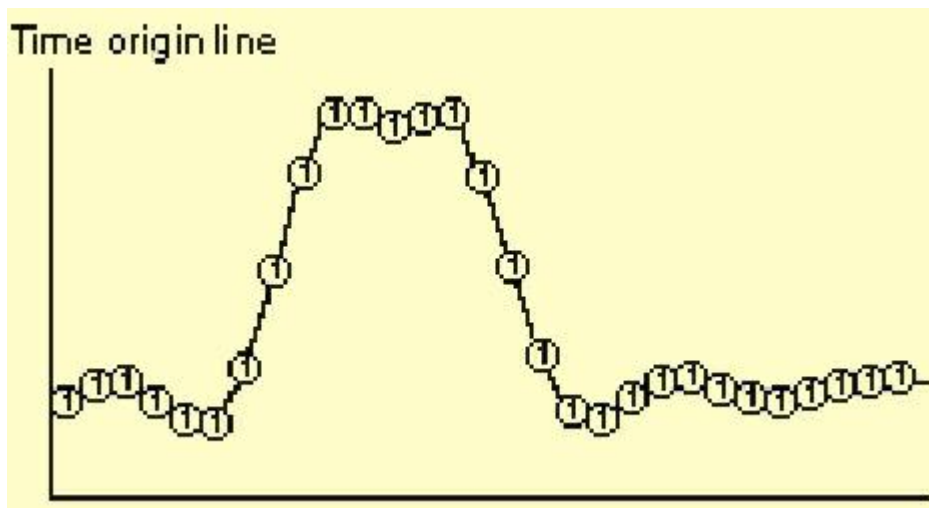
较高的采样速率提供较高的信号分辨率，可以让读者观察到断续的事件。

采样速率：表示为样点数每秒 (S/s)，指数字示波器对信号采样的频率，类似于电影摄影机中的帧的概念。示波器的采样速率越快，所显示的波形的分辨率和清晰度就越高，重要信息和事件丢失的概率就越小，如果需要观测较长时间范围内的慢变信号，则最小采样速率就变得较为重要。典型地，为了在显示的波形记录中保持固定的波形数，需要调整水平控制按钮，而所显示的采样速率也将随着水平调节按钮的调节而变化。

如何计算采样速率？计算方法取决于所测量的波形的类型，以及示波器所采用的信号重构方式。为了准确地再现信号并避免混淆，奈奎斯特定理规定，信号的采样速率必须不小于其最高频率成分的两倍。然而，这个定理的前提是基于无限长时间和连续的信号。由于没有示波器可以提供无限时间的记录长度，而且，从定义上看，低频干扰是不连续的，所以，采用两倍于最高频率成分的采样速率通常是不够的。实际上，信号的准确再现取决于其采样速率和信号采样点间隙所采用的插值法。一些示波器会为操作者提供以下选择：**测量正弦信号的正弦插值法，以及测量矩形波、脉冲和其他信号类型的线性插值法**。在使用正弦插值法时，为了准确再现信号，示波器的采样速率至少需为信号最高频率成分的 2.5 倍。使用线性插值法时，示波器的采样速率应至少是信号最高频率成分的 10 倍。一些采样速率高达 20GS/s、带宽高达 4GHz 的测量系统用 5 倍于带宽的速率来捕获高速，单脉冲和瞬态事件。

1.3.7 实时采样率

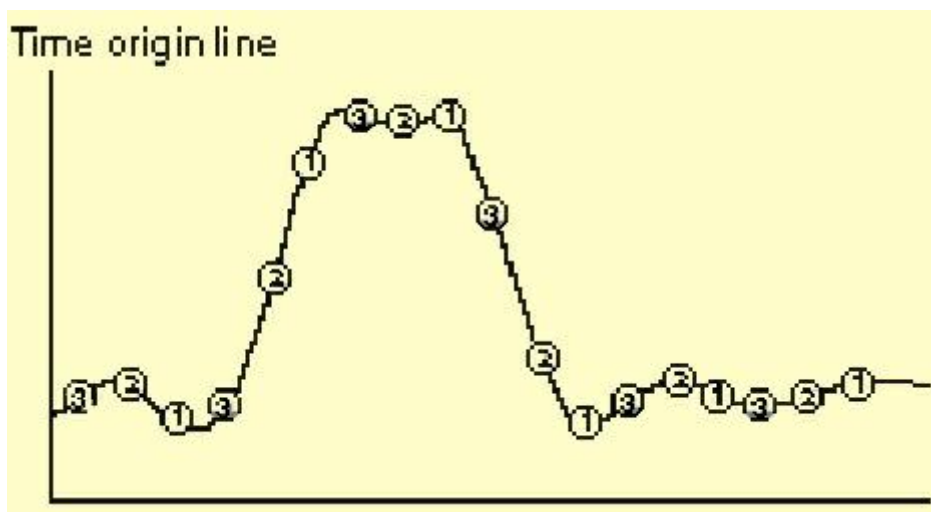
实时采样在一次触发事件期间捕获所有用于重建波形的样本点，它要求采样率至少为被测波形最高频率分量的 5 倍。



如上图所示，① 表示第一次触发所采样的数据点，并且一次就完成一个采样过程。

1.3.8 等效采样率

等效采样是在多个触发事件上捕捉样本点，要求输入的波形为重复波形。对于每个触发事件示波器会捕捉多个样本点，并把它们与原已捕捉的样本点组合到一起。



如上图所示，①②③ 第 1, 2, 3 次触发事件所采样的数据点，1, 2, 3 次触发相互间隔错开。

等效采样对信号的要求：信号必须重复并且稳定，如信号变化（如幅度）将造成显示混乱。

等效技术示波器，只适用捕获重复稳定信号，对捕获非重复信号和单次信号的能力。以及是捕获隐藏在重复信号中的毛刺和异常信号的能力。将受到实时采样率的限制。

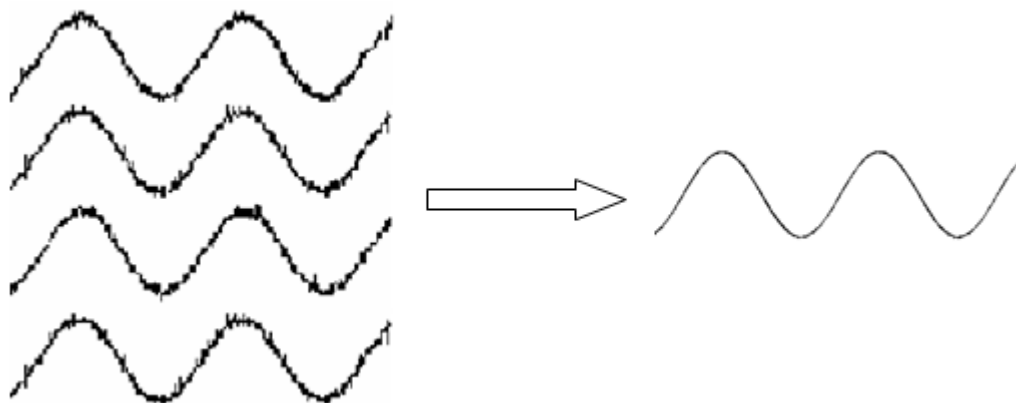
示波器标定带宽 = 重复信号带宽 > 瞬态（单次）信号带宽。

等效采样是用较低的实时采样率实现高采样率的效果的一种采样方式。它的主要特点：

1. 必须多次触发采样才能构建一个完整波形。
2. 要求信号必须是重复信号。
3. 理论上可以实现无限高的采样率。
4. 波形构建速度稍微要比实时采样慢（Normal 触发方式，1Hz 方波演示）。

1.3.9 平均采样

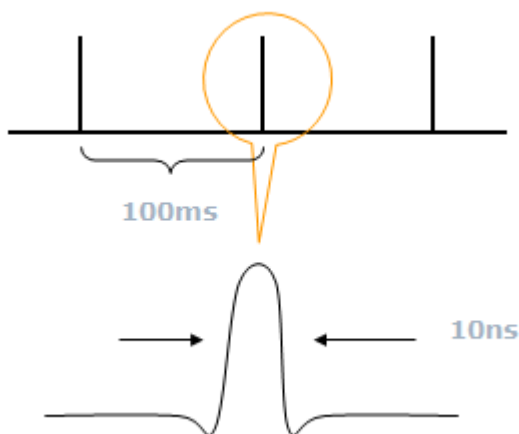
平均采样：指将多次普通采样的波形进行算术平均，多用于信号本身噪声比较大时。



1.3.10 峰值检测

峰值检测：指通过采集采样间隔信号的最大值和最小值，获取信号的包络或可能丢失的窄脉冲。

要求观察整个周期，当使用普通采样方式时，采样率为 10K，如前面描述存在波形漏失现象，无法捕获完整信号。使用峰值检测，采样率为 1G，获取间隔最大最小值，就可以获取完整的周期信号。

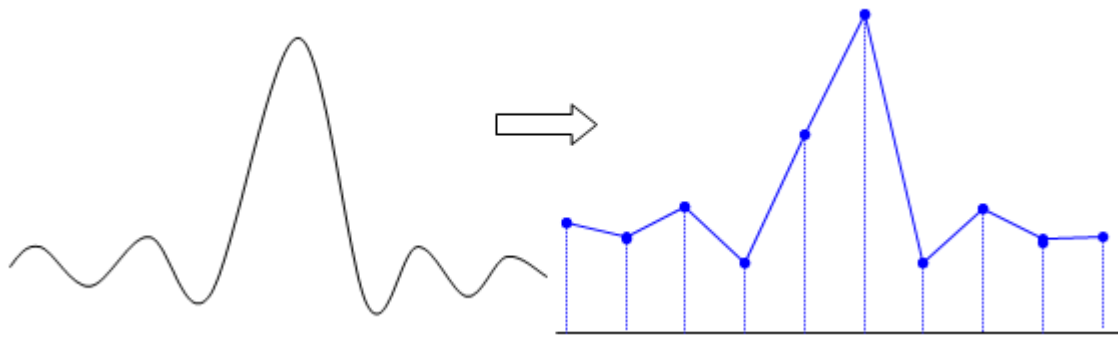


1.3.11 采样率低的影响

采样率低主要的影响是波形失真，波形混淆和波形遗漏。

1. 波形失真

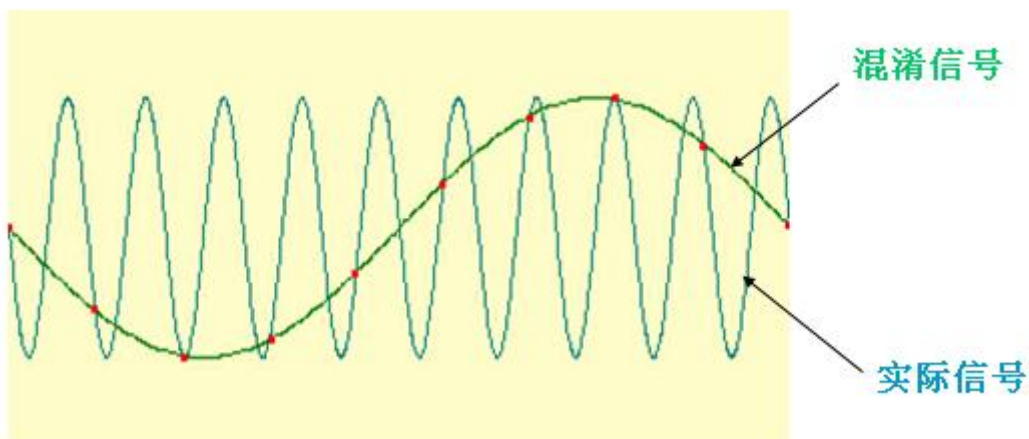
波形失真是由于某些原因导致示波器采样显示的波形与实际信号存在较大的差异。



2. 波形混淆

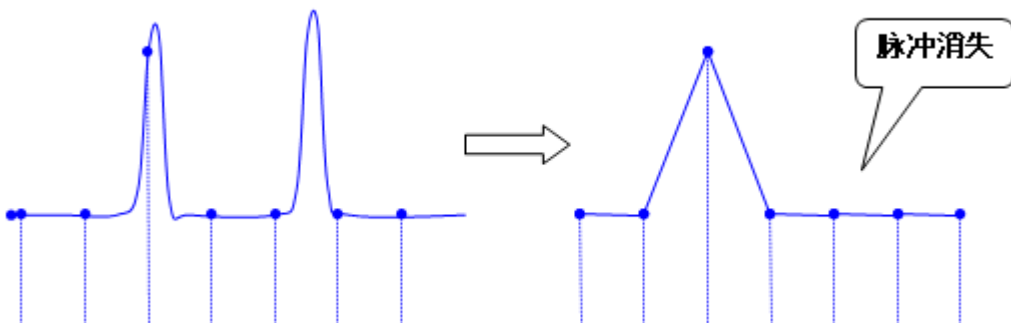
混淆是指当采样率低于实际信号最高频率 2 倍（耐奎斯特频率）时所出现的一种现象。

如下图，由于采样率低于实际信号的频率，导致结果采集的波形频率低于实际信号频率。这种信号频率与实际信号不同，它却能表示正确的波形形状，往往还具有正确的幅度。



3. 波形遗漏

波形漏失是指由于采样率低而造成的没有反映全部实际信号的一种现象。

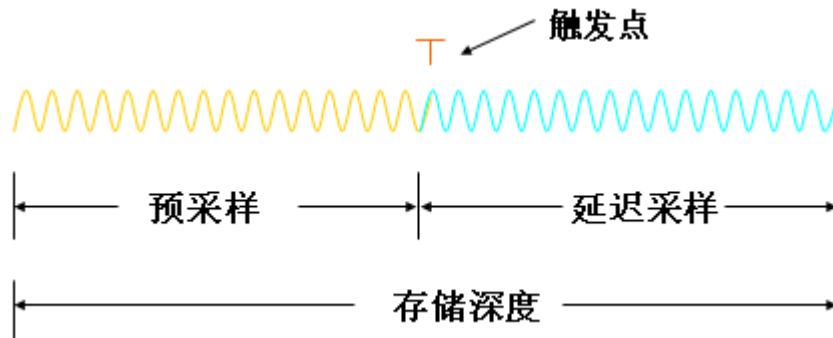


1.3.12 存储深度

存储深度：指在波形存储器中存储波形样本的数量。

波形存储时间 = 存储深度 / 采样率

示波器的存储深度将决定能采集信号的时间以及能用到的最大采样速率。



1. 记录长度

记录长度表示为构成一个完整波形记录的点数，决定了每个通道中所能捕获的数据量。由于示波器仅能存储有限数目的波形采样，波形的持续时间和示波器的采样速率成反比。

现代的示波器允许用户选择记录长度，以便对一些操作中的细节进行优化。分析一个十分稳定的正弦信号，只需要 500 点的记录长度；但如果要解析一个复杂的数字数据流，则需要有一百万个点或更多点的记录长度。最大记录长度由示波器的存储容量决定，要增加存储容量才能增加记录长度，是为捕获和显示单次信号过渡过程提供的重要指标。

示波器的存储由两个方面来完成：触发信号和延时的设定确定了示波器存储的起点。示波器的存储深度决定了数据存储的终点。记录时间 = 存储深度 / 采样率

由于时基和采样率是联动的，所以时基的速度快慢将同时改变采样率的高低。当采样率达到指标定义最高速率时，加快基速度的调整，采样率将不能加快。

时基与采样率的关系应为：存储深度（点）÷ 时间/格 × 10 = 采样间隔。 1/采样间隔 = 采样率

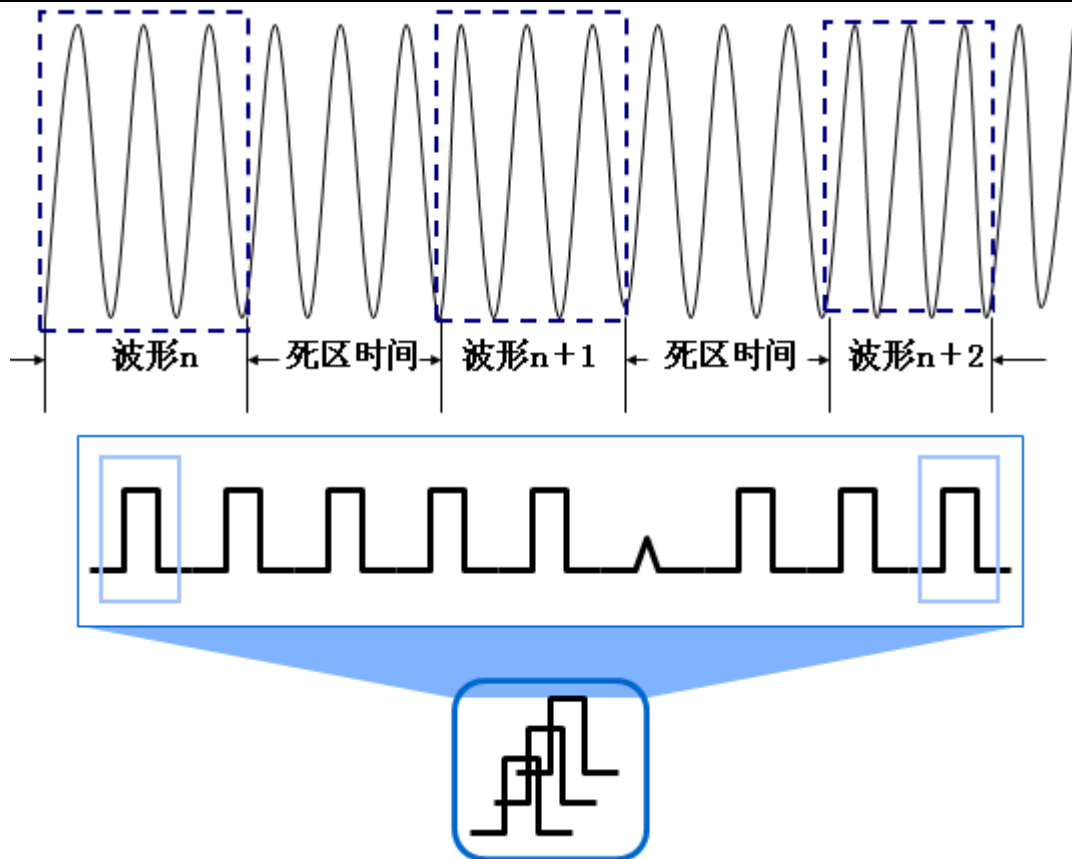
2. 采样率与存储深度的关系

示波器最高采样率决定示波器单次带宽的限制，为保证波形精确复现建议：正弦内插技术示波器以：采样率/5 = 单次带宽的公式计算单次带宽，线性内插技术示波器以：采样率/10 = 单次带宽公式计算。采样率不足将限制示波器单次带宽。如果示波器在全带宽范围内，对单次信号实现捕获和精确复现。只有采样率高于示波器带宽 5 倍以上（正弦内插），才能使示波器的重复信号带宽 = 单次信号带宽。示波器存储长度对波形的记录是以波形精确捕获为前提。当信号频率或速度超过单次带宽的限制（信号不能重组），即使示波器带宽对信号不产生影响，但由于采样不足将造成显示信号的混叠、畸变和漏失。就是示波器有在长的存储，存储的波形也是畸变的失真波形。当单次信号中的高频成份，低于示波器的单次带宽，才能保证信号的高频细节。此时存储长度越长，波形记录时间越长。存储深度短，将丢失波形部分时间的信息。

1.3.13 波形刷新率

波形捕获率也就是波形刷新率，已经成为考核一台示波器的重要参数之一。

刷新率是指 1 秒内示波器捕获波形的次数，刷新率的高低直接影响波形捕获偶然事件发生的概率。

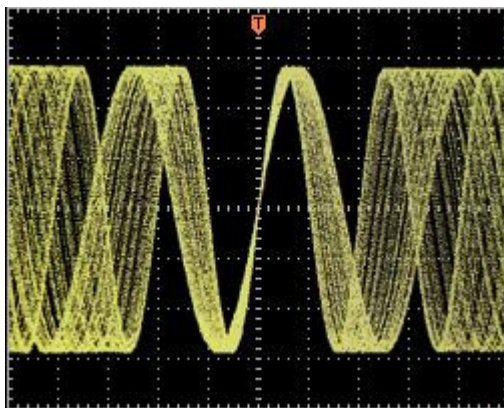


对于示波器来说，波形刷新率高，就能够组织更大数据量的波形质量信息，尤其是在动态复杂信号和隐藏在正常信号下的异常波形的捕获方面，有着特别的作用。

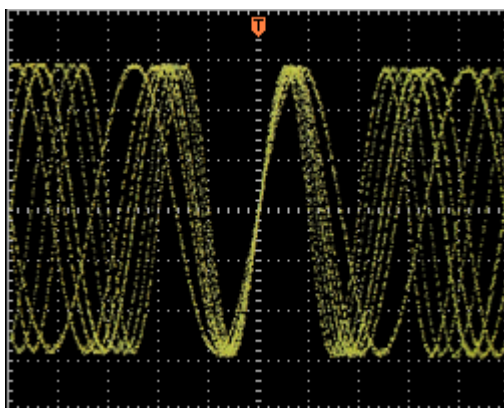
所有的示波器都会闪烁。也就是说，示波器每秒钟以特定的次数捕获信号，在这些测量点之间将不再进行测量。这就是波形捕获速率，表示为波形数每秒（wfms/s）。采样速率表示的是示波器在一个波形或周期内，采样输入信号的频率，波形捕获速率则是指示波器采集波形的速度。波形捕获速率取决于示波器的类型和性能级别，且有着很大的变化范围。高波形捕获速率的示波器将会提供更多的重要信号特性，并能极大地增加示波器快速捕获瞬时的异常情况，如抖动、矮脉冲、低频干扰和瞬时误差的概率。

数字存储示波器（DSO）使用串行处理机制，每秒钟可以捕获 10 到 5000 个波形。一些 DSO 提供一种特殊的模式，它能迅速把各种捕获信息存储到海量存储器中，暂时提供较高的波形捕获速率，而随后是较长的一段处理时间，这段处理时间内不重新活动，减少了捕获稀少和间歇事件的可能性。

大多数数字荧光示波器（DPO）采用并行处理机制来提供更高的波形捕获速率。一些 DPO 可以在一秒钟之内获得数百万个波形，大大提高了捕获间歇的和难以捕捉事件的可能性，并能让用户更快地发现信号中存在的问题。而且，DPO 的实时捕获和显示三维信号特性（如幅度、时间以及幅度的时间分布特性）的能力使其能够得到更高等级的信号特性。



1000/s 以上的刷新率



100/s – 200/s 的刷新率

1.4 触发系统

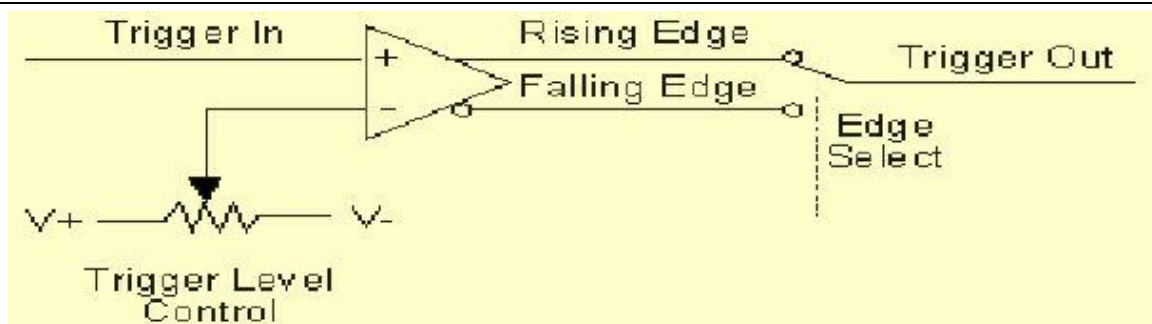
1.4.1 基本概念

对于数字示波器，工作时都是在不断地采集波形不论仪器是否稳定触发。

触发电路的作用就是保证每次时基扫描或采集的时候，都从输入信号上与定义的相同的触发条件开始，这样每一次扫描或采集的波形就同步，可以每次捕获的波形相重叠，从而显示稳定的波形，或保证单次信号的捕获：

1. 触发是使重复信号稳定显示
2. 对单次信号进行捕获
3. 对重复信号中的异常波形和单次事件中的特殊波形 进行隔离捕获

示波器的触发电路主要用于帮助对所要的波形进行定位。根据不同的信号特征和测量目的，可以选择不同的触发类型。但是，最常用的还是边沿触发：

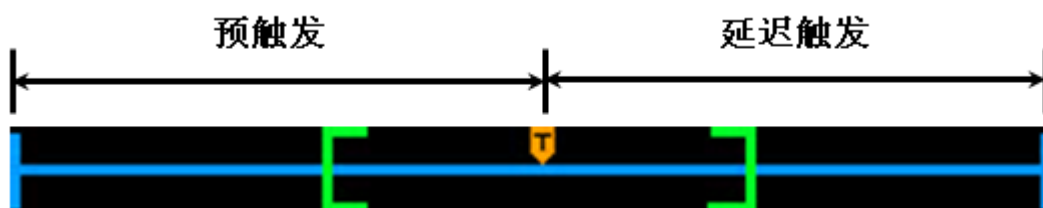


波形进入触发比较器的正输入端，在这里与另一个输入端上的触发电平电压进行比较。触发比较器有上升沿输出和下降沿输出。当您的波形的上升沿穿越触发电平时，上升沿比较器输出变为高，而下降沿输出变为低。当波形的下降沿穿越触发电平时，上升沿输出变为低，而下降沿输出变为高。示波器使用您选择的输出作为触发输出。

1.4.2 触发位置

数字示波器的一个最显著特点在于它容许用户观看触发位置之前的事件，其称之为预触发。这是因为数据被连续地存储到内存中，直到触发事件发生，并且采样数据达到存储深度一次触发结束为止。

触发点之后的数据称之为延迟触发。同时可以变更触发位置，以改变延迟触发和预触发的数据长度。



1.4.3 触发模式

触发模式主要有自动触发，普通触发和单次触发。

自动：即使没有触发，自动模式也能引起示波器的扫描。如果没有信号的输入，示波器中的定时器触发扫描。有信号显示信号，没有信号显示水平基线。

普通：当输入信号不能满足触发条件时，不扫描，示波器没有任何显示。只有当输入信号满足设置的触发点条件时，才进行扫描，并将最后捕获到的信号冻结显示在屏幕上。如符合触发条件，再次进行捕获，清除上次信号，保留冻结此次的波形。

单次：当输入的单次信号满足触发条件时，进行捕获（扫描），将波形存储和显示在屏幕上。此时再有信号输入示波器不予理会。需要进行再次捕获必须进行单次设置。

注：在实际应用中，采用普通触发模式即使触发以很慢的速率发生，也能观测感兴趣的内容。对低重复的信号捕获是非常有意义。

1.4.4 触发耦合

由于示波器的输入信号经放大器分两路，一路进入 A/D 采样器；一路到触发电路，形成触发信号。

触发耦合是触发信号与触发电路的耦合方式，就像垂直系统输入一样，可为触发信号选择各种耦合方式。正确的选择触发耦合可使示波器稳定触发。耦合方式：

DC 耦合：触发信号直接连到触发电路

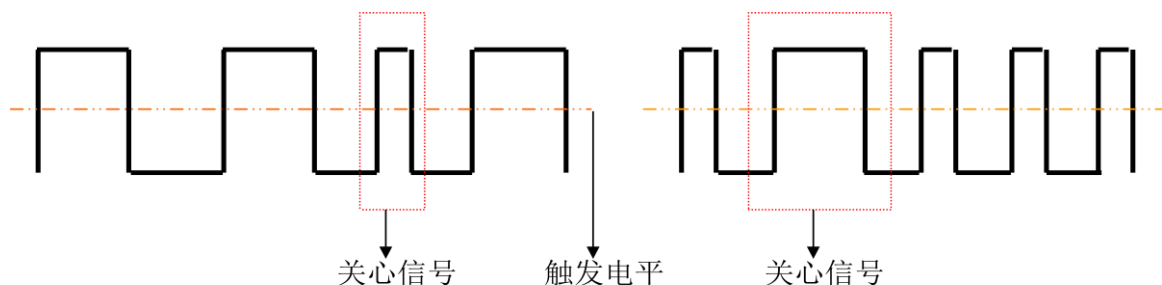
交流耦合：触发源通过一个串联的电容连到触发电路起到隔直作用

HF 抑制：使触发信号通过低通滤波器以抑制高频分量，这意味即使一个低频信号中包含很多高频噪声，仍能使其按低频信号触发。

LF 抑制：使触发源信号通过一个高通滤波器以抑制其低频成分。这意味即使一个高频信号中包含很多低频噪声，仍能使其按低频信号触发。这对于显示包含很多电源交流信号时情况是很有用处的。

1.4.5 脉宽触发

脉宽触发是让示波器通过寻找波形中比其它脉冲宽、窄或相等的脉冲来确认脉冲宽度触发。

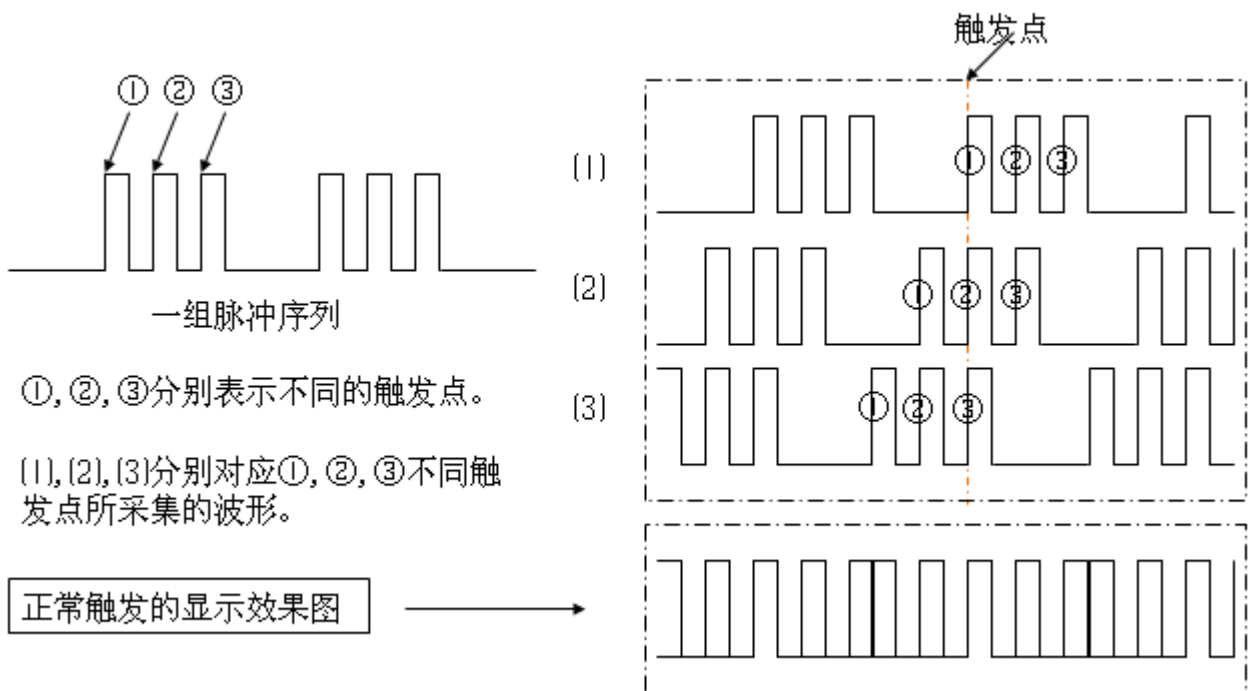


如上图，由于信号在波形的沿上都具有触发点，如果采用正常的边沿触发方式将无法准确捕捉到关心的异常脉宽的信号。

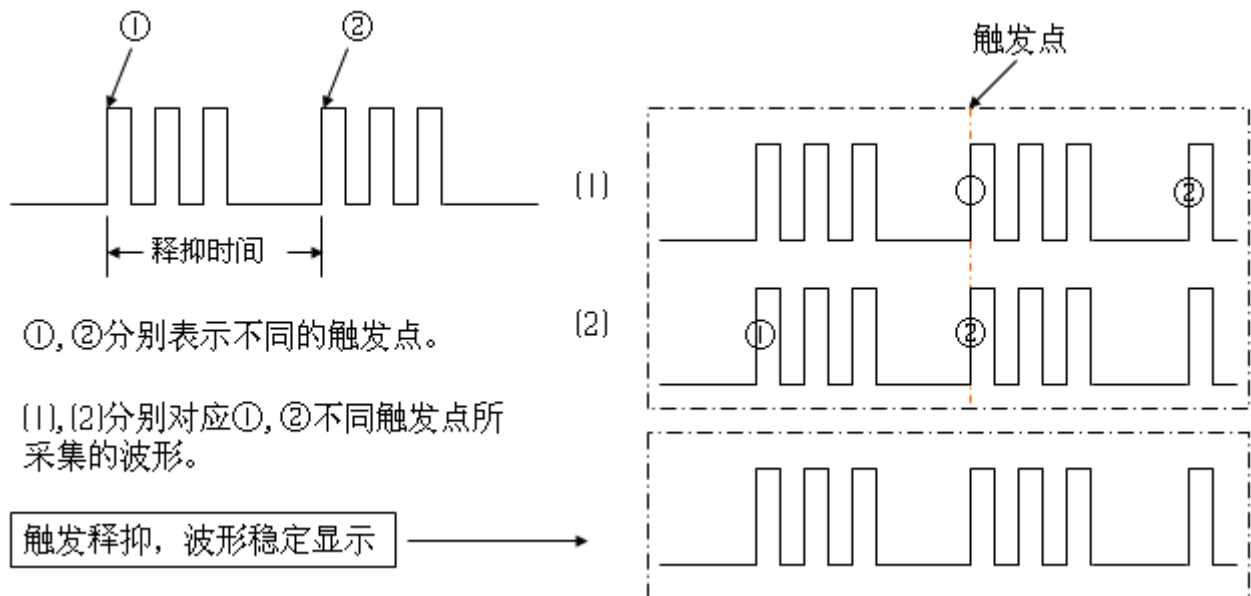
采用脉宽触发功能，监视脉冲宽度，通过与设置脉宽进行比较（>、<、=），达到捕捉异常脉冲的功能。

1.4.6 触发释抑

一组序列脉冲在采用正常的边沿触发方式下，由于每次触发时，触发点位于不同脉冲的上升沿上，因此波形显示混乱。

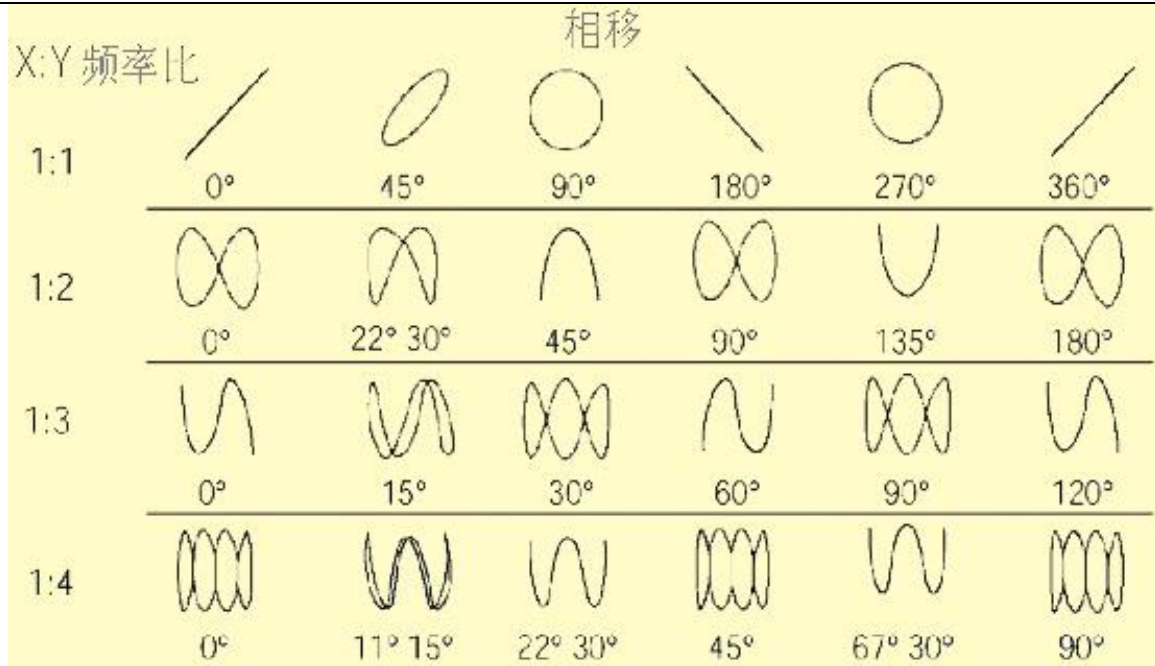


触发释抑通过控制释抑时间——触发电路重新启动的时间，可使复杂的波形稳定显示。如下图，使用触发释抑让触发点总是位于脉冲序列的某个同一脉冲上。



1.5 李沙育图形

李莎育图形是以通道 1 来表示 x 轴，通道 2 来表示 y，可通过李莎育图形来定性观察两通道之间的相位差。下表列出图形与相位的关系：



测量相位差的另一种方法：通过自动测量两通的延迟 ÷ 波形频率 × 2pi。

1.6 窗函数选择

在使用 FFT 观察频域的时候，绝大多数测量都要求使用窗函数，如 Hanning 或 Rectangular 窗函数。它们都适合典型的频率分析测量。选择这两种窗函数要在频率分辨率和幅度准确度间进行权衡。

窗	优点	缺点
Hanning	频率分辨率较好	幅度准确度较差
Hamming	幅度准确度较好	频率分辨率较差
Rectangle	频率分辨率最好	幅度准确度最差
Blackman	副值分辨率最好	频率分辨率最差

1.7 总结

本章节主要讲解了示波器的基础知识，初学的话一定要把这部分基础的知识点掌握，因为示波器是我们硬件调试必不可少的设备。

第2章 示波器操作说明及其介绍

本章节主要讲解示波器的操作说明及其相关介绍。

- 2.1 示波器使用前注意事项
- 2.2 示波器的按键操作说明
- 2.3 示波器滑动操作说明
- 2.4 对话框操作说明
- 2.5 示波器功能介绍
- 2.6 波形显示效果
- 2.8 2048 点浮点 FFT 计算的幅频显示
- 2.8 水平测量和垂直测量功能
- 2.9 普通触发以及采集波形的浏览功能
- 2.10 80 阶 Fir 低通滤波器设计
- 2.11 波形参考位置调节
- 2.12 FFT 幅频显示和测量值窗口的隐藏
- 2.13 幅值调节
- 2.14 低频采样率采集高频信号的炫酷效果
- 2.15 总结

2.1 示波器使用前注意事项

使用示波器前要特别注意以下三个问题：

2.1.1 ADC 和 DAC 的参考电压接到 3.3V 的地方

跳线帽连接如下：

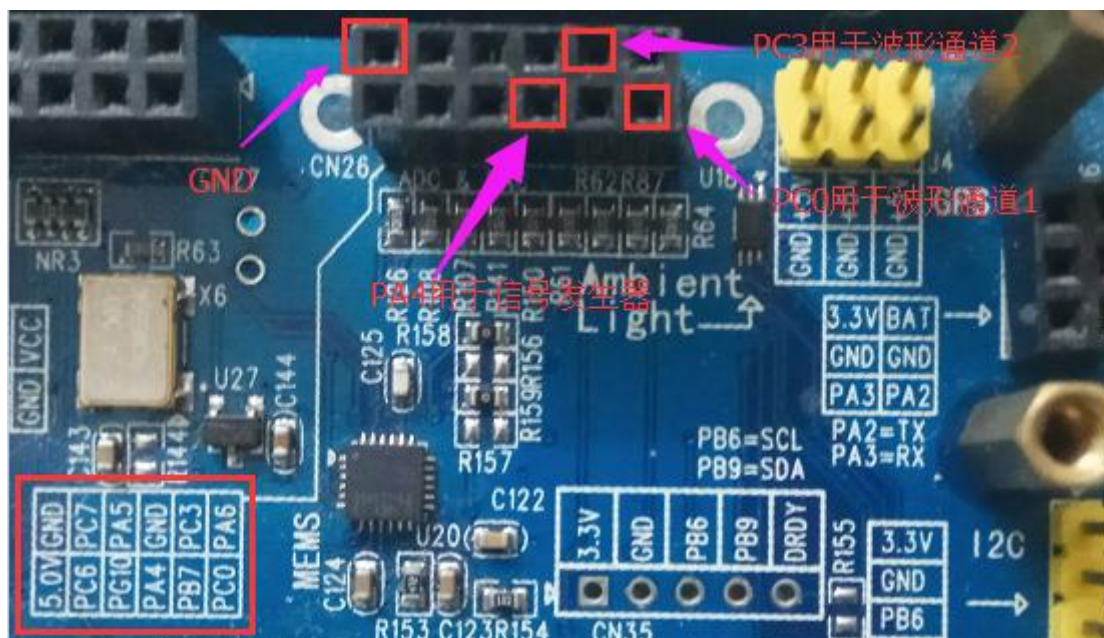


2.1.2 ADC 和 DAC 的引脚说明

双通道 ADC 和 DAC 信号发生器的引脚安排如下 (下面截图的左下角或者板子的背面也有丝印注释):

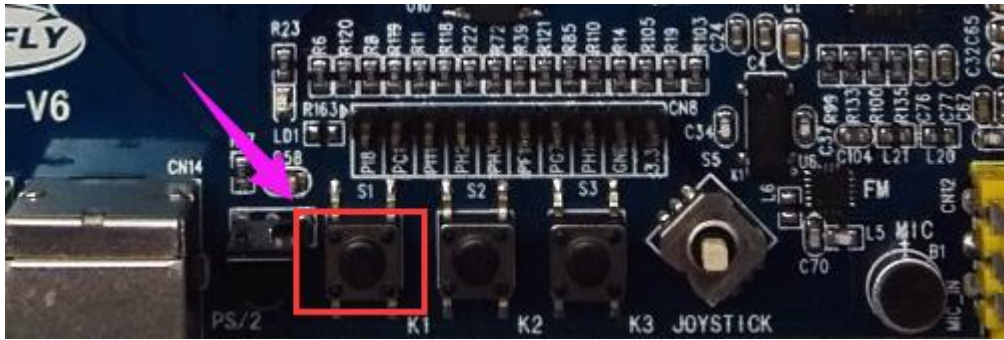
- ◆ PC0 引脚用于波形通道 1, 使用 ADC3 采样。简易电压采集也是用的这个引脚, 不过用的 ADC2 采样。
- ◆ PC3 引脚用于波形通道 2, 使用 ADC1 采样。
- ◆ PA4 引脚用于 DAC 信号发生器。

另外, 由于没有外接示波器的模拟前端, ADC 的输入电压不要超出 0-3.3V 的范围。DAC 的输出幅值范围是 0-3.3V。



2.1.3 触摸 4 点校准

第一次使用的话, 需要按下按键 K1 进行四点的触摸校准, K1 按键的位置如下, 仅电阻屏需要触摸校准, 电容屏无需校准 (详情请看本教程第 16 章附件 A):



2.2 示波器的按键操作说明

示波器使用按键 K1, K2, K3 和五向摇杆。

- K1 按键用于触摸校准。
- K2 按键用于波形显示的运行和暂停。
- K3 按键用于自动触发模式和普通触发模式的切换。
- 摇杆 OK 键用于摇杆上下左右四个方向的功能切换：
 - ◆ 设置为 AdjFreq 时，摇杆的左右按键分别用于降低采样率和提高采样率。
 - ◆ 设置为 AdjAmplitude 时，摇杆的上下按键分别用于放大和缩小幅值单位。
 - ◆ 设置为 ChangeRefPos 时，摇杆的上下按键分别用于上移和下移波形。
 - ◆ 设置为 ChangeCursorVA 时，摇杆的上下按键分别水平测量游标 a 的上移和下移。
 - ◆ 设置为 ChangeCursorVB 时，摇杆的上下按键分别水平测量游标 b 的上移和下移。
 - ◆ 设置为 ChangeCursorHA 时，摇杆的左右按键分别垂直测量游标 a 的左移和右移。
 - ◆ 设置为 ChangeCursorHB 时，摇杆的左右按键分别垂直测量游标 b 的左移和右移。
 - ◆ 设置为 ChangeTrigger 时，摇杆的上下键用于触发电压值的调节，左右键用于浏览采集的 1024*2 个 ADC 数据。

2.3 示波器的滑动操作说明

滑动功能支持水平滑动和垂直滑动，通过 Settings 按钮弹出对话框的 X Motion 和 Y Motion。

◆ Y Motion

上电后默认是垂直滑动（选择的 Y Motion），主界面波形显示区（600*480）的左半部分调节波形通道 1 的上下滑动，波形显示区的右半部分调节波形通道 2 的上下滑动。

◆ X Motion

如果选择的 X Motion，那么可以通过水平滑动浏览波形通道 1 采集的 2K 个 ADC 数据，推荐在暂停状态浏览。波形通道 2 不支持水平滑动浏览。

Settings 对话框效果：

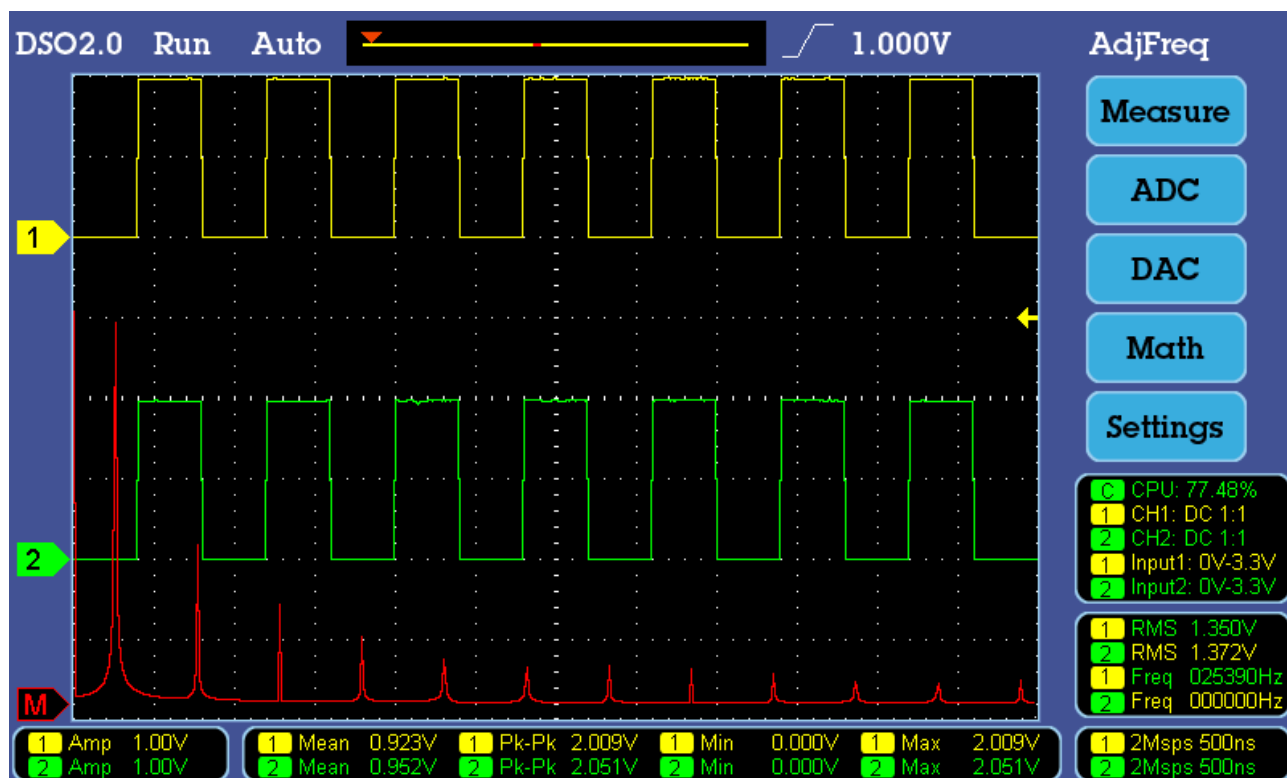


Settings



- ☒ Y Motion
- ☐ X Motion
- ☒ Hide Cursor
- ☐ Display HorizontalCursor
- ☐ Display VerticalCursor
- ☐ Hide RFFT2048 Display
- ☐ Hide AmplitudeDlg, StatusDlg, ScaleDlg and SysInfoDlg
- ☐ MoveCursorStep = 1
- ☐ MoveCursorStep = 2
- ☐ MoveCursorStep = 5
- ☒ MoveCursorStep = 10

主界面显示效果（黑色区域是波形显示区）：



2.4 对话框操作说明

示波器共提供了 5 个对话框用于波形的设置，具体说明看本教程第 11 章即可。

◆ Measure 对话框用于测量值的动态添加，当前仅支持标红的六个：

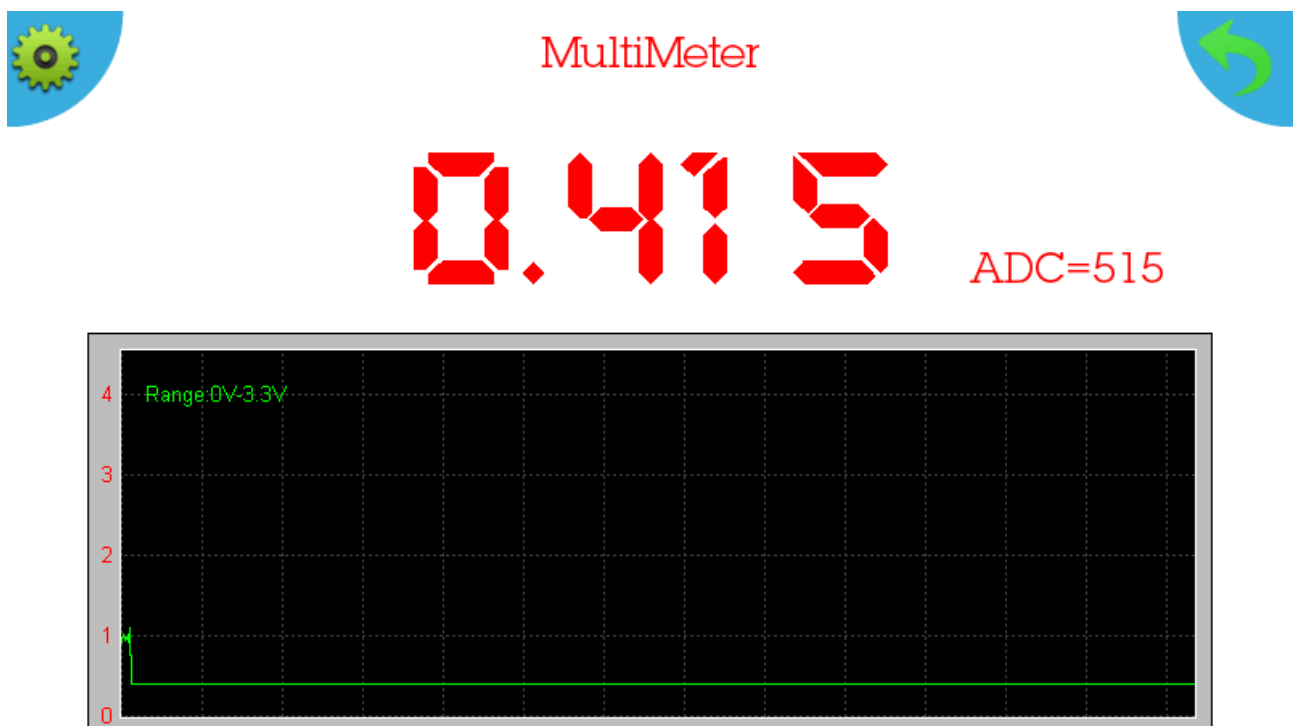


Wave Measure



☐ Snapshot All	☐ Negative Duty Cycle	☐ Positive Pulse Count
☒ Frequency	☐ Burst Width	☐ Negative Pulse Count
☐ Period	☒ Peak-to-peak	☒ RMS
☐ Rise Time	☐ Amplitude	☐ Cycle RMS
☐ Fall Time	☒ Max	☒ Mean
☐ Delay	☒ Min	☐ Cycle Mean
☐ Phase	☐ High	☐ Rising Edge Count
☐ Positive Pulse Width	☐ Low	☐ Falling Edge Count
☐ Negative Pulse Width	☐ Positive Overshoot	☐ Area
☐ Positive Duty Cycle	☐ Negative Overshoot	☐ Cycle Area

◆ ADC 对话框，实现简单的电压测量，显示波形记录：



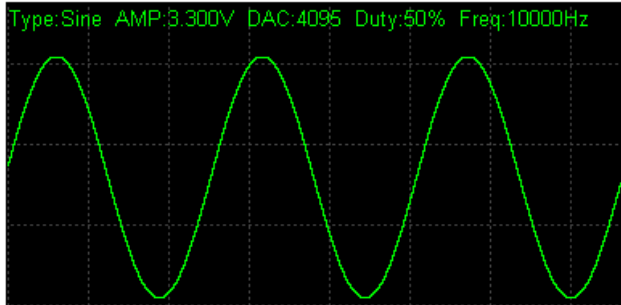
◆ DAC 信号发生器对话框，支持正弦波，方波和三角波的幅值，占空比以及频率设置。



Waveform Generator



Type: Sine AMP: 3.300V DAC: 4095 Duty: 50% Freq: 10000Hz



DAC: 0 - 4095

Duty: 2% - 98%

Sine

Square

Ramp

Input Freq: 1Hz - 50KHz

10000

1	2	3	CL
4	5	6	C
7	8	9	-
-	0	-	Output

◆ Math 对话框用于 80 阶 Fir 低通滤波的截止频率选择：



80 Order Fir Lowpass



Fs=2Msps, Scale=500ns

- ☒ None
- ☐ Fc=100KHz
- ☐ Fc=200KHz
- ☐ Fc=300KHz
- ☐ Fc=400KHz
- ☐ Fc=500KHz
- ☐ Fc=600KHz
- ☐ Fc=700KHz

Fs=200Ksps, Scale=5us

- ☒ None
- ☐ Fc=10KHz
- ☐ Fc=20KHz
- ☐ Fc=30KHz
- ☐ Fc=40KHz
- ☐ Fc=50KHz
- ☐ Fc=60KHz
- ☐ Fc=70KHz

Fs=20Ksps, Scale=50us

- ☒ None
- ☐ Fc=1KHz
- ☐ Fc=2KHz
- ☐ Fc=3KHz
- ☐ Fc=4KHz
- ☐ Fc=5KHz
- ☐ Fc=6KHz
- ☐ Fc=7KHz

◆ Settings 对话框用于幅值窗口，状态窗口，频率窗口，系统信息窗口，水平和垂直测量游标的隐藏与显示，水平和垂直滑动切换以及水平和垂直测量游标的移动步数调节。



Settings



- ☒ Y Motion
- ☐ X Motion
- ☒ Hide Cursor
- ☐ Display HorizontalCursor
- ☐ Display VerticalCursor
- ☐ Hide RFFT2048 Display
- ☐ Hide AmplitudeDlg, StatusDlg, ScaleDlg and SysInfoDlg
- ☐ MoveCursorStep = 1
- ☐ MoveCursorStep = 2
- ☐ MoveCursorStep = 5
- ☒ MoveCursorStep = 10

2.5 示波器功能介绍

二代示波器采用 STM32F429BIT6 自带的 12bit ADC 进行数据采集,当前二代示波器依然没有外接示波器模拟前端,全部采用 F429 内部资源以及软件功能实现,相比一代示波器,程序设计要更复杂些。

● 三款主流 RTOS 版本：

- ◆ 基于 uCOS-III+STemWin+FatFS+ARM_DSP_Lib 版本。
- ◆ 基于 FreeRTOS+STemWin+FatFS+ARM_DSP_Lib 版本。
- ◆ 基于 RTX+emWin+RL-FlashFS+ARM_DSP_Lib 版本。

● F429 内部资源利用以及 RAM 空间分配：

- ◆ 外部 32 位带宽 SDRAM 用于 emWin 的动态内存和 LCD 显存,刷新波形的性能强劲。
- ◆ F429 内部 192KB RAM 用于 ADC 数据存储和部分全局变量。
- ◆ F429 内部 64KB CCM RAM 用于 RTOS 的任务栈空间,FFT 计数,FIR 计算等需要频繁操作的变量。
- ◆ F429 内设使用情况如下,这些内设从开机起一直都在工作,进行大量数据的实时采集以及 DAC 的波形输出：
 - ADC1, ADC2, ADC3。
 - DAC1。
 - TIM1 的 OC1 和 OC3, TIM6, TIM8。
 - DMA2 Stream1, DMA2 Stream0 和 DMA1_Stream5

- 支持双通道 ADC 数据的实时采集

- ◆ ADC1 和 ADC3 都采用 DMA 方式进行数据传输,数据一直实时采集,只有改变采样率时才做暂停和重新启动处理。每个 ADC 分配 1024*20 字节进行 DMA 数据传输。

- 支持自动触发:

采用软件方式实现,可以在范围 0V-3.3V 内调节触发值,可在波形刷新和波形暂停状态下浏览 1024*2 个 ADC 数据。

- 支持普通触发:

采用 STM32F429 的模拟看门狗实现上升沿实时检测,可以在范围 0V-3.3V 内调节触发值,触发后记录触发值前后 1024 个 ADC 数据,即 1024*2 个。可在波形刷新和波形暂停状态下浏览 1024*2 个 ADC 数据。

- 支持自动触发和普通触发下,触发位置指示:

每个 ADC 通道实际采集的是 1024*10 个数据,实际处理的是 1024*2 个数据,通过如下方式指示当前显示波形在是 1024*2 个 ADC 数据中的起始位置,这种方式也方便采集数据的浏览指示:



横线中间的红点用来指示普通触发时触发值在 1024*2 个 ADC 数据中的位置,这个位置是固定的。

- 支持 30 种测量值的动态添加:

当前实现平均值,最大值,最小值,峰峰值, RMS 和频率估计。

- 支持水平游标尺度和垂直游标尺度:

水平游标尺度测量幅度,垂直游标尺度测量时间,且移动单位可调。

- 支持波形的运行和暂停:

暂停模式下依然支持波形的拉伸和放缩,水平测量,垂直测量和 FFT 幅频显示。

- 支持波形显示参考位置调节:

采用软件方式实现。

- 支持滑动浏览波形和滑动调节波形参考位置:

通过 Settings 对话框切换水平滑动和垂直滑动。

- 支持 2048 点浮点 FFT 计算的幅频显示:

自动触发模式支持 FFT 幅频显示,普通触发方式不支持,由于要采集 2048 点才可以实现一次 FFT 运算,限制 Scale 窗口显示范围 2.8Msps 357ns ----- 2Ksps 500us,后面的低频采样暂不做支持。

- 支持 21 种 80 阶 Fir 低通滤波器设计:

- ◆ F_s (采样率) = 2Msps 时,即 Scale 窗口显示 2Msps 500ns 时, F_c (截止频率)的 7 种选择:
 - $F_c = 100\text{KHz}$
 - $F_c = 200\text{KHz}$
 - $F_c = 300\text{KHz}$

➤ $F_c = 400\text{KHz}$

➤ $F_c = 500\text{KHz}$

➤ $F_c = 600\text{KHz}$

➤ $F_c = 700\text{KHz}$

◆ F_s (采样率) = 200Ksps 时, 即 Scale 窗口显示 200Ksps 5us 时, F_c (截止频率)的 7 种选择:

➤ $F_c = 10\text{KHz}$

➤ $F_c = 20\text{KHz}$

➤ $F_c = 30\text{KHz}$

➤ $F_c = 40\text{KHz}$

➤ $F_c = 50\text{KHz}$

➤ $F_c = 60\text{KHz}$

➤ $F_c = 70\text{KHz}$

◆ F_s (采样率) = 20Ksps 时, 即 Scale 窗口显示 50us 时, F_c (截止频率)的 7 种选择:

➤ $F_c = 1\text{KHz}$

➤ $F_c = 2\text{KHz}$

➤ $F_c = 3\text{KHz}$

➤ $F_c = 4\text{KHz}$

➤ $F_c = 5\text{KHz}$

➤ $F_c = 6\text{KHz}$

➤ $F_c = 7\text{KHz}$

● 支持如下 21 种采样率时的波形显示:

◆ 2.8Msps

◆ 2Msps

◆ 1Msps

◆ 500Ksps

◆ 200Ksps

◆ 100Ksps

◆ 50Ksps

◆ 20Ksps

◆ 10Ksps

◆ 5Ksps

◆ 2Ksps

◆ 1Ksps

◆ 500sps

- ◆ 200sps
- ◆ 100sps
- ◆ 50sps
- ◆ 20sps
- ◆ 10sps
- ◆ 5sps
- ◆ 2sps
- ◆ 1sps
- 支持如下 10 种波形幅度：
 - ◆ 10.0V
 - ◆ 5.00V
 - ◆ 2.00V
 - ◆ 1.00V
 - ◆ 500mV
 - ◆ 200mV
 - ◆ 100mV
 - ◆ 50.0mV
 - ◆ 20.0mV
 - ◆ 10.0mV

下面通过以下 10 点对示波器的功能进行一个全面的介绍：

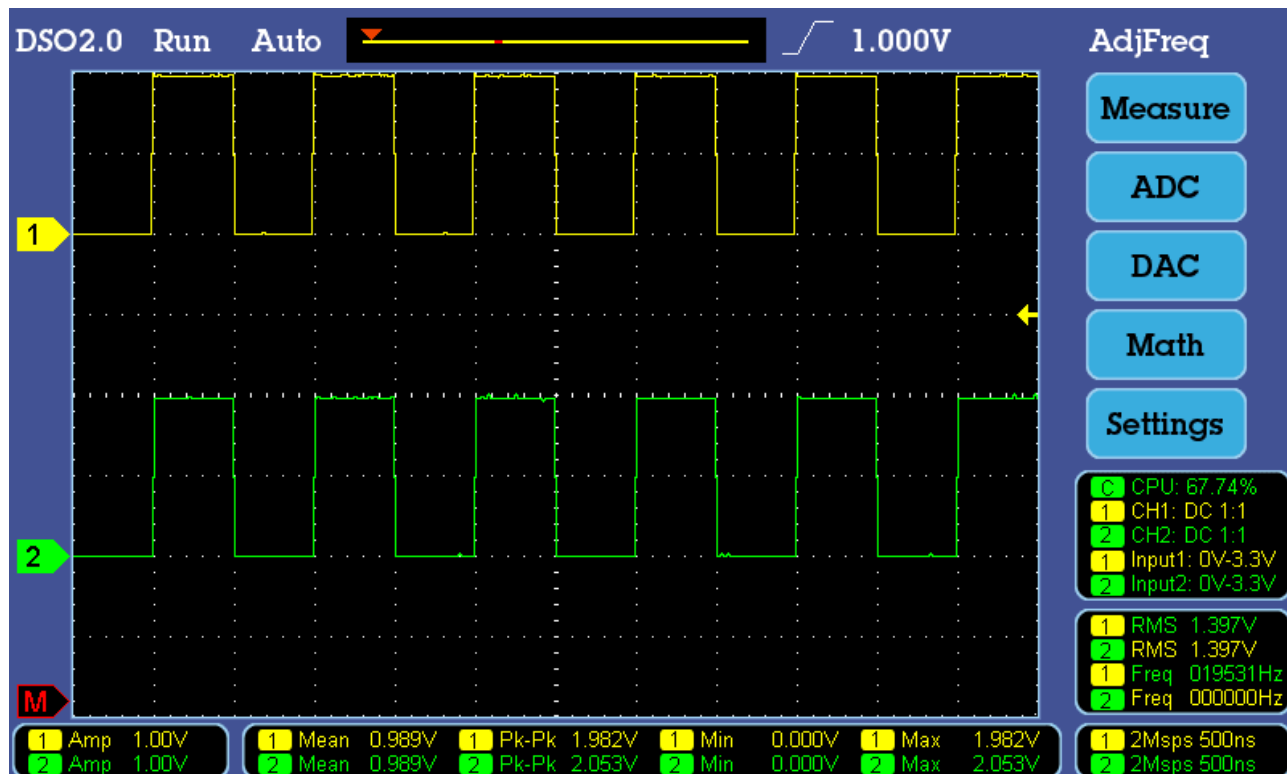
- 采集正弦波，方波和三角波效果展示。
- 2048 点浮点 FFT 计算的幅频显示。
- 水平和垂直测量功能。
- 普通触发以及采集波形的浏览功能。
- 80 阶 Fir 低通滤波设计。
- 波形参考位置调节。
- FFT 幅频显示和测量值窗口的隐藏。
- 幅值调节。
- 低频采样率采集高频信号的显示效果。
- 对话框功能介绍（在本章 2.4 小节已经做介绍）。

注意：测试波形全部由 Agilent 33220A 任意波形发生器生成。

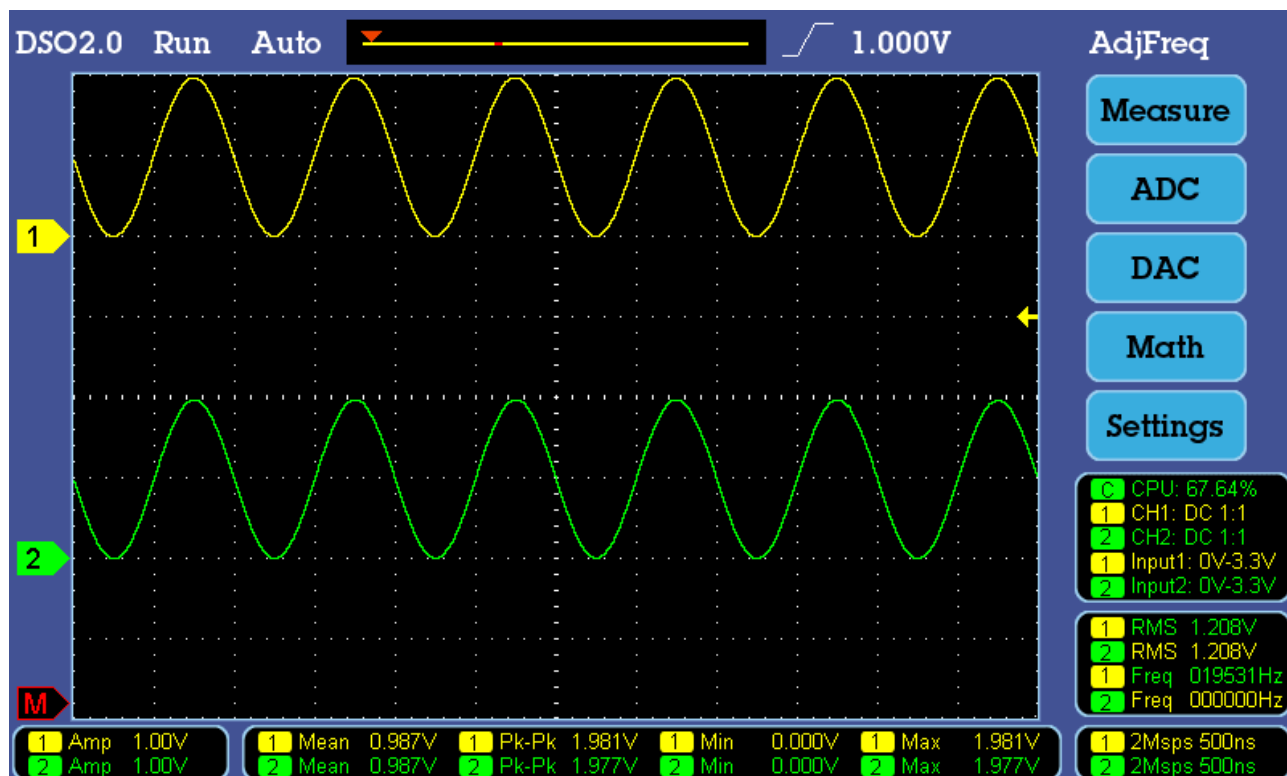
2.6 波形显示效果

双通道示波器采样率 2Msps。

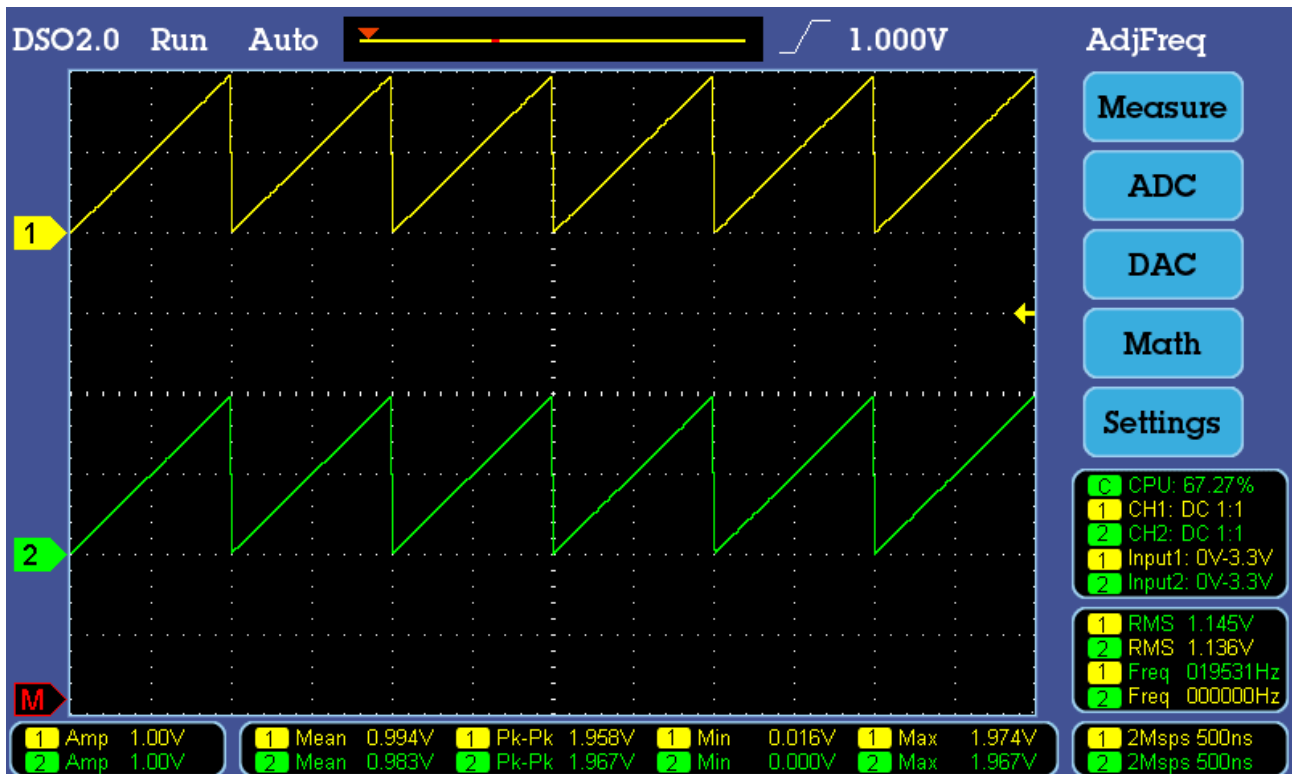
- 双通道采集同一路波形，20KHz 方波，峰峰值 2V，直流偏移 1V。



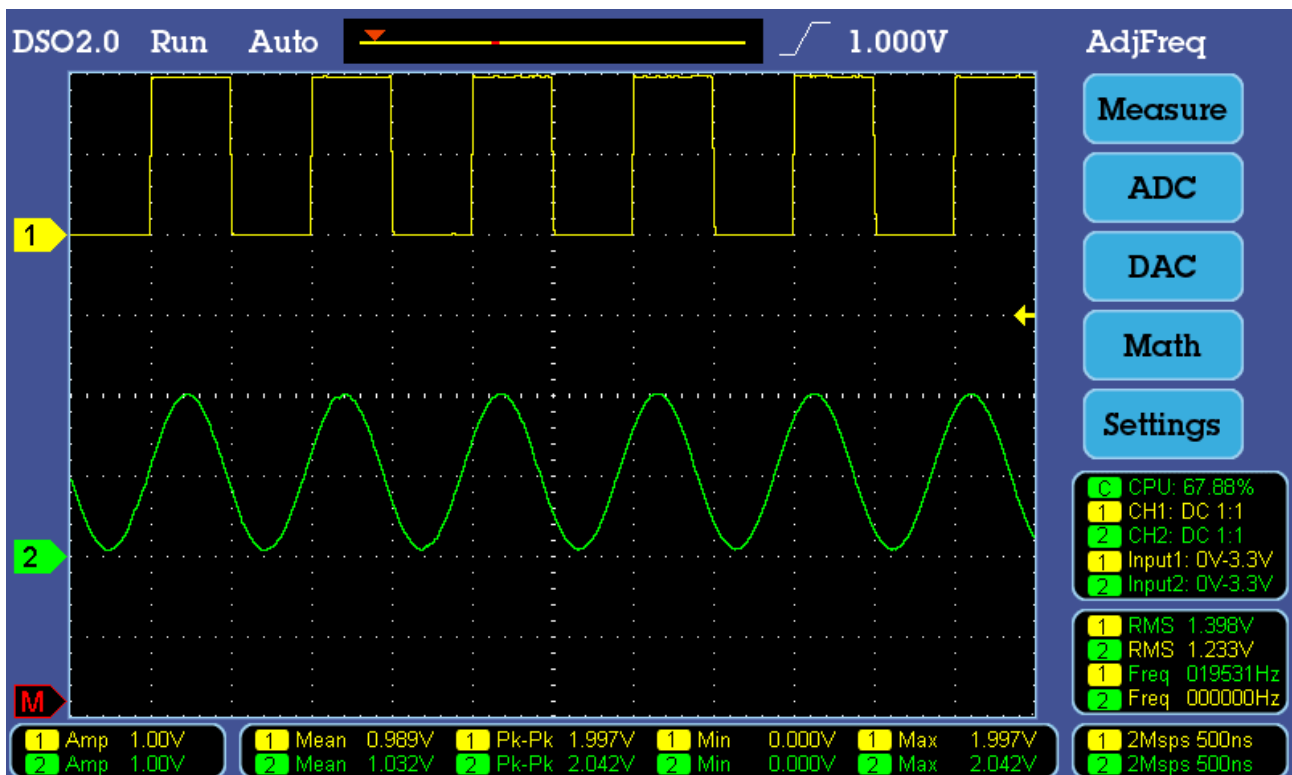
- 双通道采集同一路波形，20KHz 正弦波，峰峰值 2V，直流偏移 1V。



- 双通道采集同一波形，20KHz 三角波，峰峰值 2V，直流偏移 1V。



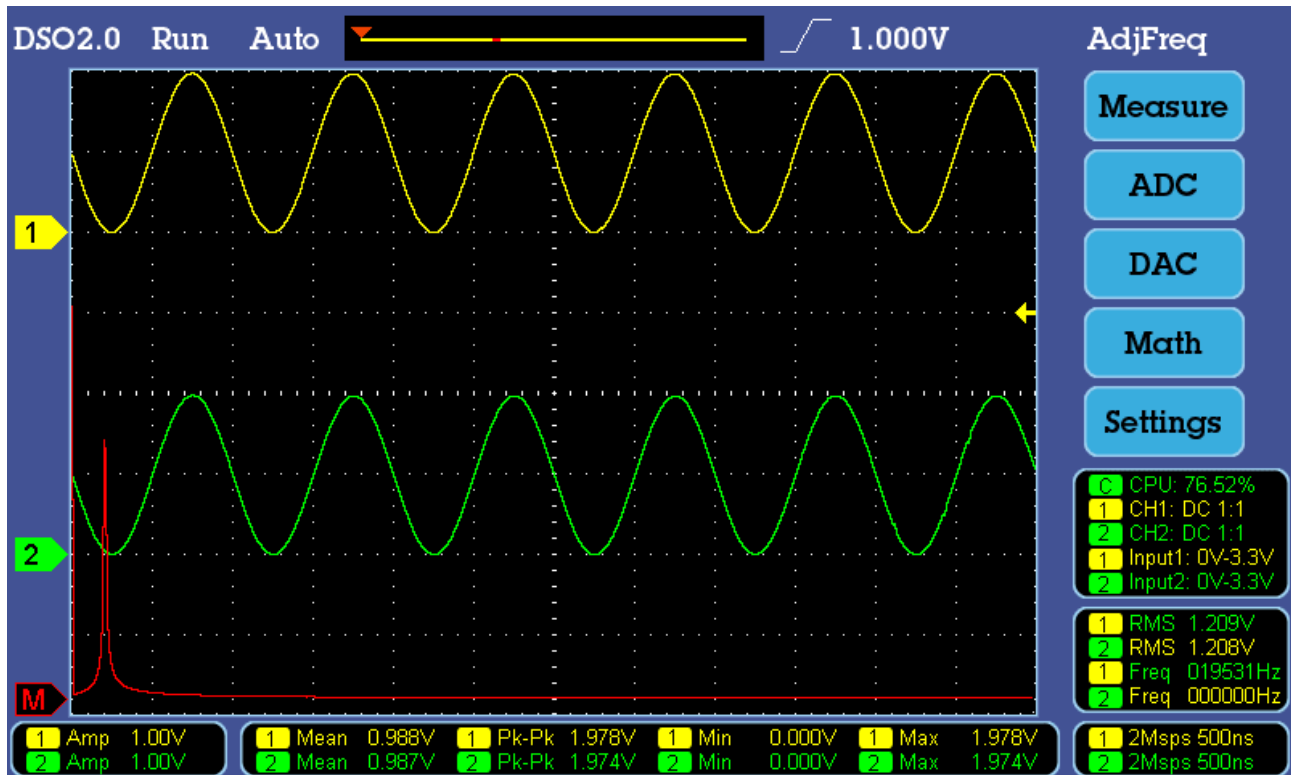
- 双通道采集一路 20KHz 方波 (由信号发生器提供)，一路 20KHz 正弦波 (由 F429 自带 DAC 提供)。



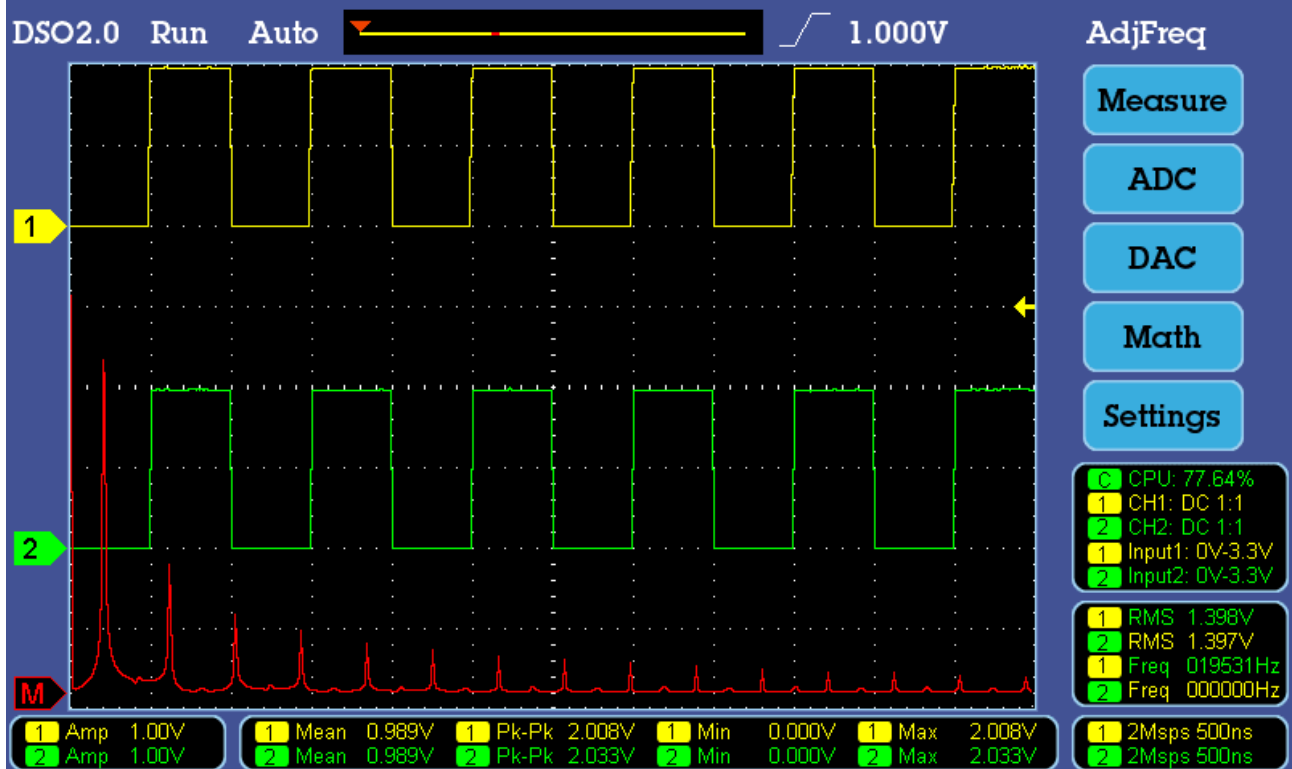
2.7 2048 点浮点 FFT 计算的幅频显示

双通道示波器采样率 2Msps。

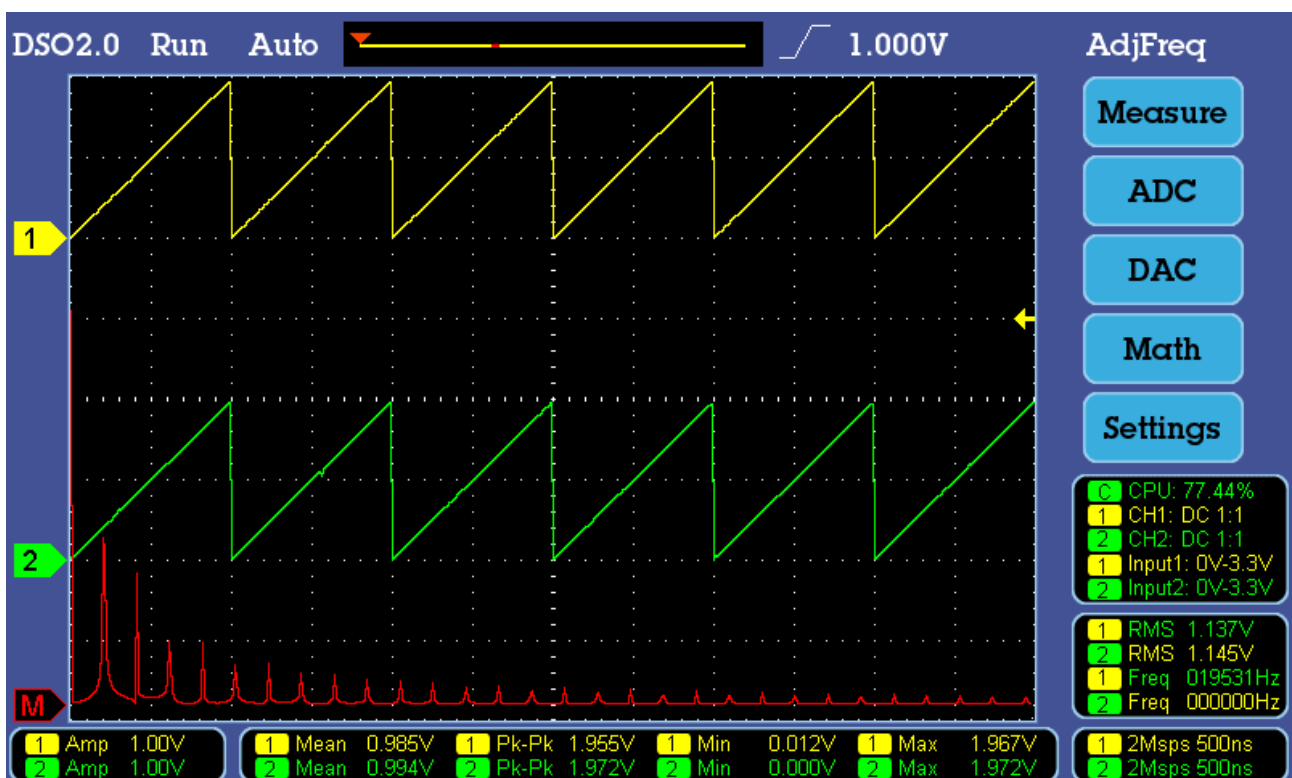
- 测试信号 1：20KHz 正弦波，峰峰值 2V，直流偏移 1V，FFT 估算频率 19531Hz。



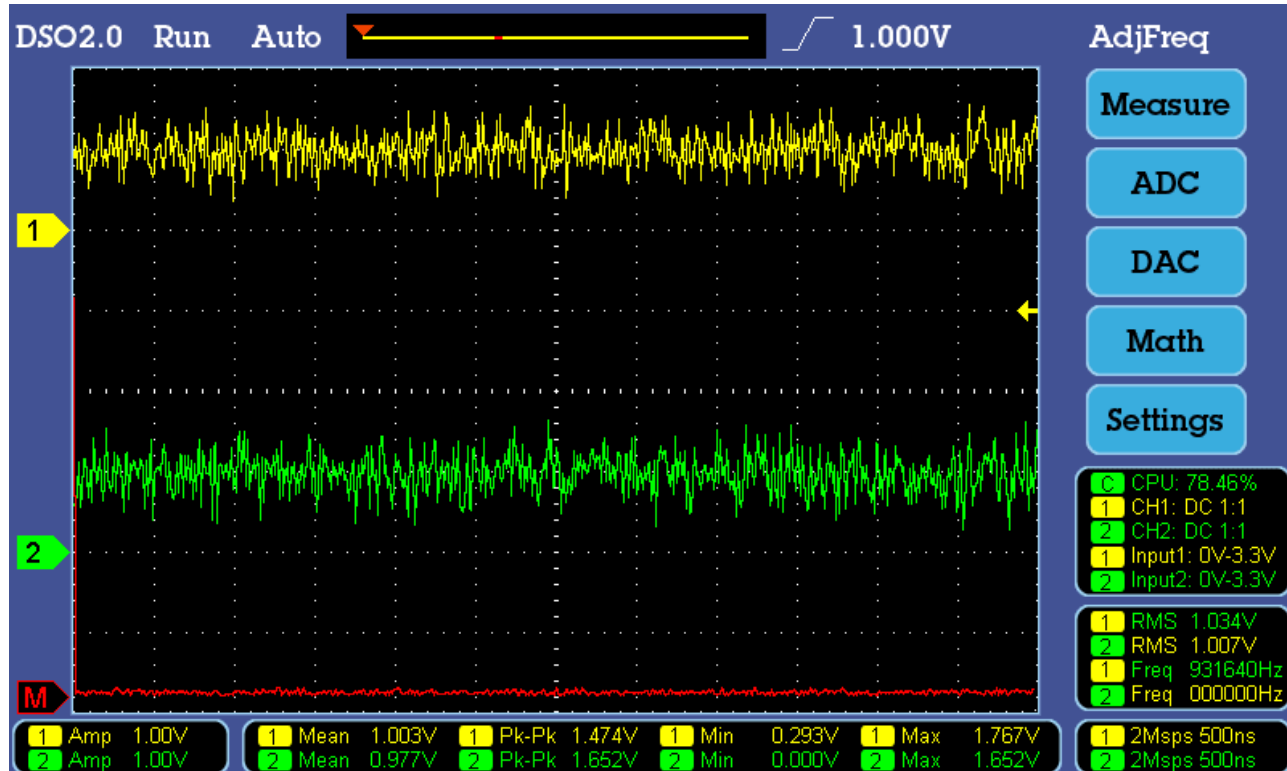
- 测试信号 2：20KHz 方波，峰峰值 2V，直流偏移 1V，FFT 估算频率 19531Hz。。



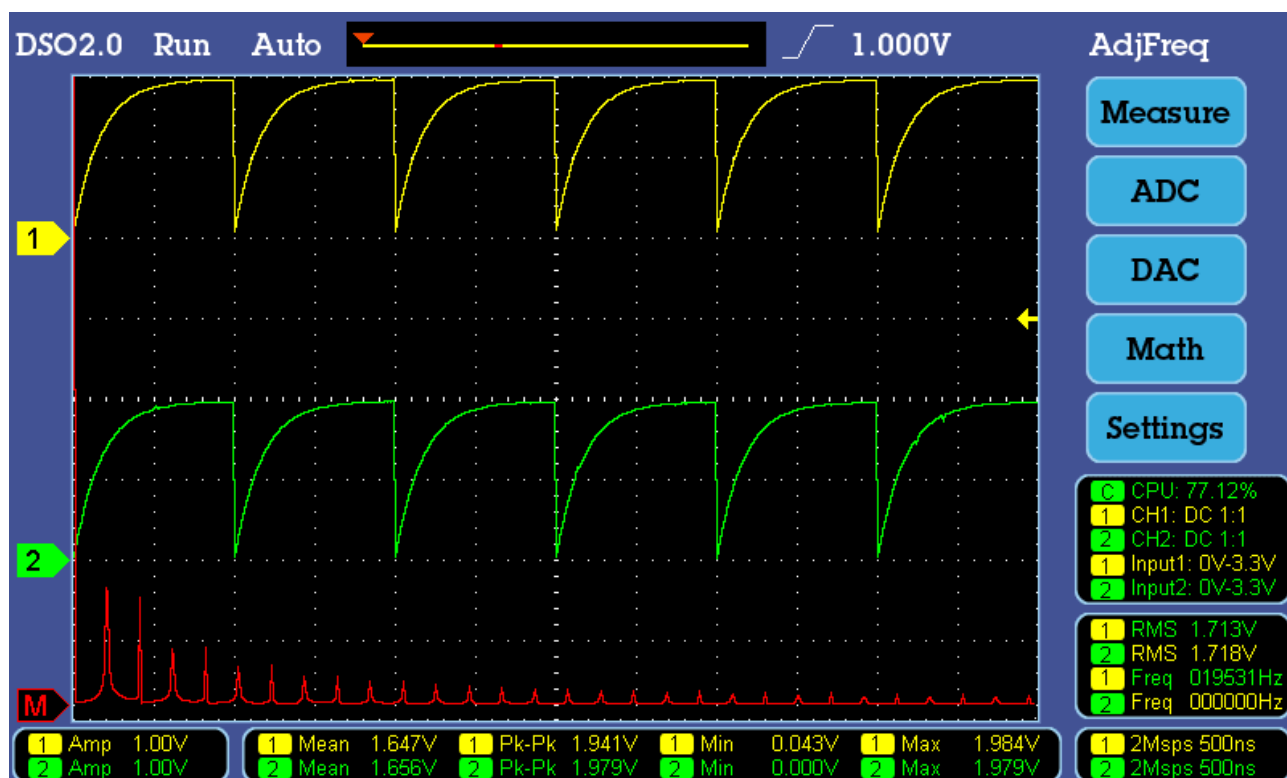
- 测试信号 3 : 20KHz 三角波, 峰峰值 2V, 直流偏移 1V, FFT 估算频率 19531Hz。



- 测试信号 4 : 噪声, 直流偏移 1V。



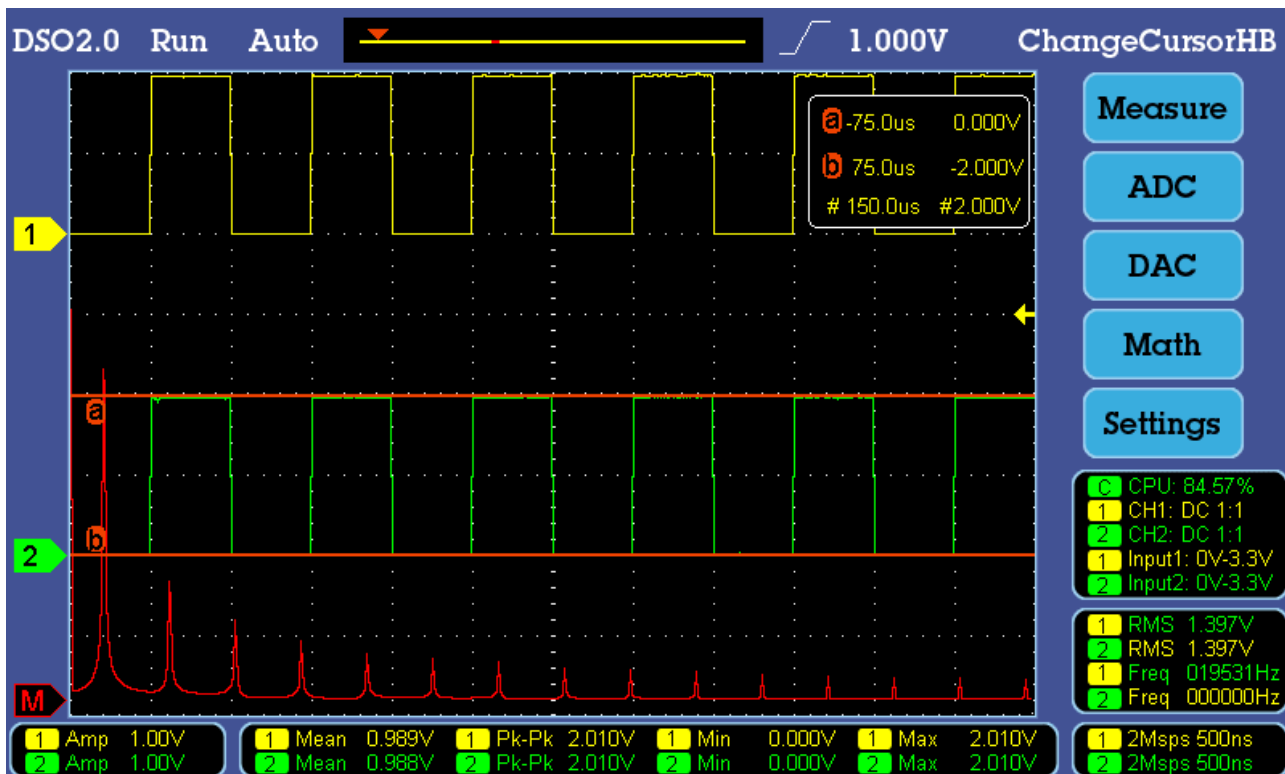
- 测试信号 5 : 20KHz 任意波, 峰峰值 2V, 直流偏移 1V, FFT 估算频率 19531Hz。



2.8 水平测量和垂直测量功能

双通道示波器采样率 2Msps, 采用水平和垂直测量功能测试正弦波和方波。

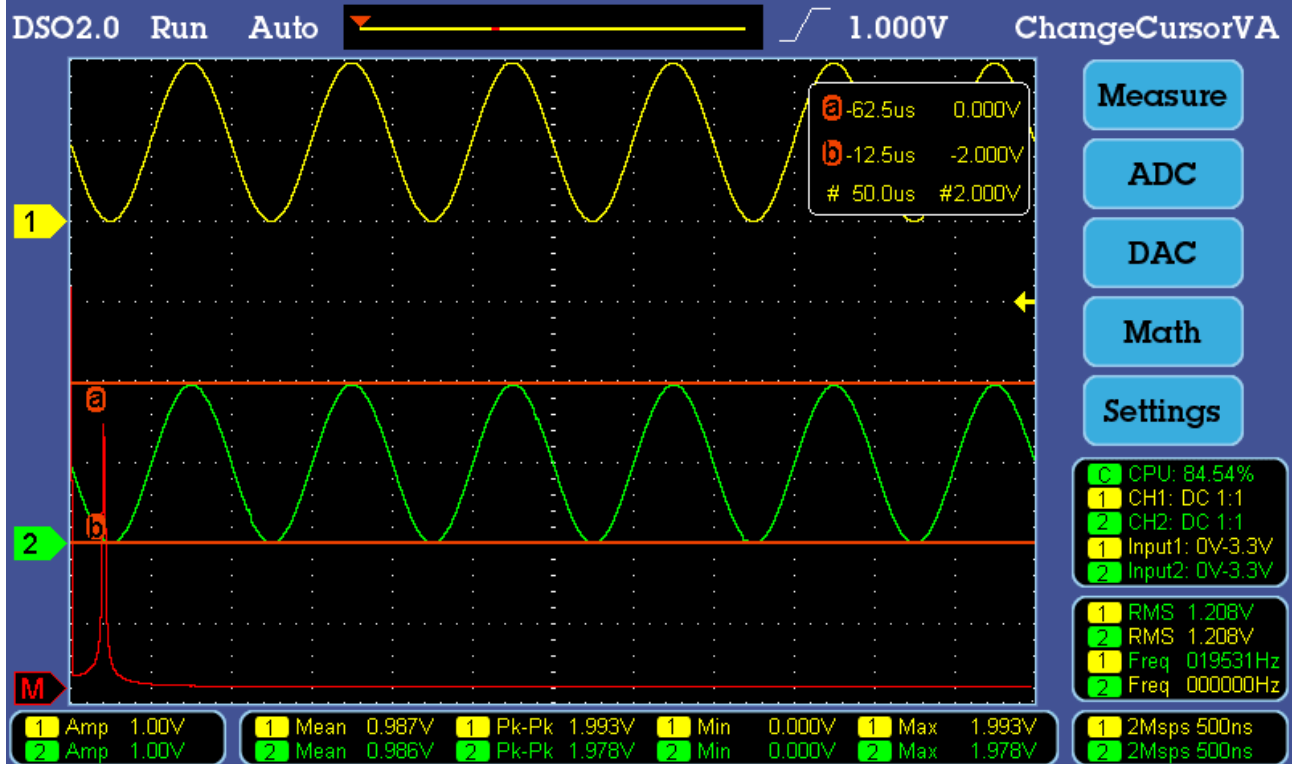
- 测量方波，20KHz，峰峰值 2V，直流偏移 1V，下面是水平测量，测得峰峰值 2V。



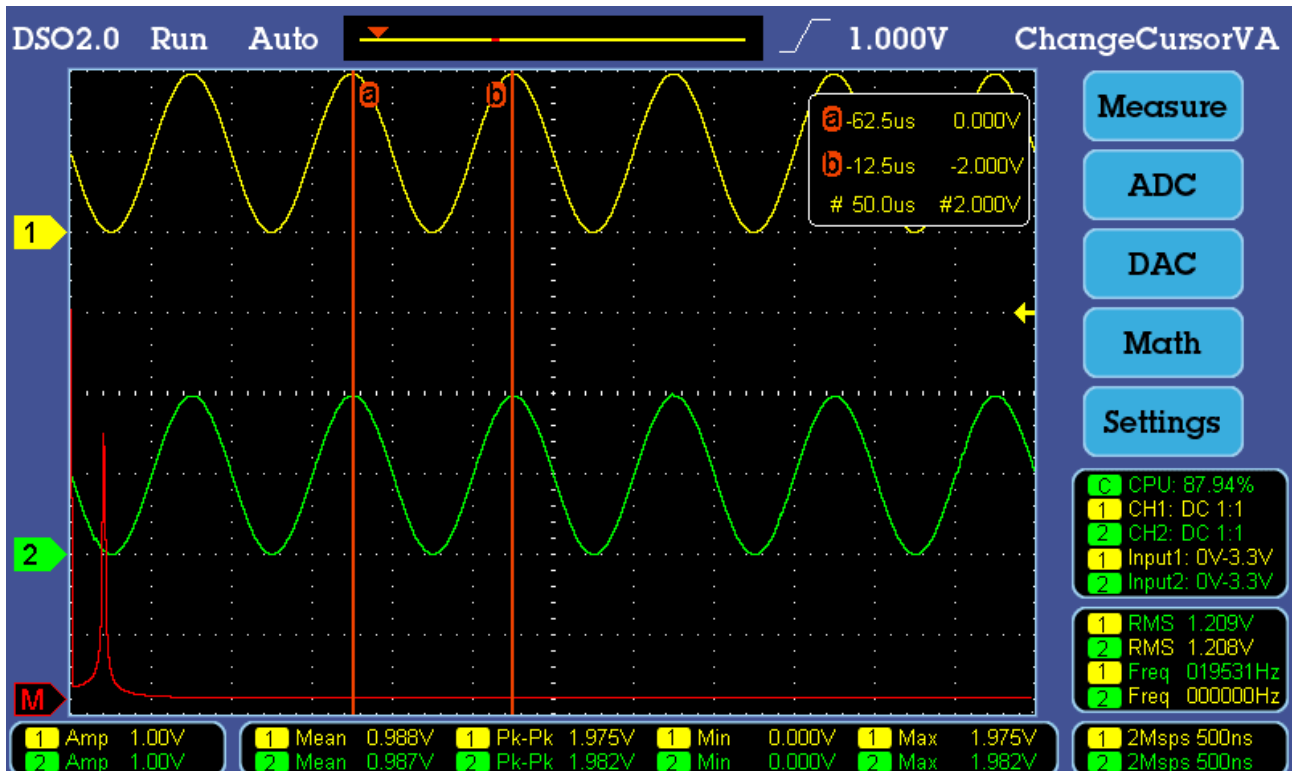
- 测量方波，20KHz，峰峰值 2V，直流偏移 1V，下面是垂直测量，测得周期 50ns，也就是 20KHz



- 测量正弦波，20KHz，峰峰值 2V，直流偏移 1V，下面是水平测量，测得峰峰值 2V



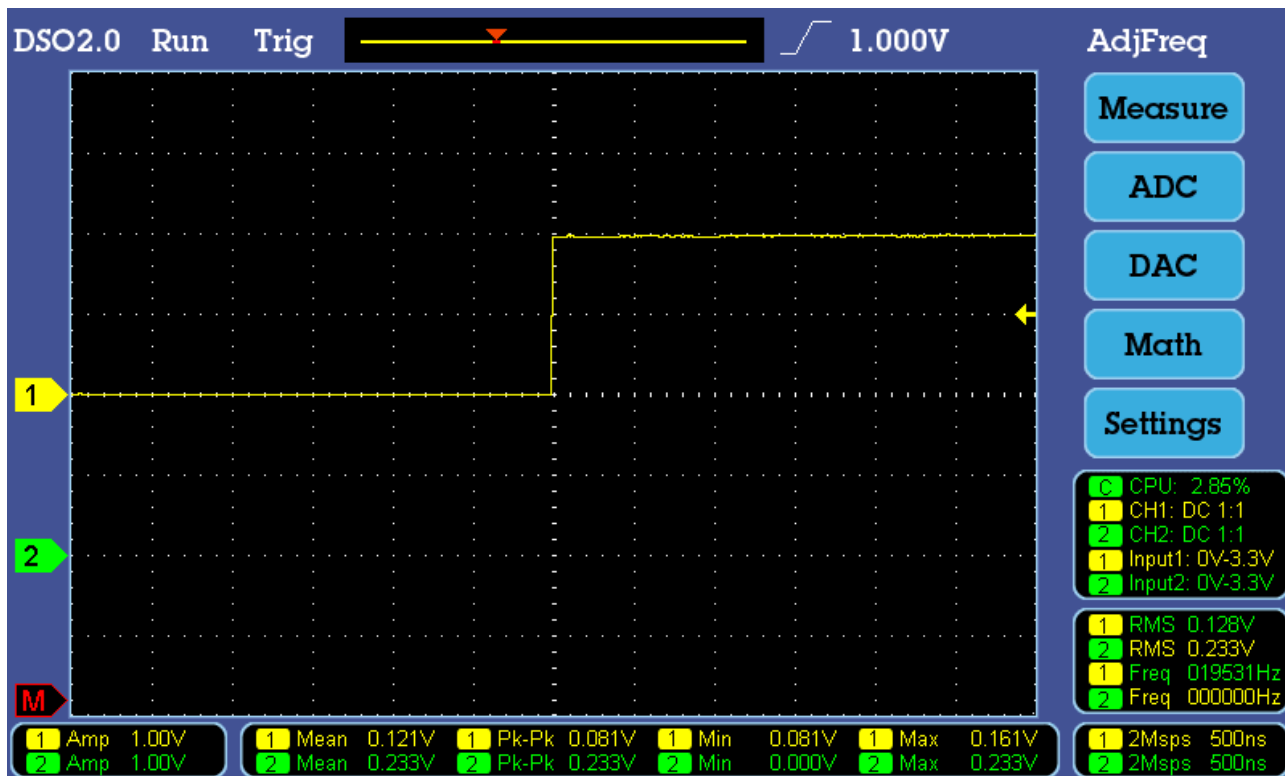
- 测量正弦波，20KHz，峰峰值 2V，直流偏移 1V，下面是垂直测量，测得周期 50ns，也就是 20KHz



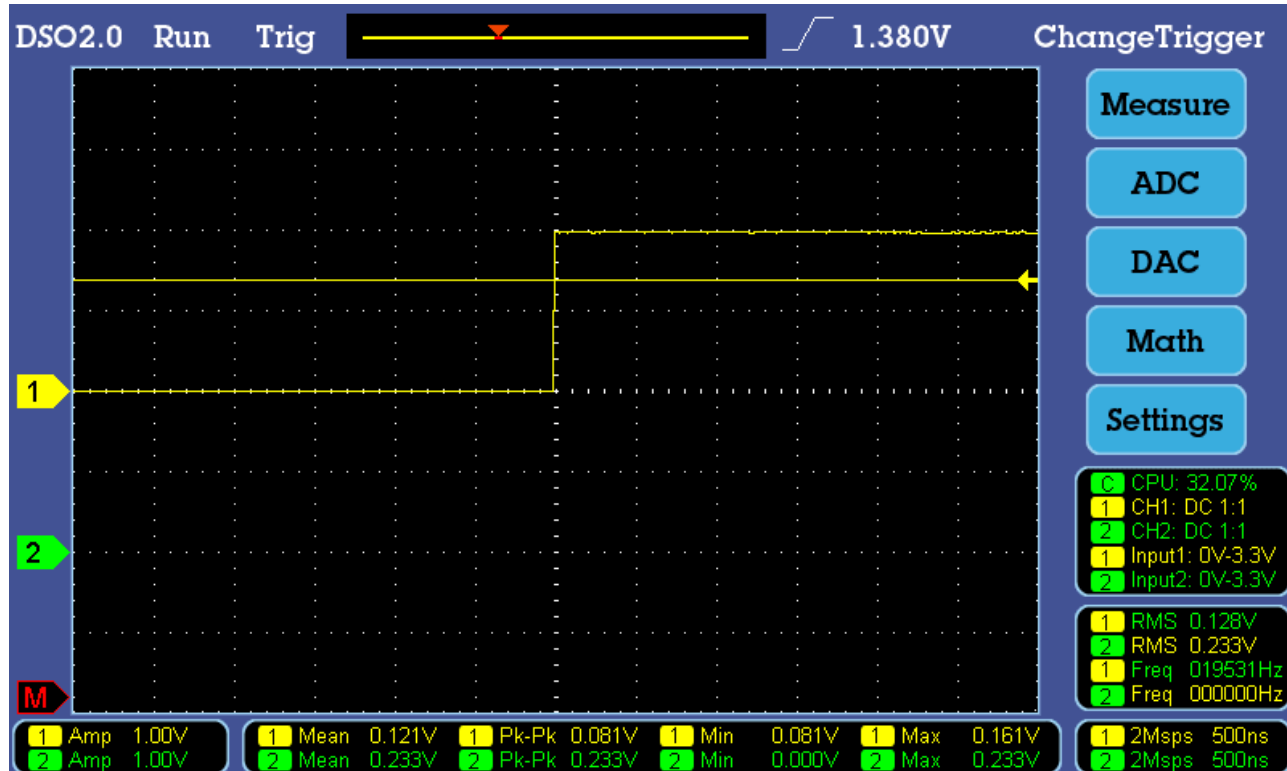
2.9 普通触发以及采集波形的浏览功能

双通道示波器采样率 2Msps，触发值设置为 1V。测量信号 5Hz 方波，峰峰值 2V，直流偏移 1V。此功能适合记录低频触发时的数据记录。

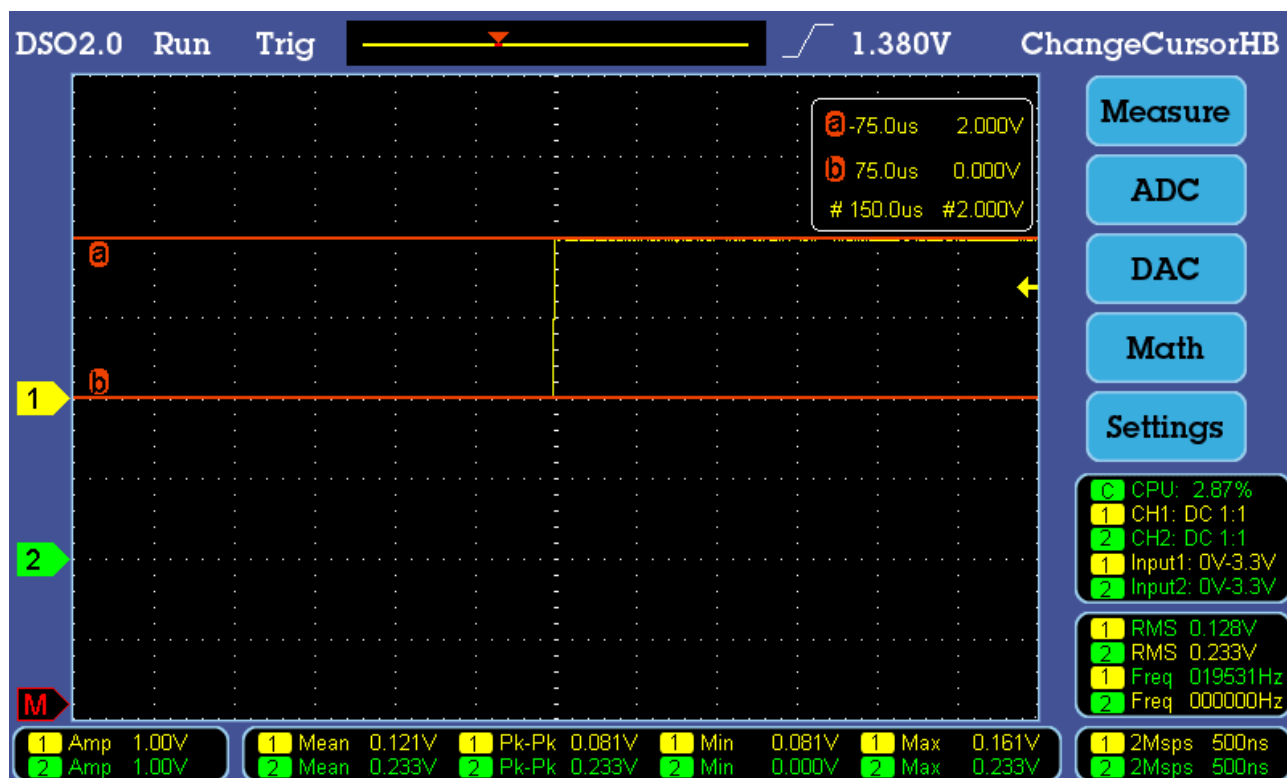
- 采集到的上升沿信号



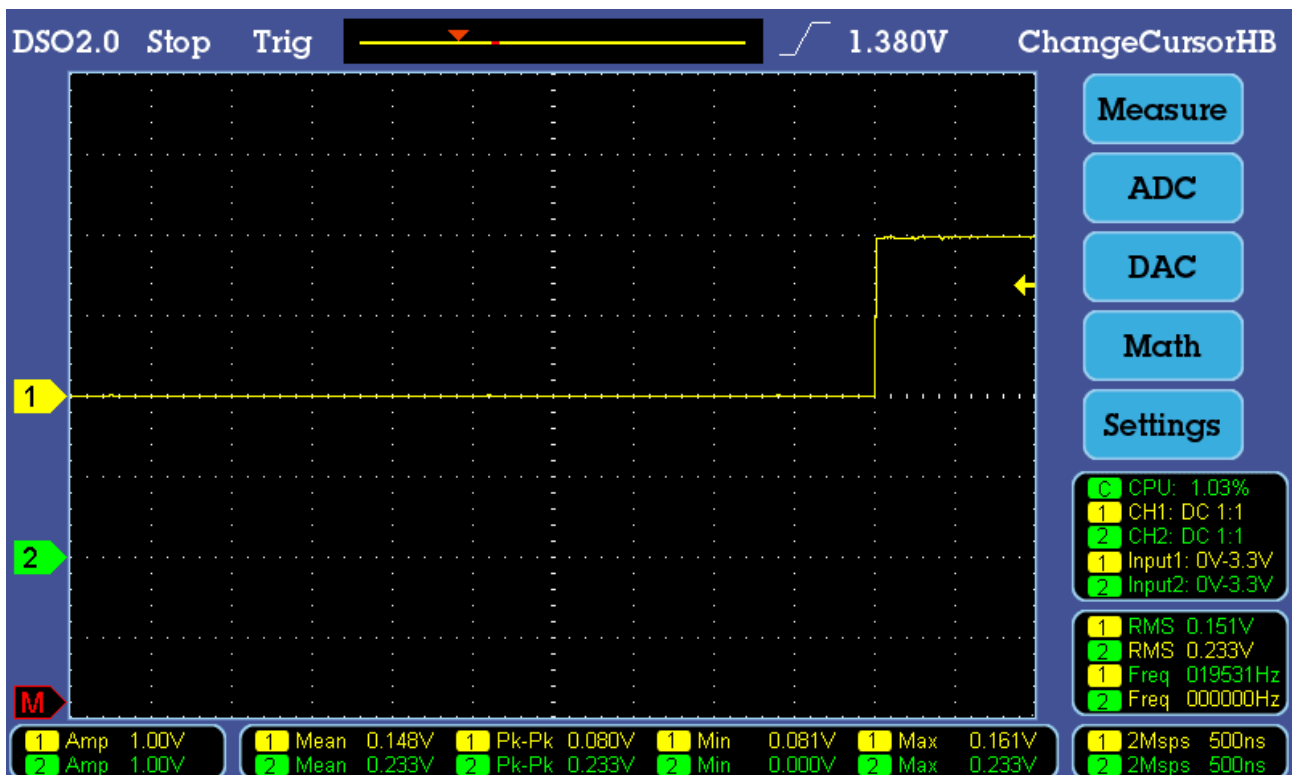
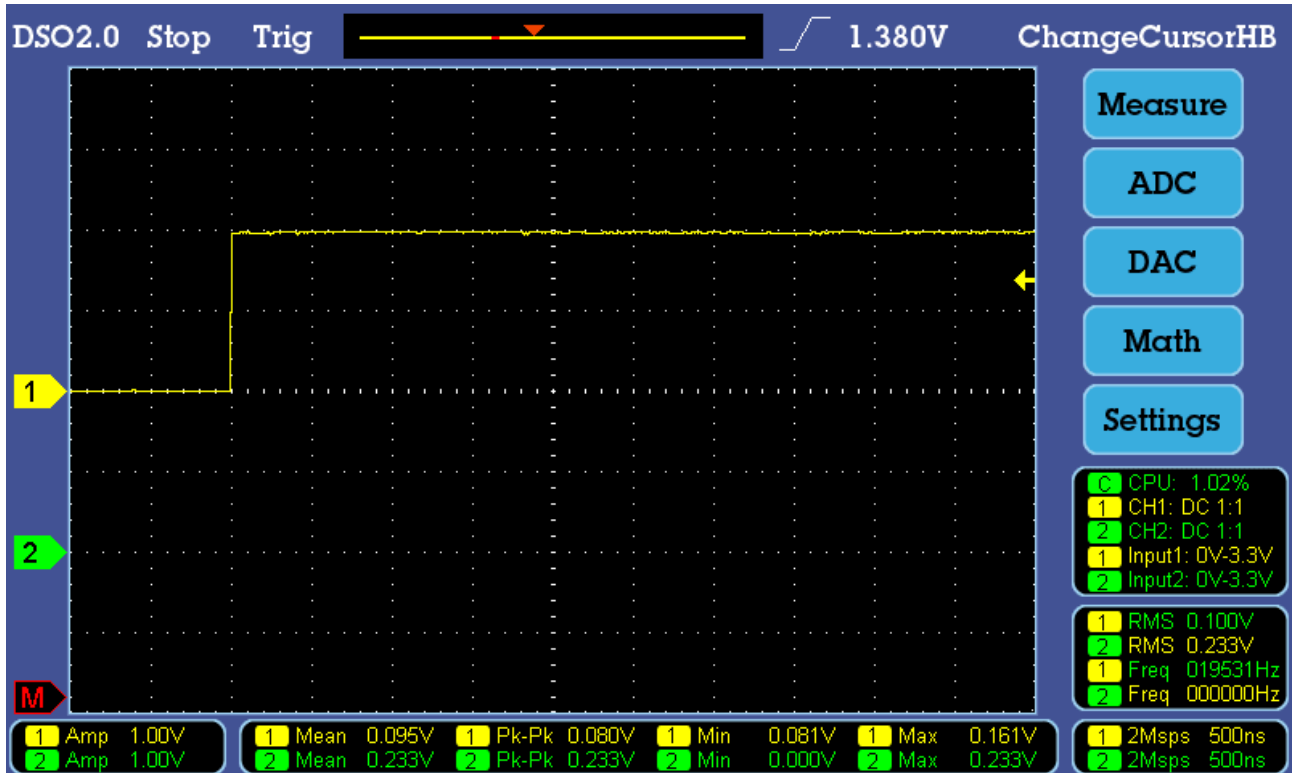
- 修改触发值



● 测量触发信号



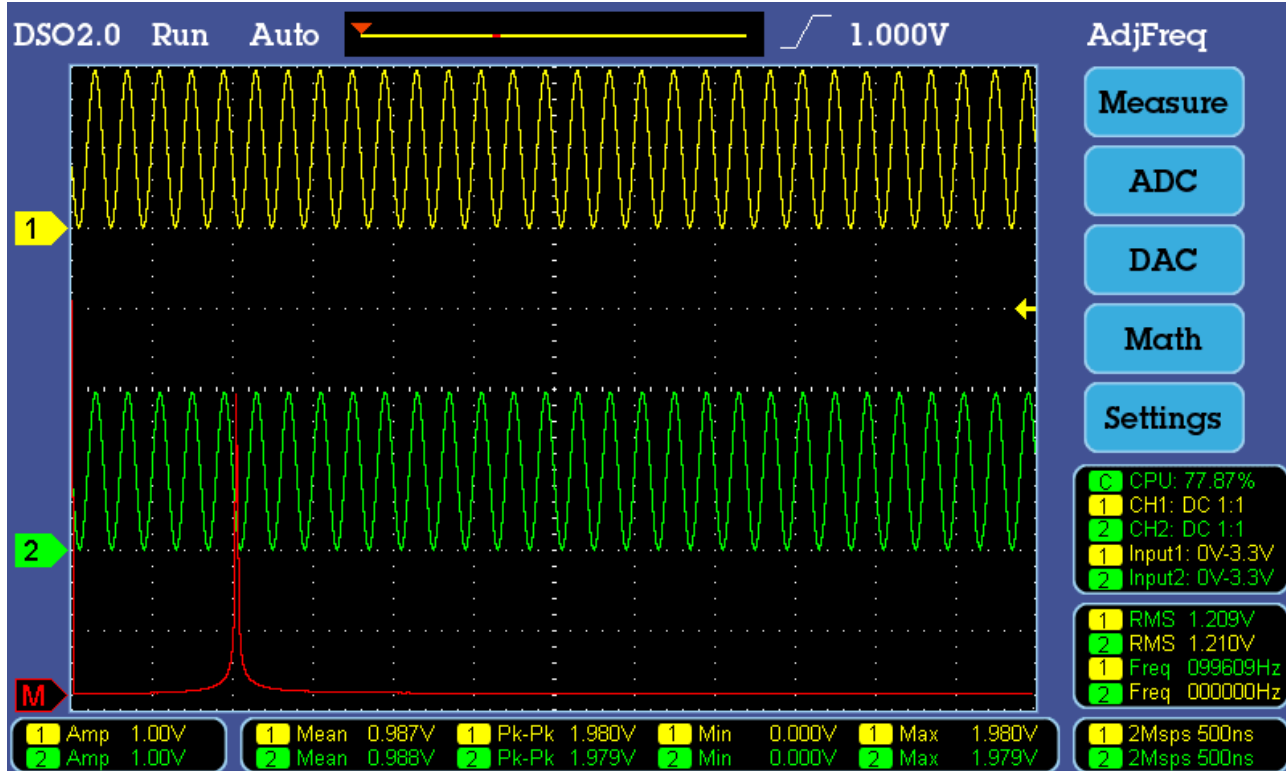
● 支持 2K 个 ADC 采集数据的浏览功能，下面是左移和右移 200 个数据的效果



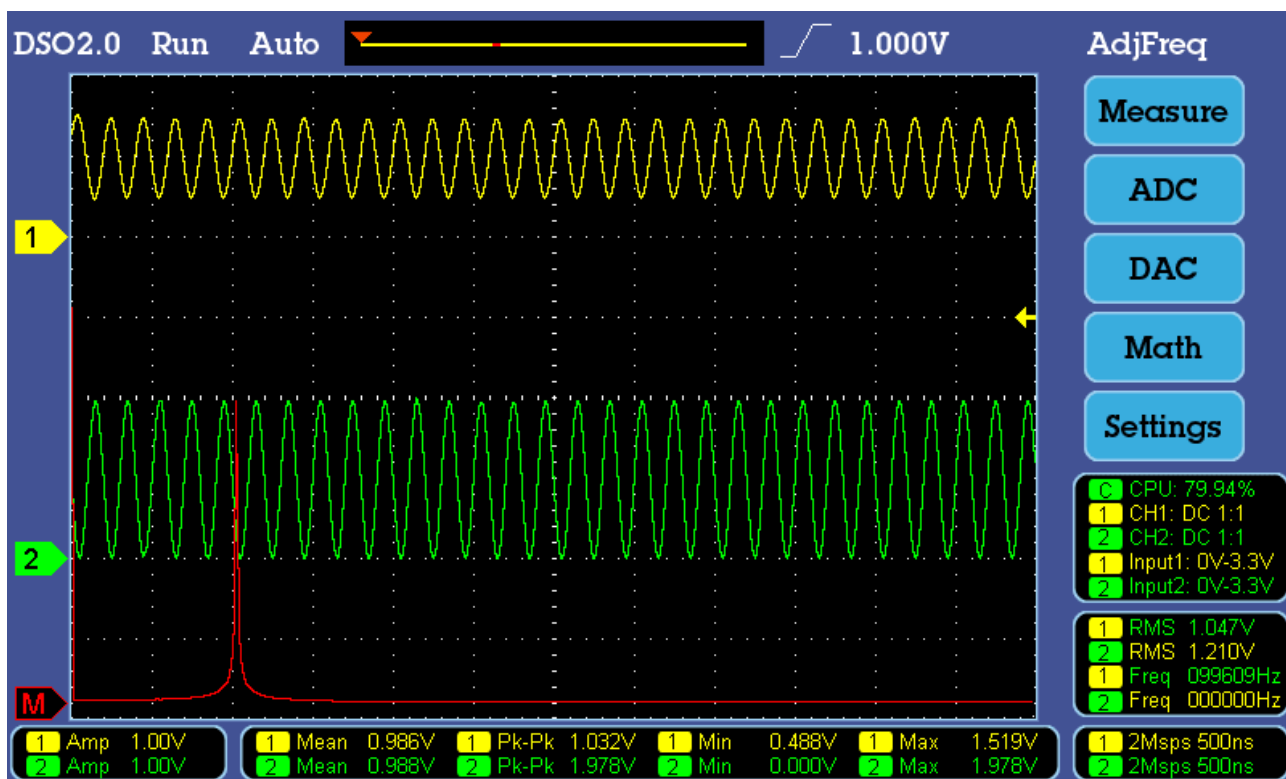
2.10 80 阶 Fir 低通滤波设计

双通道示波器采样率 2Msps，分别测量了正弦波和方波在不同的截止频率下的滤波效果。

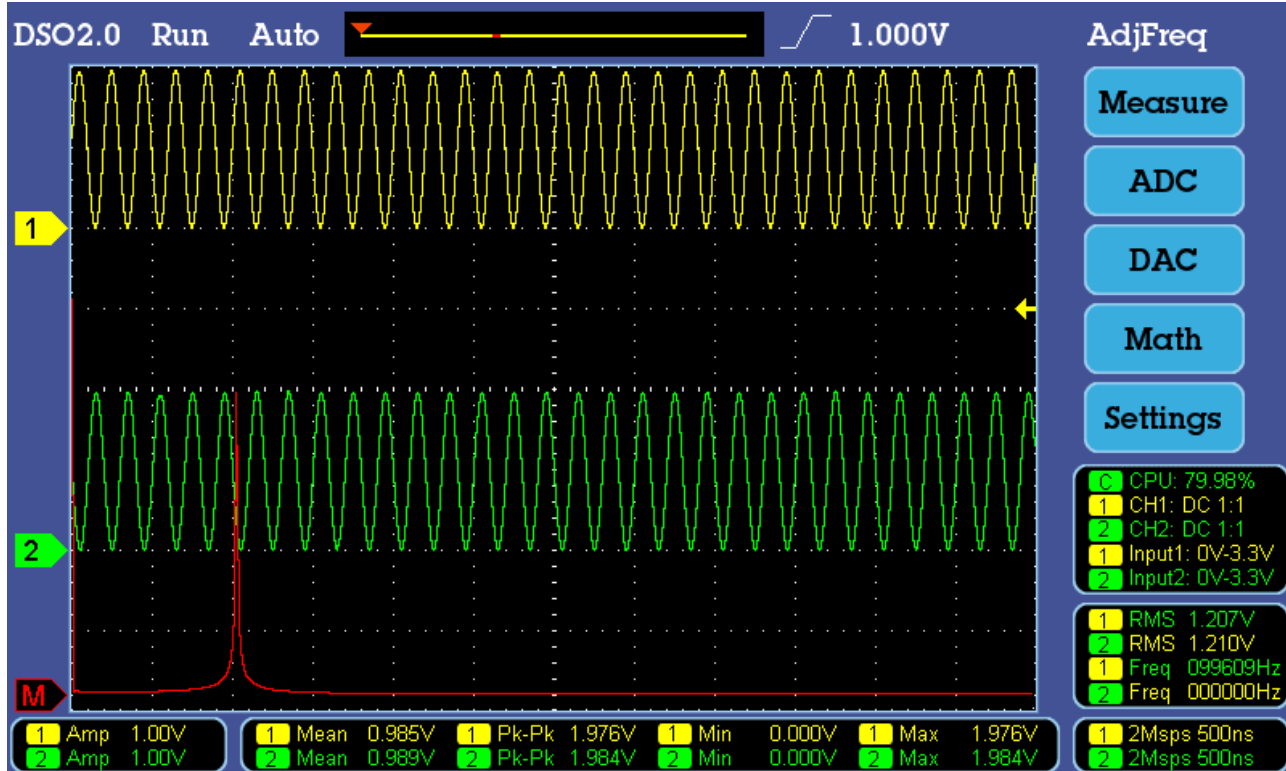
- 测试正弦波 100KHz，峰峰值 2V，直流偏移 1V 的原始信号。通道 1 和通道 2 采样同一路波形。



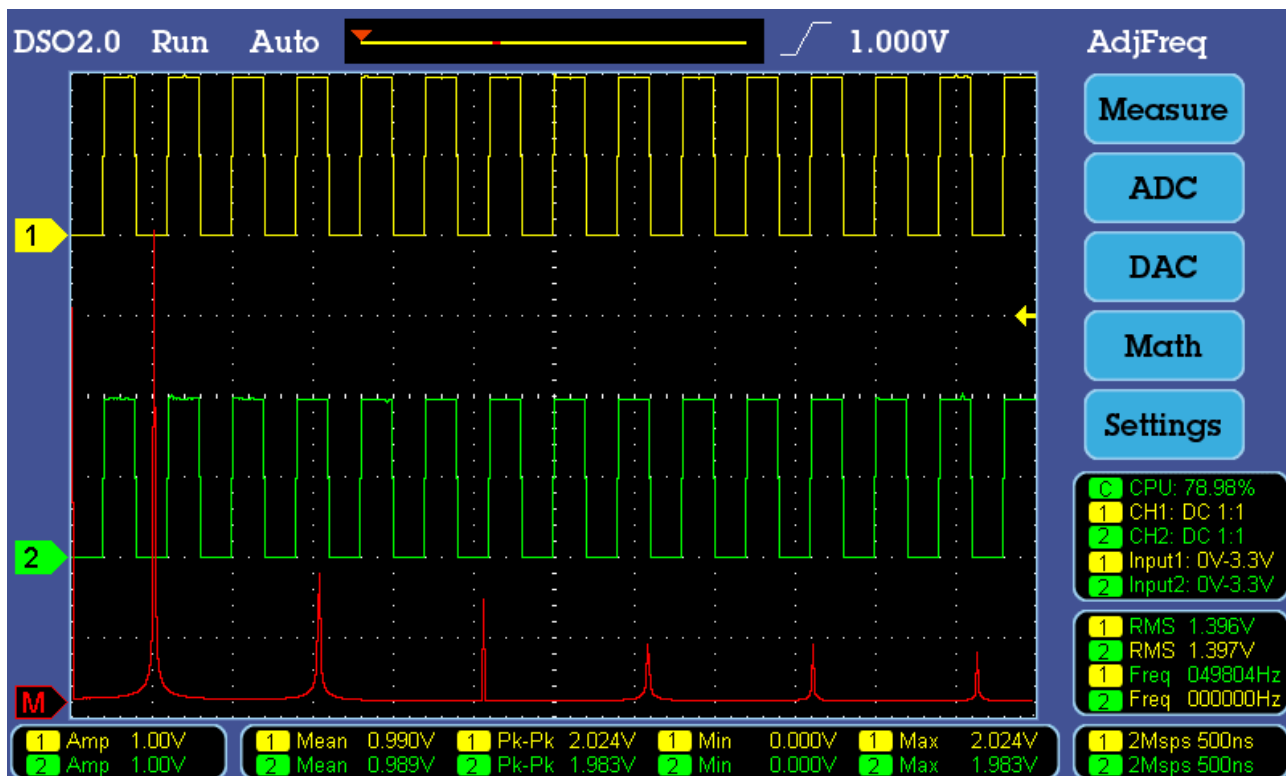
- 测试正弦波 100KHz，峰峰值 2V，直流偏移 1V 的在 100KHz 截止频率下的效果。通道 1 被滤波，通道 2 还是原始波形。



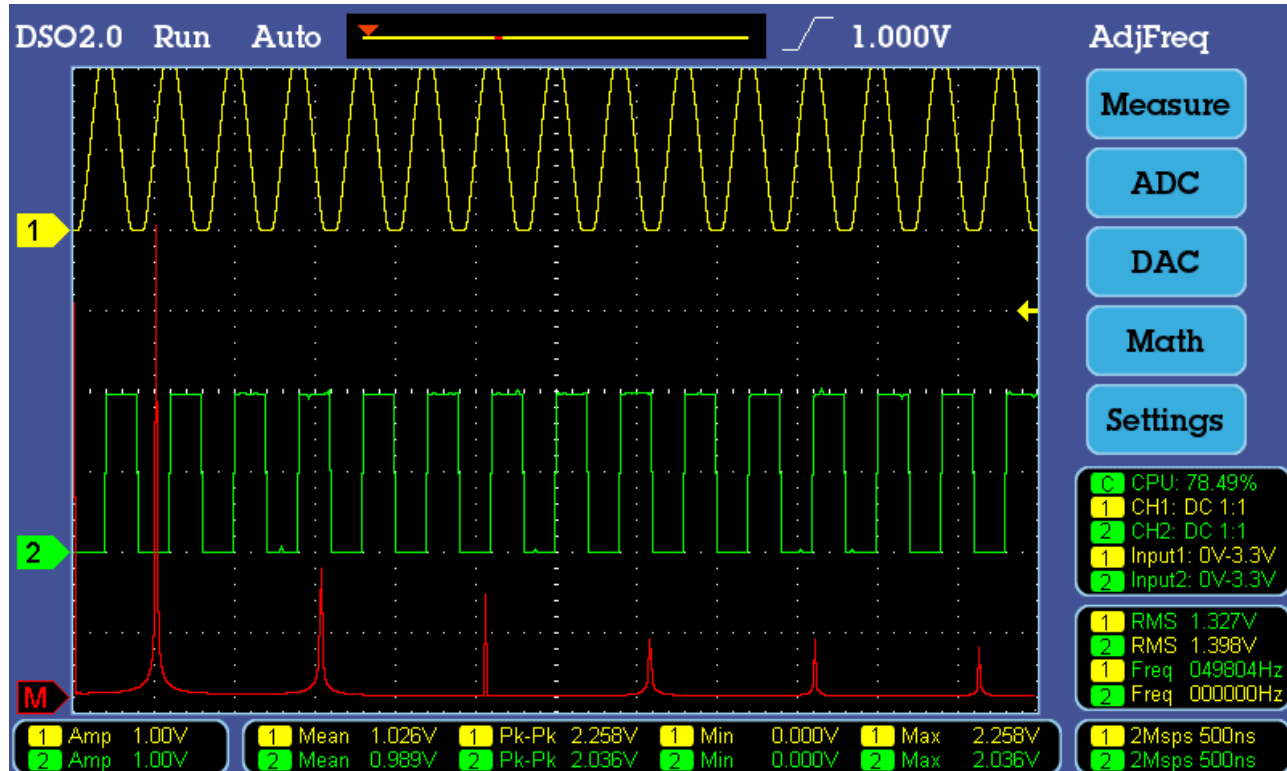
- 测试正弦波 100KHz，峰峰值 2V，直流偏移 1V 的在 400KHz 截止频率下的效果。通道 1 被滤波，通道 2 还是原始波形。



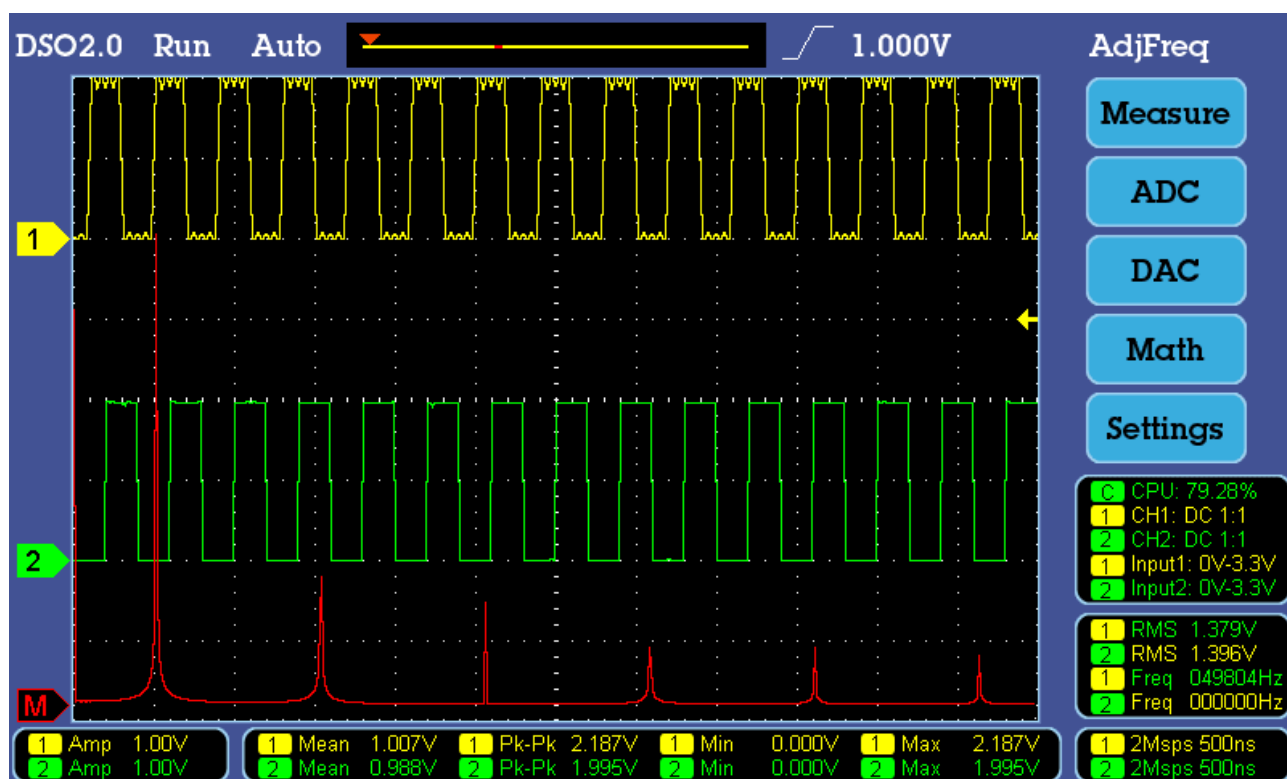
- 测试方波 50KHz，峰峰值 2V，直流偏移 1V 的原始信号，通道 1 和通道 2 采样同一路波形。



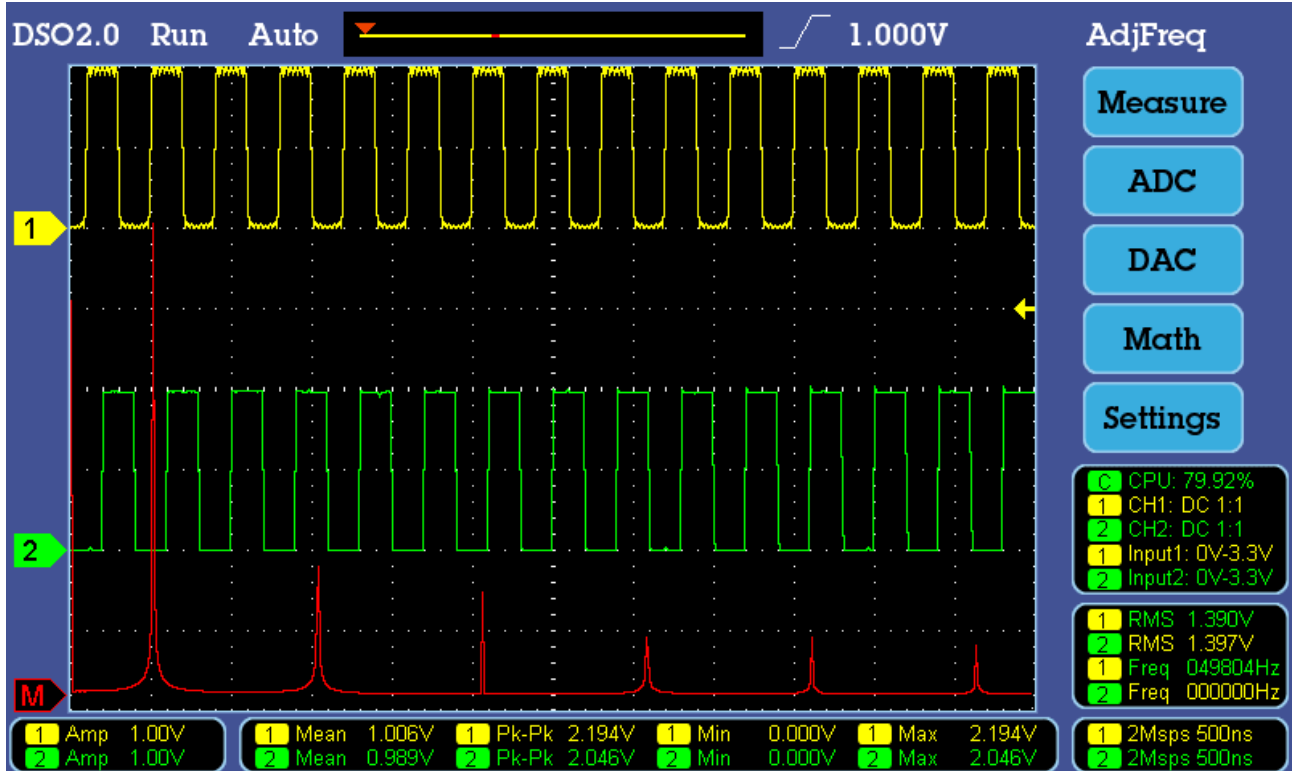
- 测试方波 50KHz，峰峰值 2V，直流偏移 1V 的在 100KHz 截止频率下的效果。通道 1 被滤波，通道 2 还是原始波形。



- 测试方波 50KHz，峰峰值 2V，直流偏移 1V 的在 400KHz 截止频率下的效果，通道 1 被滤波，通道 2 还是原始波形。



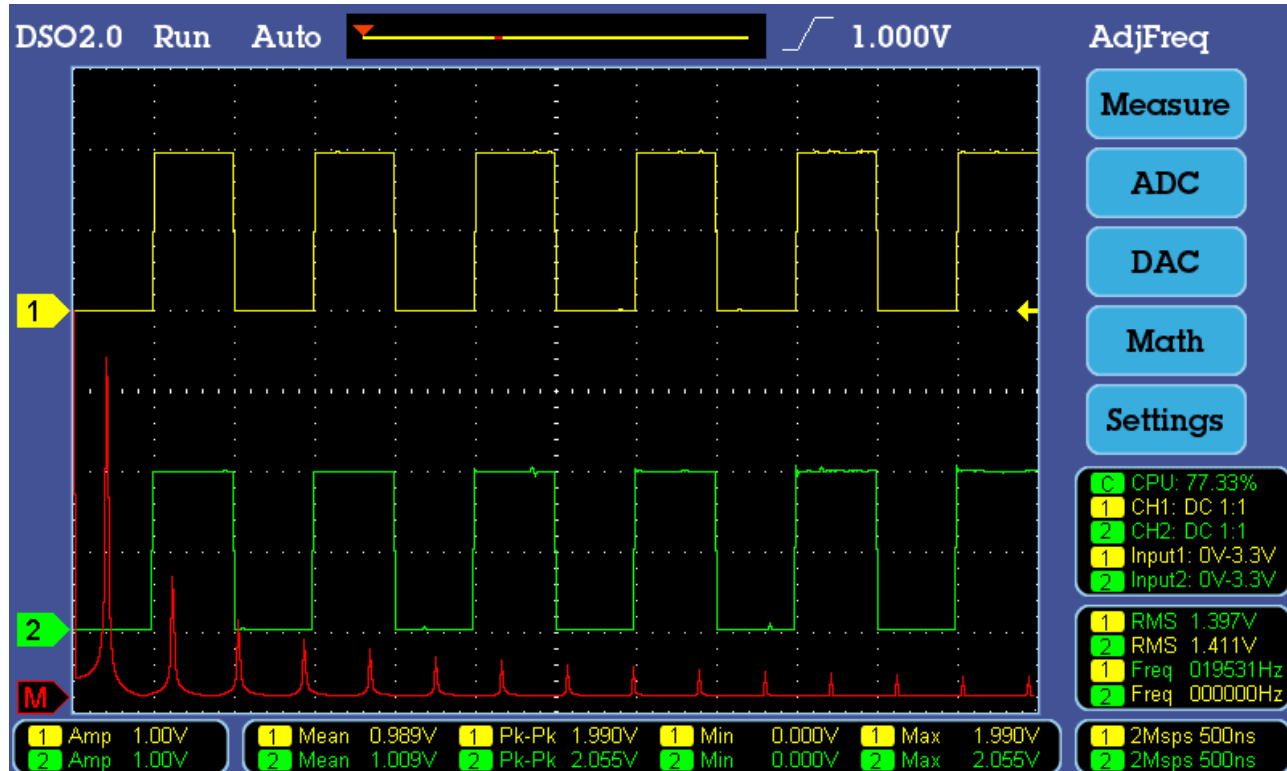
- 测试方波 50KHz，峰峰值 2V，直流偏移 1V 的在 700KHz 截止频率下的效果，通道 1 被滤波，通道 2 还是原始波形。



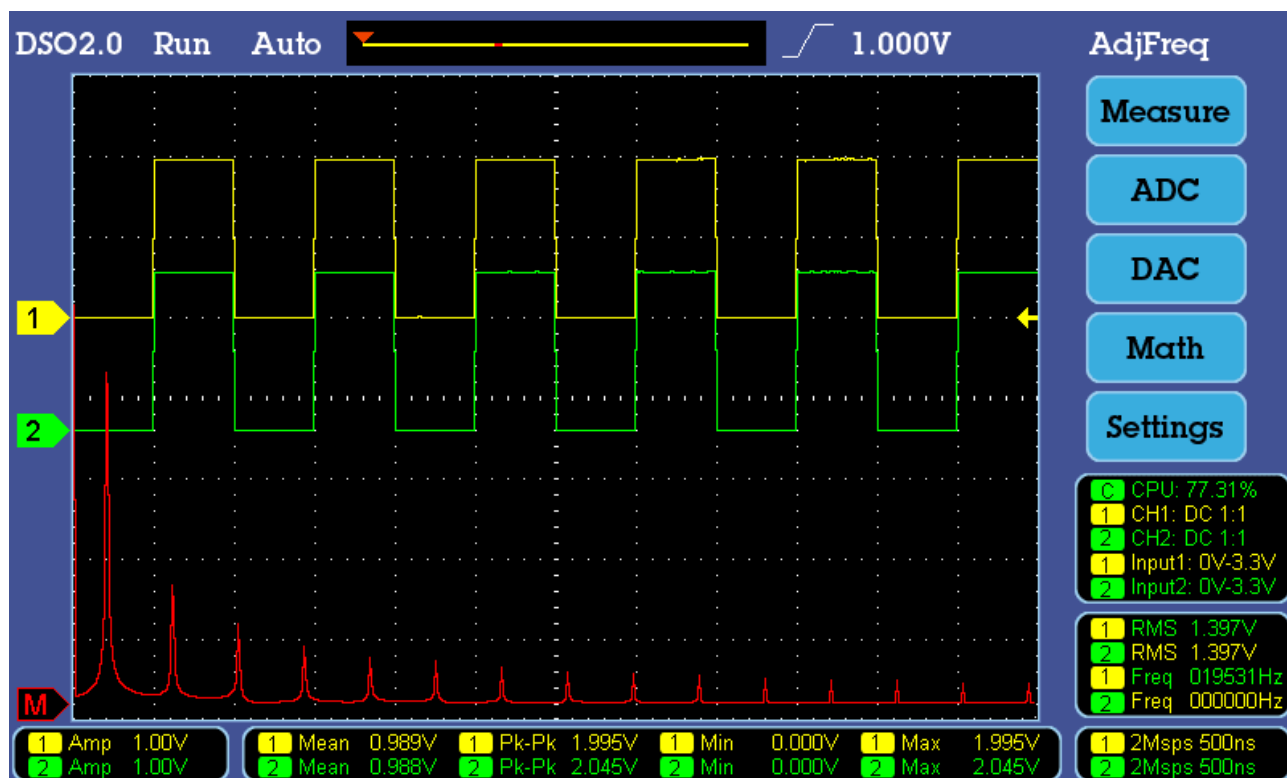
2.11 波形参考位置调节

双通道示波器采样率 2Msps，都采样 20KHz 正弦波，峰峰值 2V，直流偏移 1V，此方法是软件方式实现并非硬件调节。因为支持垂直滑动调节，所以上移或者下移位置比较简单。

- 通道 1 和通道 2 的波形都下移

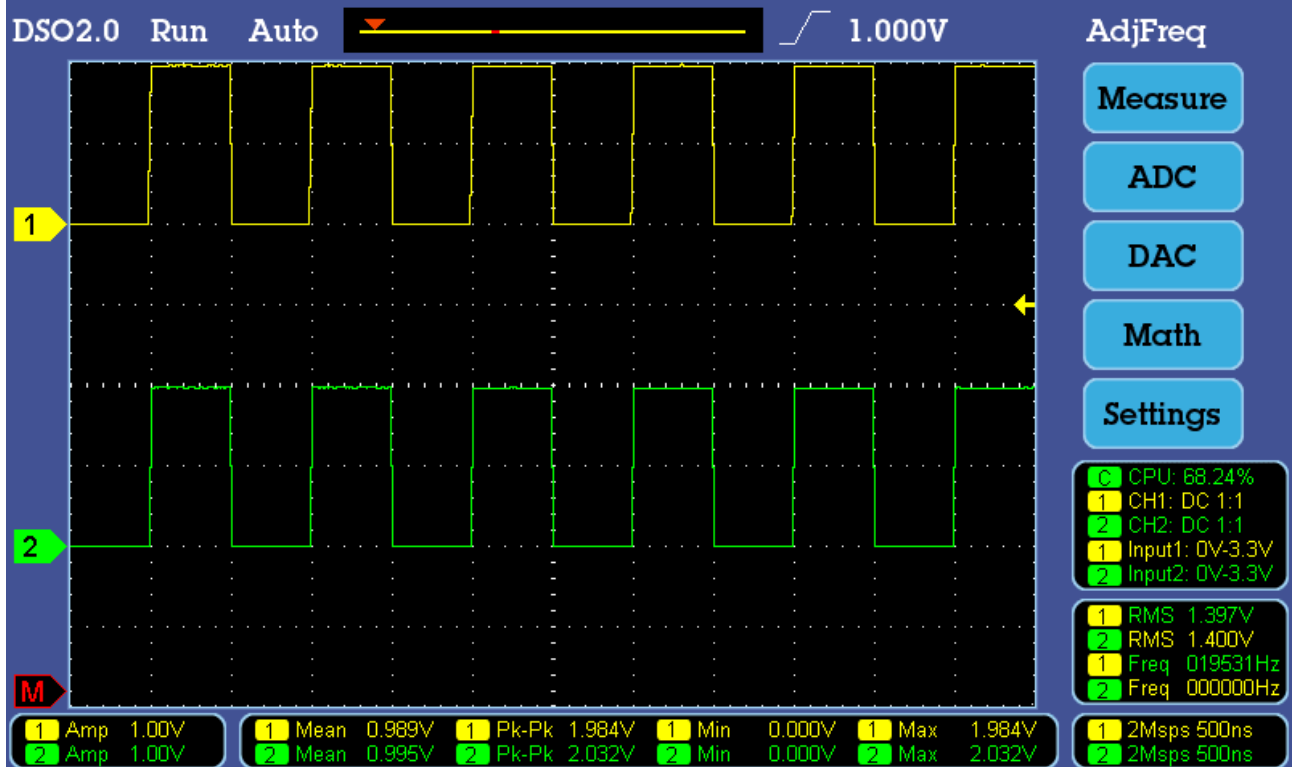


- 通道 1 下移，通道 2 上移

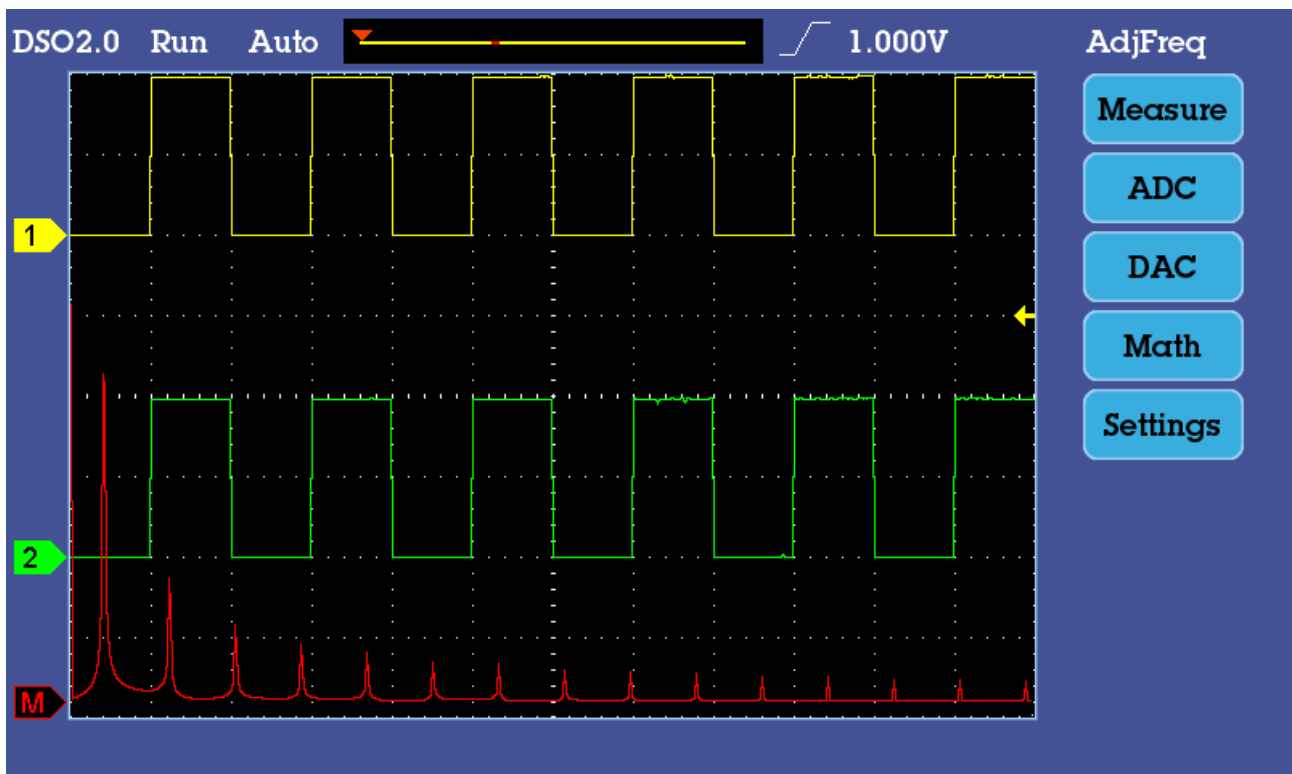


2.12 FFT 幅频显示和测量值窗口的隐藏

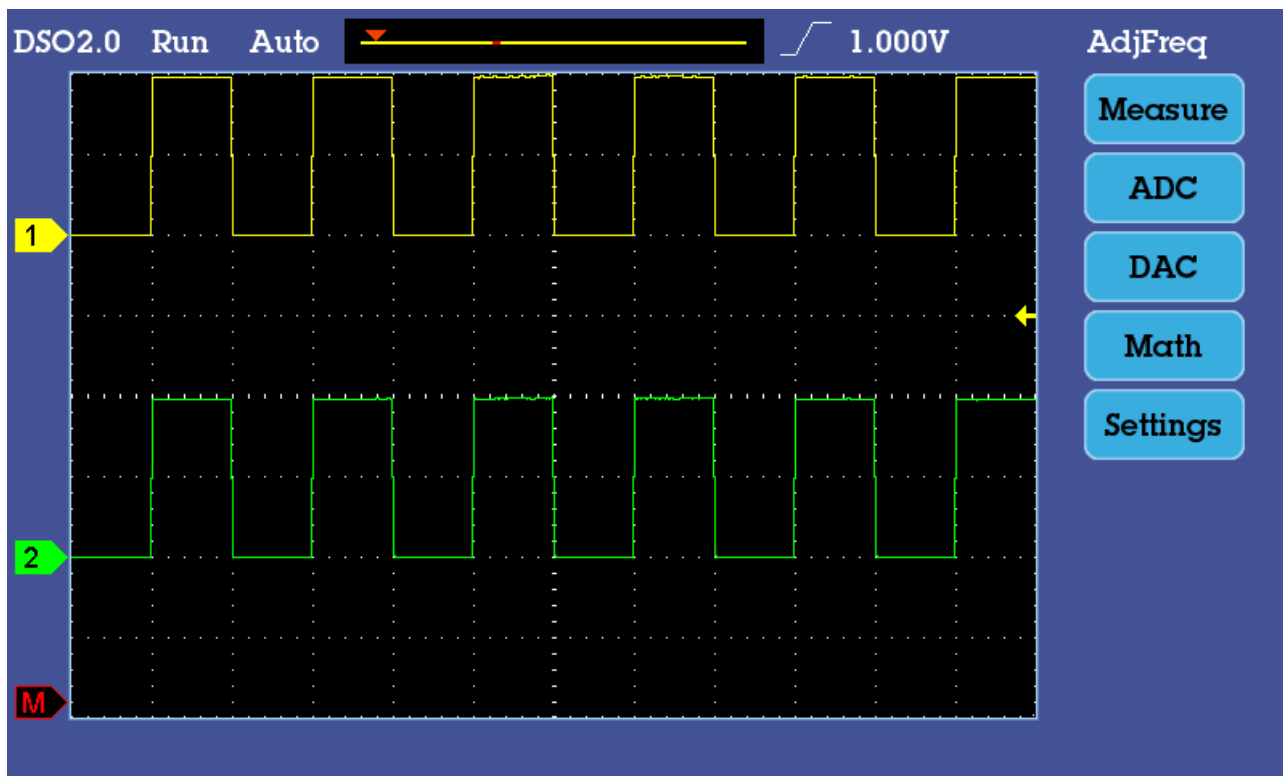
- 隐藏 FFT 幅频的显示。



- 隐藏幅值窗口，状态窗口，频率窗口和系统信息窗口。



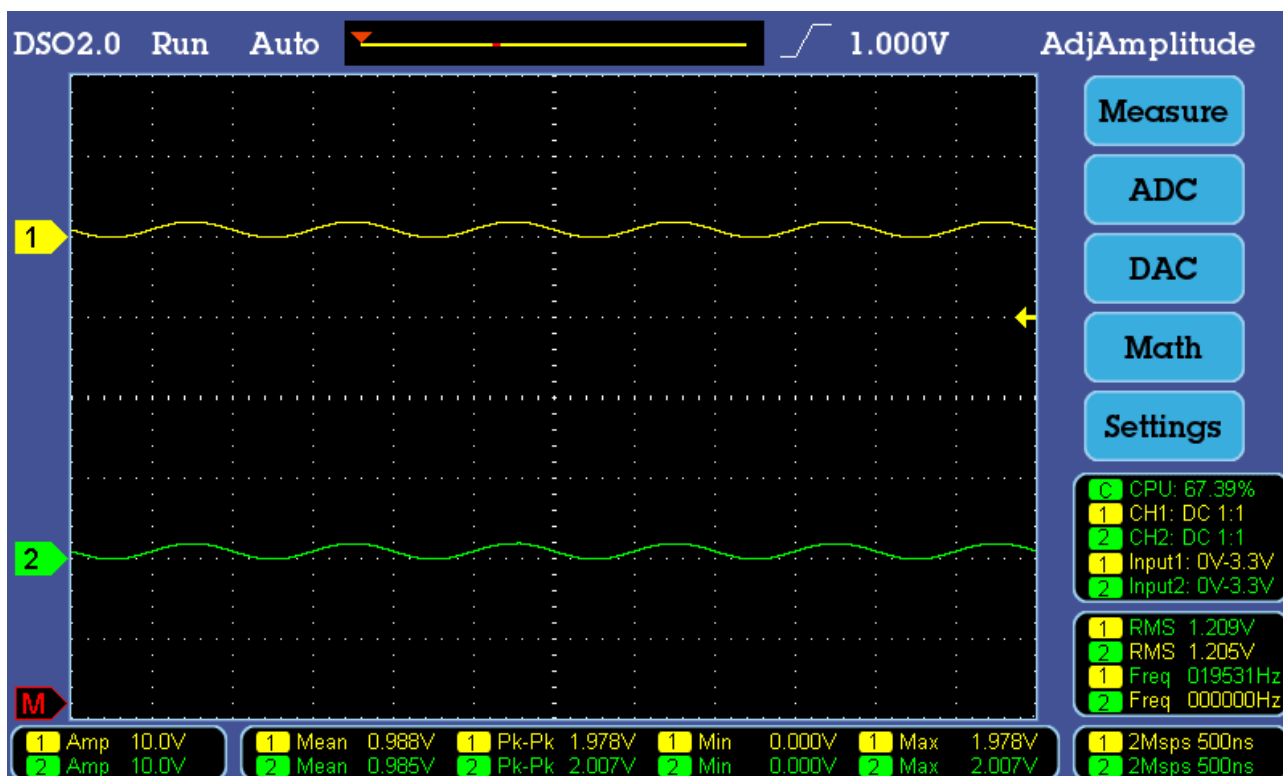
- FFT 幅频显示，幅值窗口，状态窗口，频率窗口和系统信息窗口的隐藏。



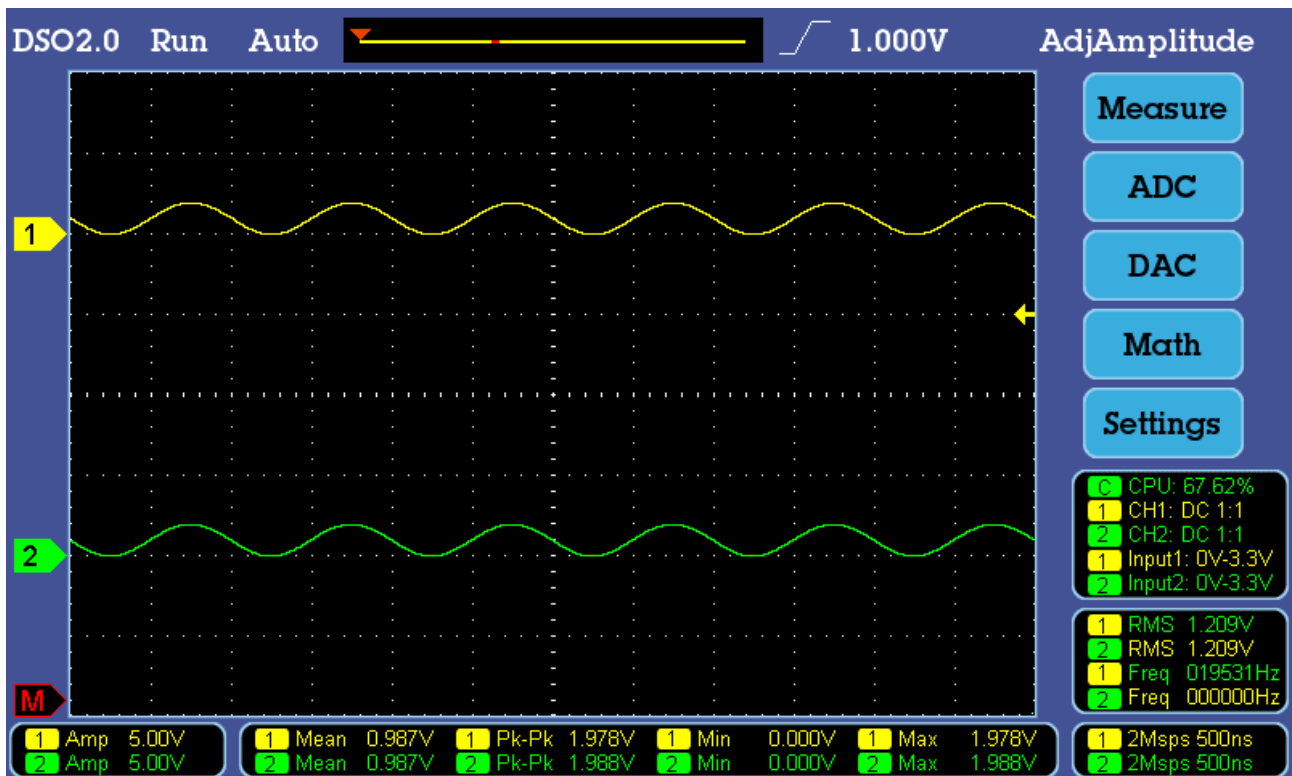
2.13 幅值调节

双通道示波器采样率 2Msps，都采样 20KHz 正弦波，峰峰值 2V，直流偏移 1V 时的幅值调整。

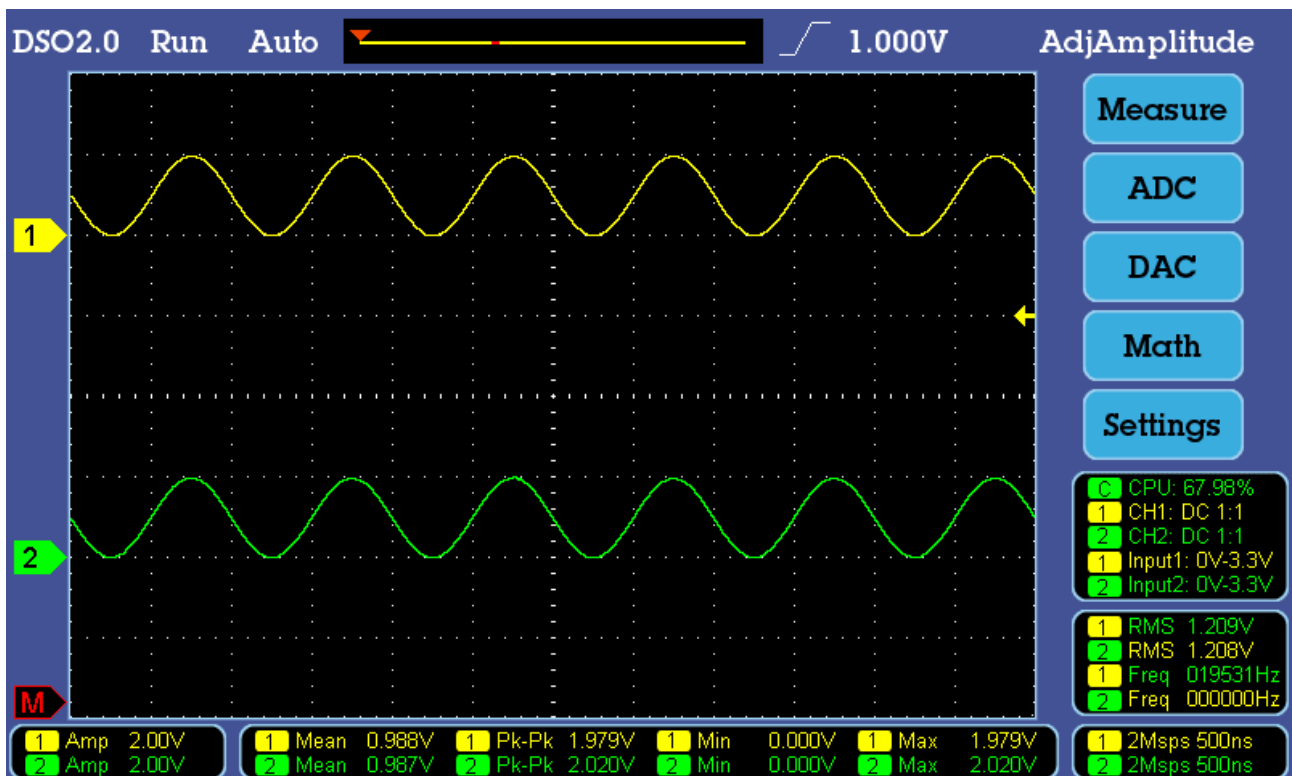
- 幅值单位 10V，波形显示效果



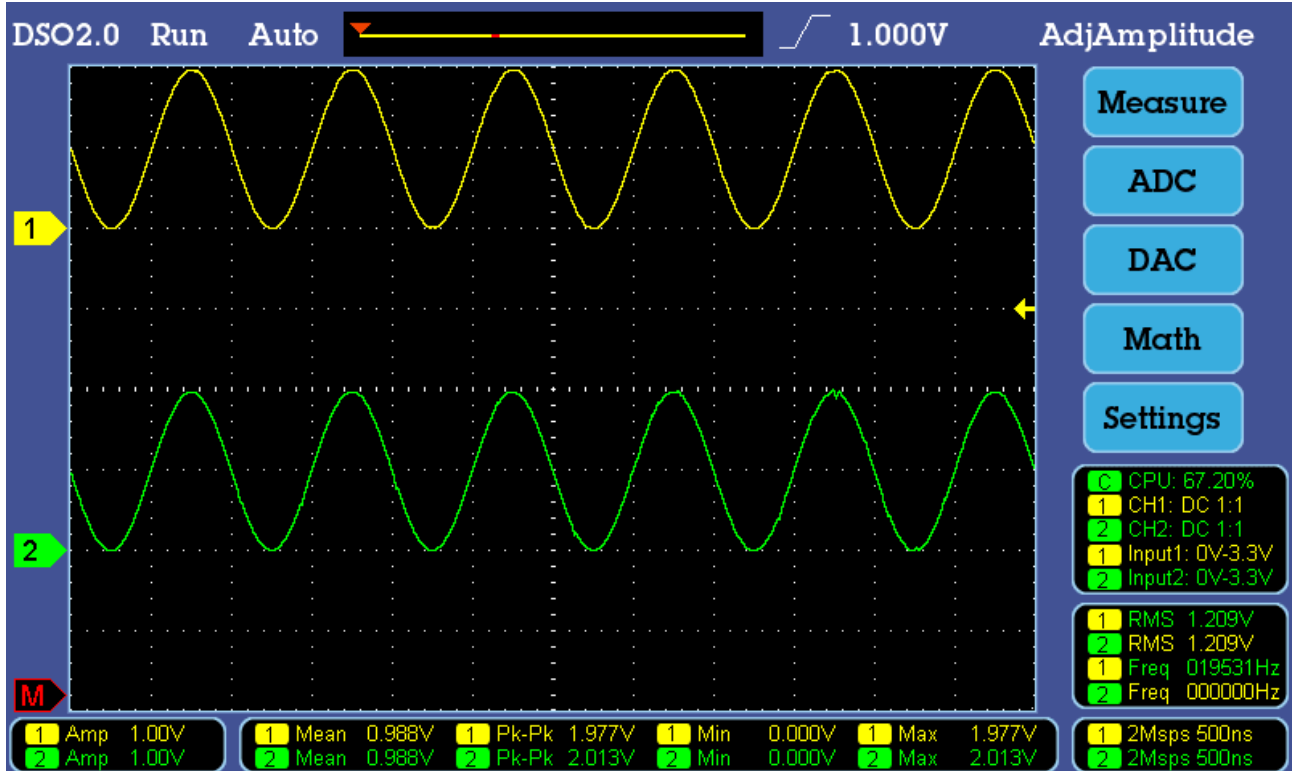
- 幅值单位 5V，波形显示效果



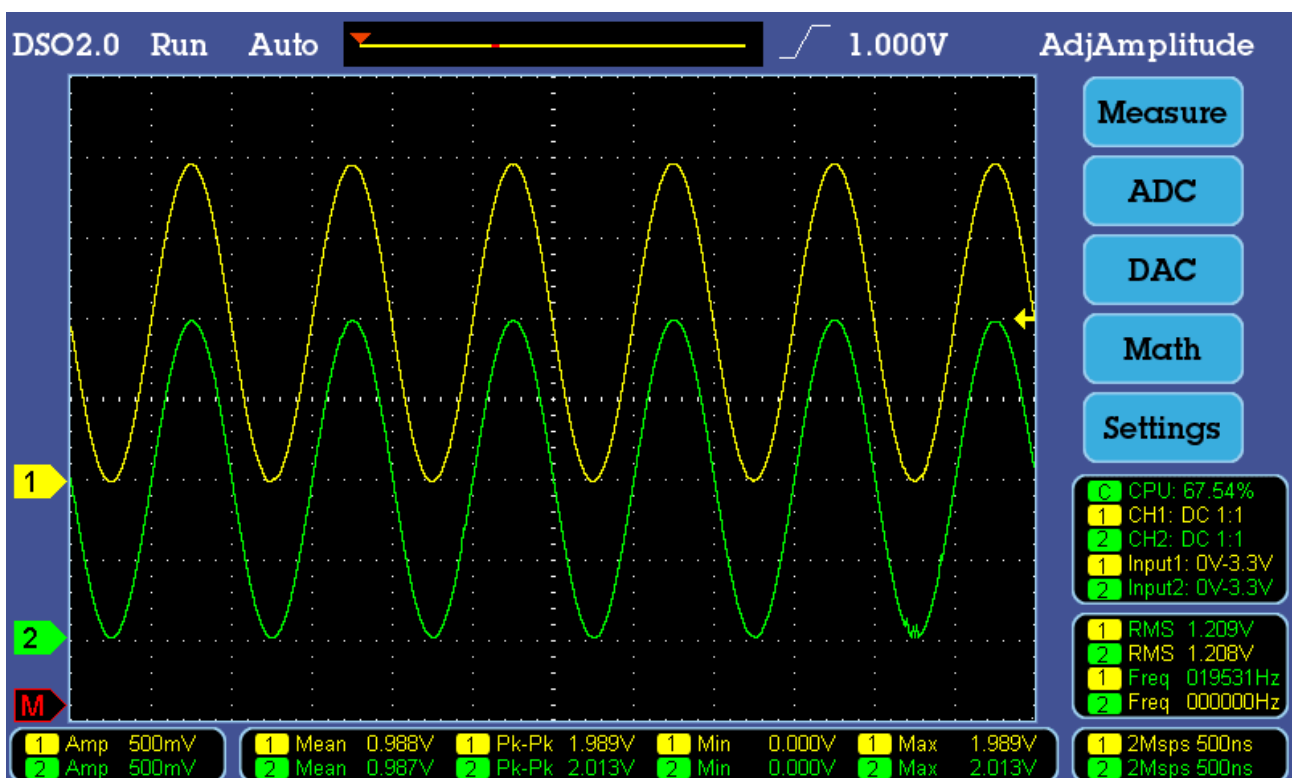
- 幅值单位 2V，波形显示效果



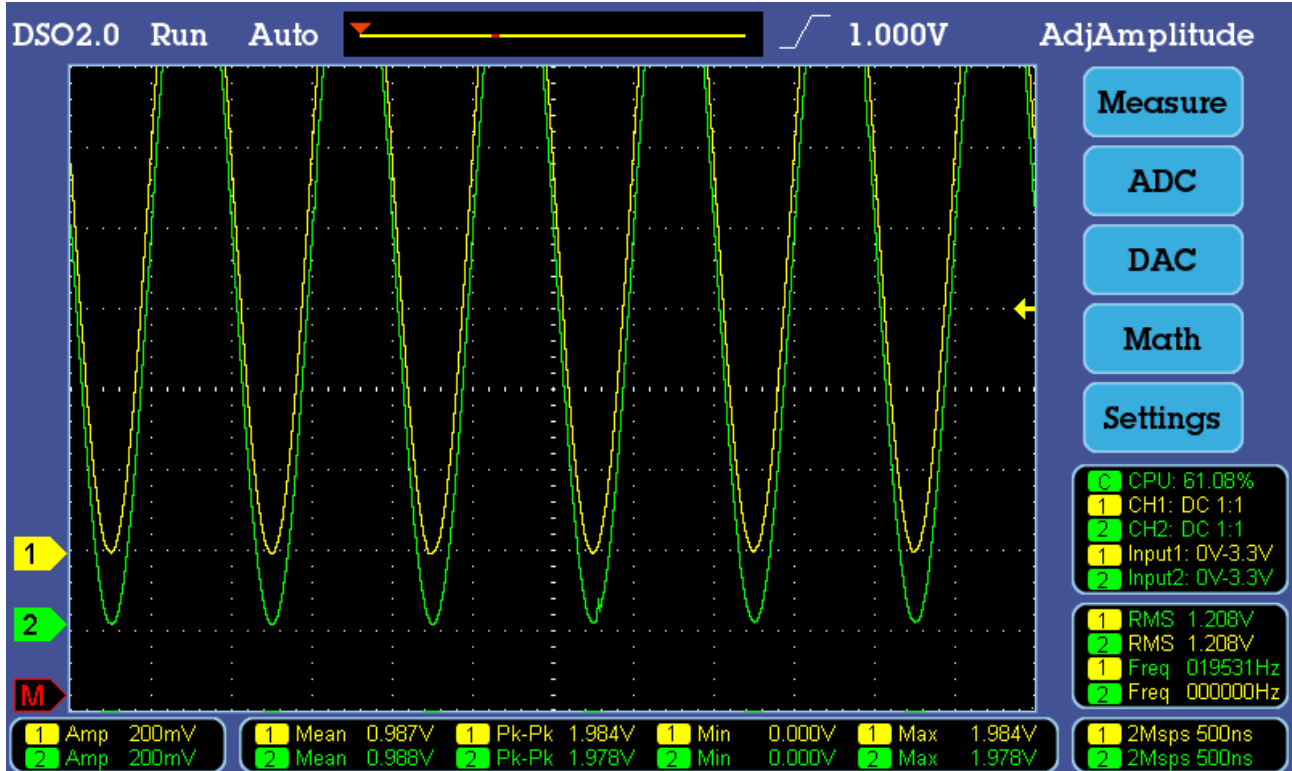
- 幅值单位 1V，波形显示效果



- 幅值单位 500mV，波形显示效果

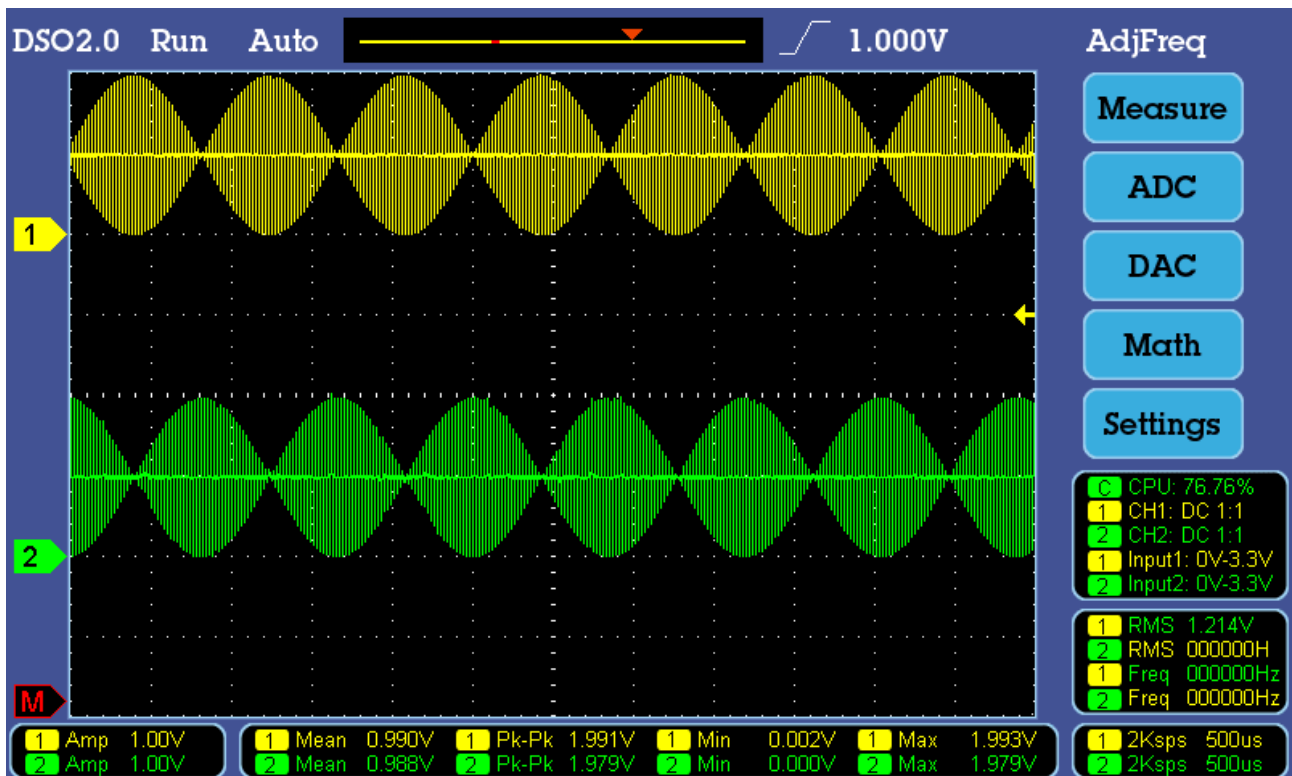


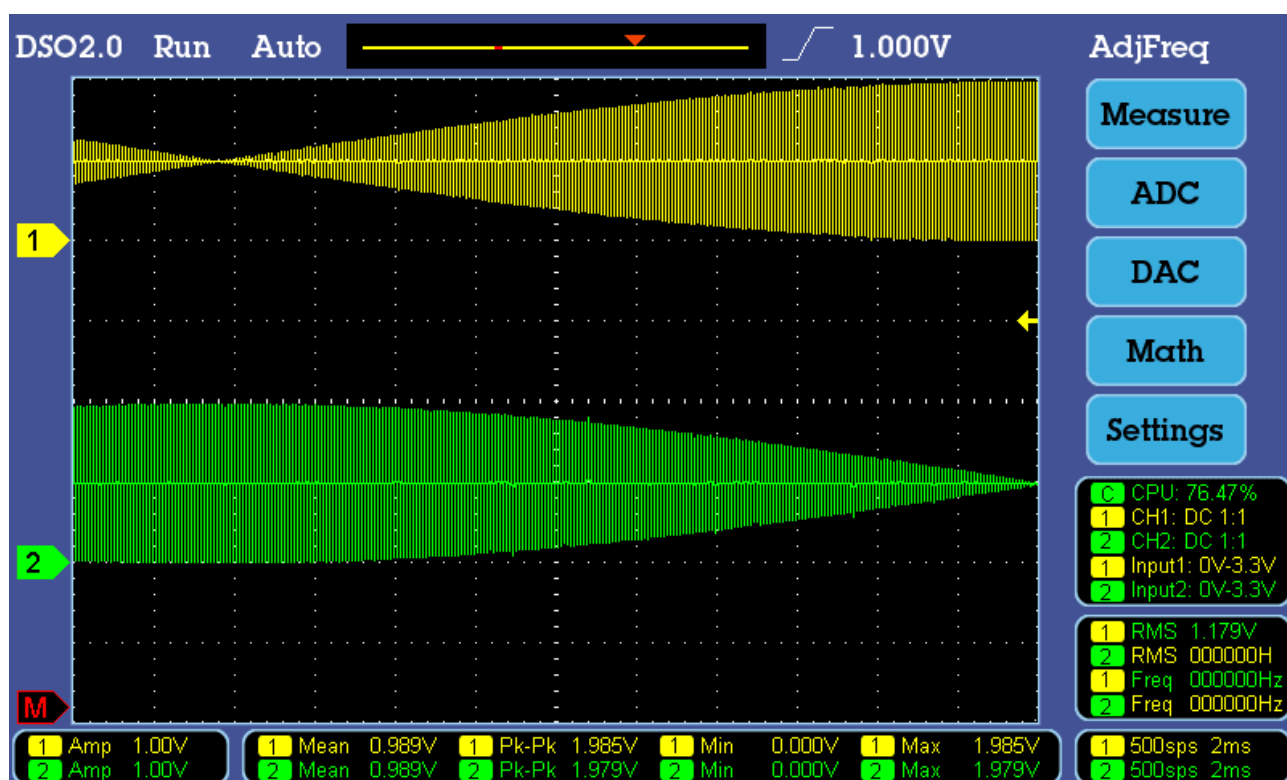
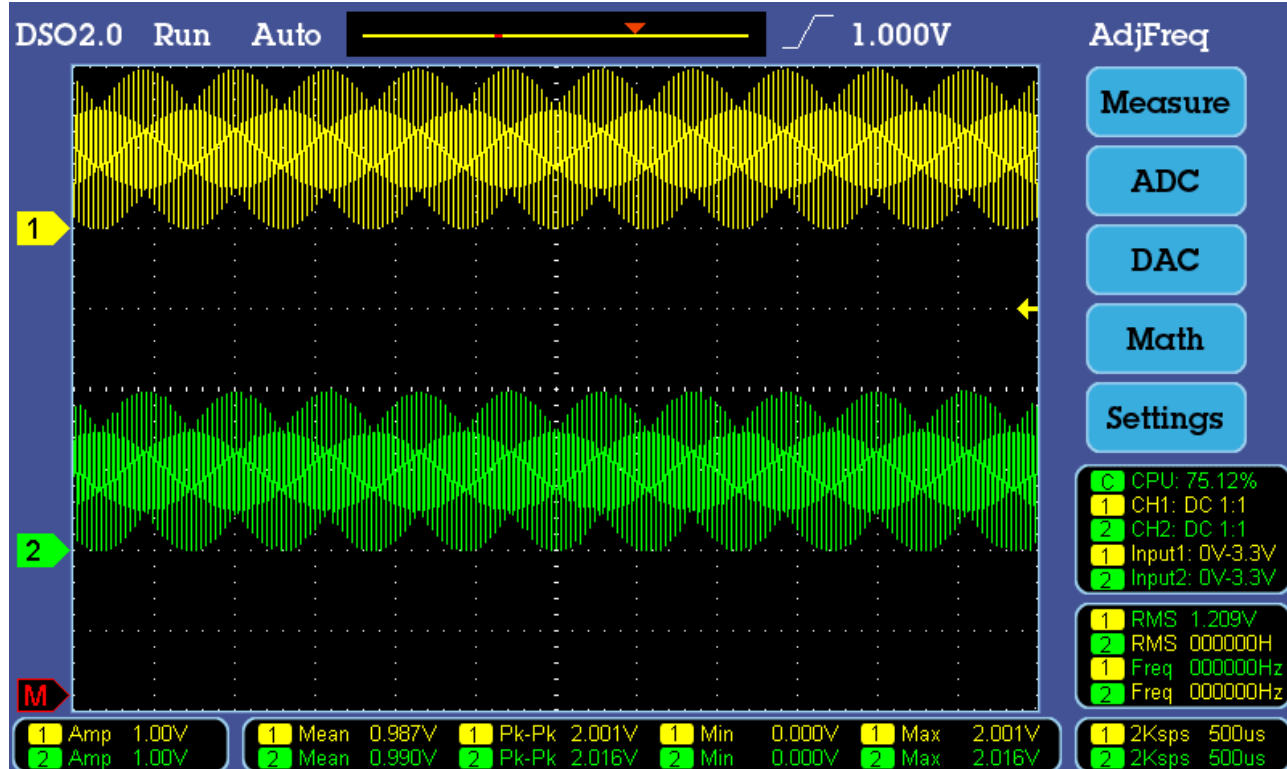
- 幅值单位 200mV，波形显示效果

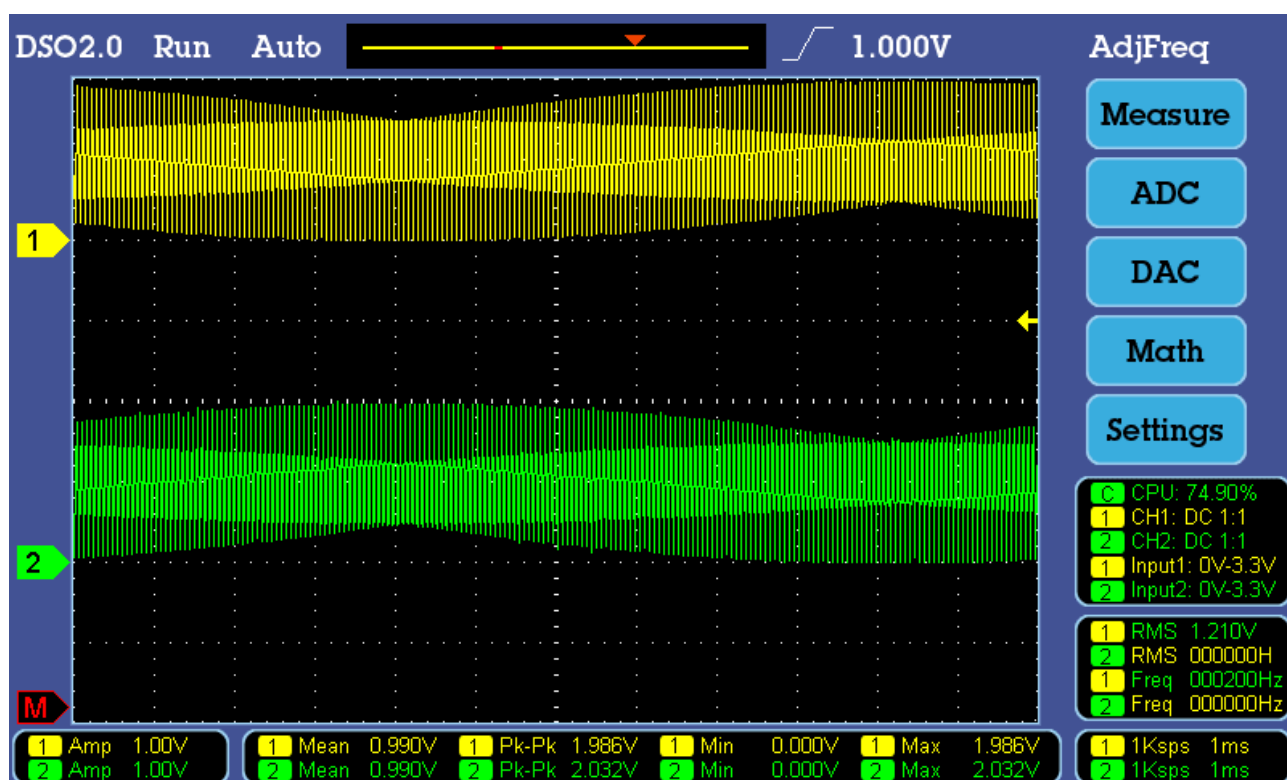
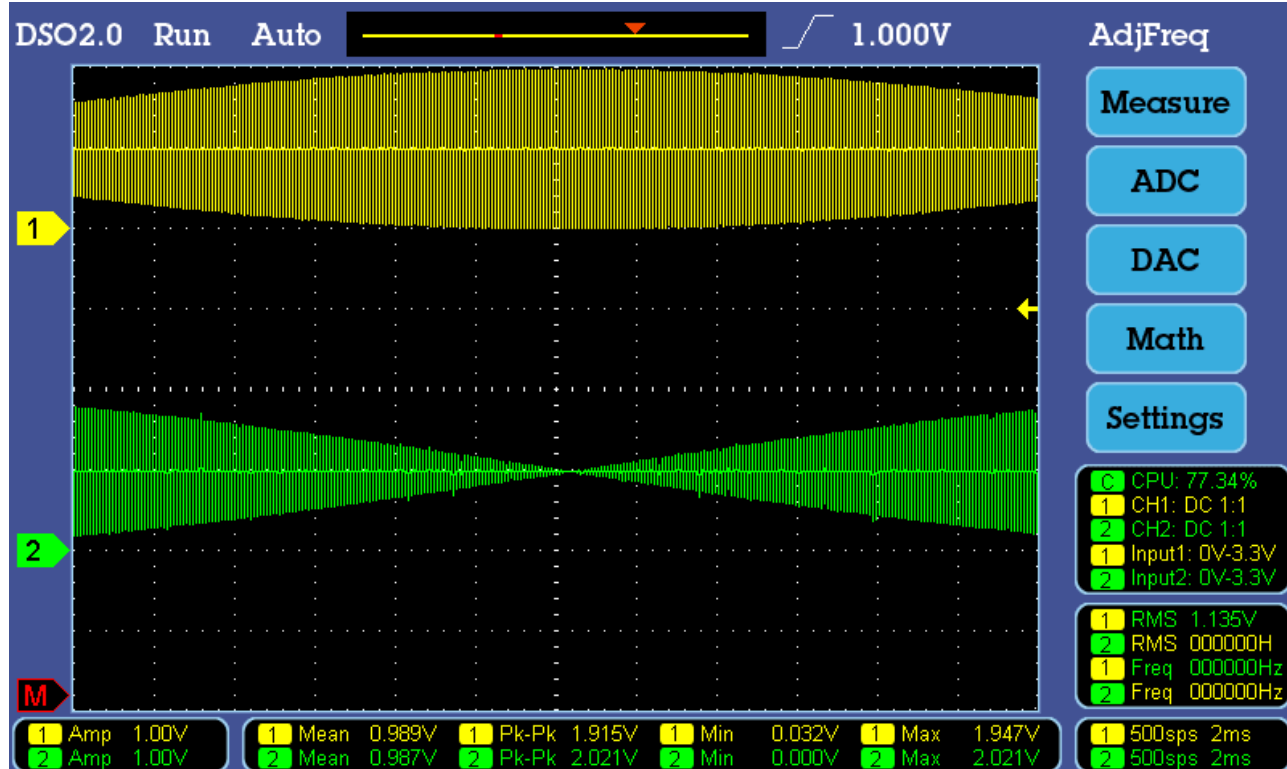


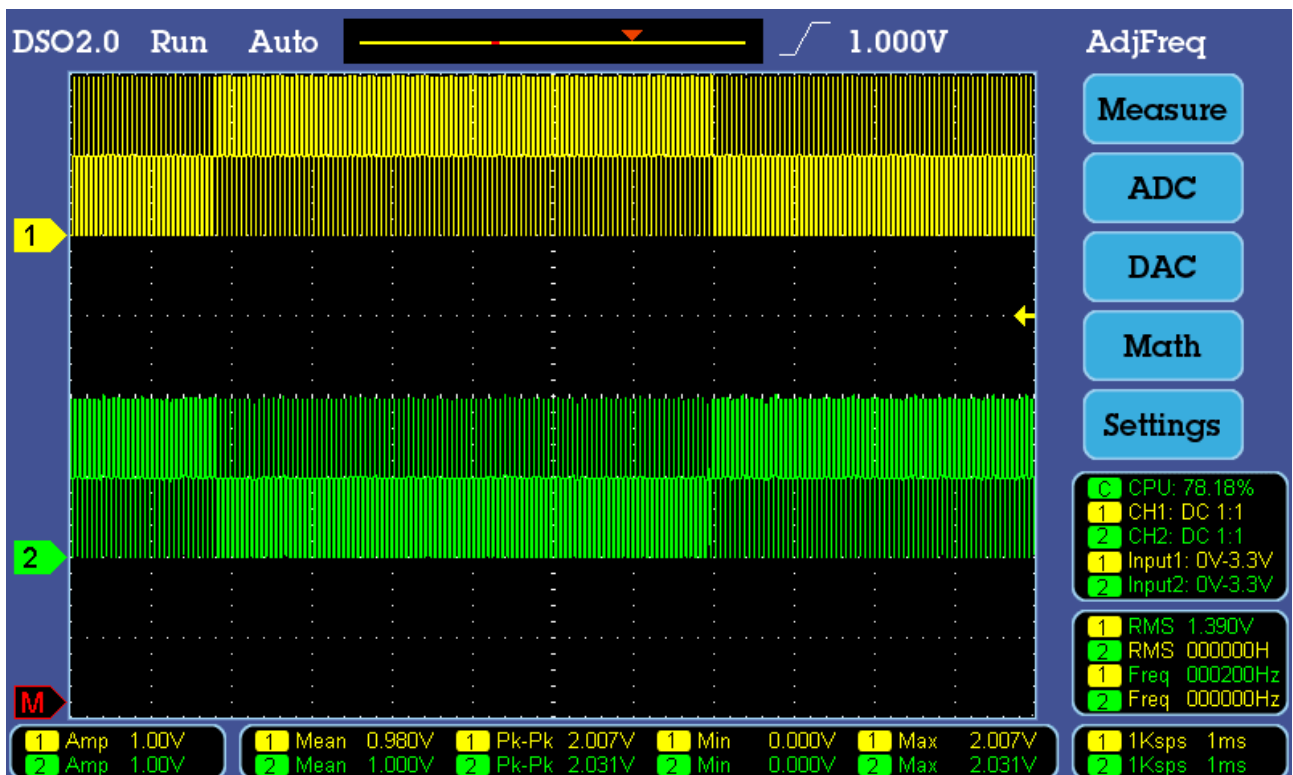
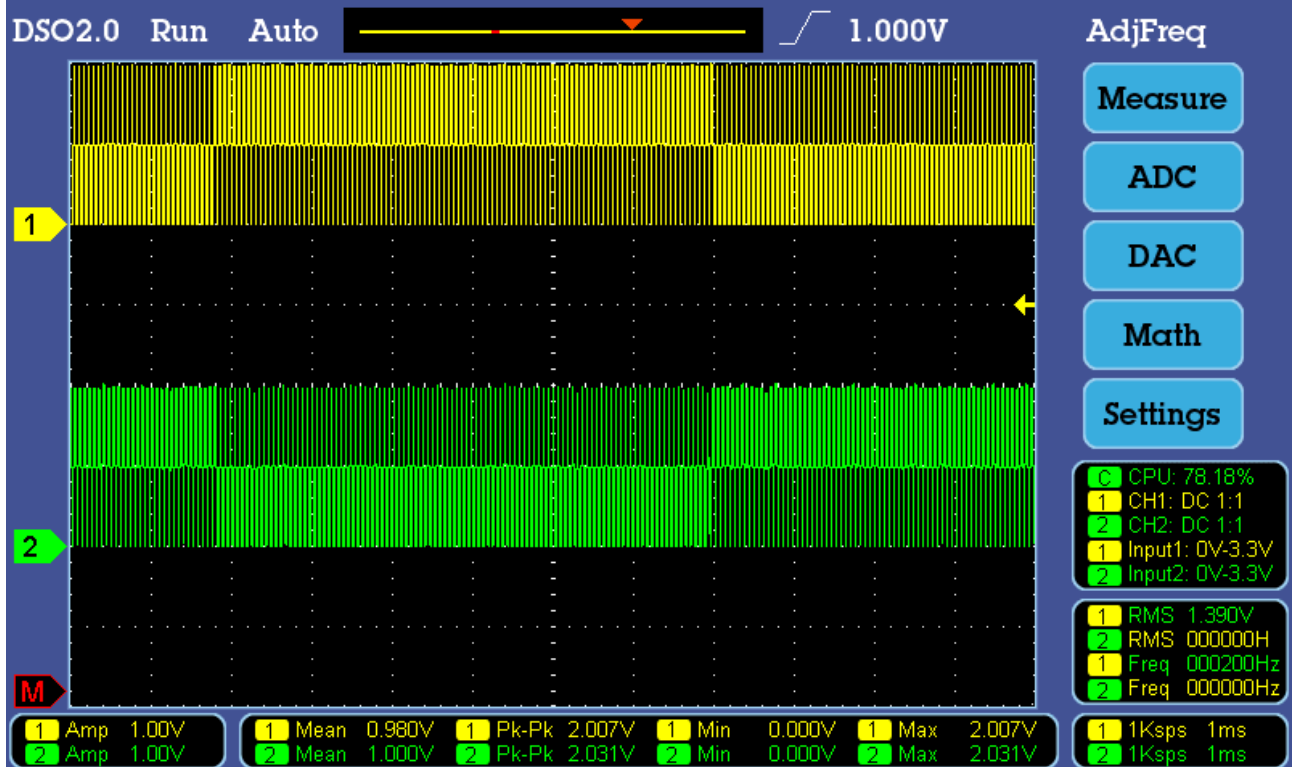
2.14 低频采样率采集高频信号的炫酷效果

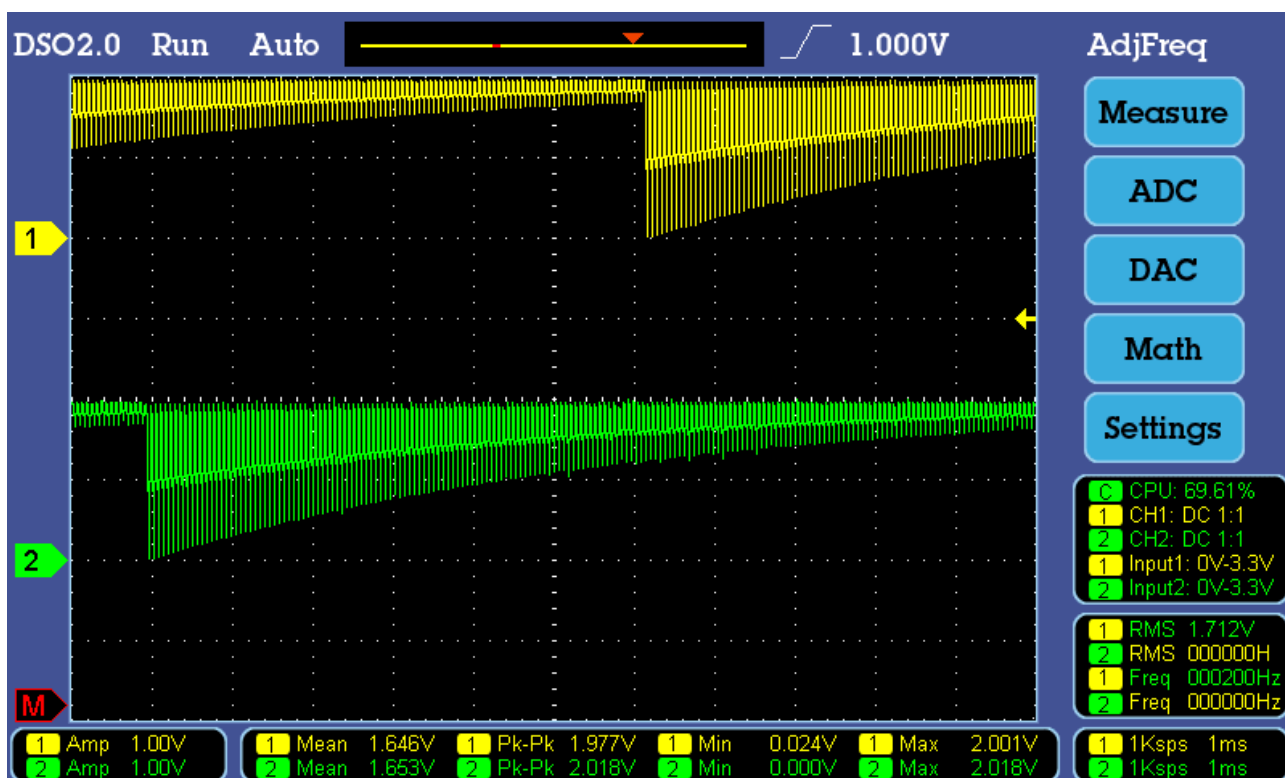
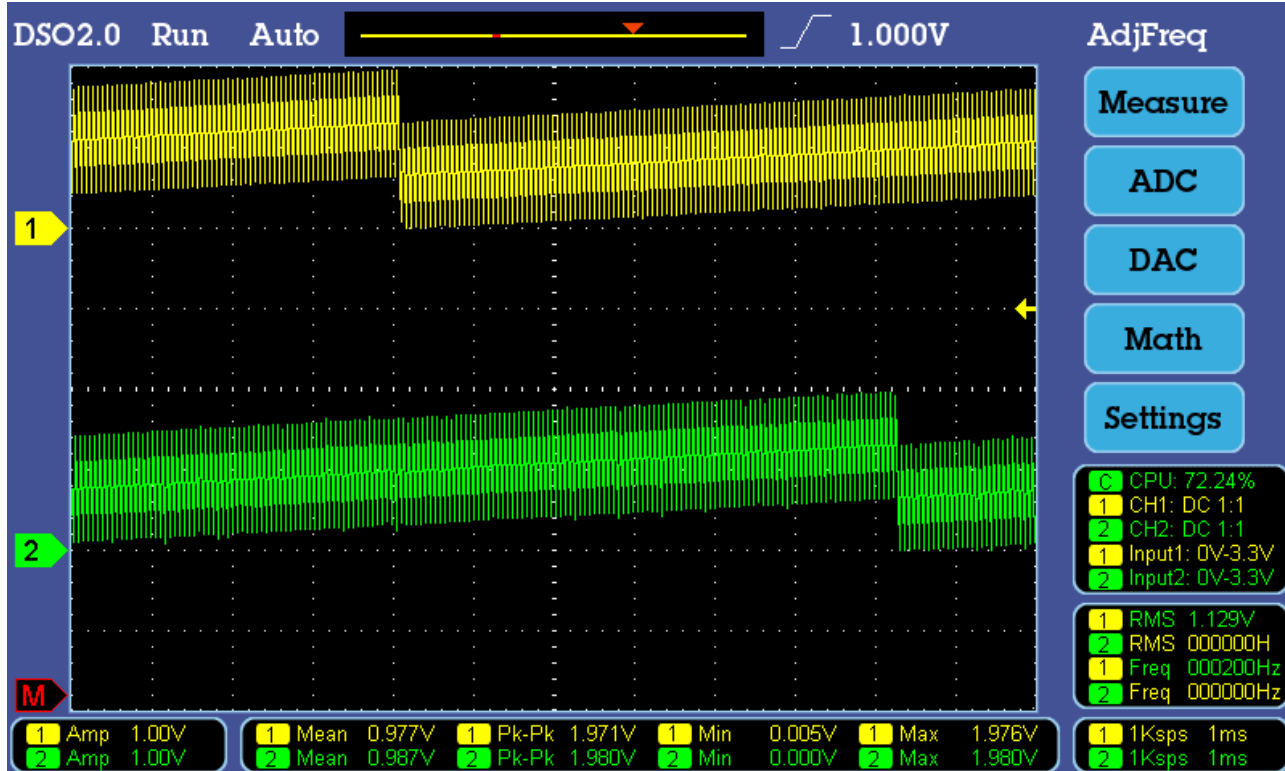
下面为大家展示几个低频采样高频信号，由于混叠而产生的炫酷效果：











2.15 总结

本章节主要讲解了示波器的使用说明及其介绍，如果大家在使用中遇到了什么问题，登录我们论坛发帖即可。

第3章 示波器设计—功能划分和准备工作

本章节主要为大家讲解示波器的系统框架设计。

- 3.1 功能划分
- 3.2 任务优先级安排
- 3.3 按分类封装各种变量
- 3.4 选择编译器最高等级优化
- 3.5 制作工程模板
- 3.6 总结

3.1 功能划分

示波器的实现大致可以分为模拟信号的采集，波形处理，波形刷新和测量值显示，波形的控制调节和波形的存储浏览。将这些功能的实现落实到实际硬件上，详情如下：

◆ 模拟信号采集

STM32 自带 3 个独立的 ADC，分别是 ADC1，ADC2 和 ADC3。实现双通道示波器，我们需要用到两个 ADC，另外一个 ADC 用于简易电压采集。实际的使用中由于要采样频率可调，需要配置成定时器触发。虽然每个通道最高速度只有 2.4Msps 的采样率（实际应用中是配置到 2.8Msps），但是每秒的采集的数据高达 4.8MB 左右，所以还得配合 DMA 一起使用，大大降低 CPU 的负担。

◆ 波形处理

波形处理就是指 FFT，FIR，RMS，平均值，最大值，最小值和峰峰值的计算，这个功能需要交给一个单独的任务去做。

◆ 波形刷新和测量值显示

这个功能通过 emWin 任务就可以实现。

◆ 波形的控制调节

由于需要按键和触摸都可以调节，所以单独安排一个任务。

◆ 波形的存储浏览

波形的浏览继续放在 emWin 任务里面实现，而波形的存储记录需要放在一个单独的文件系统任务里面。

下面通过框图将这几个功能做个规划：

GUI 任务

波形刷新
测量值显示
波形调节响应
波形浏览等

DSP 任务

计算 FFT ,FIR ,
RMS, 最大值,
最小值, 平均值,
峰峰值等

触摸和按键任务

电阻屏和电容屏
的触摸扫描,
按键扫描

文件系统任务

截图功能
数据记录

上面是初步的功能划分,有了这些划分后,实际程序设计的时候就会方便很多,在这个基础上再进一步调整。具体任务之间的通信机制在我们实现了各个功能后再实现。

3.2 任务优先级安排

首先 emWin 任务一定要是除了空闲任务,统计任务以外最低优先级的,因为 emWin 极其占用系统资源,而且时间长,如果这个任务设置为高优先级,会直接影响低优先级任务的执行。

DSP 任务一定要是优先级最高的,因为采集的数据要实时处理。

触摸和按键任务,以及文件系统任务的优先级谁高谁低都没有关系。

- 知识点拓展

关于任务优先级的安排,在我们 RTX 操作系统教程第 8 章的 8.2 小节有些拓展:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。

同时在我们 FreeRTOS 操作系统教程的第 13 章的 13.2 小节有些拓展:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

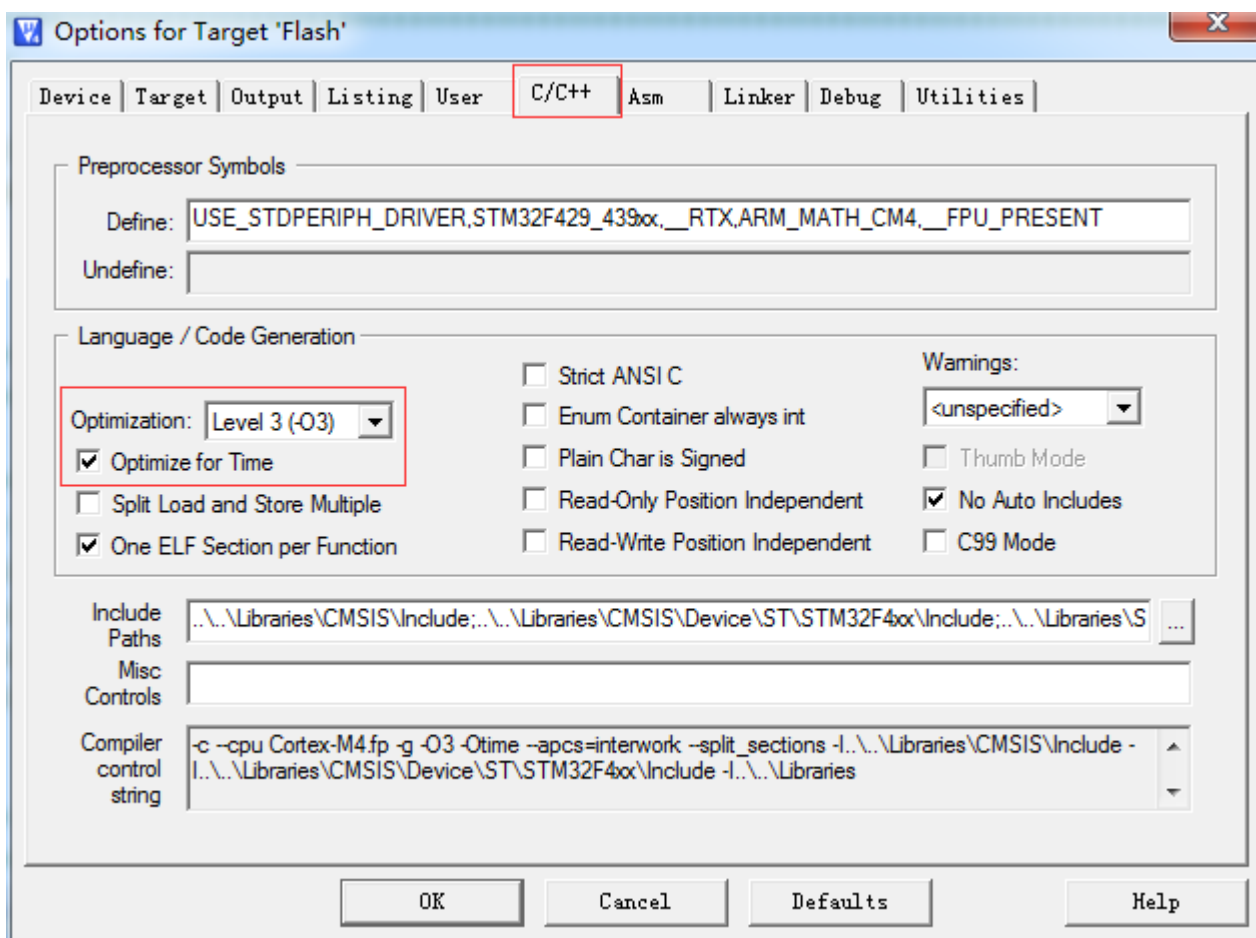
3.3 按分类封装各种变量

示波器的设计需要很多变量进行逻辑管理,从设计之初就需要将变量分类进行结构体封装,方便以后的维护升级。这一步至关重要,实际中差不多要定义上百个变量,如果不进行分类管理,以后的升级维护将非常麻烦。

这种方式还有一个好处是方便我们将 F429 的 CCM RAM 空间分配给这些变量使用。使用 CCM RAM 的好处是速度比通用 RAM 要快些,缺点是这部分空间不支持 DMA 操作。初次使用的用户比较容易在这个地方犯错误。所以在使用局部变量时,切勿将局部变量用于 DMA 传输。

3.4 选择编译器最高等级优化

为了发挥 STM32F429 的最高性能,需要大家开启最高等级优化和时间优化,即下面两个选项:



Optimization 选择 Level 3 最高优先等级，并且开启 Optimize for Time 时间优化。

● 知识点拓展

关于如何使用 MDK 实现代码最小优化和性能最佳优化可以看这个帖子中 KEIL 做的专题教程：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=1794>。

3.5 制作工程模板

关于工程模板推荐如下三个例子：

- ◆ V6-1016_STemWin 提高篇实验_WAV，MP3 软解播放器，带类似干干静听频谱（uCOS-III）
这个例子包含了 uCOS-III + STemWin + FatFS。
- ◆ V6-1017_STemWin 提高篇实验_WAV，MP3 软解播放器，带类似干干静听频谱（FreeRTOS）
这个例子包含了 FreeRTOS + STemWin + FatFS。
- ◆ V6-1018_STemWin 提高篇实验_视频播放器（RTX 版本，仅支持 MDK4.74）
这个例子包含了 RTX + emWin + RL-FlashFS。

当然，这三个只是推荐的，大家实际制作不必限制。也可以自己新建一个工程，但要包含 RTOS，GUI 和 FS。另外这三个例程中原有的功能删掉即可，仅留下程序模板。整理好模板后，开启 MDK 的最高等级优化 3 和时间优化，测试系统是否正常运行。测试正常后就可以作为模板了。

- 知识点拓展

工程模板推荐例子的下载地址：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=23687> 。

3.6 总结

工程设计初期的任务划分，任务优先级安排以及变量的统一管理还是比较重要的，这样各个任务可以在一个合理的框架下工作。不过设计的不好也没有关系，随着各个任务功能的实现，逐步修改完善即可。

第4章 示波器设计—主界面布局和框架

本章简单为大家说明主界面的布局和框架。

- 4.1 主界面框架选择
- 4.2 主界面布局
- 4.3 总结

4.1 主界面框架选择

emWin 主界面框架主要有三种 (当然, 还有很多其它种类, 我们这里仅说明以下三种):

◆ 第 1 种类似 ATM 机操作界面

大家在 ATM 机取钱的时候应该都有印象, ATM 机有一个主界面, 然后一路点击下去, 直到把钱取出来, 退出后回到主界面。这种形式简单总结下就是多级窗口之间任意切换, 删除之前的窗口, 创建新窗口, 这里有个演示效果, 让大家有个感性认识: <http://bbs.armfly.com/searcher.php>。

◆ 第 2 种类似电脑端, 手机端的主界面

这种方式有一个主界面, 主界面上面有很多图标, 点击某个图标进入功能界面, 这个功能界面也可以有多级子界面, 比如我们 STM32-V6 开发板的综合 Demo 就是这种形式:

<http://bbs.armfly.com/read.php?tid=17330>。想返回到主界面, 直接关闭这些子界面即可。

◆ 第 3 种采用隐藏和显示的方式

这种方式简单实用, 系统启动后先把所有需要显示的窗口全部创建好, 并用函数 WM_HideWindow 将这些窗口全部隐藏起来, 需要显示哪个窗口的时候, 调用函数 WM_ShowWindow 即可, 缺点是所有的窗口要一直占用着内存。

二代示波器的界面是在第二种方案的基础上实现的, 不同的是需要我们在桌面窗口上刷新波形。之所以在桌面窗口上刷新, 而没有选择在子窗口上刷新, 主要是方便程序操作。具体的代码实现会在后面章节讲解。

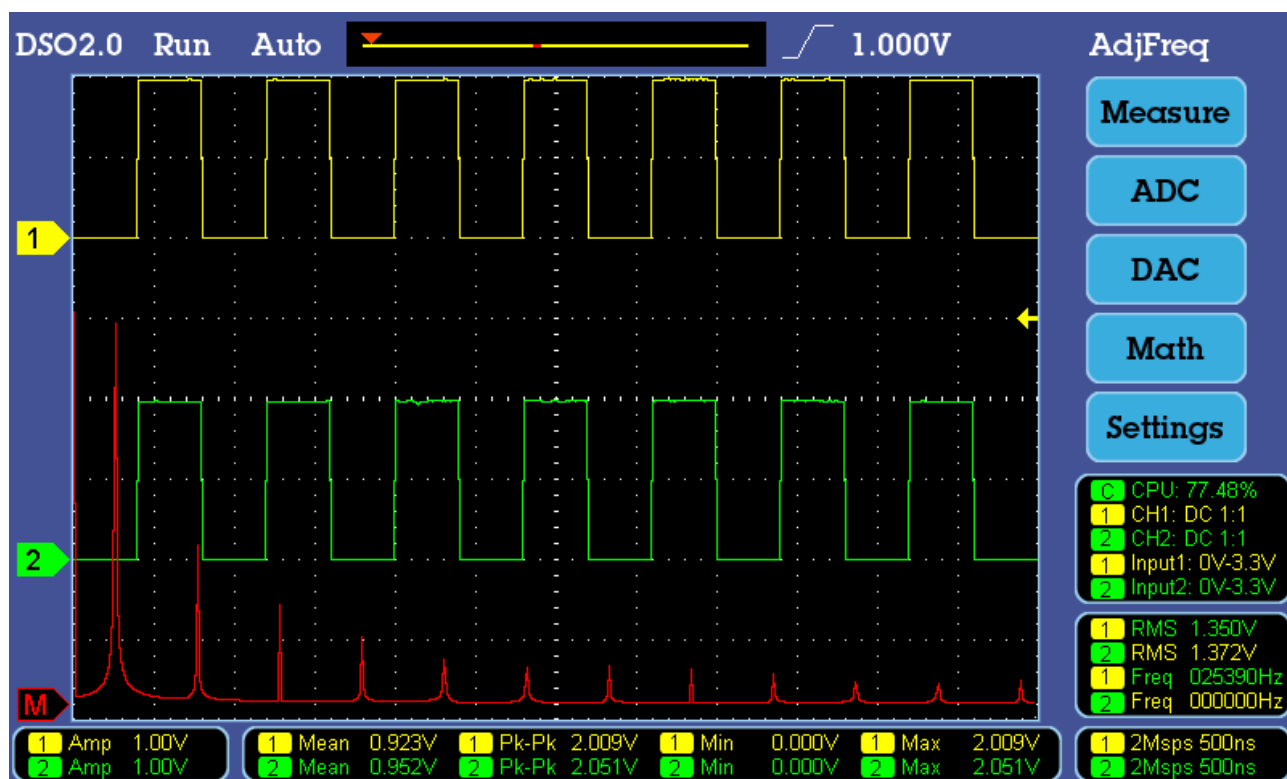
● 知识点拓展

新版 emWin 教程第 64 章: STemWin 多个窗口切换方法:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834>。

4.2 主界面布局

emWin 主界面布局如下：



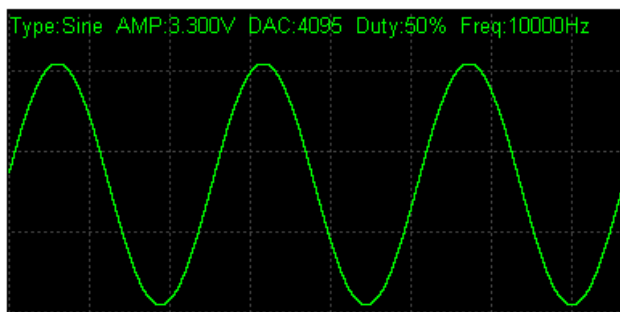
设计这个界面的布局耽误了很多时间，一方面是界面各个窗口的位置调整，还有界面效果的优化。最主要的原因还是 emWin 配套的 GUIBuilder 功能太弱，不支持这种类型的主界面建立。所以这个时候有个美工帮我们把界面布局做好就尤其重要了，特别是每个窗口和控件的坐标位置也给标出来。

主界面主要分为以下几部分：

- ◆ 桌面窗口，主要用于波形显示区。
- ◆ 五个圆角矩形对话框。
- ◆ 右上角的五个按钮，每个按钮都可以独立打开一个对话框，用于示波器控制，比如按下 DAC 按钮，弹出的这个界面主要用于信号发生器设置。



Waveform Generator



Input Freq: 1Hz - 50KHz

10000

DAC:0 - 4095



Duty:2% - 98%



Sine

Square

Ramp

1

2

3

CL

4

5

6

C

7

8

9

-

-

0

-

Output

4.3 总结

本章节主要把主界面框架和布局为大家做了一个简单介绍，方便后续章节的学习。

第5章 示波器设计—波形快速刷新方案

本章节比较重要，推荐的波形刷新方式都经过了大量测试验证。

- 5.1 波形快速刷新方案
- 5.2 示波器背景的快速刷新
- 5.3 系统上电，主界面无闪烁感
- 5.4 测量窗口的刷新
- 5.5 开关对话框时，界面的刷新方式
- 5.6 五个数值显示窗口的快速刷新
- 5.7 总结

5.1 波形快速刷新方案选择

波形快速刷新有很多方案需要测试，由于我们的 GUI 是采用的 emWin，所以下面的这些测试都是基于 emWin 实现的。

- ◆ 选择 8 位色还是 16 位色。
- ◆ 使能三缓冲还是窗口存储设备。
- ◆ 选用 ARGB 格式的 emWin 库还是 ABGR 格式的库。
- ◆ 将 STM32F429 超频后刷新是否有提升。
- ◆ 使用存储设备函数 GUI_MEMDEV_Draw 绘制还是多缓冲函数 GUI_MULTIBUF_Begin()和 GUI_MULTIBUF_End()。
- ◆ 使用函数 GUI_DrawGraph 还是 GUI_DrawPolyLine 绘制波形。

下面将这几个方案以此为大家做个说明。

5.1.1 选择 8 位色还是 16 位色

之前测试 emWin 刷颜色块，8 位色可以跑到 2 亿多点每秒，而 16 位色是 1 亿多点每秒。实际测试发现 8 位色刷颜色块还行，刷波形却不行，比较卡，所以采用 16 位色。

也许有读者会问为什么不使用 32 位色？对于单片机来说，刷 32 位色比 16 位色要吃力。所以 32 位色也不考虑。

- 知识点拓展

新版 emWin 教程第 34 章: STemWin 支持的颜色格式:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834> 。

5.1.2 使用三缓冲还是窗口存储设备

对于 STM32F429 而言, 使能三缓冲是指的用户要在 LCDConf_Lin_Template.c 文件中配置多缓冲, 并在应用程序中调用函数 WM_MULTIBUF_Enable(1)来使能。

使用内存设备是调用函数 WM_SetCreateFlags(WM_CF_MEMDEV)来实现。

如果 emWin 的配置支持多缓冲和窗口存储设备, 务必优先选择使用多缓冲, 实际使用 STM32F429BIT6 + 32 位 SDRAM + RGB565/RGB888 平台测试, 多缓冲可以有效地降低窗口移动或者滑动时的撕裂感, 并有效地提高流畅性, 通过使能窗口使用内存设备是做不到的。所以我们示波器也是选择使用三缓冲。使能多缓冲方式如下:

```
/*
*****
*   函数名: MainTask
*   功能说明: GUI 主函数
*   形参: 无
*   返回值: 无
*****
*/
void MainTask(void)
{
    /* emWin 初始化 */
    GUI_Init();

    /* 使能多缓冲 */
    WM_MULTIBUF_Enable(1);

    /* 省略 */
}
```

5.1.3 使用 ARGB 格式 emWin 库还是 ABGR 格式库

由于前面我们已经选择了使用 RGB565 颜色格式, 对于这个颜色格式, 使用 ARGB 格式的 emWin 库并没有性能提升, 而且实际做应用代码还要注意额外的事项。所以继续使用 ABGR 格式库。

- 知识点拓展

专题教程第 1 期: 基于 STM32 的硬件 RGB888 接口实现 emWin 的快速刷新方案, 32 位色或 24 位色:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=44512> 。

5.1.4 将 STM32F429 超频后刷新是否有提升

STM32F429 操作自带的 Flash 无法实现 0 延迟，随着设置的主频越高，延迟参数就要设置的越大，参数手册中给的测试结果如下：

Wait states (WS) (LATENCY)	HCLK (MHz)			
	Voltage range 2.7 V - 3.6 V	Voltage range 2.4 V - 2.7 V	Voltage range 2.1 V - 2.4 V	Voltage range 1.8 V - 2.1 V Prefetch OFF
0 WS (1 CPU cycle)	0 <HCLK ≤ 30	0 <HCLK ≤ 24	0 <HCLK ≤ 22	0 < HCLK ≤ 20
1 WS (2 CPU cycles)	30 <HCLK ≤ 60	24 < HCLK ≤ 48	22 <HCLK ≤ 44	20 <HCLK ≤ 40
2 WS (3 CPU cycles)	60 <HCLK ≤ 90	48 < HCLK ≤ 72	44 < HCLK ≤ 66	40 < HCLK ≤ 60
3 WS (4 CPU cycles)	90 <HCLK ≤ 120	72 < HCLK ≤ 96	66 <HCLK ≤ 88	60 < HCLK ≤ 80
4 WS (5 CPU cycles)	120 <HCLK ≤ 150	96 < HCLK ≤ 120	88 < HCLK ≤ 110	80 < HCLK ≤ 100
5 WS (6 CPU cycles)	150 <HCLK ≤ 180	120 <HCLK ≤ 144	110 < HCLK ≤ 132	100 < HCLK ≤ 120
6 WS (7 CPU cycles)		144 <HCLK ≤ 168	132 < HCLK ≤ 154	120 < HCLK ≤ 140
7 WS (8 CPU cycles)		168 <HCLK ≤ 180	154 <HCLK ≤ 176	140 < HCLK ≤ 160
8 WS (9 CPU cycles)			176 <HCLK ≤ 180	160 < HCLK ≤ 168

实际测试，将其超频到 216MHz 并没有任何性能提升，为了稳定起见，依然使用的 168MHz。至于为什么不使用 F429 支持的 180MHz，其原因在这个帖子里面有描述：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=16830>。

5.1.5 使用存储设备函数还是三缓冲函数做整体刷新

之前的一代示波器是采用下面的方式进行绘制的：

```
GUI_MEMDEV_Draw(&Rect, _Draw, &Param, 0, GUI_MEMDEV_NOTRANS);
```

在函数_Draw 里面实现波形显示区和波形的绘制，但是速度比较慢，600*480 显示区的刷新率差不多 10 帧左右，现在做二代示波器显然不能再使用这种方法了，速度太慢，而且实际测试发现 F429 使用这种方式比一代示波器中 F407 采用这种方式要慢一点，这样的结果显然不是我们想要的。现在的绘制方式是采用的多缓冲函数：

```
GUI_MULTIBUF_Begin();
/* 在这两个函数之间实现波形绘制 */
GUI_MULTIBUF_End();
```

使用这两个函数做整体刷新，可以有效的避免波形刷新时的闪烁和撕裂感。

5.1.6 使用哪个函数绘制波形

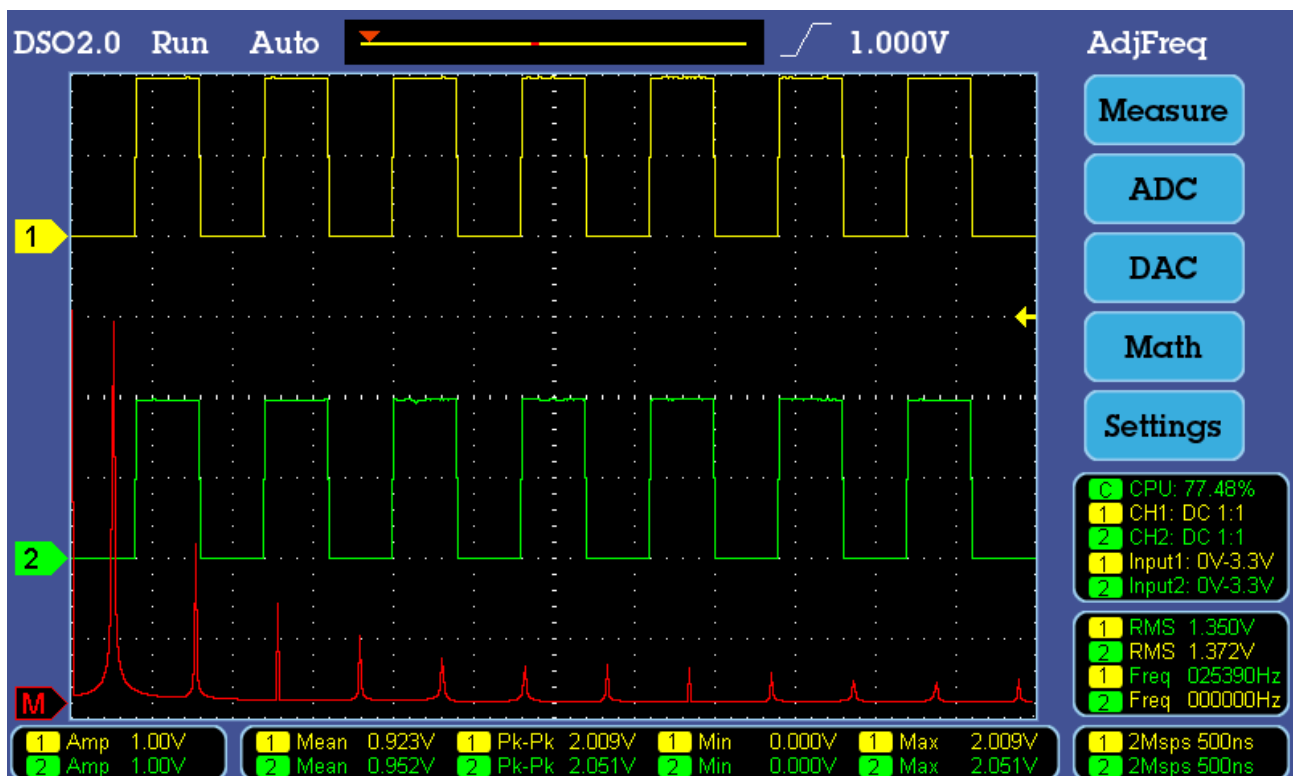
之前一代示波器中波形的绘制是采用的函数 GUI_DrawPolyLine，二代示波器中也采用这个函数的话，发现速度慢了好多，而使用函数 GUI_DrawGraph 却不存在这个问题。而且重新优化底层，函数 GUI_DrawPolyLine 的性能并没有提升，估计是内部执行机制的问题。使用函数 GUI_DrawPolyLine 的好处就是可以实现各种波形效果，鉴于这个函数性能比较差，所以还是继续使用 GUI_DrawGraph。

5.1.7 使用单图层还是双图层

由于 STM32F429 的硬件双图层性能有限，而且使用 emWin 管理比较麻烦，所以二代示波器就采用单图层来实现。

5.2 示波器背景的快速刷新

示波器的界面显示效果如下：



波形显示区背景是固定的，所以上电后就将其绘制到存储设备里面，以后显示背景就可以直接调用存储设备的 API 函数。下面是示波器背景的绘制：

```
/*
*****
* 函数名: CreateWindowAmplitude
* 功能说明: 创建幅值窗口
* 形参: x0 左上角 x 坐标
```

```
*          y0  左上角 y 坐标
*          x1  右下角 x 坐标
*          y1  右下角 y 坐标
*   返 回 值: 无
*****
*/
void DSO_DrawBakFrame(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1)
{
    uint16_t x;
    uint16_t y;

    /* 填充背景 */
    GUI_SetBkColor(GUI_BLACK);
    GUI_ClearRect(x0, y0, x1, y1);

    GUI_SetColor(GUI_WHITE);

    /* 绘制水平刻度点 */
    for (y = 0; y < 9; y++)
    {
        for (x = 0; x < 61; x++)
        {
            GUI_DrawPoint(x0 + (x * 10), y0 + (y * 50));
        }
    }

    for (x = 0; x < 61; x++)
    {
        GUI_DrawPoint(x0 + (x * 10), y1);
    }

    /* 绘制垂直刻度点 */
    for (x = 0; x < 12; x++)
    {
        for (y = 0; y < 41; y++)
        {
            GUI_DrawPoint(x0 + (x * 50), y0 + (y * 10));
        }
    }

    for (y = 0; y < 41; y++)
    {
        GUI_DrawPixel(x1, y0 + (y * 10));
    }

    /* 绘制最后脚上的那个点 */
    GUI_DrawPixel(x1 - 1, y1 - 1);

    /* 绘制垂直中心刻度点 */
    for (y = 0; y < 41; y++)
    {
        GUI_DrawPixel(x0 - 1 + (300), y0 + (y * 10));
        GUI_DrawPixel(x0 + 1 + (300), y0 + (y * 10));
    }

    GUI_DrawPixel(x0 - 1 + (300), y1);
    GUI_DrawPixel(x0 + 1 + (300), y1);
}
```

```
/* 绘制水平中心刻度点 */
for (x = 0; x < 61; x++)
{
    GUI_DrawPixel(x0 + (x * 10), y0 - 1 + (200));
    GUI_DrawPixel(x0 + (x * 10), y0 + 1 + (200));
}

GUI_DrawPixel(x1, y0 - 1 + (200));
GUI_DrawPixel(x1, y0 + 1 + (200));
}
```

通过下面的方式将示波器背景绘制到存储设备里面：

```
hMemDSO = GUI_MEMDEV_CreateFixed(0,
                                0,
                                600,
                                400,
                                GUI_MEMDEV_HASTRANS,
                                GUI_MEMDEV_APILIST_16,
                                GUICC_M565);

/* 绘制示波器窗口背景 */
GUI_MEMDEV_Select(hMemDSO);
DSO_DrawBakFrame(0, 0, 599, 399);
GUI_MEMDEV_Select(0);
```

以后程序中绘制示波器背景，仅需调用如下函数就可以实现

```
GUI_MEMDEV_WriteAt(hMemDSO, 40, 40);
```

实际测试这个函数仅需 9-10ms 就可以完成。

5.3 系统上电，主界面无闪烁感

为了追求极致的用户体验，需要解决板子开机后存在的两个问题。

◆ 界面整体加载

有时候界面设计比较复杂，开机后不能保证所有的控件同时加载出来，这个时候有个非常简单的解决办法，绘制前隐藏桌面窗口，绘制完毕后显示桌面窗口。

```
WM_HideWin(WM_HBKWIN);
```

```
/* 初始化 DSO */
DSO_Init(1);
```

```
WM_ShowWin(WM_HBKWIN);
```

◆ 上电瞬间产生的高亮

这个问题不仔细注意的话，不容易看到。起初以为是软件配置问题，调试了下没有效果，后来又设置背光 PWM 的不同占空比，发现也不行。

后来干脆关闭 PWM，而且也关闭 emWin 和 LCD 的图层，仅显示背景层，直接操作 PWM 引脚的高低电平。测试发现将其配置为低电平后，要延迟 200ms 左右再点亮 LCD 就没有问题了，延迟时间短了则会有个瞬间的高亮。知道了这个原因，程序中就好解决了，可以直接延迟 200ms 后再点亮，而这个二代

示波器无需这么做，因为上电后需要将各种测量窗口的背景和波形显示区背景绘制到存储设备里面，正好用于替代者 200ms 延迟。

```
GUI_Init();

/* 先关闭背光 */
LCD_SetBackLight(0);

/* 在这里加载各种窗口背景 */

/* 显示出所有的窗口 */
GUI_Exec();

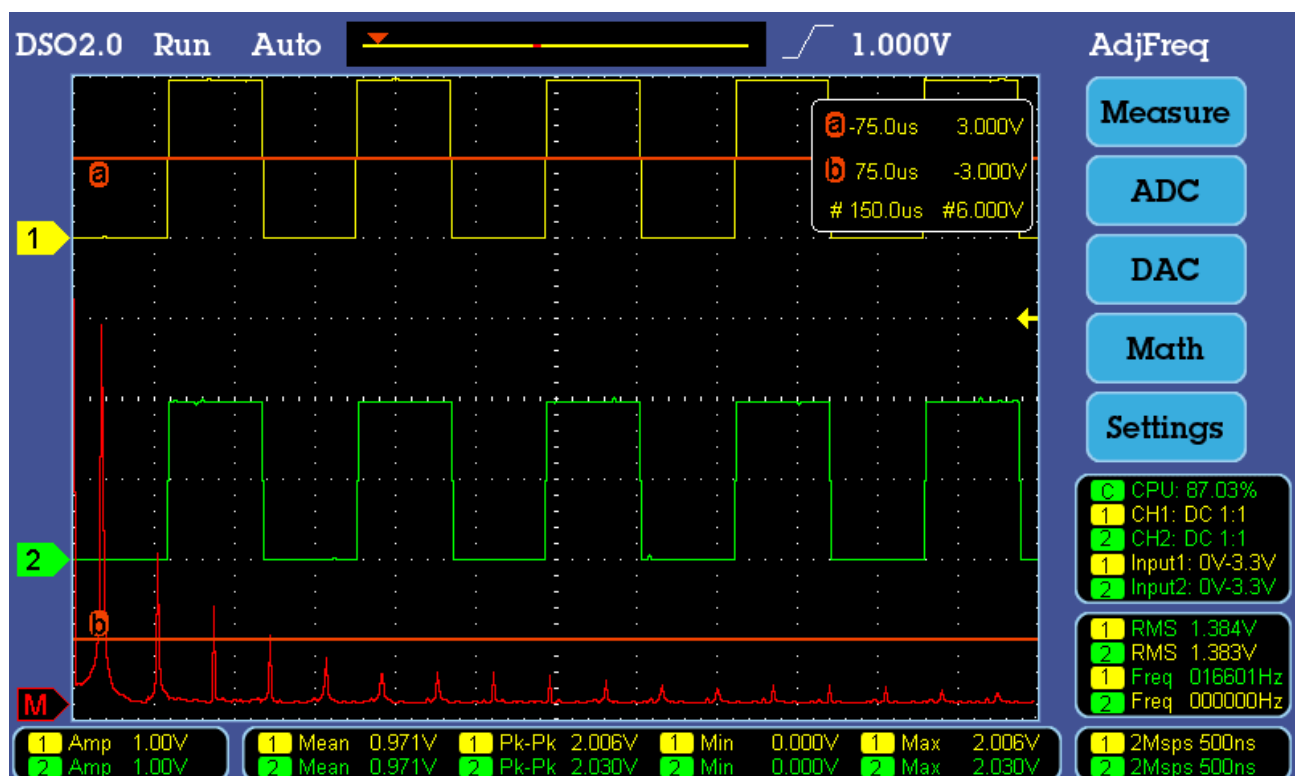
/* 再开启背光 */
LCD_SetBackLight(255);

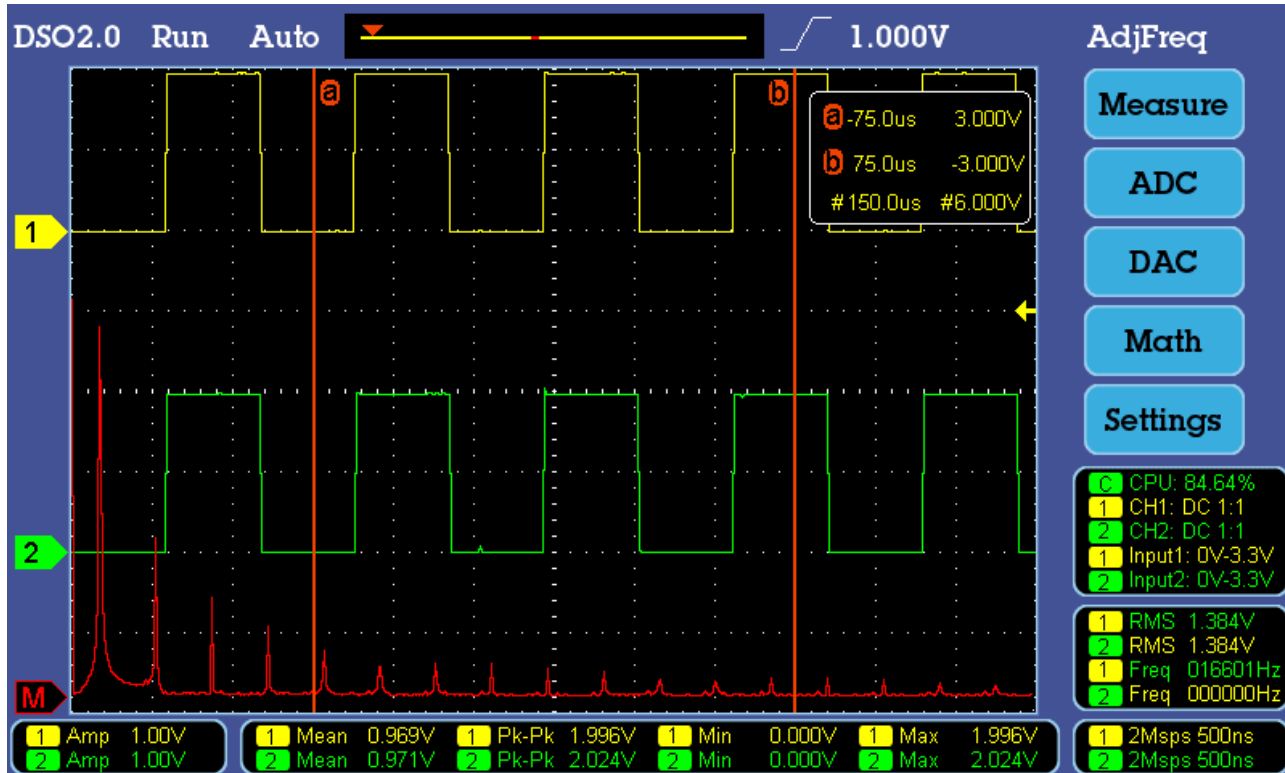
/* 开启其它操作 */
```

通过这种方式就完美解决了瞬间高亮的问题。

5.4 测量窗口的刷新

测量功能是指的下面的水平测量和垂直测量：

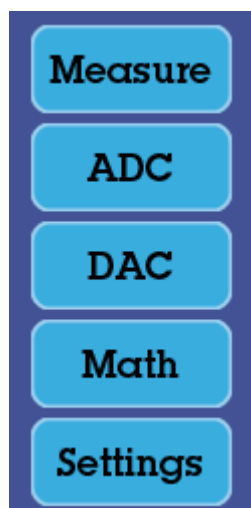




测量功能的数据显示不要以窗口的形式呈现，因为将窗口显示在波形显示区上会造成波形刷新慢。当前的方案是在绘制完毕波形后，直接 2D 函数绘制测量窗口，这种方式的实际效果好很多。

5.5 开关对话框时，界面的刷新方式

二代示波器主界面上有如下五个按键，点击后会弹出一个对话框。



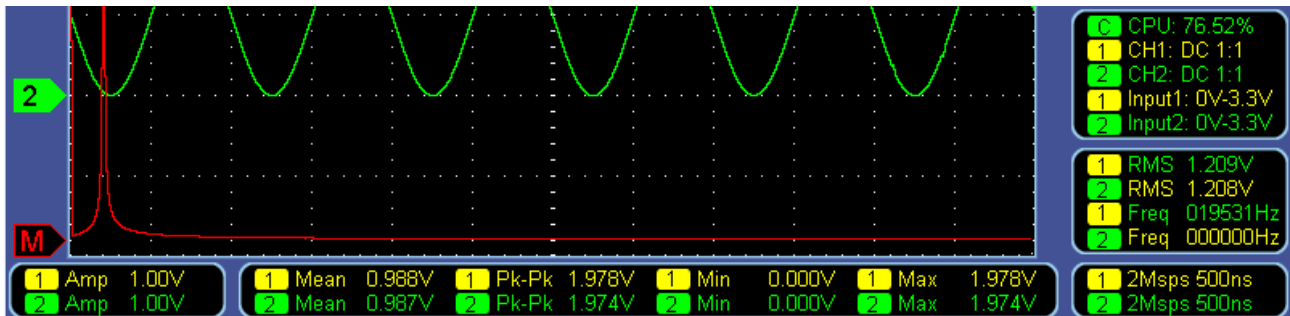
我们这里要讨论的是关闭这个对话框时存在的问题。关闭这个对话框时，为了保证主界面整体的刷新效果，需要清背景的同时，将示波器波形显示区背景也一起刷出来（文件 DSO_Init.c）：

```
GUI_SetBkColor(0x905040);  
GUI_Clear();  
GUI_MEMDEV_WriteAt(hMemDSO, 40, 40);
```

通过这种方式，关闭对话框绘制的效果就好很多。

5.6 五个数值显示窗口的快速刷新

五个数值显示窗口是指的下面这五个：



主要有幅值窗口，两个状态窗口，频率窗口和系统信息窗口。这几个窗口的创建都是以对话框的形式创建，方便管理。对话框回调函数的 WM_PAINT 消息里面通过大量的 2D 函数进行绘制，每次刷新数值还是比较影响系统性能的，为了降低影响，需要提前将其绘制到存储设备里面，跟本章节 5.2 小节的方法类似。我们这里以其中一个状态窗口为例进行说明，WM_PAINT 消息里面要绘制如下这些图形和字符：

```
/*
*****
*   函数名: PaintDialogStatus1
*   功能说明: 状态窗口的回调函数重绘消息
*   形参: pMsg 指针地址
*   返回值: 无
*****
*/
void PaintDialogStatus1(WM_MESSAGE * pMsg)
{
    /* 清背景色 */
    GUI_SetBkColor(0x905040);
    GUI_Clear();

    /* 绘制填充的抗锯齿圆角矩形 */
    GUI_SetColor(GUI_BLACK);
    GUI_AA_FillRoundedRect(0, 0, 509, 34, 10);

    /* 绘制抗锯齿圆角矩形 */
    GUI_SetColor(0XEBCD9E);
    GUI_SetPenSize(2);
    GUI_AA_DrawRoundedRect(0, 0, 509, 34, 10);

    /* 绘制抗锯齿圆角矩形，并填数值 1 */
    GUI_SetColor(GUI_YELLOW);
    GUI_AA_FillRoundedRect(10, 4, 30, 16, 3);

    GUI_SetColor(GUI_BLACK);
    GUI_SetFont(&GUI_Font16_1);
    GUI_SetTextMode(GUI_TEXTMODE_TRANS);
    GUI_DispCharAt('1', 16, 3);

    /* 绘制抗锯齿圆角矩形，并填数值 1 */
}
```

```
GUI_SetColor(GUI_YELLOW);
GUI_AA_FillRoundedRect(135, 4, 155, 16, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('1', 141, 3);

/* 绘制抗锯齿圆角矩形，并填数值 1 */
GUI_SetColor(GUI_YELLOW);
GUI_AA_FillRoundedRect(260, 4, 280, 16, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('1', 266, 2);

/* 绘制抗锯齿圆角矩形，并填数值 1 */
GUI_SetColor(GUI_YELLOW);
GUI_AA_FillRoundedRect(385, 4, 405, 16, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('1', 391, 2);

/* 绘制抗锯齿圆角矩形，并填数值 2 */
GUI_SetColor(GUI_GREEN);
GUI_AA_FillRoundedRect(10, 19, 30, 31, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('2', 16, 18);

/* 绘制抗锯齿圆角矩形，并填数值 2 */
GUI_SetColor(GUI_GREEN);
GUI_AA_FillRoundedRect(135, 19, 155, 31, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('2', 141, 18);

/* 绘制抗锯齿圆角矩形，并填数值 2 */
GUI_SetColor(GUI_GREEN);
GUI_AA_FillRoundedRect(260, 19, 280, 31, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font16_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispCharAt('2', 266, 18);

/* 绘制抗锯齿圆角矩形，并填数值 2 */
GUI_SetColor(GUI_GREEN);
GUI_AA_FillRoundedRect(385, 19, 405, 31, 3);

GUI_SetColor(GUI_BLACK);
```

```
GUI_SetFont(&GUI_Font16_1);  
GUI_SetTextMode(GUI_TEXTMODE_TRANS);  
GUI_DispCharAt('2', 391, 18);  
}
```

通过下面的方式将其绘制到存储设备里面（系统上电时就进行加载）：

```
/*  
*****  
*   函 数 名: DrawWinStatus1Bk  
*   功能说明: 将窗口背景绘制到存储设备里面  
*   形    参: 无  
*   返 回 值: 无  
*****  
*/  
void DrawWinStatus1Bk(void)  
{  
    hMemStatus1Dlg = GUI_MEMDEV_CreateFixed(0,  
                                              0,  
                                              510,  
                                              35,  
                                              GUI_MEMDEV_HASTRANS,  
                                              GUI_MEMDEV_APILIST_16,  
                                              GUICC_M565);  
  
    GUI_MEMDEV_Select(hMemStatus1Dlg);  
    PaintDialogStatus1(NULL);  
    GUI_MEMDEV_Select(0);  
}
```

之后将如下函数填到状态窗口回调函数 WM_PAINT 消息里面即可

```
GUI_MEMDEV_WriteAt(hMemStatus1Dlg, 145, 444);
```

其它四个窗口的设计方法同理，通过这样的优化，一定程度上降低了 GUI 刷新的负担。

5.7 总结

建议大家实际测试下不同方案，这样会有一个深刻的体会。

第6章 示波器设计—双通道 ADC 驱动

本章节为大家讲解示波器的 ADC 驱动，采用 STM32 自带 ADC 实现。关于 STM32F429 的 ADC，可以说处处有地雷，不小心就踩上了，如果简单的使用，不会发现，复杂使用就很容易踩到了。

6.1 3 个 ADC 的快速交替采样

6.2 双通道 ADC 采样

6.3 拓展阅读

6.4 总结

6.1 3 个 ADC 的快速交替采样

起初二代示波器是打算像一代示波器那样，准备做成 3ADC (ADC1, ADC2 和 ADC3) 快速交替采样，后期才改成双通道。这里将 3ADC 的各种奇葩问题也给大家做个说明，防止大家踩坑。

3 个 ADC 快速交替采样的两个可选的方案及其存在的问题。

6.1.1 方案一

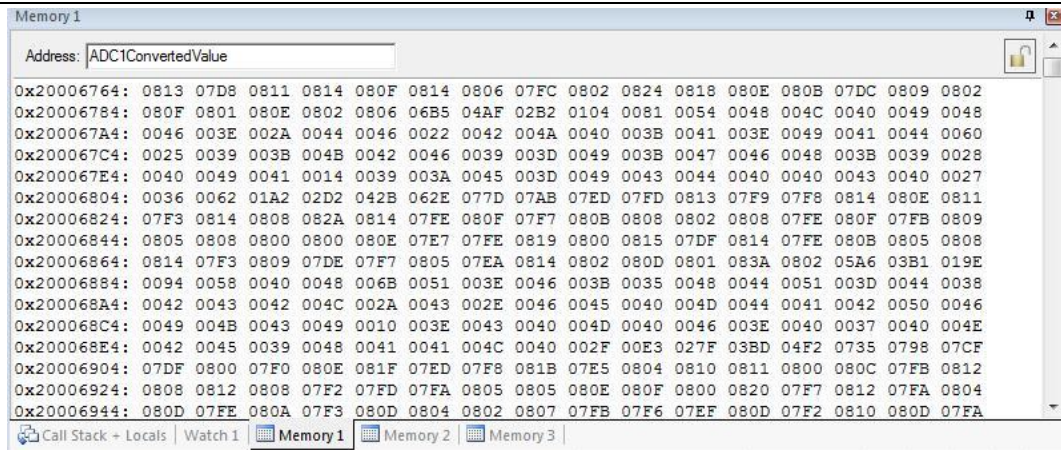
依然采用一代示波器那种方式，3 个 ADC 都独立配置自己的 DMA 通道和相应的定时器进行触发。经过详细的测试发现，在最高采样率 $2.8\text{Mps} * 3 = 8.4\text{Mps}$ 的情况下，F429 的总线矩阵已经处理不过来了，导致的现象就是 3 个 ADC 中有两个已经停止工作。

下面的测试都是在我们 STM32-V6 开发板上进行的，主频 168MHz。

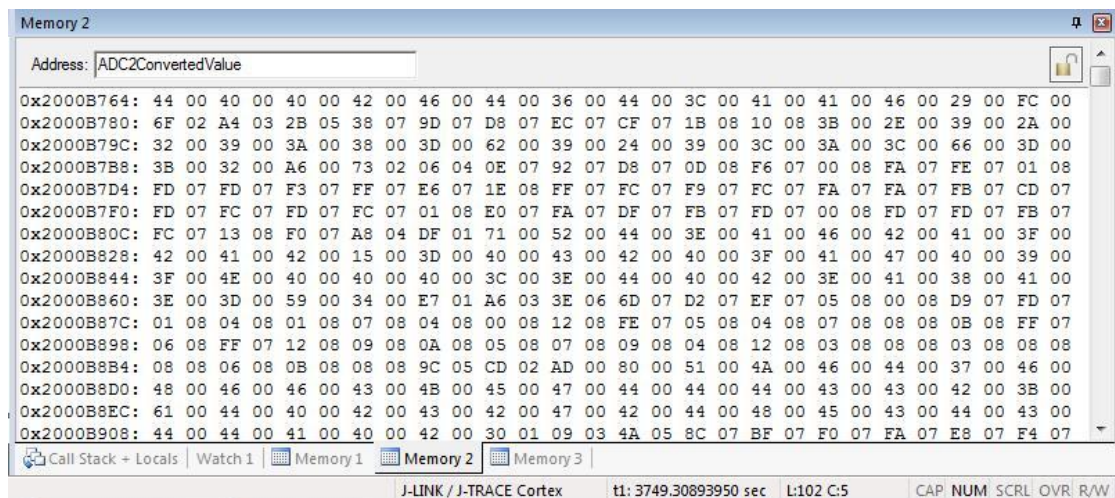
◆ 测试方法

直接调试状态看 ADC1, 2, 3 的三个大缓冲即可，看数据缓冲的数据是否在变化。

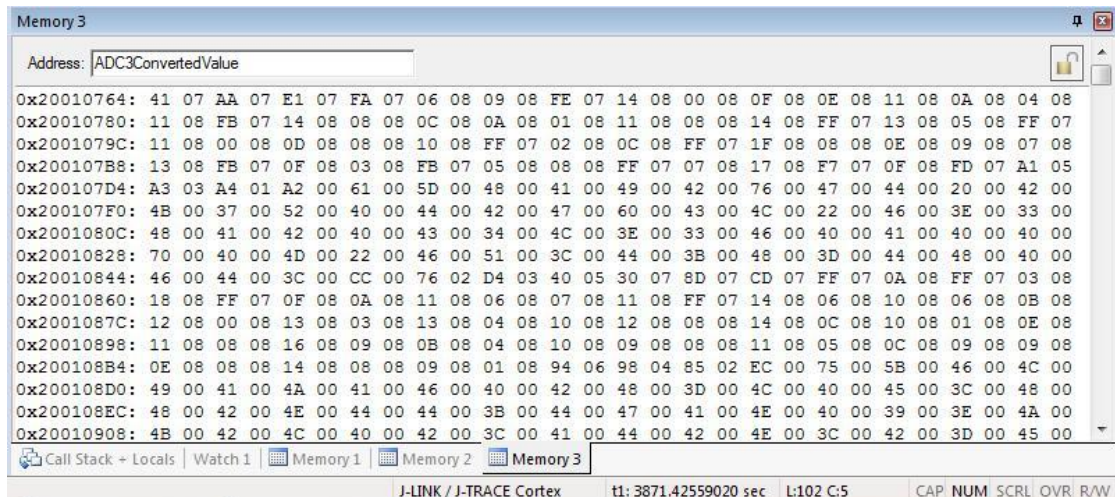
ADC1：



ADC2 :



ADC3 :



◆ 测试一

条件：

开启 ADC1，ADC2 和 ADC3 及其配套的 DMA，采用定时器触发，未开启 emWin（要用到 LTDC，DMA2D 和 FMC 外接的 SDRAM）。

现象：

这种情况下，3 个 ADC 可以正常工作。

◆ 测试二

条件：

开启 ADC1 ,ADC2 和 ADC3 及其配套的 DMA ,采用定时器触发。仅使用 ADC1 ,开启 emWin (要用到 LTDC ,DMA2D 和 FMC 外接的 SDRAM)。

现象：

ADC1 在最高采样率 2.8Msps 的情况下，工作几秒钟，停止工作。

◆ 测试三

条件：

开启 ADC1 ,ADC2 和 ADC3 及其配套的 DMA ,采用定时器触发。仅使用 ADC2 ,开启 emWin (要用到 LTDC ,DMA2D 和 FMC 外接的 SDRAM)。

现象：

ADC2 在最高采样率 2.8Msps 的情况下，停止工作。

◆ 测试四

条件：

开启 ADC1 ,ADC2 和 ADC3 及其配套的 DMA ,采用定时器触发。仅使用 ADC3 ,开启 emWin (要用到 LTDC ,DMA2D 和 FMC 外接的 SDRAM)。

现象：

ADC3 在最高采样率 2.8Msps 的情况下，正常工作。

◆ 测试五

条件：

开启 ADC1 ,ADC2 和 ADC3 及其配套的 DMA ,采用定时器触发。同时使用 ADC3 ,ADC2 和 ADC1 ,开启 emWin (要用到 LTDC ,DMA2D 和 FMC 外接的 SDRAM)。

现象：

每个 ADC 都是在最高采样率 2.8Msps ,刚开始 ADC1 还工作 ,过会 ADC1 停止工作 ,ADC2 一直没有工作 ,ADC3 一直在工作。

◆ 测试现象

ADC 工作不正常时，二代示波器波形显示效果如下：



◆ 总结

对于 STM32F429 来说，如果三个 ADC 配合自己的 DMA 采用最高采样率 $2.8\text{Mps} \times 3$ ，且采用的定时器触发，在未使用 emWin（要用到 LTDC，DMA2D 和 FMC 外接的 SDRAM）时，可以正常工作，使用了的话，将会出现 ADC1 和 ADC2 不工作的情况。

可能的原因是定时器触发太快，F429 总线矩阵在这种情况下已经无法正常工作。

6.1.2 方案二

采用 F429 自带的 3 个 ADC 快速交替采样模式，这个模式的问题最多，表现在以下三个方面：

- ◆ 这种方式不能用于定时器触发，因为这三个 ADC 之间的采样间隔只有 5 个 ADC 时钟周期到 20 个 ADC 时钟周期可选，这样采用定时器触发的时候就没有办法做到等间隔采样。
- ◆ 采集出来的波形效果比较差，基本没有使用价值。
- ◆ 这种方式的 3 个 ADC 快速交替采样外加开启 emWin（使能 DMA2D，LTDC 和 FMC 驱动的 SDRAM），轻轻松松就将 F429 的总线矩阵干趴下了，直接导致 ADC 不工作了。

测试条件：STM32-V6 开发板，采用 STM32F429 支持的 3 个 ADC 快速交替采样，使能 DMA，采用的软件触发。

◆ 测试一

配置采样率 2.1Mps 。

```
/*  
PCLK2 = HCLK / 2  
下面选择的是 2 分频  
ADCCLK = PCLK2 / 4 = HCLK / 4 = 168 / 4 = 42M  
三 ADC 采样频率： 42 / 20 = 2.1Mbps  
*/
```

*****/

/* ADC 公共部分初始化-----*/

```
ADC_CommonInitStructure.ADC_Mode = ADC_TripleMode_Interl;
ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_20Cycles;
ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_1;
ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div2;
ADC_CommonInit(&ADC_CommonInitStructure);
```

凑合还能干活，但是显示效果很差：



◆ 测试二

配置采样率 4.2Msps，直接停止工作。

PCLK2 = HCLK / 2

下面选择的是 2 分频

ADCCLK = PCLK2 / 4 = HCLK / 4 = 168 / 4 = 42M

三 ADC 采样频率：42 / 10 = 4.2Mbps

*****/

/* ADC 公共部分初始化-----*/

```
ADC_CommonInitStructure.ADC_Mode = ADC_TripleMode_Interl;
ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_10Cycles;
ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_1;
ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div2;
ADC_CommonInit(&ADC_CommonInitStructure);
```

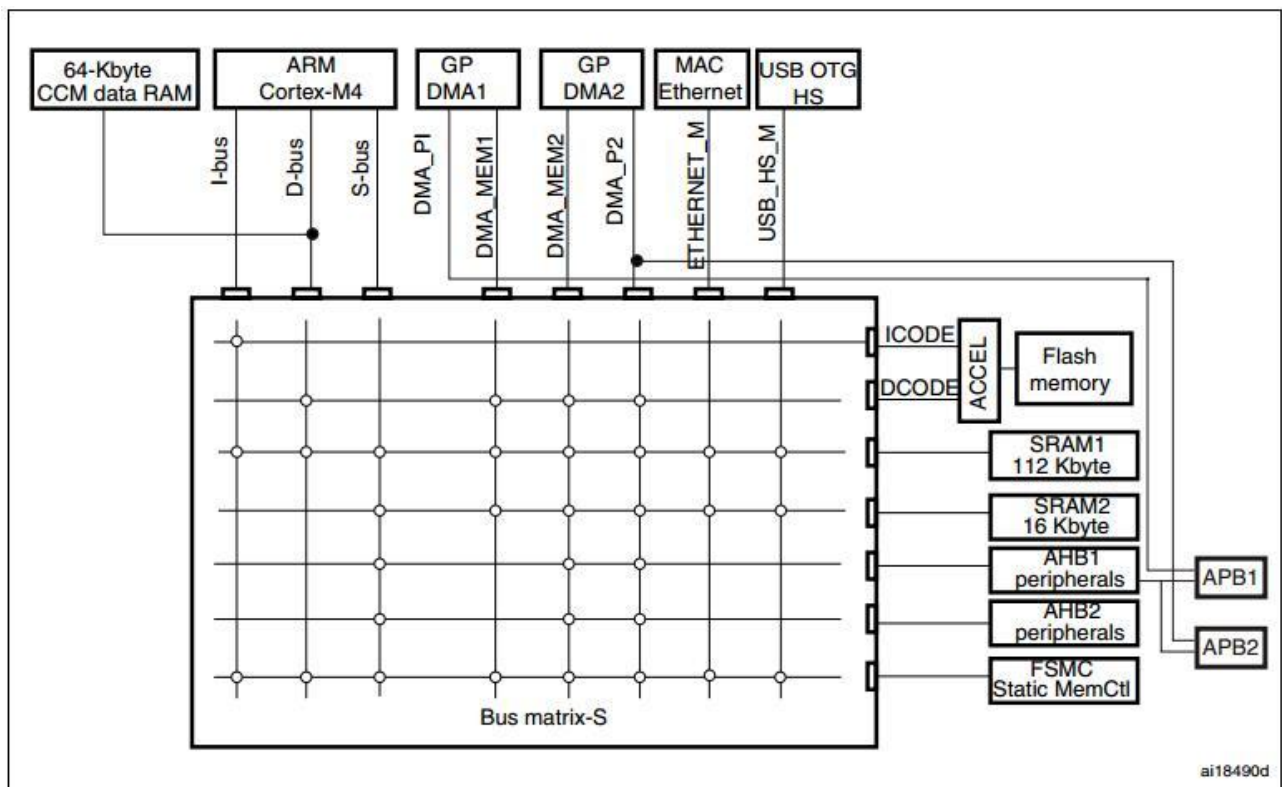
显示效果直接死机：



◆ 总结

相比于三个 ADC 独立配置实现快速交替采样，F429 自带的这种 3ADC 快速交替采样的显示效果极其差，基本没有利用价值，而且最重要的是这种方式轻轻松松就将 F429 的总线矩阵干趴下了。结果就是 ADC 不干活了。

再来欣赏下 F429 的总线矩阵长什么样子：



6.1.3 选择方案一还是方案二

最终决定继续采用方案一，将最高采样率锁定在 6Msps，这样才稳定些。

6.2 双通道 ADC 采样

相对于前面三通道 ADC 实现的快速交替采样，通过 ADC1 和 ADC3 实现双通道示波器，F429 的硬件负担就稍轻松一些，不过依然存在一些问题。下面优先为大家说明 ADC1 和 ADC3 的驱动设计问题。

- ◆ ADC1 和 ADC3 被设置为定时器触发，DMA 方式。其中 ADC3 使能了模拟看门狗，用于示波器上升沿的正常触发功能。另外，使用定时器触发是为了设置不同的采样率。
- ◆ ADC2 用于简单电压采集，未使用定时器和 DMA。

```
/*
*****
*   函数名: bsp_InitADC
*   功能说明: ADC 初始化
*   形参: 无
*   返回值: 无
*****
*/
void bsp_InitADC(void)
{
    ADC_InitTypeDef      ADC_InitStructure;
    ADC_CommonInitTypeDef ADC_CommonInitStructure;
    DMA_InitTypeDef      DMA_InitStructure;
    GPIO_InitTypeDef     GPIO_InitStructure;
    NVIC_InitTypeDef     NVIC_InitStructure;

    /* 配置模拟看门狗中断 NVIC */
    NVIC_InitStructure.NVIC_IRQChannel = ADC_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    /* 使能 ADC1, ADC2, ADC3, DMA2 和 GPIO 时钟 *****/
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2 | RCC_AHB1Periph_GPIOC, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1 | RCC_APB2Periph_ADC2 | RCC_APB2Periph_ADC3, ENABLE);

    /* DMA2 Stream1 channel1 配置用于 ADC3 *****/
    //DMA_DeInit(DMA2_Stream1);
    DMA_InitStructure.DMA_Channel = DMA_Channel_2;
    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)ADC3_DR_ADDRESS;
    DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)&ADC3ConvertedValue;
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
    DMA_InitStructure.DMA_BufferSize = 1024*10;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_PeripheralDataSize_HalfWord;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
```



```
DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
DMA_Init(DMA2_Stream1, &DMA_InitStructure);
DMA_Cmd(DMA2_Stream1, ENABLE);

/* DMA2 Stream0 channel0 配置用于 ADC1 *****/
//DMA_DeInit(DMA2_Stream0);
DMA_InitStructure.DMA_Channel = DMA_Channel_0;
DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)ADC1_DR_ADDRESS;
DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)&ADC1ConvertedValue;
DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
DMA_InitStructure.DMA_BufferSize = 1024*10;
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
DMA_InitStructure.DMA_MemoryDataSize = DMA_PeripheralDataSize_HalfWord;
DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
DMA_InitStructure.DMA_Priority = DMA_Priority_High;
DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
DMA_Init(DMA2_Stream0, &DMA_InitStructure);
DMA_Cmd(DMA2_Stream0, ENABLE);
```

```
/* 配置 ADC 引脚为模拟输入模式*****/
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
GPIO_Init(GPIOC, &GPIO_InitStructure);
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
GPIO_Init(GPIOC, &GPIO_InitStructure);
```

```
/*
```

```
*****
```

```
    PCLK2 = HCLK / 2
```

```
    下面选择的是 2 分频
```

```
    ADCCLK = PCLK2 / 2 = HCLK / 4 = 168 / 4 = 42M
```

```
    ADC 采样频率: Sampling Time + Conversion Time = 3 + 12 cycles = 15cyc
```

```
    Conversion Time = 42MHz / 15cyc = 2.8Mbps.
```

```
*****
```

```
*/
```

```
/* ADC 公共部分初始化*****/
```

```
ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;
ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_8Cycles;
ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled;
ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div2;
ADC_CommonInit(&ADC_CommonInitStructure);
```

```
////////////////////////////////////
////////////////////////////////////
```

```
/*ADC3 的配置*****//
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC3;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfConversion = 1;

/* ADC3 规则通道配置 */
ADC_Init(ADC3, &ADC_InitStructure);
ADC-RegularChannelConfig(ADC3, ADC_Channel_10, 1, ADC_SampleTime_3Cycles);

/* 使能 ADC3 DMA */
ADC_DMACmd(ADC3, ENABLE);

/* 配置模拟看门狗的阈值 注意别配置反了, 要不一直进入中断 */
ADC_AnalogWatchdogThresholdsConfig(ADC3, 4095, 0);

/* 配置模拟看门狗监测 ADC3 的通道 10 */
ADC_AnalogWatchdogSingleChannelConfig(ADC3, ADC_Channel_10);

/* 使能一个规则通道的看门狗 */
ADC_AnalogWatchdogCmd(ADC3, ADC_AnalogWatchdog_SingleRegEnable);

/* 使能模拟看门狗中断 */
ADC_ITConfig(ADC3, ADC_IT_AWD, ENABLE);

/* 使能 DMA 请求 (多 ADC 模式) -----*/
ADC_DMAResquestAfterLastTransferCmd(ADC3, ENABLE);

/* Enable ADC1 -----*/
ADC_Cmd(ADC3, ENABLE);

////////////////////////////////////
////////////////////////////////////

/*ADC1 的配置*****//
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfConversion = 1;

/* ADC1 规则通道配置 -----*/
ADC_Init(ADC1, &ADC_InitStructure);
ADC-RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_3Cycles);

/* 使能 ADC1 DMA */
ADC_DMACmd(ADC1, ENABLE);

/* 使能 DMA 请求 (多 ADC 模式) -----*/
ADC_DMAResquestAfterLastTransferCmd(ADC1, ENABLE);

/* Enable ADC1 -----*/
ADC_Cmd(ADC1, ENABLE);
```



```

////////////////////////////////////
////////////////////////////////////

/*ADC2 的配置*****/
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC2;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfConversion = 1;

/* ADC2 规则通道配置 */
ADC_Init(ADC2, &ADC_InitStructure);
ADC-RegularChannelConfig(ADC2, ADC_Channel_10, 1, ADC_SampleTime_480Cycles);

/* 使能 ADC2 */
ADC_Cmd(ADC2, ENABLE);

/**定时器配置*****/
TIM1_Config();
}

```

使用这个驱动要注意以下四个问题：

- ◆ 当前 ADC1 和 ADC3 的 DMA 缓冲区都设置了 10240 个，这个大小已经够用。
- ◆ DMA 的 FIFO 功能一定要关闭，这是一个大坑。详情在这个帖子里面做了总结，必看：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=77359>。
- ◆ ADC 的采样率不要设置低了，ADC 设置的时钟速度一定要大于等于定时器触发速度。当前 ADC1 和 ADC3 配置的采样率都是 2.8Msps。如果设置的定时器触发速度大于这个，ADC 的采样速度是跟不上的。
- ◆ ADC2 的配置没有使用外部定时器触发和 DMA，而且这里关闭了连续采样
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE。
如果用户使能为连续转换，调用函数 ADC_SoftwareStartConv(ADC2)做一次软件触发即可。而我们将它关闭掉，触发一次读取一次，更方便些，适合做一些慢速的电压测量的工作。

说完了上面的问题，还有一个定时器触发的问題，看下面的驱动代码：

```

/*
*****
* 函数名: TIM1_Config
* 功能说明: 配置定时器 1，用于触发 ADC1 和 ADC3。
*          当外部触发信号被选为 ADC 规则或注入转换时，只有它的上升沿可以启动转换。
* 形参: 无
* 返回值: 无
*****
*/
static void TIM1_Config(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    TIM_OCInitTypeDef TIM_OCInitStructure;
}

```

```

/* 使能定时器 1 */
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);

/* 先禁能再配置 */
TIM_Cmd(TIM1, DISABLE);

/*
*****
system_stm32f4xx.c 文件中 void SetSysClock(void) 函数对时钟的配置如下:

HCLK = SYSCLK / 1      (AHB1Periph)
PCLK2 = HCLK / 2       (APB2Periph)
PCLK1 = HCLK / 4       (APB1Periph)

因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = PCLK1 x 2 = SystemCoreClock / 2;
因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = PCLK2 x 2 = SystemCoreClock;

APB1 定时器有 TIM2, TIM3, TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14
APB2 定时器有 TIM1, TIM8, TIM9, TIM10, TIM11

TIM1 更新周期是 = TIM1CLK / (TIM_Period + 1) / (TIM_Prescaler + 1)
*****
*/
//初始化定时器 1 的寄存器为复位值
TIM_TimeBaseStructInit(&TIM_TimeBaseStructure);
//ARR 自动重装载寄存器周期的值(定时时间) 到设置频率后产生个更新或者中断(就是说定时时间到)
TIM_TimeBaseStructure.TIM_Period = 168000000/g_SampleFreqTable[TimeBaseId][0] - 1;
//PSC 时钟预分频数 例如: 时钟频率=TIM1CLK/(时钟预分频+1)
TIM_TimeBaseStructure.TIM_Prescaler = g_SampleFreqTable[TimeBaseId][1]-1;
TIM_TimeBaseStructure.TIM_ClockDivision = 0x0;           //CR1->CKD 时间分割值
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //CR1->CMS[1:0]和 DIR 定时器模式 向上计数
TIM_TimeBaseStructure.TIM_RepetitionCounter = 0x0000;    /* TIM1 和 TIM8 必须设置 */
TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);

/*****ADC1 的触发*****/
//CCMR2 在向上计数时, 一旦 TIMx_CNT<TIMx_CCR1 时通道 1 为有效电平, 否则为无效电平
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //CCER 输出使能
//CCR3 同计数器 TIMx_CNT 的比较, 并在 OC1 端口上产生输出信号
TIM_OCInitStructure.TIM_Pulse = TIM_TimeBaseStructure.TIM_Period/2;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;      //CCER 输出极性设置
/* only for TIM1 and TIM8. 此处和正相引脚不同 */
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High; /* only for TIM1 and TIM8. */
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset; /* only for TIM1 and TIM8. */
TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset; /* only for TIM1 and TIM8. */
TIM_OC1Init(TIM1, &TIM_OCInitStructure);

/*****ADC3 的触发*****/
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_Pulse = TIM_TimeBaseStructure.TIM_Period-1;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;
/* only for TIM1 and TIM8. 此处和正相引脚不同 */
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High; /* only for TIM1 and TIM8. */
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset; /* only for TIM1 and TIM8. */

```

```
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset; /* only for TIM1 and TIM8. */
TIM_OC3Init(TIM1, &TIM_OCInitStructure);
//TIM_OC3PreloadConfig(TIM1, TIM_OCPreload_Enable); //CMR2 设置预装载使能 更新事件产生时写入有效
//TIM_ARRPreloadConfig(TIM1, ENABLE); //CR1 设置 ARR 自动重装 更新事件产生时写入有效
TIM_Cmd(TIM1, ENABLE);

/* 使能 PWM 输出 */
TIM_CtrlPWMOutputs(TIM1, ENABLE);
}
```

关于这个驱动设计，注意以下三个问题：

- ◆ 定时器 TIM1 是 16 位定时器，下面两个参数的配置切勿超过 65535。

TIM_TimeBaseStructure.TIM_Period

TIM_TimeBaseStructure.TIM_Prescaler

- ◆ 关于定时器触发的配置，要特别注意下面参数的设置：

TIM_OCInitStructure.TIM_Pulse

如果大家将 TIM1 的 OC1 和 OC3 设置成一样的，那么定时器的触发频率大于 2MHz 时，很容易导致 ADC 触发失败。而且这种触发失败的现象还比较神奇，程序初始的触发频率大于 2MHz，上电后导致 ADC 触发失败，但是切换几次不同频率，再切换回来，又正常工作了。为了安全起见这里将其区分开了。

- ◆ 另外使用 TIM1 或者 TIM8 做 ADC 采样触发，要注意结构体成员的配置，这两个定时器比其他定时器多一些参数要配置，大家配置的时候切勿忘记初始化，因为我们这里用的是局部变量，未初始化的话，它的值是随机的。

TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;

TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High;

TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset;

TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset;

6.3 拓展阅读

ADC 的外部输入阻抗只有 50KΩ，大家测量时要特别注意。另外，这里重点推荐两篇提高 ADC 精度的文档，是 ST 官方做的，对于阻抗和精度问题做了详细说明，下载地址：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=82543>。

6.4 总结

关于 ADC 的驱动就为大家讲解这么多，一定要实际操作配置下。如果需要 ADC 其它的配置方式，在我们论坛置顶帖 STM32-V5 的裸机例子基础篇里面下载即可：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=1285>。

- ☐  V5-012a_ADC实验_ADC温度测量.rar
- ☐  V5-012b_ADC实验_ADC3的DMA工作方式.rar
- ☐  V5-012c_ADC实验_双ADC快速交叉采样.rar
- ☐  V5-012d_ADC实验_3ADC快速交叉采样.rar
- ☐  V5-012e_ADC实验_ADC定时器触发.rar
- ☐  V5-012f_ADC实验_板载电子电压监测.rar
- ☐  V5-012g_ADC实验_单ADC多通道扫描模式.rar
- ☐  V5-012h_ADC实验_模拟看门狗.rar
- ☐  V5-012i_ADC实验_双ADC多通道规则采样.rar
- ☐  V5-012j_ADC实验_单通道注入模式.rar

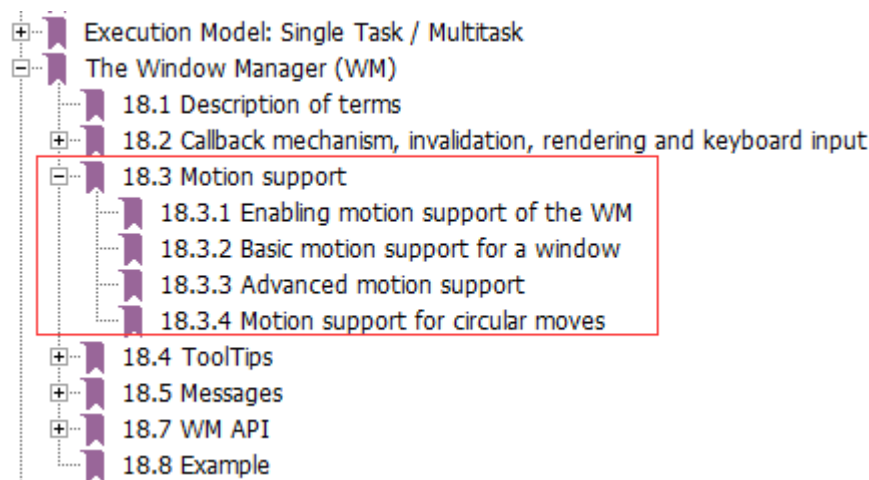
第7章 示波器设计—波形滑动浏览

类似手机的滑动功能用在示波器上还是比较实用的，可以用来调节波形位置和滑动浏览波形的存储记录。

- 7.1 滑动基础知识
- 7.2 滑动基础知识总结
- 7.3 滑动功能在窗口上的实现
- 7.4 总结

7.1 滑动基础知识

滑动的实现是基于 emWin 提供的 Motion 功能。在 emWin 的官方手册中对这部分功能讲解的比较少，但是仍然需要大家去读一下，有一个大概的了解。



7.2 滑动基础知识总结

望大家务必看了 emWin 官方手册中 Motion 章节的内容，这里将这些知识点做个汇总，方便大家后面使用。

WM_MOTION_Enable()	使能 motion
WM_MOTION_SetMoveable()	使能窗口 X 方向或者 Y 方向的 Motion
WM_MOTION_SetMovement()	设置窗口以指定的速度移动一段距离。
=====	
WM_MOTION_SetDeceleration()	设置窗口减速度

WM_MOTION_SetSpeed() 设置窗口的移动速度。
WM_MOTION_SetMotion() 设置初始速度和加速度。等同于上面两个函数之和

=====

WM_MOTION_SetDefaultPeriod()
设置指针输入设备（触摸，鼠标，键盘等）等释放后，减速度默认持续的时间。

- 1、如果窗口在移动中，将减速，直到停止运行。
- 2、如果窗口没有移动，但是使能了 snap 功能，那么窗口会在设置的时间内移动到下一个栅格内。
- 3、如果窗口在移动，且使能了 snap 功能，那么窗口会减速运动直到停止在最近的一个栅格上。

上面的这些函数都是独立的操作 Motion 功能，而二代示波器中使用 Motion 功能是基于窗口的，其实就是把上面这些函数的功能在窗口上实现。下面我们就讲解二代示波器的 Motion 功能实现。

7.3 滑动功能在窗口上的实现

我们这里以二代示波器的滑动功能实现过程为例，给大家做个说明。

◆ 第 1 步：使能滑动，设置周期。

```
WM_MOTION_Enable(1);            /* 使能滑动 */
WM_MOTION_SetDefaultPeriod(100); /* 设置滑动周期 */
```

◆ 第 2 步：创建一个透明的窗口，正好覆盖 600*400 的波形显示区。

```
hMotion = WM_CreateWindowAsChild(40,
                                   40,
                                   600,
                                   400,
                                   WM_HBKWIN,
                                   WM_CF_MOTION_Y | WM_CF_SHOW | WM_CF_HASTRANS,
                                   _cbMotion,
                                   sizeof(pPara));

WM_SetUserData(hMotion, &pPara, sizeof(pPara));
```

函数里面的标志 WM_CF_MOTION_Y | WM_CF_SHOW | WM_CF_HASTRANS 比较重要，滑动标志既可以在这里设置，也可以在窗口回调函数的 WM_MOTION 消息里面设置。

WM_CF_MOTION_Y：表示窗口支持垂直滑动。

WM_CF_MOTION_X：表示窗口支持水平滑动（在窗口回调里面进行了设置）。

WM_CF_HASTRANS：表示窗口透明，这个标志不要忘记设置，这样使用滑动功能的时候对示波器显示区没有任何影响。

◆ 第 3 步：也是最后一步，滑动窗口的回调函数。

源代码是如下这样的：

```
/*
*****
* 函数名: _cbMotion
* 功能说明: Motion 窗口的回调函数，主要是桌面图标的滑动处理
* 形参: pMsg WM_MESSAGE 类型指针变量
*****
*/
```



```
* 返回值: 无
*****
*/
static void _cbMotion(WM_MESSAGE * pMsg)
{
    WM_MOTION_INFO * pInfo;
    WM_HWIN          hWin = pMsg->hWin;
    PARA             * pPara;

    switch (pMsg->MsgId)
    {
        case WM_MOTION:
            WM_GetUserData(hWin, &pPara, sizeof(pPara));
            pInfo = (WM_MOTION_INFO *)pMsg->Data.p;
            switch (pInfo->Cmd)
            {
                /* Motion 功能初始化, 设置 X 方向和 Y 方向都支持滑动 */
                case WM_MOTION_INIT:
                    pInfo->Flags = WM_CF_MOTION_X | WM_CF_MOTION_Y | WM_MOTION_MANAGE_BY_WINDOW;
                    pInfo->SnapY = 1;
                    pInfo->SnapX = 1;
                    break;

                case WM_MOTION_MOVE:
                    pPara->FinalMove = pInfo->FinalMove;
                    /* Y 轴方向, 上下滑动 */
                    if(g_Flag->ucMotionXY == 0)
                    {
                        if((pInfo->pState->Pressed == 1)&&(pInfo->pState->x <= 340))
                        {
                            pPara->iCH1Pos += pInfo->dy;
                            if(pPara->iCH1Pos >= 440)
                            {
                                pPara->iCH1Pos = 440;
                            }
                            if(pPara->iCH1Pos <= 40)
                            {
                                pPara->iCH1Pos = 40;
                            }

                            g_DS01->usRefPos = pPara->iCH1Pos;
                        }
                        else if((pInfo->pState->Pressed == 1)&&(pInfo->pState->x > 340))
                        {
                            pPara->iCH2Pos += pInfo->dy;
                            if(pPara->iCH2Pos >= 440)
                            {
                                pPara->iCH2Pos = 440;
                            }
                            if(pPara->iCH2Pos <= 40)
                            {
                                pPara->iCH2Pos = 40;
                            }

                            g_DS02->usRefPos = pPara->iCH2Pos;
                        }
                    }

                    WM_InvalidateArea(&rRefPos);
            }
    }
}
```

```
    }  
    /* X 轴方向，左右滑动 */  
    else  
    {  
        if(pInfo->pState->Pressed == 1)  
        {  
            g_DS01->sCurTriStep -= pInfo->dx;  
        }  
  
        if(TriggerFlag == 0)  
        {  
        }  
    }  
    else  
    {  
        if(g_DS01->sCurTriStep <= -724)  
        {  
            g_DS01->sCurTriStep = -724;  
        }  
  
        if(g_DS01->sCurTriStep >= 724)  
        {  
            g_DS01->sCurTriStep = 724;  
        }  
  
        TriggerFlag = 2;  
    }  
  
    WM_InvalidateArea(&rTrigPos);  
}  
  
/* 更新当前的数据位置的箭头 */  
g_Flag->ucWaveRefresh = 1;  
break;  
  
case WM_MOTION_GETPOS:  
    pInfo->yPos = pPara->iCH1Pos;  
    //pInfo->xPos = pPara->iCH1Pos;  
    break;  
}  
break;  
}  
}
```

为了方便看这个函数实现的功能，这里将其进一步简化成如下这样：

```
static void _cbMotion(WM_MESSAGE * pMsg)  
{  
    WM_MOTION_INFO * pInfo;  
    WM_HWIN          hWin = pMsg->hWin;  
    PARA             * pPara;  
  
    switch (pMsg->MsgId)  
    {  
        case WM_MOTION:  
            WM_GetUserData(hWin, &pPara, sizeof(pPara));  
            pInfo = (WM_MOTION_INFO *)pMsg->Data.p;  
            switch (pInfo->Cmd)  
            {
```

```
/* Motion 功能初始化, 设置 X 方向和 Y 方向都支持滑动 */
case WM_MOTION_INIT: //----- (1)
    pInfo->Flags = WM_CF_MOTION_X | WM_CF_MOTION_Y | WM_MOTION_MANAGE_BY_WINDOW;
    pInfo->SnapY = 1;
    pInfo->SnapX = 1;
    break;

case WM_MOTION_MOVE: //----- (2)
    pPara->FinalMove = pInfo->FinalMove;
    /* Y 轴方向, 上下滑动 */
    if(g_Flag->ucMotionXY == 0)
    {
        if(pInfo->pState->Pressed == 1)
        {

        }
    }
    /* X 轴方向, 左右滑动 */
    else
    {
        if(pInfo->pState->Pressed == 1)
        {

        }
    }

    /* 更新当前的数据位置的箭头 */
    g_Flag->ucWaveRefresh = 1;
    break;

case WM_MOTION_GETPOS: //----- (3)
    pInfo->yPos = pPara->iCH1Pos;
    //pInfo->xPos = pPara->iCH1Pos;
    break;

}
break;
}
```

滑动功能实现的关键就在这个 WM_MOTION 消息里面。

1. WM_MOTION_INIT 消息

在初始化消息里面设置 pInfo->Flags 来使能水平滑动和垂直滑动, 并且通过标志 WM_MOTION_MANAGE_BY_WINDOW 使能窗口管理器来管理 Motion 功能。标志 pInfo->SnapY 和 pInfo->SnapX 是用来设置滑动停止后的栅格区域, 详情看 7.2 小节中函数 WM_MOTION_SetDefaultPeriod 的作用说明。

2. WM_MOTION_MOVE 消息

这个消息在二代示波器中的使用比较简单, 主要是判断是否按下, 然后根据按下的位置设置波形的水平或者上下移动即可。当前按下的位置在参数 pInfo->pState->x 和 pInfo->pState->y 里面。实际的波形更新是通过设置标志 g_Flag->ucWaveRefresh = 1 实现的。

3. WM_MOTION_GETPOS 消息

这个消息未用到, 用于获取按下的位置。



7.4 总结

关于滑动的实现，关键的知识点就这么多，需要大家实际操作下，有个深入的认识。

第8章 示波器设计—测量功能

二代示波器测量功能实现比较简单，使用 2D 函数绘制即可。不过也专门开辟一个章节，为大家做一个简单的说明，方便理解。

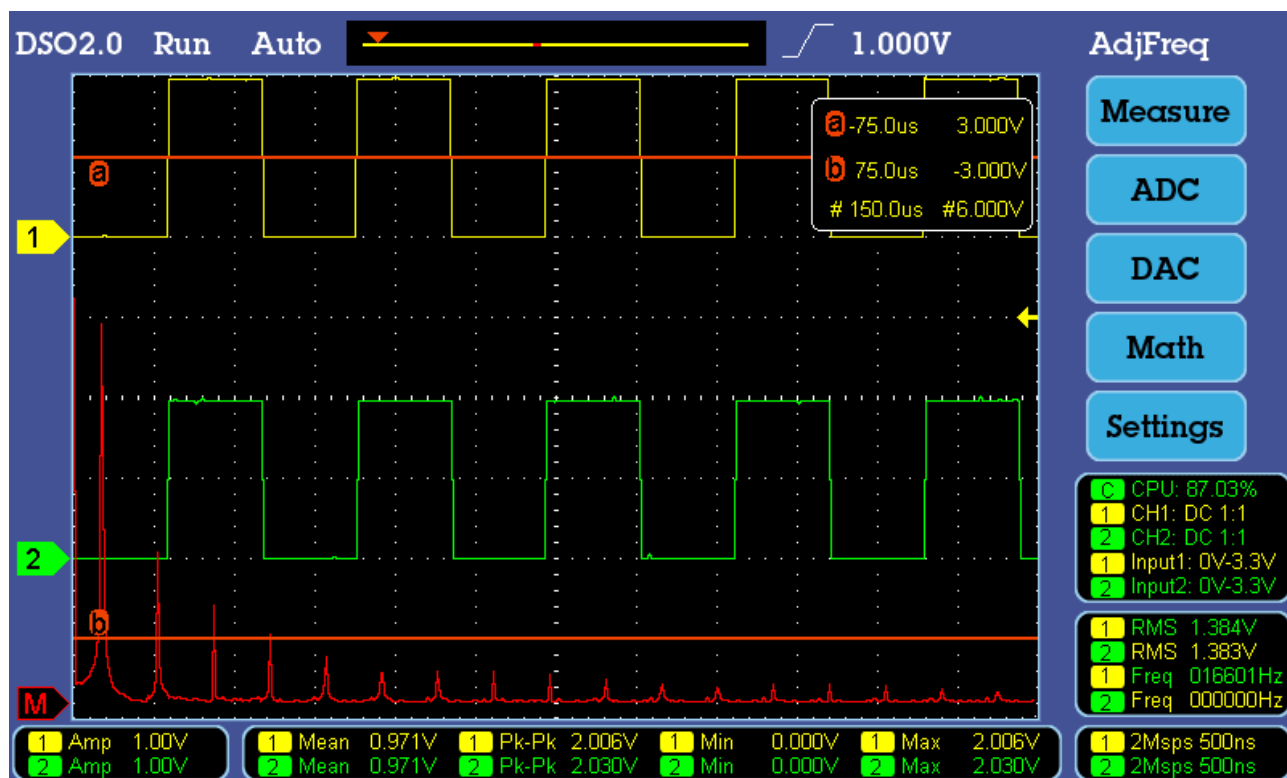
8.1 水平测量功能

8.2 垂直测量功能

8.3 总结

8.1 水平测量功能

水平测量方式的效果如下：



水平测量功能的数据显示不要以窗口的形式呈现，因为将窗口显示在波形显示区上会造成波形刷新慢。当前的方案是在绘制完毕波形后，直接 2D 函数绘制测量窗口，这种方式的实际效果好很多。对应的代码如下：

```
/*  
*****  
* 函数名: DSO_DrawCursorH  
* 功能说明: 绘制示波器的水平测量游标，用于测量幅值。  
* 形参: 无  
*****  
*/
```

```
* 返回值: 无
*****
*/
void DSO_DrawCursorH(void)
{
    char buf[60];

    /* 第 1 步: 绘制用于测量的两个游标线***** */
    GUI_SetColor(0x0040f0);

    /* 绘制横线 A */
    GUI_DrawHLine(g_Cursors->sCursorHA, DSOSCREEN_STARTX, DSOSCREEN_ENDX);
    GUI_DrawHLine(g_Cursors->sCursorHA+1, DSOSCREEN_STARTX, DSOSCREEN_ENDX);
    GUI_FillRoundedRect(50, g_Cursors->sCursorHA+3, 61, g_Cursors->sCursorHA+18, 3);

    /* 绘制横线 B */
    GUI_DrawHLine(g_Cursors->sCursorHB, DSOSCREEN_STARTX, DSOSCREEN_ENDX);
    GUI_DrawHLine(g_Cursors->sCursorHB-1, DSOSCREEN_STARTX, DSOSCREEN_ENDX);
    GUI_FillRoundedRect(50, g_Cursors->sCursorHB-18, 61, g_Cursors->sCursorHB-3, 3);

    /* 在横线 A 和横线 B 旁边的小圆圈中显示字母 a 和字母 b */
    GUI_SetColor(GUI_BLACK);
    GUI_SetFont(&GUI_Font20_ASCII);
    GUI_SetTextMode(GUI_TEXTMODE_TRANS);
    GUI_DispatchAt('a', 51, g_Cursors->sCursorHA);
    GUI_DispatchAt('b', 51, g_Cursors->sCursorHB-19);

    /* 第 2 步: 绘制游标数据显示窗口***** */
    GUI_SetColor(GUI_BLACK);
    GUI_FillRoundedRect(500, 55, 635, 135, 4);
    GUI_SetColor(GUI_WHITE);
    GUI_DrawRoundedRect(499, 54, 636, 136, 4);

    GUI_SetColor(0x0040f0);
    GUI_FillRoundedRect(75 + 433, 47 + 15, 86 + 433, 62 + 15, 3);
    GUI_FillRoundedRect(75 + 433, 45 + 45, 86 + 433, 60 + 45, 3);

    GUI_SetColor(GUI_BLACK);
    GUI_SetFont(&GUI_Font20_1);
    GUI_SetTextMode(GUI_TEXTMODE_TRANS);
    GUI_DispatchAt('a', 509, 59);
    GUI_DispatchAt('b', 509, 88);

    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font16_1);
    GUI_DispatchAt('#', 511, 115);

    /* 第 3 步: 计算竖线 A 的数值***** */
    g_Cursors->WaveCursorA = (float)((g_Cursors->sCursorVA - 340) * g_CursorUnitTable[TimeBaseId][0])/1000;
    if(g_CursorUnitTable[TimeBaseId][1] == 1)
    {
        sprintf(buf, "%5.1fus", g_Cursors->WaveCursorA);
    }
    else if(g_CursorUnitTable[TimeBaseId][1] == 2)
    {
        sprintf(buf, "%5.3fms", g_Cursors->WaveCursorA);
    }
    else if(g_CursorUnitTable[TimeBaseId][1] == 3)
```



```
{
    sprintf(buf, "%5.2fms", g_Cursors->WaveCursorA);
}
else if(g_CursorUintTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", g_Cursors->WaveCursorA);
}
else if(g_CursorUintTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", g_Cursors->WaveCursorA);
}
else if(g_CursorUintTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", g_Cursors->WaveCursorA);
}
else if(g_CursorUintTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", g_Cursors->WaveCursorA);
}

GUI_DispStringAt(buf, 523, 63);

/* 第3步：计算竖线B的数值*****/
g_Cursors->WaveCursorB = (float)((g_Cursors->sCursorVB - 340) * g_CursorUintTable[TimeBaseId][0])/1000;
if(g_CursorUintTable[TimeBaseId][1] == 1)
{
    sprintf(buf, "%5.1fus", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 2)
{
    sprintf(buf, "%5.3fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 3)
{
    sprintf(buf, "%5.2fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", g_Cursors->WaveCursorB);
}

GUI_DispStringAt(buf, 523, 91);

/* 第5步：计算竖线A竖线B的差值*****/
if(g_CursorUintTable[TimeBaseId][1] == 1)
{

```

```
    sprintf(buf, "%5.1fus", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 2)
{
    sprintf(buf, "%5.3fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 3)
{
    sprintf(buf, "%5.2fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
GUI_DispStringAt(buf, 523, 115);

/* 第 6 步：显示水平线的计算数值***** */
GUI_DispCharAt(' ', 509+72, 115);

g_Cursors->WaveCursorA = (float)((240 - g_Cursors->sCursorHA) * g_AttTable[ChlAmpId][1]) / 50000;
sprintf(buf, "%5.3fV", g_Cursors->WaveCursorA);
GUI_DispStringAt(buf, 518+72, 63);

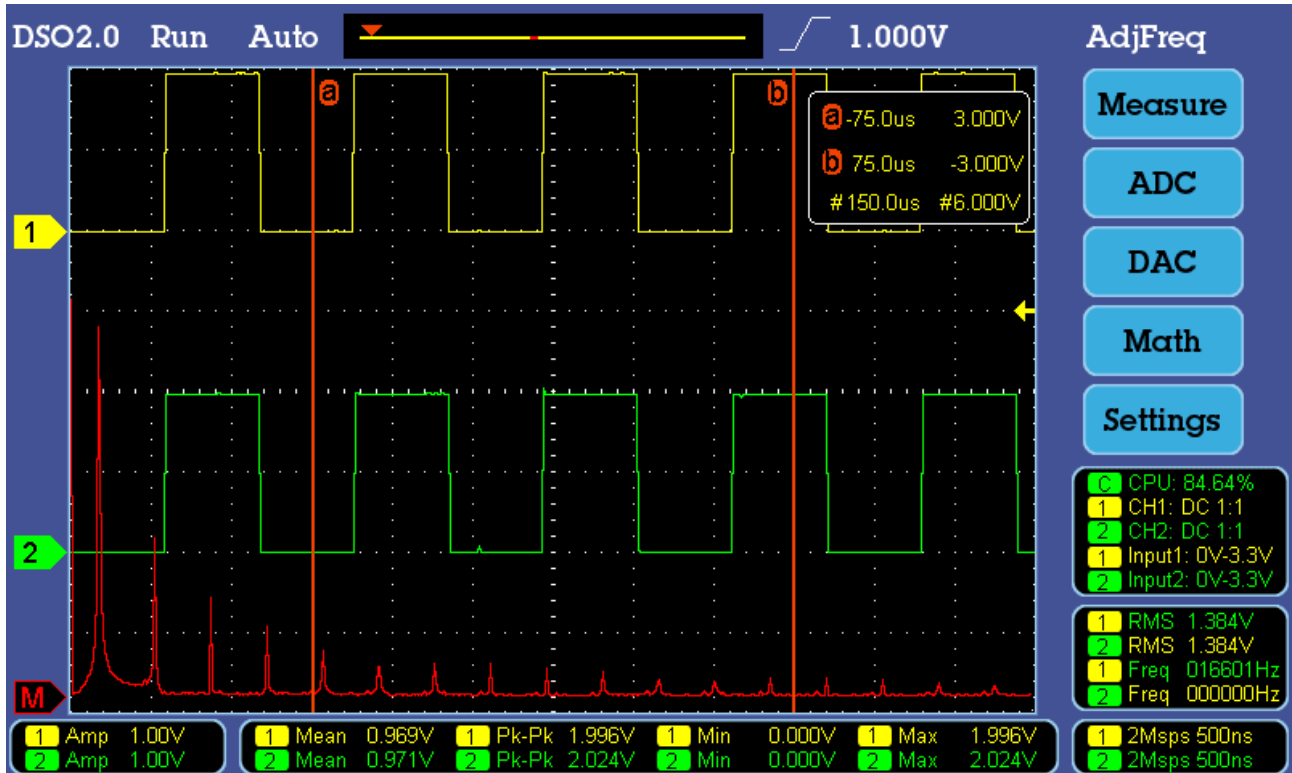
g_Cursors->WaveCursorB = (float)((240 - g_Cursors->sCursorHB) * g_AttTable[ChlAmpId][1]) / 50000;
sprintf(buf, "%5.3fV", g_Cursors->WaveCursorB);
GUI_DispStringAt(buf, 518+70, 91);

sprintf(buf, "%5.3fV", g_Cursors->WaveCursorA - g_Cursors->WaveCursorB);
GUI_DispStringAt(buf, 520+70, 115);
}
```

水平测量的实现比较简单，就是 2D 函数调用和字符显示。如果大家要自己实现，按照代码中注释的步骤制作即可。测量功能的调节是通过开发板上面的摇杆实现的。

8.2 垂直测量功能

垂直测量方式的效果如下：



垂直测量功能的数据显示不要以窗口的形式呈现，因为将窗口显示在波形显示区上会造成波形刷新慢。当前的方案是在绘制完毕波形后，直接 2D 函数绘制测量窗口，这种方式的实际效果好很多。对应的代码如下：

```

/*
*****
*   函数名: DSO_DrawCursorV
*   功能说明: 绘制示波器的垂直测量游标，用于测量的时间。
*   形参: 无
*   返回值: 无
*****
*/
void DSO_DrawCursorV(void)
{
    char buf[60];

    /* 第 1 步: 绘制用于测量的两个游标线***** */
    GUI_SetColor(0x0040f0);

    /* 绘制竖线 A */
    GUI_DrawVLine(g_Cursors->sCursorVA, DSOSCREEN_STARTY, DSOSCREEN_ENDY);
    GUI_DrawVLine(g_Cursors->sCursorVA+1, DSOSCREEN_STARTY, DSOSCREEN_ENDY);
    GUI_FillRoundedRect(g_Cursors->sCursorVA+5, 47, g_Cursors->sCursorVA+16, 62, 3);

    /* 绘制竖线 B */
    GUI_DrawVLine(g_Cursors->sCursorVB, DSOSCREEN_STARTY, DSOSCREEN_ENDY);
    GUI_DrawVLine(g_Cursors->sCursorVB-1, DSOSCREEN_STARTY, DSOSCREEN_ENDY);
    GUI_FillRoundedRect(g_Cursors->sCursorVB-16, 47, g_Cursors->sCursorVB-5, 62, 3);

    /* 在竖线 A 和竖线 B 旁边的小圆圈中显示字母 a 和字母 b */
    GUI_SetColor(GUI_BLACK);
    GUI_SetFont(&GUI_Font20_ASCII);

```

```
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispatchAt('a', g_Cursors->CursorVA+6, 44);
GUI_DispatchAt('b', g_Cursors->CursorVB-15, 45);

/* 第 2 步：绘制游标数据显示窗口*****/
GUI_SetColor(GUI_BLACK);
GUI_FillRoundedRect(500, 55, 635, 135, 4);
GUI_SetColor(GUI_WHITE);
GUI_DrawRoundedRect( 499, 54, 636, 136, 4);

GUI_SetColor(0x0040f0);
GUI_FillRoundedRect(75 + 433 , 47 + 15, 86 + 433, 62 + 15, 3);
GUI_FillRoundedRect(75 + 433 , 45 + 45, 86 + 433, 60 + 45, 3);

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font20_1);
GUI_SetTextMode(GUI_TEXTMODE_TRANS);
GUI_DispatchAt('a', 509, 59);
GUI_DispatchAt('b', 509, 88);

GUI_SetColor(GUI_YELLOW);
GUI_SetFont(&GUI_Font16_1);
GUI_DispatchAt('#', 513, 115);

/* 第 3 步：计算竖线 A 的数值*****/
g_Cursors->WaveCursorA = (float)((g_Cursors->CursorVA - 340) * g_CursorUnitTable[TimeBaseId][0])/1000;
if(g_CursorUnitTable[TimeBaseId][1] == 1)
{
    sprintf(buf, "%5.1fus", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 2)
{
    sprintf(buf, "%5.3fms", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 3)
{
    sprintf(buf, "%5.2fms", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", g_Cursors->WaveCursorA);
}
else if(g_CursorUnitTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", g_Cursors->WaveCursorA);
}

/* 显示竖线 A 数值 */
GUI_DispatchStringAt(buf, 523, 63);
```

```
/* 第4步：计算竖线B的数值***** */
g_Cursors->WaveCursorB = (float)((g_Cursors->sCursorVB - 340) * g_CursorUintTable[TimeBaseId][0])/1000;
if(g_CursorUintTable[TimeBaseId][1] == 1)
{
    sprintf(buf, "%5.1fus", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 2)
{
    sprintf(buf, "%5.3fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 3)
{
    sprintf(buf, "%5.2fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", g_Cursors->WaveCursorB);
}
else if(g_CursorUintTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", g_Cursors->WaveCursorB);
}

/* 显示竖线B数值 */
GUI_DispStringAt(buf, 523, 91);

/* 第5步：计算竖线A竖线B的差值***** */
if(g_CursorUintTable[TimeBaseId][1] == 1)
{
    sprintf(buf, "%5.1fus", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 2)
{
    sprintf(buf, "%5.3fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 3)
{
    sprintf(buf, "%5.2fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 4)
{
    sprintf(buf, "%5.1fms", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 5)
{
    sprintf(buf, "%5.3fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
else if(g_CursorUintTable[TimeBaseId][1] == 6)
{
    sprintf(buf, "%5.2fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}
```

```
}
else if(g_CursorUintTable[TimeBaseId][1] == 7)
{
    sprintf(buf, "%5.1fs", (g_Cursors->WaveCursorB - g_Cursors->WaveCursorA));
}

/* 显示竖线 A 和竖线 B 的差值 */
GUI_DispStringAt(buf, 523, 115);

/* 第 6 步：显示水平线的计算数值*****/
GUI_DispCharAt('#', 509+72, 115);

g_Cursors->WaveCursorA = (float)((240 - g_Cursors->sCursorHA) * g_AttTable[Ch1AmpId][1]) / 50000;
sprintf(buf, "%5.3fV", g_Cursors->WaveCursorA);
GUI_DispStringAt(buf, 518+72, 63);

g_Cursors->WaveCursorB = (float)((240 - g_Cursors->sCursorHB) * g_AttTable[Ch1AmpId][1]) / 50000;
sprintf(buf, "%5.3fV", g_Cursors->WaveCursorB);
GUI_DispStringAt(buf, 518+70, 91);

sprintf(buf, "%5.3fV", g_Cursors->WaveCursorA - g_Cursors->WaveCursorB);
GUI_DispStringAt(buf, 520+70, 115);
}
```

垂直测量的实现比较简单，就是 2D 函数调用和字符显示。如果大家要自己实现，按照代码中注释的步骤制作即可。测量功能的调节是通过开发板上面的摇杆实现的。

8.3 总结

由于测量功能的实现比较简单，所以我们这里只是简单的进行了说明。如果大家要自己制作的话，建议实际操作下。

第9章 示波器设计—自动触发和普通触发

自带触发和普通触发是示波器设计中比较重要的两个功能，本章节为大家讲解二代示波器中自动触发和普通触发的实现。

9.1 自动触发

9.2 普通触发

9.3 总结

9.1 自动触发

由于示波器模拟前端模块稍有些问题，所以自动触发功能是用软件实现的。软件实现自动触发比较容易实现，具体的实现代码如下：

```
/* 通过软件检测实现上升沿触发，并保留最后 600 的数据不做检测，用于直接显示 2048-600 = 1448; */  
j = 0;  
for(i = 0; i < 1448; i++)  
{  
    j++;  
    if((g_DS01->usWaveBufTemp[i] > g_TrigVol->usTrigValue) &&  
        (g_DS01->usWaveBufTemp[i+1] < g_TrigVol->usTrigValue))  
    {  
        break;  
    }  
}
```

`g_DS01->usWaveBufTemp` 是 2048 个 ADC 数据的缓冲区，`g_TrigVol->usTrigValue` 是上升沿触发值。for 循环的作用就是从 1448 个数据中找到满足触发值的位置，判断方法也比较简单，大于前一个值小于后一个值即可。

保留 600 个数据是因为这个大小是波形显示区一次可以显示的波形个数。如果从前 $2048-600 = 1448$ 个数据中检索不到满足要求的数据将不再检索，直接显示末尾的 600 个数据，如果检测到将直接从 for 循环里面退出。

这个方法在实际测试中比较好用，另外上升沿阈值的判断还不够严谨，大家有兴趣可以继续完善下。

9.2 普通触发

普通触发的实现是基于 ADC 的模拟看门狗功能，通过设置不同看门狗阈值实现不同的触发电压。由于使能了看门狗中断，检测到外部触发电压后会进入 ADC 模拟看门狗中断。在中断里面判断是否是上升沿触发，如果是的话，会关闭模拟看门狗中断并开启一个定时器计数功能，目的是为了采集这个触发电压前后各 1024 个 ADC 数据，基本的实现思路就是这个样子的。

下面把实际的实现为大家做个说明：

◆ 第 1 步：将 ADC3 配置使能模拟看门狗功能。

```
/*ADC3 的配置******/
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC3;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfConversion = 1;

/* ADC3 规则通道配置 */
ADC_Init(ADC3, &ADC_InitStructure);
ADC-RegularChannelConfig(ADC3, ADC_Channel_10, 1, ADC_SampleTime_3Cycles);

/* 使能 ADC3 DMA */
ADC_DMACmd(ADC3, ENABLE);

/* 配置模拟看门狗的阈值 注意别配置反了，要不一直进入中断 */
ADC_AnalogWatchdogThresholdsConfig(ADC3, 4095, 0);

/* 配置模拟看门狗监测 ADC3 的通道 10 */
ADC_AnalogWatchdogSingleChannelConfig(ADC3, ADC_Channel_10);

/* 使能一个规则通道的看门狗 */
ADC_AnalogWatchdogCmd(ADC3, ADC_AnalogWatchdog_SingleRegEnable);

/* 使能模拟看门狗中断 */
ADC_ITConfig(ADC3, ADC_IT_AWD, ENABLE);

/* 使能 DMA 请求 -----*/
ADC_DMAResquestAfterLastTransferCmd(ADC3, ENABLE);

/* Enable ADC3 -----*/
ADC_Cmd(ADC3, ENABLE);
```

特别注意函数 ADC_AnalogWatchdogThresholdsConfig 的设置，因为是 12 位分辨率的 ADC，最大值就是 $2^{12} - 1 = 4095$ ，这里设置为 4095 表示超过 4095 才会触发模拟看门狗中断，由于已经是最大值了，所以不会触发模拟看门狗中断。

◆ 第 2 步：模拟看门狗中断。

达到设置的模拟看门狗触发值会进入到这个中断里面。

```
/*
*****
* 函数名: ADC_IRQHandler
* 功能说明: 模拟看门狗中断服务程序。
* 形参: 无
* 返回值: 无
*****
*/
void ADC_IRQHandler()
{
    /* 读取 DMA 剩余要传输的数目 */
    g_usTrigCount = DMA2_Stream1->NDTR;
```

```

/* 确认是否是 ADC3 的看门狗中断 */
if((ADC3->SR)&0x01)
{
    /* 取触发值的前一个点，查看是否是在阈值范围内，从而判断是上升沿还是下降沿 */
    if(g_usTrigCount == 10240)
    {
        /* 剩余 10240 表示触发值是 ADC3ConvertedValue[10239]是触发值，那么上一个点就是 10238 */
        g_usTrigTempFlag = ADC3ConvertedValue[10238];
    }
    else if(g_usTrigCount == 10239)
    {
        /* 剩余 10239 表示触发值是 ADC3ConvertedValue[0]是触发值，那么上一个点就是 10239 */
        g_usTrigTempFlag = ADC3ConvertedValue[10239];
    }
    else
    {
        /* 剩余 10239 表示触发值是 ADC3ConvertedValue[0]是触发值，那么上一个点就是 10239 */
        g_usTrigTempFlag = ADC3ConvertedValue[10238 - g_usTrigCount];
    }

    /* 判断是否是上升沿，是的话开启定时器记录 ADC 数据 */
    if(g_usTrigTempFlag <= g_TrigVol->usTrigValue)
    {
        /* 关闭 ADC3 的看门狗中断 */
        ADC3->CR1 &= 0xfffffbf;
        TriggerFlag = 1;

        /* 启动定时器计数 */
        TIM8->CR1 |= 0x01;
    }
    /* 清除挂起标志 */
    ADC3->SR &= 0xfe;
}
}

```

进入到这个中断后，主要做了一件事，判断是否是上升沿，如果是上升沿的话，将关闭模拟看门狗并开启定时器测量功能。也就是下面第 3 步要讲解的。

◆ 第 3 步：初始化一个定时器做时间测量，表示检测到触发值后记录一段时间的波形。

```

/*
*****
* 函数名: TIM8_MeasureTrigConfig
* 功能说明: 使用 TIM8 为普通触发模式下数据采集计时，定时采集触发值前后的 1024 个 ADC 数据
* 形参: 无
* 返回值: 无
*****
*/
/*
    每次捕获采集触发值前后的 1024 个 ADC 数据(单通道)。
*/
const uint32_t g_TrigFreqTable[][2] =
{
    {60, 1024}, //2.8Msps 168000000/2800000 = 60 => 60 * 1024
    {84, 1024}, //2Msps 168000000/2000000 = 84 => 84 * 1024
    {168, 1024}, //1Msps 168000000/1000000 = 168 => 168 * 1024
    {336, 1024}, //500Ksps 168000000/500000 = 336 => 336 * 1024
    {840, 1024}, //200Ksps 168000000/200000 = 840 => 840 * 1024
}

```

```
{1680, 1024}, //100Ksps 168000000/100000 = 1680 => 1680 * 1024
{3360, 1024}, //50Ksps 168000000/50000 = 3360 => 3360 * 1024
{8400, 1024}, //20Ksps 168000000/20000 = 8400 => 8400 * 1024
{16800, 1024}, //10Ksps 168000000/10000 = 16800 => 16800 * 1024
{33600, 1024}, //5Ksps 168000000/5000 = 33600 => 33600 * 1024

/* 下面 5 种采样频率下刷新较慢, 因为采集前后 1024 个 ADC 的时间较长 */
{42000, 2048}, //2Ksps 168000000/2000 = 84000 => 84000 * 1024
{42000, 4096}, //1Ksps 168000000/1000 = 168000 => 168000 * 1024
{42000, 8192}, //500sps 168000000/500 = 336000 => 336000 * 1024
{42000, 20480}, //200sps 168000000/200 = 840000 => 840000 * 1024
{42000, 40960}, //100sps 168000000/100 = 1680000 => 1680000 * 1024

/* 下面这几种采样率不做触发支持 */
{42000, 40960}, //50sps
{42000, 40960}, //20sps
{42000, 40960}, //10sps
{42000, 40960}, //5sps
{42000, 40960}, //2sps
{42000, 40960}, //1sps
};

void TIM8_MeasureTrigConfig(void)
{
    TIM_TimeBaseInitTypeDef TIM_BaseInitStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    /* 开启时钟 */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM8, ENABLE);

    /* 加上第一次进入中断的标志, 进入中断后将其置 1 */
    g_usFirstTimeIRQFlag = 0;

    /* 使能定时器 8 中断 */
    NVIC_InitStructure.NVIC_IRQChannel = TIM8_UP_TIM13_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    /* 定时器配置 */
    TimeBaseId = 1; /* 开机后按照 ADC 单通道 1Msps 进行配置 */
    TIM_DeInit(TIM8);
    TIM_BaseInitStructure.TIM_Period = g_TrigFreqTable[TimeBaseId][0] - 1;
    TIM_BaseInitStructure.TIM_Prescaler = g_TrigFreqTable[TimeBaseId][1] - 1;
    TIM_BaseInitStructure.TIM_ClockDivision = 0;
    TIM_BaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM8, &TIM_BaseInitStructure);
    TIM_ITConfig(TIM8, TIM_IT_Update, ENABLE);
    TIM_Cmd(TIM8, DISABLE);
}

/*
*****
* 函数名: Time8Recorder
* 功能说明: 使用 TIM8 为普通触发模式下数据采集计时, 定时采集触发值前后的 1024 个 ADC 数据
* 形参: 无
* 返回值: 无
*****
*/
```

```
*****
*/
static void Time8Recorder(void)
{
    TIM_TimeBaseInitTypeDef TIM_BaseInitStructure;

    /* 加上第一次进入中断的标志 */
    g_usFirstTimeIRQFlag = 0;

    TIM_DeInit(TIM8);
    TIM_BaseInitStructure.TIM_Period = g_TrigFreqTable[TimeBaseId][0] - 1;
    TIM_BaseInitStructure.TIM_Prescaler = g_TrigFreqTable[TimeBaseId][1] - 1;
    TIM_BaseInitStructure.TIM_ClockDivision = 0;
    TIM_BaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM8, &TIM_BaseInitStructure);
    TIM_ITConfig(TIM8, TIM_IT_Update, ENABLE);
    TIM_Cmd(TIM8, DISABLE);
}
```

首先讲解数组 g_TrigFreqTable 的作用，因为不同采样率下，采样 1024 个数据时间是不同的，所以我们要将定时器设置为不同的参数配置，以此实现这段时间内采集 1024 个数据（也许有读者会问，我们不是要采集 2048 个数据吗，为什么这里只采集了 1024 个。原因在于另外 1024 个数据直接从这个触发值所在位置的前面获取即可，也就是已经采集的，最近的 1024 个数据）。比如第一组是 60 和 1024，他的作用就是将定时器结构体成员配置为：

`TIM_BaseInitStructure.TIM_Period = 60 - 1`

`TIM_BaseInitStructure.TIM_Prescaler = 1024 - 1`

那么定时器的更新周期 = $TIMCLK / (TIM_Period + 1) / (TIM_Prescaler + 1) = TIMCLK/60/1024$ 。

到了这个时间后，就会进入到定时器中断，也就是在这个采样率下，已经完成了 1024 个数据的采集。其它采样率的参数配置同理。具体取值不限制，只要大家的配置满足采集了 1024 个数据即可。

这个理解了，其它的就是 TIM8 的配置，这个就相对容易些，我们这里就不做介绍了。另外特别注意变量标志 g_usFirstTimeIRQFlag，下一步里面会说明为什么要做这个。

◆ 第 4 步：也是最后一步，测量时间到后，进入到定时器中断。

进入到定时器中断后，要关闭定时器，并将触发值前后各 1024 个采样数据从缓冲区中提取出来。具体实现代码如下：

```
/*
*****
*   函数名: TIM8_UP_TIM13_IRQHandler
*   功能说明: 定时器中断函数
*   形参: 无
*   返回值: 无
*****
*/
void TIM8_UP_TIM13_IRQHandler(void)
{
    uint16_t i, k, usCurPos;

    /* 第一次进入 TIM8 中断执行退出操作，因为定时器更新中断有个小 bug，
       一旦初始化了定时器并且使能中断就会有立即进入一次中断的情况，为了防止这种情况的
       出现，作如下操作：
    */
}
```

```
*/
if(g_usFirstTimeIRQFlag == 0)
{
    g_usFirstTimeIRQFlag = 1;

    TIM_ClearITPendingBit(TIM8, TIM_IT_Update);

    /* 如果是第一次进入就退出 */
    return;
}

/* 定时器时间中断 */
if (TIM_GetITStatus(TIM8, TIM_IT_Update) != RESET)
{
    /* 先关闭定时器 */
    TIM_Cmd(TIM8, DISABLE);

    /* 确保开启的定时器计时采集 ADC 数据 */
    if(TriggerFlag == 1)
    {
        /* 定时器采集 ADC 数据结束 */
        TriggerFlag = 2;

        /* 开启定时器计数前 DMA 传输数据的位置 */
        usCurPos = 10240 - g_usTrigCount;

        /*
            下面分为三种情况获取 2K 的数据(三通道，单通道就是前后 1024 个 ADC 数据):
            1. 当前的位置 < 1024.
            2. 1024 <= 当前的位置 <= 10240 - 1024(9216)
            3. 当前位置 > 10240 - 1024(9216)
        */
        /* 第一种情况: 当前的位置 < 1024. */
        if(usCurPos < 1024)
        {
            k = 1024 - usCurPos;
            usCurPos = 10240 - k;

            /* 前部分数据 */
            for(i = 0; i < k; i++)
            {
                g_DS01->usWaveBufTemp[i] = ADC3ConvertedValue[i+usCurPos];
            }

            usCurPos = i;
            k = 10240 - g_usTrigCount + 1024;

            /* 后部分数据 */
            for(i = 0; i < k; i++)
            {
                g_DS01->usWaveBufTemp[i + usCurPos] = ADC3ConvertedValue[i];
            }
        }
        /* 第三种情况: 当前位置 > 10240 - 1024(9216) */
        else if(usCurPos > 9216)
        {
            usCurPos = usCurPos - 1024;
        }
    }
}
```

```

/* 采集前 1024+g_usTrigCount 的数据 */
for(i = 0; i < (g_usTrigCount + 1024); i++)
{
    g_DS01->usWaveBufTemp[i] = ADC3ConvertedValue[i+usCurPos];
}

k = i;
usCurPos = (1024 - g_usTrigCount);

/* 剩余数据的采集 */
for(i = 0; i < usCurPos; i++)
{
    g_DS01->usWaveBufTemp[i + k] = ADC3ConvertedValue[i];
}
}
/* 第二种情况：1024 <= 当前的位置 <= 10240 - 1024(9216) */
else
{
    usCurPos = usCurPos - 1024;
    for(i = 0; i < 2048; i++)
    {
        g_DS01->usWaveBufTemp[i] = ADC3ConvertedValue[i+usCurPos];
    }
}

TIM_ClearITPendingBit(TIM8, TIM_IT_Update);
}
}

```

变量标志 g_usFirstTimeIRQFlag 的作用就是代码前面的注释，第一次进入 TIM8 中断执行退出操作，因为定时器更新中断有个小 bug，一旦初始化了定时器并且使能中断就会有立即进入一次中断的情况，为了防止这种情况而做的这个变量。

定时器中断里面最重要的功能还是从 10240 大小的缓冲区里面提取 2048 个采样数据，代码里面分为三种情况进行实现：

1. 当前的位置 < 1024。
2. 1024 <= 当前的位置 <= 10240 - 1024(9216)。
3. 当前位置 > 10240 - 1024(9216)。

具体实现需要大家拿笔实际计算下，捋一遍基本就都明白了，我们这里就不做说明了。

9.3 总结

本章为大家讲解的普通触发更适合做波形故障记录，如果波形触发速度太快了，有点处理不过来，因为我们需要记录 2048 个数据，大家可以根据实际需要调整记录的大小。

第10章 示波器设计—数字信号处理

本章节为大家讲解二代示波器中用到的 FFT 和 FIR。单纯从应用上来说，比较省事，调用 API 函数即可，从学习的角度来说，需要大家花点精力。

10.1 FFT 快速傅里叶变换

10.2 FIR 有限冲击响应滤波器

10.3 总结

10.1 FFT 快速傅里叶变换

二代示波器中做了 2048 点的浮点 FFT。具体实现是采用 ARM 的 DSP 库函数 arm_rfft_fast_f32。

```
/*
*****
*   函 数 名: DS01_WaveProcess
*   功能说明: 波形通道 1 的 FFT, FIR, 最大值, 最小值, 平均值, 峰峰值, 频率和 RMS 的计算
*   形    参: 无。
*   返 回 值: 无
*****
*/
void DS01_WaveProcess(void)
{
    uint16_t i;
    uint32_t uiCycle, uiCount = 0;

    /* 省略其它 */

    /* 自动触发模式才计算 FFT */
    if(TriggerFlag == 0)
    {
        /* 前 10 种采样率计算 FFT */
        if(TimeBaseId < 10)
        {
            /* 执行 2048 点浮点 FFT 运算 */
            for(i=0; i<2048; i++)
            {
                testInput_fft_2048[i] = g_DS01->usWaveBufTemp[i];
            }

            /* 2048 点实序列快速 FFT, 浮点 */
            arm_rfft_fast_f32(&S, testInput_fft_2048, testOutput_fft_2048, ifftFlag);

            /* 求 1204 点的幅值 */
            arm_cmplx_mag_f32(testOutput_fft_2048, testInput_fft_2048, 1024);

            /* 获得最大的幅值和所在的位置 */
            uiCycle = testInput_fft_2048[1];
        }
    }
}
```



```
for(i = 2; i < 1024; i++)
{
    if(uiCycle < testInput_fft_2048[i])
    {
        uiCycle = testInput_fft_2048[i];
        uiCount = i;
    }
}

/* 估算频率 */
g_DS01->uiFreq = uiCount * g_SampleFreqTable[TimeBaseId][0] / 2048;
}
/* 省略其它 */
}
/* 省略其它 */
}
```

上面的代码中对采集的 2048 点数据做浮点 FFT 处理，并求出幅频响应，方便示波器上频谱的展示。同时代码中也做了一个简单的波形频率估计，这个估计仅适合周期性的正弦波，方波。

- ◆ 对于正弦波而言，它经过 FFT 后，理想的幅频响应就是一根线，所以根据这个特性，只要找出幅频响应中幅值最大的值即可，它所对应的频率位置就是波形的频率。
- ◆ 对于方波而言，它经过 FFT 后，理想的幅频响应就跟手机信号一样（中间有间隔），方波是由无限多个正弦波叠加而成的，其中第一个幅值最高的就是基波，也是方波的频率。

当前提供的方法仅适合估算，要提高精度的话需要对更多的点做 FFT，或者采样率正好是波形频率的整数倍，这样可以有效降低频谱泄露，准确率较高。当然，这种方法的操作难度有点大，因为我们不知道波形频率是多少，求得就是波形频率。

关于 FFT 的计算，大家可以直接看我们的 DSP 教程，有详细讲解，务必优先看第 25 章，对 FFT 变换结果的物理意义有个认识，然后再看其它章节：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=3886>。

10.2 FIR 有限冲击响应滤波器

二代示波器的 FIR 滤波控制界面如下：



80 Order Fir Lowpass



Fs=2Msps, Scale=500ns

- ☒ None
- ☐ Fc=100KHz
- ☐ Fc=200KHz
- ☐ Fc=300KHz
- ☐ Fc=400KHz
- ☐ Fc=500KHz
- ☐ Fc=600KHz
- ☐ Fc=700KHz

Fs=200Ksps, Scale=5us

- ☒ None
- ☐ Fc=10KHz
- ☐ Fc=20KHz
- ☐ Fc=30KHz
- ☐ Fc=40KHz
- ☐ Fc=50KHz
- ☐ Fc=60KHz
- ☐ Fc=70KHz

Fs=20Ksps, Scale=50us

- ☒ None
- ☐ Fc=1KHz
- ☐ Fc=2KHz
- ☐ Fc=3KHz
- ☐ Fc=4KHz
- ☐ Fc=5KHz
- ☐ Fc=6KHz
- ☐ Fc=7KHz

做了一个 80 阶低通 FIR 滤波器，分别可以在 2Msps 采样率，200Ksps 采样率和 20Ksps 采样率下工作。每个采样率下做了 7 种截止频率。

FIR 滤波在信号处理任务里面被调用：

```
/*
*****
* 函数名: DS01_WaveProcess
* 功能说明: 波形通道 1 的 FFT, FIR, 最大值, 最小值, 平均值, 峰峰值, 频率和 RMS 的计算
* 形参: 无。
* 返回值: 无
*****
*/
void DS01_WaveProcess(void)
{
    uint16_t i;
    uint32_t uiCycle, uiCount = 0;

    /* 省略其它 */

    /* 自动触发模式才计算 FFT */
    if(TriggerFlag == 0)
    {
        /* 省略其它 */

        /* 前 10 种采样率计算 FFT */
        if(TimeBaseId < 10)
        {
            /* 执行 650 点, 80 阶 Fir 滤波器 */
            /* 单通道 2Msps */
            if((TimeBaseId == 1)&&(g_DS01->ucFirFilter_Step100KHz !=0 ))
            {
```

```

        DSO_FirFilter_Step100KHz();
    }

    /* 单通道 200Ksps */
    if((TimeBaseId == 4)&&(g_DS01->ucFirFIter_Step10KHz !=0 ))
    {
        DSO_FirFilter_Step10KHz();
    }

    /* 单通道 20Ksps */
    if((TimeBaseId == 7)&&(g_DS01->ucFirFIter_Step1KHz !=0 ))
    {
        DSO_FirFilter_Step1KHz();
    }
}
/* 省略其它 */
}

```

根据不同的采样率，分别做了个处理，我们这里以函数 DSO_FirFilter_Step100KHz()为例进行说明。这个函数的实现如下：

```

#define FIR_LENGTH_SAMPLES    650    /* 采样点数 */
#define BLOCK_SIZE            650    /* 调用一次 arm_fir_f32 处理的采样点个数 */
#define NUM_TAPS               81     /* 滤波器系数个数 */
#define FIR_StateBufSize      BLOCK_SIZE + NUM_TAPS - 1

/*
*****
*                               滤波器系数
*****
*/
/* 80 阶 FIR 低通滤波器系数 通过 fadtool 获取系数 Fs = 2Msps, Fc = 100KHz */
const float32_t firCoeffs32LP_100KHz[NUM_TAPS] = {
    -3.126438727e-19f, -0.0002058673272f, -0.0004228431499f, -0.0006468905485f, -0.0008642434841f,
    -0.001048665727f, -0.001161655295f, -0.001156177954f, -0.0009838859551f, -0.0006050768425f,
    8.391786605e-19f, 0.0008203662583f, 0.00180617522f, 0.002865221584f, 0.003865677863f,
    0.004645895679f, 0.005031134468f, 0.004856021609f, 0.00399050815f, 0.002366269007f,
    -2.110346192e-18f, -0.002989985514f, -0.006377025973f, -0.009831513278f, -0.01293939352f,
    -0.0152332196f, -0.01623356342f, -0.01549717132f, -0.01266706176f, -0.007519159f,
    3.381513724e-18f, 0.009749332443f, 0.0213866625f, 0.03438147902f, 0.04804687202f,
    0.06158847734f, 0.07416618615f, 0.08496309072f, 0.09325480461f, 0.09847255051f,
    0.1002533659f, 0.09847255051f, 0.09325480461f, 0.08496309072f, 0.07416618615f,
    0.06158847734f, 0.04804687202f, 0.03438147902f, 0.0213866625f, 0.009749332443f,
    3.381513724e-18f, -0.007519159f, -0.01266706176f, -0.01549717132f, -0.01623356342f,
    -0.0152332196f, -0.01293939352f, -0.009831513278f, -0.006377025973f, -0.002989985514f,
    -2.110346192e-18f, 0.002366269007f, 0.00399050815f, 0.004856021609f, 0.005031134468f,
    0.004645895679f, 0.003865677863f, 0.002865221584f, 0.00180617522f, 0.0008203662583f,
    8.391786605e-19f, -0.0006050768425f, -0.0009838859551f, -0.001156177954f, -0.001161655295f,
    -0.001048665727f, -0.0008642434841f, -0.0006468905485f, -0.0004228431499f, -0.0002058673272f,
    -3.126438727e-19f
};

/* 其它截止频率对应的系数未贴出 */

/*
*****
* 函 数 名: DSO_FirFilter_Step100KHz
*****
*/

```

```
* 功能说明: FIR 滤波器。
* 形 参: 无
* 返 回 值: 无
*****
*/
void DSO_FirFilter_Step100KHz(void)
{
    arm_fir_instance_f32 S;
    uint16_t i;

    /* 获取要滤波的数值 */
    for(i = 0; i < BLOCK_SIZE; i++)
    {
        FirDataInput[i] = g_DS01->usWaveBufTemp[i+g_DS01->sCurTriPos+g_DS01->sCurTriStep];
    }

    /* FIR 低通滤波器, 截止频率 100KHz */
    if(g_DS01->ucFirFlter_Step100KHz == 1)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_100KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 200KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 2)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_200KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 300KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 3)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_300KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 400KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 4)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_400KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 500KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 5)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_500KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 600KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 6)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_600KHz[0], &firStateF32[0], BLOCK_SIZE);
    }
    /* FIR 低通滤波器, 截止频率 700KHz */
    else if(g_DS01->ucFirFilter_Step100KHz == 7)
    {
        arm_fir_init_f32(&S, NUM_TAPS, (float32_t *)&firCoeffs32LP_700KHz[0], &firStateF32[0], BLOCK_SIZE);
    }

    /* 80 阶 FIR 滤波 */
    arm_fir_f32(&S, FirDataInput, FirDataOutput, BLOCK_SIZE);

    .
    for(i = 0; i < DSOSCREEN_LENGTH; i++)
    {
```

```
g_DS01->usWaveBuf[i] =FirDataOutput[i+50];  
}  
}
```

上面的代码中有三个重要的地方：

- ◆ 滤波器系数的生成,使用 Matlab 的 fdatool 工具箱即可,详情可以学习我们 DSP 教程的第 37 章节,有详细讲解：<http://forum.armfly.com/forum.php?mod=viewthread&tid=3886>。如果初学的话,可以按照教程讲解实际动手生成一次,看看跟代码中的系数是否一致,一致的话,说明已经成功生成。
- ◆ FIR 滤波器有一个重要的概念就是群延迟,当波形经过 FIR 滤波器后,输出的波形会有一定的延迟。对于线性相位的 FIR,这个群延迟就是一个常数。但是实际应用中这个群延迟是多少呢,关于群延迟的数值,Matlab 的 fdatool 工具箱会根据用户的配置计算好。比如我们这里设计的 80 阶 FIR 低通滤波器的群延迟就是 40,反应在实际的采样值上就是滤波后输出数据的第 41 个才是实际滤波后的波形数据起始点。程序中为了方便起见,每次都是处理 650 个数据,直接使用滤波结果的后 600 个数据。更多关于延迟的知识,看我们 DSP 教程第 37 章的 37.8 小节即可。
- ◆ FIR 滤波器涉及到的函数 arm_fir_init_f32 和 arm_fir_f32 也是在 DSP 教程第 37 章进行了讲解。

关于二代示波器的 FIR 滤波器设计就为大家说这么多,更多这方面的知识需要学习我们的 DSP 教程。

10.3 总结

如果大家不熟悉 FFT 和 FIR 的话,务必要优先学习我们 DSP 教程的相关章节,磨刀不费砍柴工。有了一定的认识后,再来二代示波器的基础上做实战演练即可。

第11章 示波器设计—功能模块划分

二代示波器的界面上做了五个按钮，分别用于不同功能的配置，本章节就为大家讲解这五个按钮实现的功能。

11.1 主界面上的五个按钮

11.2 Measure 测量功能

11.3 ADC 电压测量功能

11.4 DAC 信号发生器

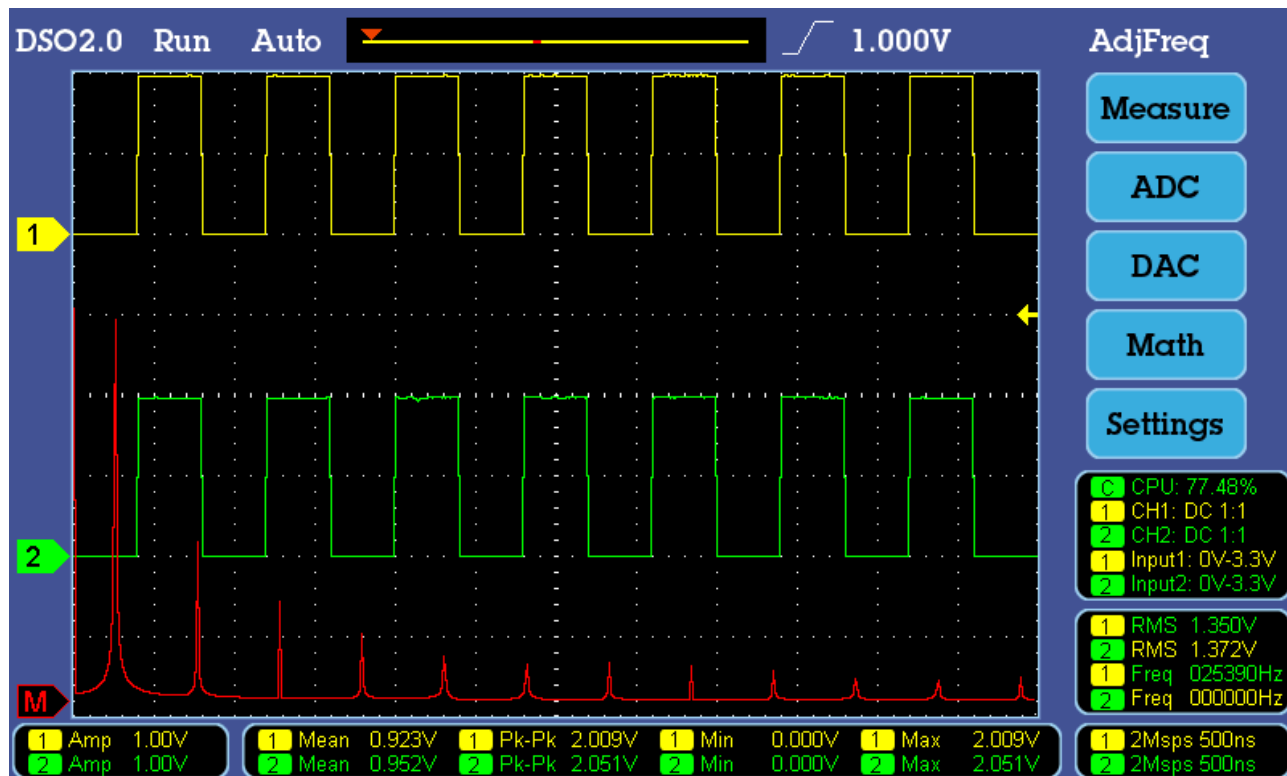
11.5 Math 数字信号处理

11.6 Settings 设置

11.7 总结

11.1 主界面上的五个按钮

为了方便各个功能的配置，主界面右侧做了五个按钮，用于实现五个不同功能的配置。



代码如下：

```
/* 创建需要的按钮*****  
hButton0 = BUTTON_Create(670, 40, 100, 44, GUI_ID_BUTTON0, WM_CF_SHOW);
```

```
BUTTON_SetText(hButton0, "Measure");
WM_SetHasTrans(hButton0);
WM_SetCallback(hButton0, _cbButton);

hButton1 = BUTTON_Create(670, 90-1, 100, 44, GUI_ID_BUTTON1, WM_CF_SHOW);
BUTTON_SetText(hButton1, "ADC");
WM_SetHasTrans(hButton1);
WM_SetCallback(hButton1, _cbButton);

hButton2 = BUTTON_Create(670, 140-2, 100, 44, GUI_ID_BUTTON2, WM_CF_SHOW);
BUTTON_SetText(hButton2, "DAC");
WM_SetHasTrans(hButton2);
WM_SetCallback(hButton2, _cbButton);

hButton3 = BUTTON_Create(670, 190-3, 100, 44, GUI_ID_BUTTON3, WM_CF_SHOW);
BUTTON_SetText(hButton3, "Math");
WM_SetHasTrans(hButton3);
WM_SetCallback(hButton3, _cbButton);

hButton4 = BUTTON_Create(670, 240-4, 100, 44, GUI_ID_BUTTON4, WM_CF_SHOW);
BUTTON_SetText(hButton4, "Settings");
WM_SetHasTrans(hButton4);
WM_SetCallback(hButton4, _cbButton);
```

这些按钮都是创建在桌面窗口上的，点击这些按钮后，会触发桌面窗口回调函数中的按钮消息，从而弹出相应的设置对话框：

```
/*
*****
*   函数名: _cbBkWin
*   功能说明: 桌面窗口的回调函数
*   形参: pMsg
*   返回值: 无
*****
*/
static void _cbBkWin(WM_MESSAGE * pMsg)
{
    int NCode, Id;

    switch (pMsg->MsgId)
    {

        /* 桌面背景重绘 */
        case WM_PAINT:
            DSO_Init(0);
            break;

        /* 通过设置触发标志来更新一次触发游标的显示 */
        case WM_TIMER:
            g_Flag->ucWaveRefresh = 1;
            //WM_RestartTimer(pMsg->Data.v, 1000);
            break;

        case WM_NOTIFY_PARENT:
            Id = WM_GetId(pMsg->hWinSrc);
            NCode = pMsg->Data.v;
            switch (Id)
            {
```



```
/* 按钮 0: 打开 Measure 对话框 */
case GUI_ID_BUTTON0:
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            DSO_CreateMeasureDlg();
            break;
    }
    break;

/* 按钮 1: 打开 ADC 对话框 */
case GUI_ID_BUTTON1:
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            DSO_CreateAdcDlg();
            break;
    }
    break;

/* 按钮 2: 打开 DAC 对话框 */
case GUI_ID_BUTTON2:
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            DSO_CreateDacDlg();
            break;
    }
    break;

/* 按钮 3: 打开 Math 对话框 */
case GUI_ID_BUTTON3:
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            DSO_CreateMathDlg();
            break;
    }
    break;

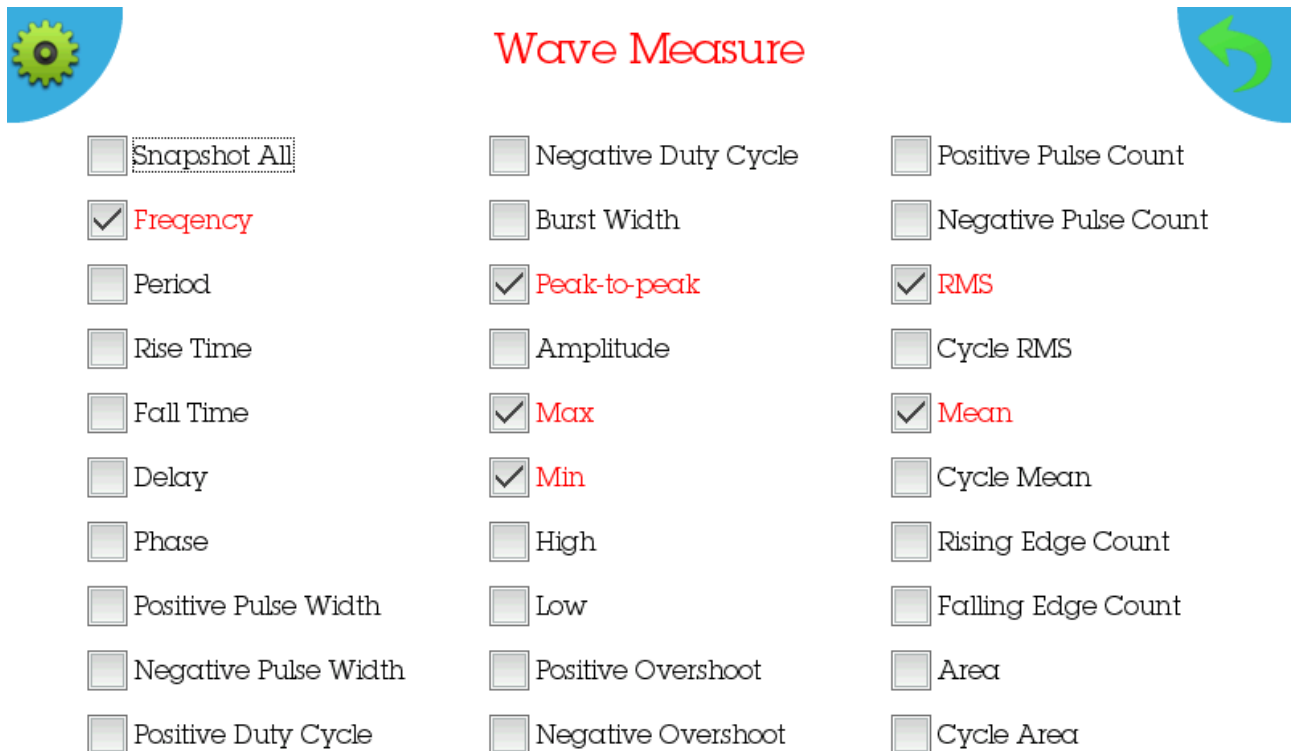
/* 按钮 4: 打开 Settings 对话框 */
case GUI_ID_BUTTON4:
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            DSO_CreateSettingsDlg();
            break;
    }
    break;

default:
    WM_DefaultProc(pMsg);
}
```

下面分别将这五个对话框所实现的功能为大家做个讲解。

11.2 Measure 测量功能

测量界面的效果如下：



当前二代示波器仅支持打对勾的这六种测量值，分别是频率，峰峰值，最大值，最小值，均方根和平均值。其它的测量选项还不支持，勾选或者取消对主界面没有影响。而支持的这六个测量功能取消勾选的话，主界面对应的数值将停止更新，勾选上将继续更新。

测量对话框的创建也比较简单，采用非阻塞方式创建：

```

/*
*****
*   函 数 名: DSO_CreateMeasureDlg
*   功能说明: 创建测量对话框
*   形    参: 无
*   返 回 值: 无
*****
*/
WM_HWIN DSO_CreateMeasureDlg(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreateMeasure,
                               GUI_COUNTOF(_aDialogCreateMeasure),
                               _cbDialogMeasure,
                               WM_HBKWIN,
                               0,
                               0);

    return hWin;
}

```

11.3 ADC 电压测量功能

ADC 电压测量功能的界面比较简单，就是显示 ADC2 所采集的电压，并显示电压曲线。

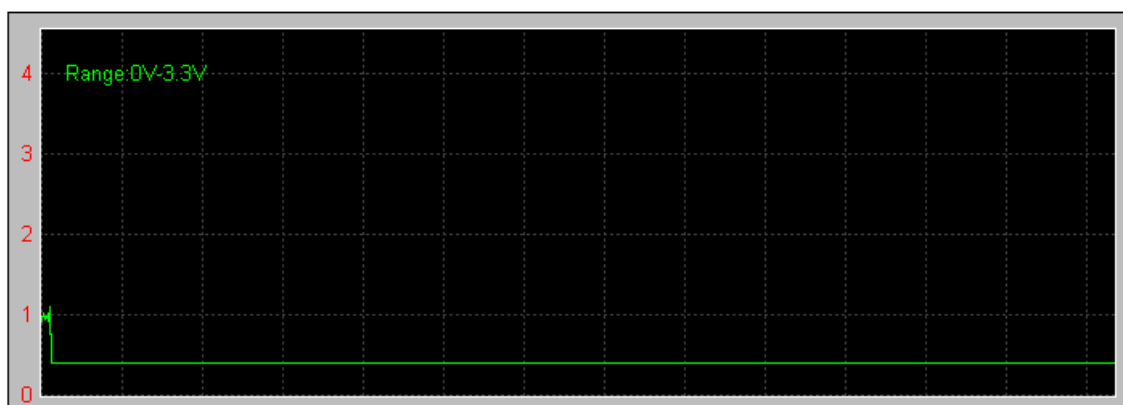


MultiMeter



0.415

ADC=515



ADC 电压测量对话框的创建也比较简单，采用非阻塞方式创建：

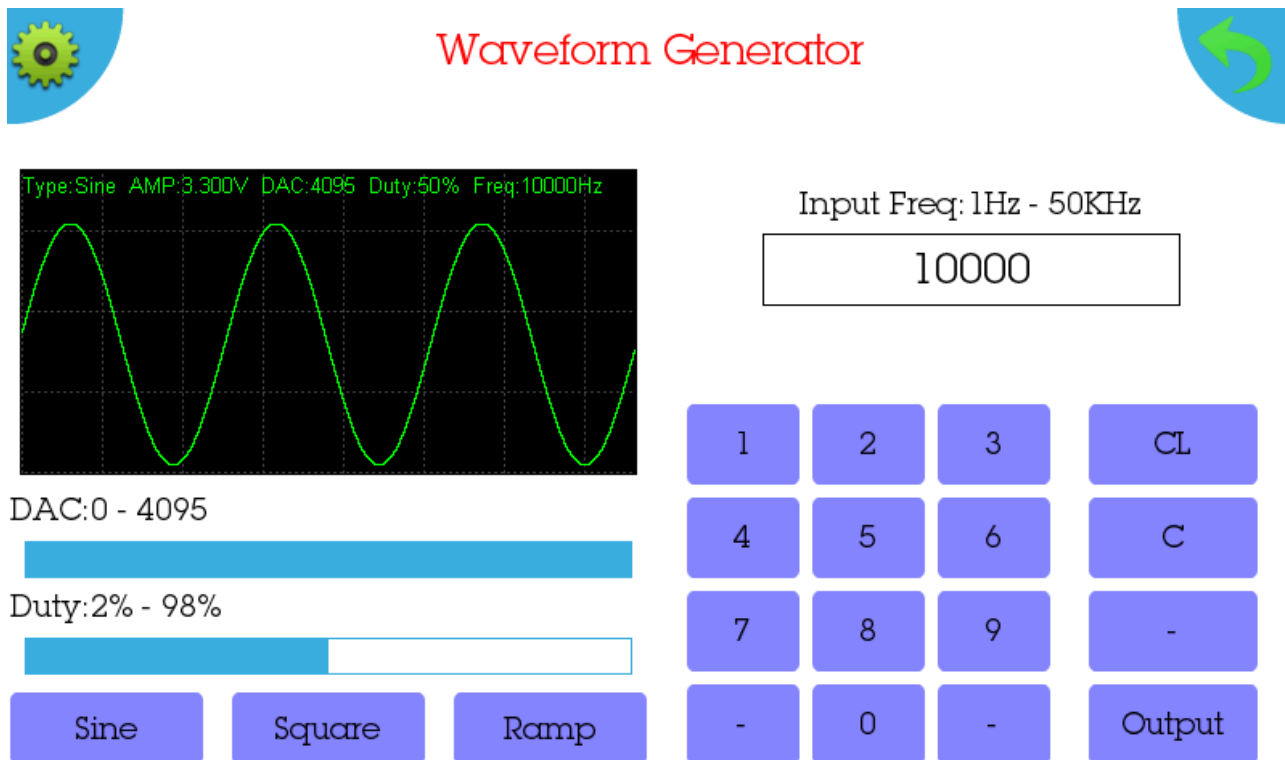
```
/*
*****
*   函 数 名: DSO_CreateAdcDlg
*   功能说明: 创建对话框
*   形    参: 无
*   返 回 值: 无
*****
*/
WM_HWIN DSO_CreateAdcDlg(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreateADC,
                                GUI_COUNTOF(_aDialogCreateADC),
                                _cbCallbackADC,
                                WM_HBKWIN,
                                0,
                                0);

    return hWin;
}
```

11.4 DAC 信号发生器

DAC 信号发生器对话框主要实现了正弦波,方波和三角波的幅值,频率以及占空比的设置,界面效果如下:



这个界面的实现代码要稍复杂些,具体实现在第 14 章会为大家详细讲解。DAC 信号发生器对话框的创建采用非阻塞方式创建:

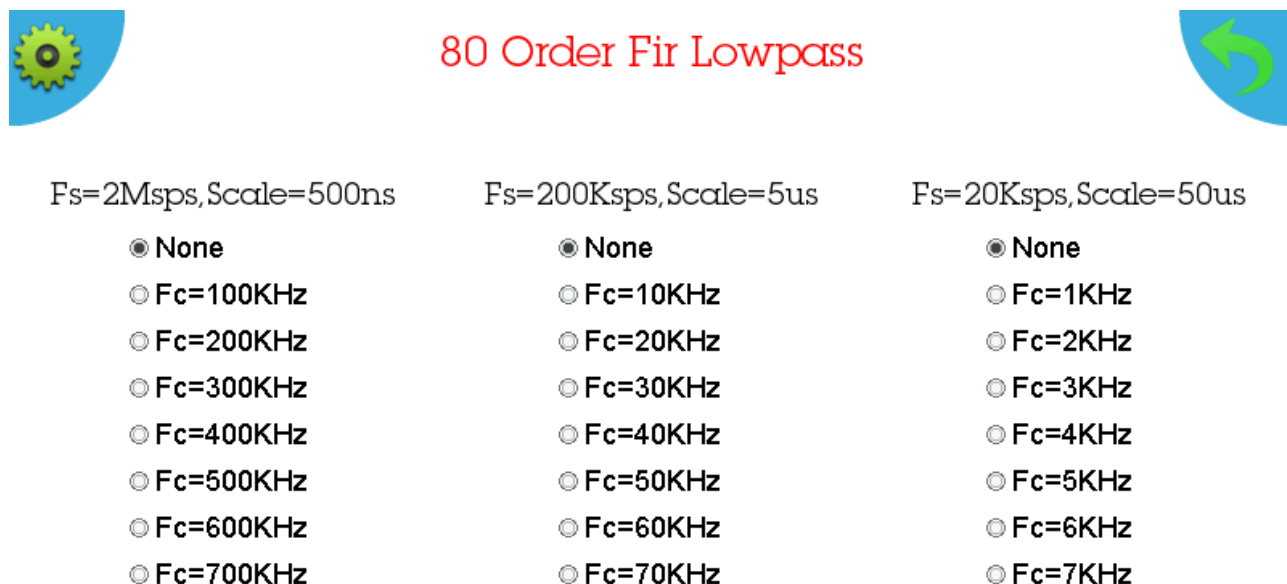
```
/*
*****
*   函数名: DSO_CreateDacDlg
*   功能说明: 创建对话框
*   形参: 无
*   返回值: 无
*****
*/
WM_HWIN DSO_CreateDacDlg(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreatedAC,
                                GUI_COUNTOF(_aDialogCreatedAC),
                                _cbCallbackDAC,
                                WM_HBKWIN,
                                0,
                                0);

    return hWin;
}
```

11.5 Math 数字信号处理

Math 数字信号处理界面用于 FIR 低通滤波器的截止频率配置，界面效果如下：



具体的功能实现已经在前面第 10 章为大家进行了讲解。DAC 信号发生器对话框的创建采用非阻塞方式创建：

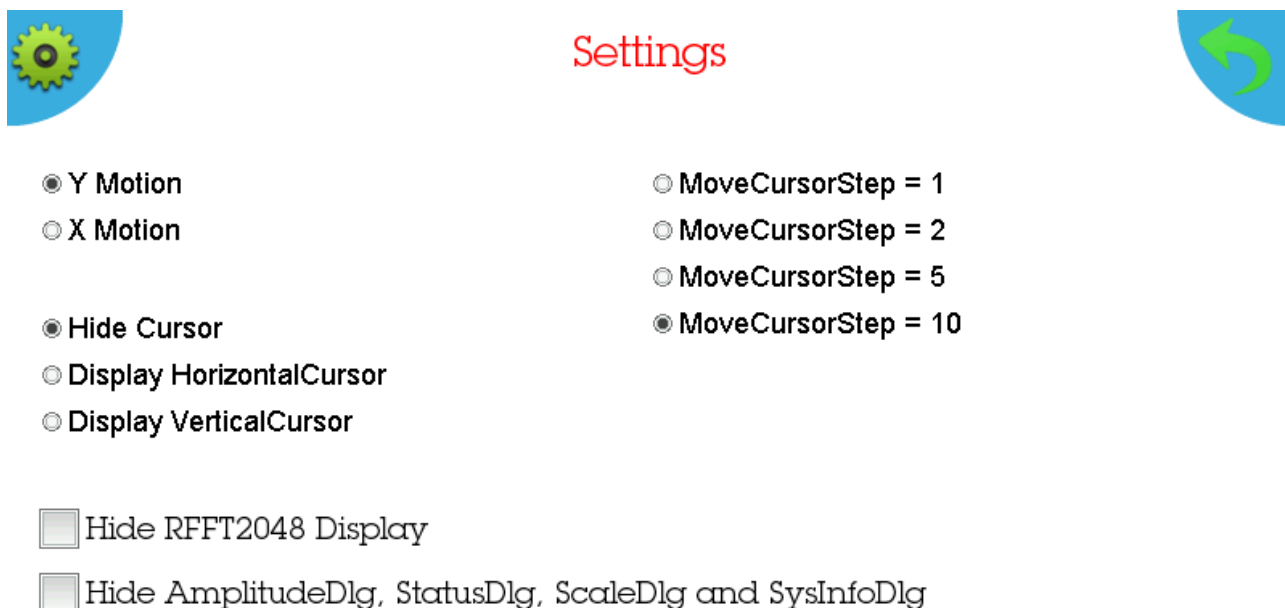
```
/*
*****
*   函数名: DSO_CreateMathDlg
*   功能说明: 创建对话框
*   形参: 无
*   返回值: 无
*****
*/
WM_HWIN DSO_CreateMathDlg(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreateMath,
                               GUI_COUNTOF(_aDialogCreateMath),
                               _cbDialogMath,
                               WM_HBKWIN,
                               0,
                               0);

    return hWin;
}
```

11.6 Settings 设置

Settings 设置对话框的界面效果如下：



设置对话框主要实现了以下四个功能的配置：

- ◆ Motion 功能设置
 - Y motion 表示选择波形垂直方向滑动。
 - X motion 表示选择波形水平方向滑动。
- ◆ 水平和垂直测量功能的选择
 - Hide Cursor 表示隐藏水平测量和垂直测量。
 - Display HorizontalCursor 表示显示水平测量。
 - Display VerticalCursor 表示显示垂直测量。
- ◆ 设置水平和垂直测量游标每次移动的像素个数
 - MoveCursorStep = 1 表示每次移动 1 个像素。
 - MoveCursorStep = 2 表示每次移动 2 个像素。
 - MoveCursorStep = 5 表示每次移动 5 个像素。
 - MoveCursorStep = 10 表示每次移动 10 个像素。
- ◆ Hide RFFT2048 Display 表示隐藏 FFT 频谱显示。
- ◆ Hide AmplitudeDlg, StatusDlg, ScaleDlg and SysInfoDlg 表示隐藏幅值窗口，状态窗口，频率窗口和系统信息窗口。

Settings 设置对话框的创建也采用非阻塞方式创建：

```
/*
*****
*   函数名: DSO_CreateSettingsDlg
*   功能说明: 创建设置对话框
*   形参: 无
*   返回值: 无
*****
*/
WM_HWIN DSO_CreateSettingsDlg(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreateSettings,
                               GUI_COUNTOF(_aDialogCreateSettings),
                               _cbDialogSettings,
                               WM_HBKWIN,
                               0,
                               0);

    return hWin;
}
```

11.7 总结

本章节的内容比较简单，主要是让大家对这五个对话框的功能有个了解，看代码时就比较容易理解了。

第12章 示波器设计—DAC 信号发生器的实现

本章节为大家讲解二代示波器中信号发生器的实现。这个功能还是比较实用的，方便为二代示波器提供测试信号。实现了正弦波，方波和三角波的频率，幅度以及占空比设置。

12.1 DAC 的输出阻抗和使能缓冲问题

12.2 DAC 驱动实现

12.3 信号发生器配置界面设计

12.4 信号发生器波形显示效果

12.5 总结

12.1 DAC 的输出阻抗和使能缓冲问题

我们这里把 F429 的输出阻抗和使能缓冲问题放在最前面说。

使能了多缓冲后发现失真问题，即满幅输出的时候有削顶和削底，而禁止了输出缓冲会导致输出阻抗仅有 10KΩ左右，外接负载很容易造成分压（可以根据实际情况，外接运放输出）。

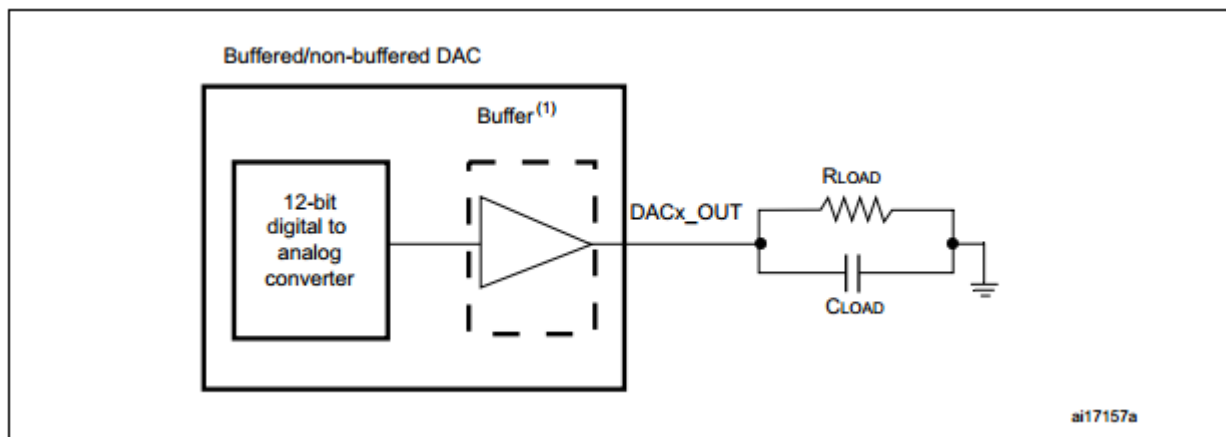
F429 的手册中对于 DAC 的几个关键特性说明如下：

- ◆ 开启缓冲的时候，外接的负载阻抗最小得是 5KΩ。
- ◆ 禁止缓冲的时候，DAC 输出阻抗最大可达 15KΩ，比如要实现 1%精度的输出，外接负载阻抗至少得是 1.5MΩ。
- ◆ 开启缓冲的时候，最小输出电压 0.2V，最大 $V_{DDA} - 0.2V$ ，这个应该是造成削顶问题的根本原因。
- ◆ 禁止缓冲的时候，最小输出电压的典型值是 0.5mV，最大输出是 $V_{REF} - 1LSB$ 。基本正好满幅输出，所以效果比较好。

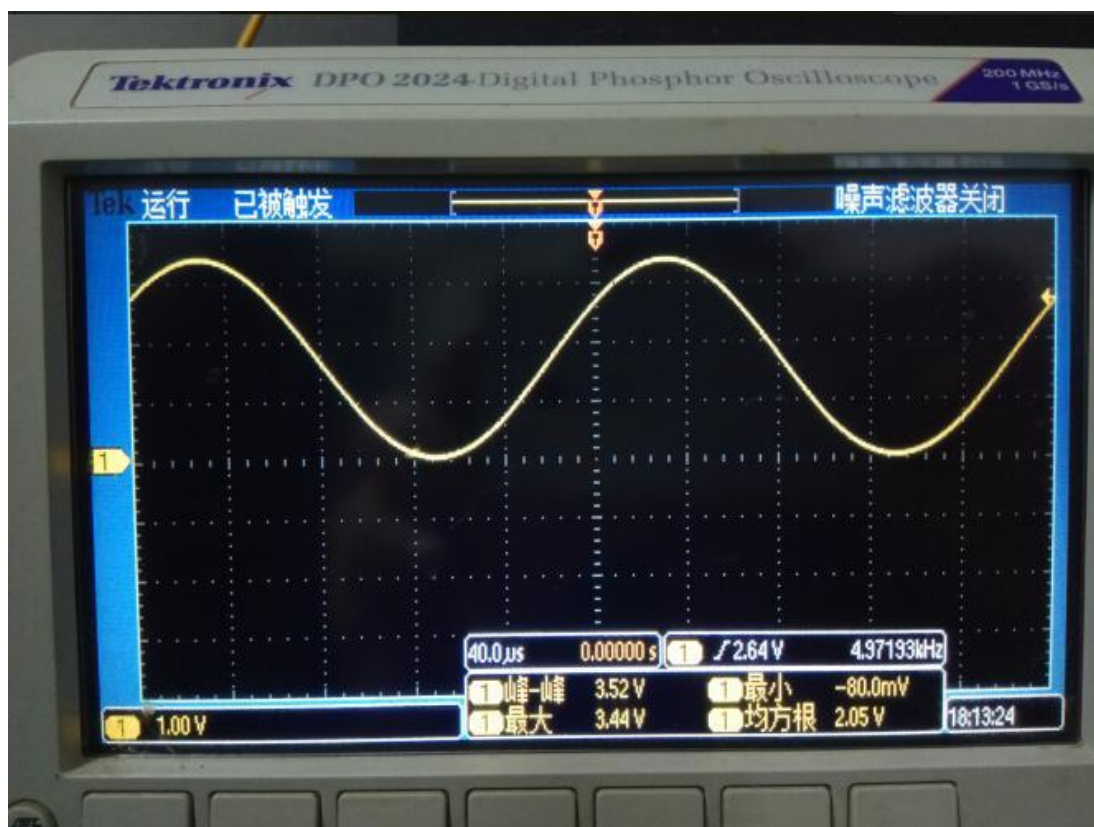
F429 数据手册中几个关键参数的截图：

Symbol	Parameter	Min	Typ	Max	Unit	Comments
$R_{LOAD}^{(2)}$	Resistive load with buffer ON	5	-	-	kΩ	
$R_O^{(2)}$	Impedance output with buffer OFF	-	-	15	kΩ	When the buffer is OFF, the Minimum resistive load between DAC_OUT and V_{SS} to have a 1% accuracy is 1.5 MΩ
$C_{LOAD}^{(2)}$	Capacitive load	-	-	50	pF	Maximum capacitive load at DAC_OUT pin (when the buffer is ON).
DAC_OUT_min ⁽²⁾	Lower DAC_OUT voltage with buffer ON	0.2	-	-	V	It gives the maximum output excursion of the DAC. It corresponds to 12-bit input code (0x0E0) to (0xF1C) at $V_{REF+} = 3.6 V$ and (0x1C7) to (0xE38) at $V_{REF+} = 1.7 V$
DAC_OUT_max ⁽²⁾	Higher DAC_OUT voltage with buffer ON	-	-	$V_{DDA} - 0.2$	V	
DAC_OUT_min ⁽²⁾	Lower DAC_OUT voltage with buffer OFF	-	0.5	-	mV	It gives the maximum output excursion of the DAC.
DAC_OUT_max ⁽²⁾	Higher DAC_OUT voltage with buffer OFF	-	-	$V_{REF+} - 1LSB$	V	

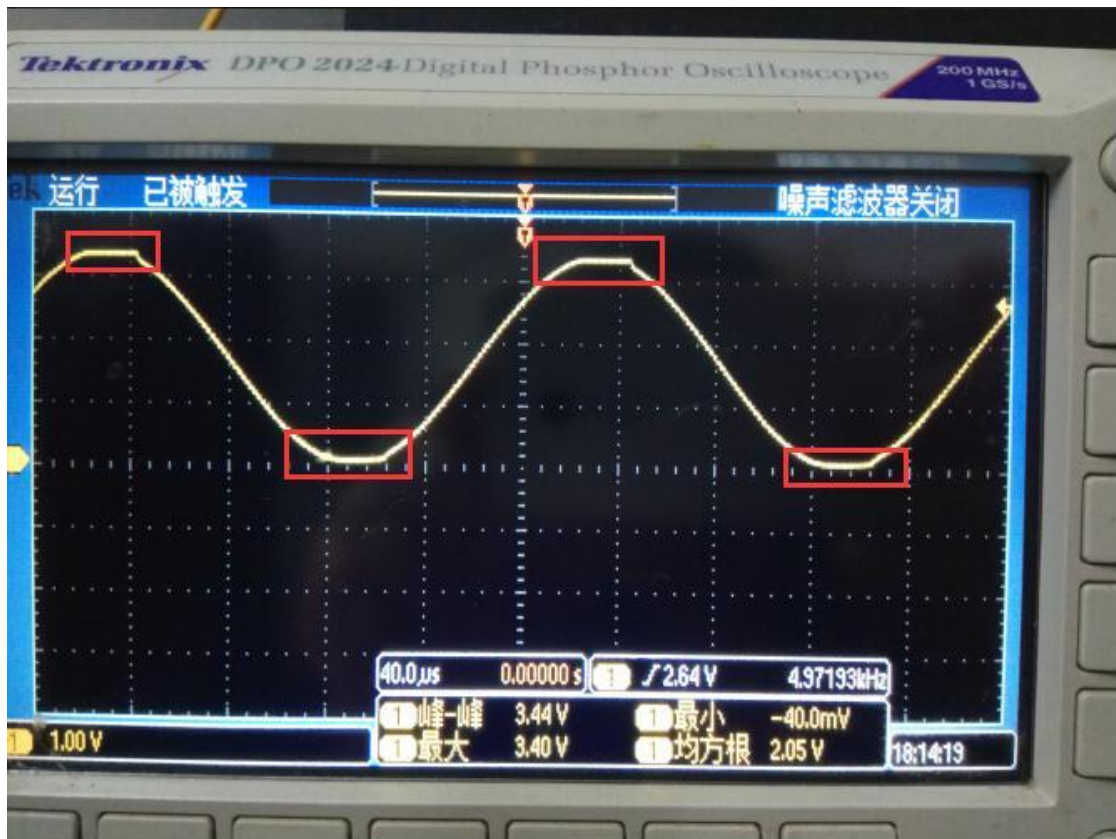
缓冲和外接负载时的框图：



禁止缓冲时，满幅输出效果比较漂亮：



使能缓冲时，满幅输出效果，出现削顶问题：



有了上面的感性认识后，下面为大家讲解 DAC 的驱动实现和相应的 GUI 界面实现。

12.2 DAC 驱动实现

F429 带有两个 DAC，分别是 DAC1 和 DAC2，我们这里使用了 DAC1，驱动中还需要用到 TIM6 和 DMA，方便我们配置不同的频率，占空比和幅值。

12.2.1 第 1 步：引脚配置和 DAC 配置

配置代码如下，使用的 PA4 引脚做输出：

```
/*
*****
* 函数名: bsp_InitDAC1
* 功能说明: 配置 PA4/DAC1
* 形参: 无
* 返回值: 无
*****
*/
void bsp_InitDAC1(void)
{
    /* 配置 GPIO */
    {
        GPIO_InitTypeDef GPIO_InitStructure;

        RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);
```

```

/* 配置 DAC 引脚为模拟模式 PA4 / DAC_OUT1 */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
GPIO_Init(GPIOA, &GPIO_InitStructure);
}

/* DAC 通道 1 配置 */
{
    DAC_InitTypeDef DAC_InitStructure;

    /* 使能 DAC 时钟 */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);

    DAC_InitStructure.DAC_Trigger = DAC_Trigger_None; /* 选择软件触发, 软件修改 DAC 数据寄存器 */
    DAC_InitStructure.DAC_WaveGeneration = DAC_WaveGeneration_None;
    DAC_InitStructure.DAC_LFSRUnmask_TriangleAmplitude = DAC_LFSRUnmask_Bit0;
    //DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
    DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Disable;
    DAC_Init(DAC_Channel_1, &DAC_InitStructure);
    DAC_Cmd(DAC_Channel_1, ENABLE);
}
}

```

特别注意。程序中关闭了 DAC 输出缓冲，即 DAC 参数成员 DAC_InitStructure.DAC_OutputBuffer。关于 DAC 的缓冲问题，看前面 12.1 小节说明即可。

12.2.2 第 2 步：DAC 的定时器触发和 DMA 配置

DAC 的定时器触发和 DMA 配置如下：

```

/*
*****
* 函数名: dac1_InitForDMA
* 功能说明: 配置 PA4 为 DAC_OUT1, 启用 DMA2
* 形参: _BufAddr : DMA 数据缓冲区地址
*       _Count   : 缓冲区样本个数
*       _DacFreq  : DAC 样本更新频率
* 返回值: 无
*****
*/
void dac1_InitForDMA(uint32_t _BufAddr, uint32_t _Count, uint32_t _DacFreq)
{
    uint16_t usPeriod;
    uint16_t usPrescaler;
    uint32_t uiTIMxCLK;
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;

    DMA_Cmd(DMA1_Stream5, DISABLE);
    DAC_DMAMCmd(DAC_Channel_1, DISABLE);
    TIM_Cmd(TIM6, DISABLE);

    /* TIM6 配置 */
    {

```

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM6, ENABLE);

uiTIMxCLK = SystemCoreClock / 2;

if (_DacFreq < 100)
{
    usPrescaler = 10000 - 1; /* 分频比 = 10000 */
    usPeriod = (uiTIMxCLK / 10000) / _DacFreq - 1; /* 自动重装的值 */
}
else if (_DacFreq < 3000)
{
    usPrescaler = 100 - 1; /* 分频比 = 100 */
    usPeriod = (uiTIMxCLK / 100) / _DacFreq - 1; /* 自动重装的值 */
}
else /* 大于 4K 的频率, 无需分频 */
{
    usPrescaler = 0; /* 分频比 = 1 */
    usPeriod = uiTIMxCLK / _DacFreq - 1; /* 自动重装的值 */
}

TIM_TimeBaseStructure.TIM_Period = usPeriod;
TIM_TimeBaseStructure.TIM_Prescaler = usPrescaler;
TIM_TimeBaseStructure.TIM_ClockDivision = 0;
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseStructure.TIM_RepetitionCounter = 0x0000; /* TIM1 和 TIM8 必须设置 */

TIM_TimeBaseInit(TIM6, &TIM_TimeBaseStructure);

/* 选择 TIM6 做 DAC 的触发时钟 */
TIM_SelectOutputTrigger(TIM6, TIM_TRGOSource_Update);
}

/* DAC 通道 1 配置 */
{
    DAC_InitTypeDef DAC_InitStructure;

    /* 使能 DAC 时钟 */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);

    DAC_InitStructure.DAC_Trigger = DAC_Trigger_T6_TRGO;
    DAC_InitStructure.DAC_WaveGeneration = DAC_WaveGeneration_None;
    DAC_InitStructure.DAC_LFSRUnmask_TriangleAmplitude = DAC_LFSRUnmask_Bit0;
    //DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
    DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Disable;
    DAC_Init(DAC_Channel_1, &DAC_InitStructure);
    DAC_Cmd(DAC_Channel_1, ENABLE);
}

/* DMA1_Stream5 配置 */
{
    DMA_InitTypeDef DMA_InitStructure;

    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA1, ENABLE);

    /* 配置 DMA1 Stream 5 channel 7 用于 DAC1 */
    DMA_InitStructure.DMA_Channel = DMA_Channel_7;
    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&DAC->DHR12R1;
    DMA_InitStructure.DMA_Memory0BaseAddr = _BufAddr;
```

```

DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral;
DMA_InitStructure.DMA_BufferSize = _Count;
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
DMA_InitStructure.DMA_MemoryDataSize = DMA_PeripheralDataSize_HalfWord;
DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
DMA_InitStructure.DMA_Priority = DMA_Priority_High;
DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_HalfFull;
DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
DMA_Init(DMA1_Stream5, &DMA_InitStructure);
DMA_Cmd(DMA1_Stream5, ENABLE);

/* 使能 DAC 通道 1 的 DMA */
DAC_DMAMCmd(DAC_Channel1_1, ENABLE);
}

/* 使能定时器 */
TIM_Cmd(TIM6, ENABLE);
}

```

通过这个函数可以方便的设置 DAC 的输出波形频率。计算方法是：

配置的定时器触发频率 / DMA 的缓冲个数 = 输出波形频率

其中 DMA 缓冲数据的个数就是输出波形一个周期的采样点数。程序中统一将其配置为 128 个点代表一个周期的波形，大家实际应用中配置的点数不要太少，否则波形不够漂亮。比如我们要出 10KHz 的波形，这个函数的配置就是：dac1_InitForDMA((uint32_t)&g_Wave1, 128, 10000 * 128); 数组 g_Wave1 里面是 128 个波形采样点。

关于这个驱动代码，要注意 TIM6 的配置。F429 的定时器从 TIM1 到 TIM14 的主频如下：

```

/*
*****
system_stm32f4xx.c 文件中 void SetSysClock(void) 函数对时钟的配置如下：

HCLK = SYSCLK / 1      (AHB1Periph)
PCLK2 = HCLK / 2       (APB2Periph)
PCLK1 = HCLK / 4       (APB1Periph)

因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = PCLK1 x 2 = SystemCoreClock / 2;
因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = PCLK2 x 2 = SystemCoreClock;

APB1 定时器有 TIM2, TIM3, TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14
APB2 定时器有 TIM1, TIM8, TIM9, TIM10, TIM11

TIM 更新周期是 = TIMCLK / (TIM_Period + 1) / (TIM_Prescaler + 1)
*****
*/

```

由此可知，TIM6 的主频是 SystemCoreClock / 2。当主频是 168MHz 时，TIM6 的时钟就是 84MHz，TIM6 更新周期 = TIM6CLK / (TIM_Period + 1) / (TIM_Prescaler + 1)，其中 TIM_Period 就是定时器结构体成员 TIM_TimeBaseStructure.TIM_Period。

TIM_Prescaler 就是定时器结构体成员 TIM_TimeBaseStructure.TIM_Prescaler。

另外还有非常重要的一点，TIM6 是 16 位定时器，这两个参范围是 0-65535，切不要超过 65535。正是因为这个原因，程序中对不同的输出频率做了范围区分。

12.2.3 第 3 步：正弦波输出配置

正弦波的输出配置如下：

```
/*
*****
*   函 数 名: dac1_SetSinWave
*   功能说明: DAC1 输出正弦波
*   形    参: _vpp : 幅度 0-4095;
*             _freq : 频率
*   返 回 值: 无
*****
*/
void dac1_SetSinWave(uint16_t _vpp, uint32_t _freq)
{
    uint32_t i;
    uint32_t dac;

    TIM_Cmd(TIM6, DISABLE);

    /* 调整正弦波幅度 */
    for (i = 0; i < 128; i++)
    {
        dac = (g_SineWave128[i] * _vpp) / 4095;
        if (dac > 4095)
        {
            dac = 4095;
        }
        g_Wave1[i] = dac;
    }

    dac1_InitForDMA((uint32_t)&g_Wave1, 128, _freq * 128);
}
```

正弦波输出 128 个采样点代表一个周期，同时程序里面增加了一个幅值设置功能，范围 0 到 4095。实际 DAC 输出的波形频率由前面第 2 步函数 dac1_InitForDMA 实现。比如我们要实现频率 10KHz 幅值 4095 正弦波，那么配置就是：dac1_SetSinWave(4095, 10000)。

12.2.4 第 4 步：方波输出配置

方波的输出配置如下：

```
/*
*****
*   函 数 名: dac1_SetRectWave
*   功能说明: DAC1 输出方波
*   形    参: _low : 低电平时 DAC,
*             _high : 高电平时 DAC
*****
*/
```



```

*          _freq : 频率 Hz
*          _duty : 占空比 2% - 98%, 调节步数 1%
*   返回值: 无
*****
*/
void dac1_SetRectWave(uint16_t _low, uint16_t _high, uint32_t _freq, uint16_t _duty)
{
    uint16_t i;
    TIM_Cmd(TIM6, DISABLE);

    for (i = 0; i < (_duty * 128) / 100; i++)
    {
        g_Wave1[i] = _high;
    }
    for (; i < 128; i++)
    {
        g_Wave1[i] = _low;
    }

    dac1_InitForDMA((uint32_t)&g_Wave1, 128, _freq * 128);
}

```

方波也是输出 128 个采样点代表一个周期,同时支持幅值和占空比的配置,其中占空比可以配置 2%到 98%,直接填数值 2 到 98 就可以了。实际 DAC 输出的波形频率由前面第 2 步函数 dac1_InitForDMA 实现。比如我们要实现频率 10KHz,幅值 4095,占空比 50%的方波,那么配置就是:

dac1_SetRectWave (0, 4095, 10000, 50)。

12.2.5 第 5 步：三角波输出配置

三角波的输出配置如下：

```

/*
*****
*   函数名: dac1_SetTriWave
*   功能说明: DAC1 输出三角波
*   形参: _low : 低电平时 DAC,
*         _high : 高电平时 DAC
*         _freq : 频率 Hz
*         _duty : 占空比
*   返回值: 无
*****
*/
void dac1_SetTriWave(uint16_t _low, uint16_t _high, uint32_t _freq, uint16_t _duty)
{
    uint32_t i;
    uint16_t dac;
    uint16_t m;

    TIM_Cmd(TIM6, DISABLE);

    /* 构造三角波数组, 128 个样本, 从 _low 到 _high */
    m = (_duty * 128) / 100;

    if (m == 0)

```

```
{
    m = 1;
}

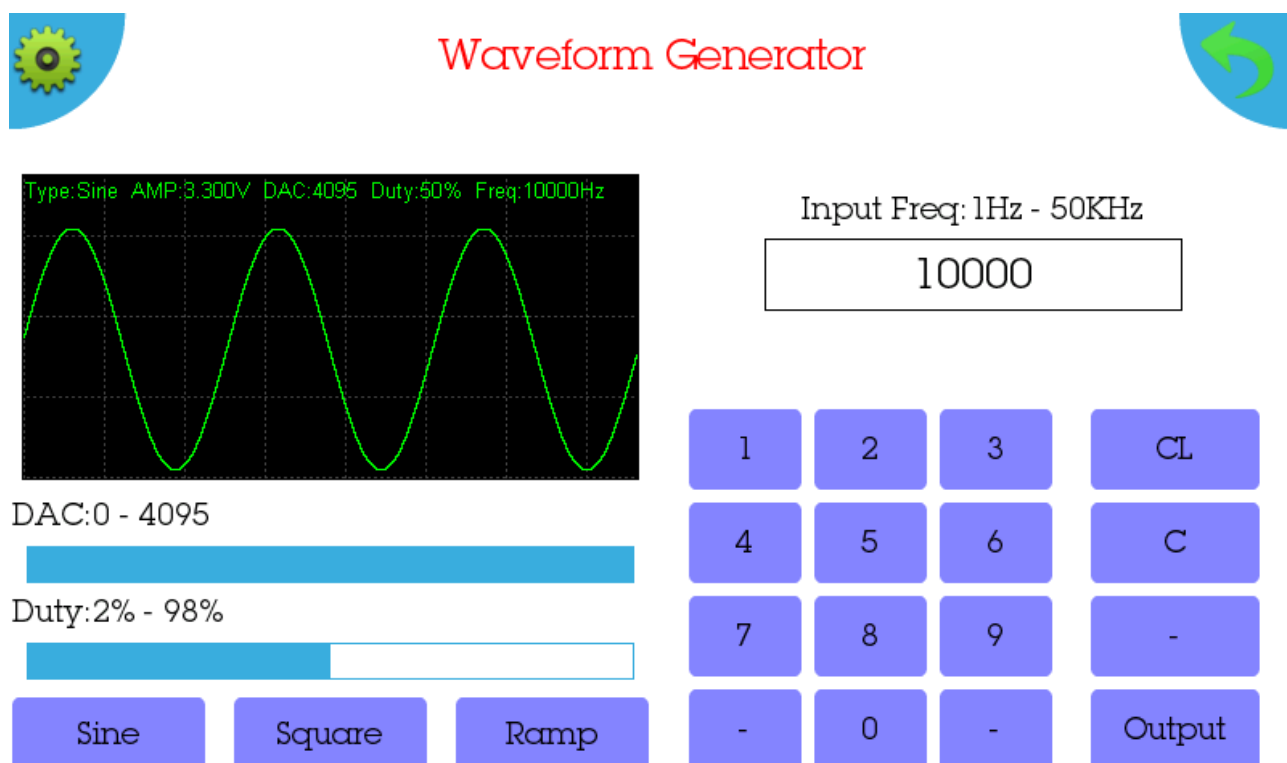
if (m > 127)
{
    m = 127;
}
for (i = 0; i < m; i++)
{
    dac = _low + ((_high - _low) * i) / m;
    g_Wave1[i] = dac;
}
for (; i < 128; i++)
{
    dac = _high - ((_high - _low) * (i - m)) / (128 - m);
    g_Wave1[i] = dac;
}

dac1_InitForDMA((uint32_t)&g_Wave1, 128, _freq * 128);
}
```

三角波也是输出 128 个采样点代表一个周期，同时支持幅值和占空比的配置，其中占空比可以配置 0%到 100%，不过程序中对 0%和 100%做了一个特殊处理。实际 DAC 输出的波形频率由前面第 2 步函数 dac1_InitForDMA 实现。比如我们要实现频率 10KHz，幅值 4095，占空比 50%的三角波，那么配置就是：dac1_SetTriWave (0, 4095, 10000, 50)。

12.3 信号发生器配置界面设计

信号发生器的界面设计如下：



这个操作界面简单易用，支持正弦波，方波和三角波的切换，支持占空比设置，支持幅值设置，同时也支持频率设置，限制频率范围 1Hz 到 50KHz。超过 50KHz 的话，波形效果会变的越来越差。

关于这个对话框的代码实现就不在教程里面做讲解了，我们这里主要讲解下对话框上的小键盘实现。这里小键盘是一个独立的窗口，父窗口是信号发生器主窗口，通过函数 WM_SendMessageNoPara 发自定义消息给父窗口，在父窗口里面更新 Graph 控件的波形和波形信息，同时 DAC 的波形输出也得到更新。了解了这知识点后，再看代码就比较容易了。

- 知识点拓展

新版 emWin 教程第 51 章：实用的官方小键盘实例讲解：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834>。

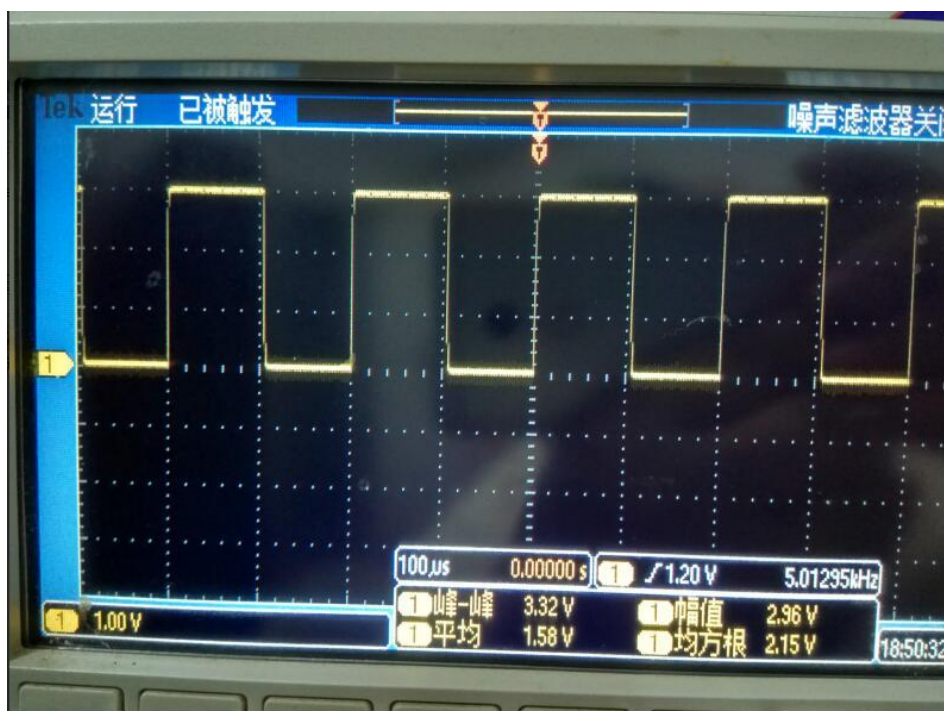
另外还有 emWin 提高篇例子的第一期 ATM 机里面也有用到小键盘。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=23687>。

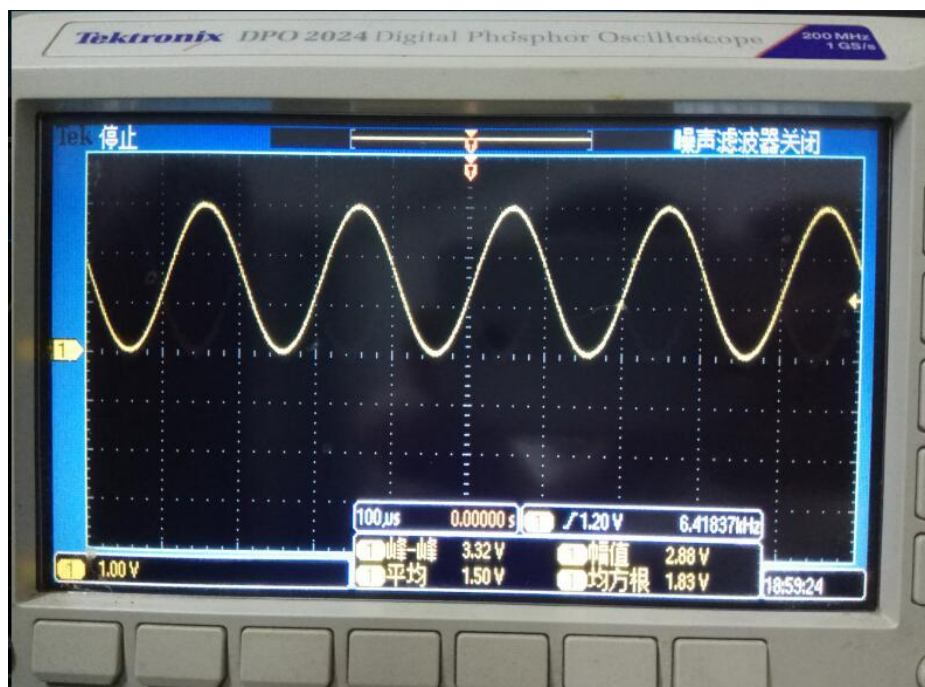
12.4 信号发生器波形显示效果

下面为大家展示信号发生器输出波形效果：

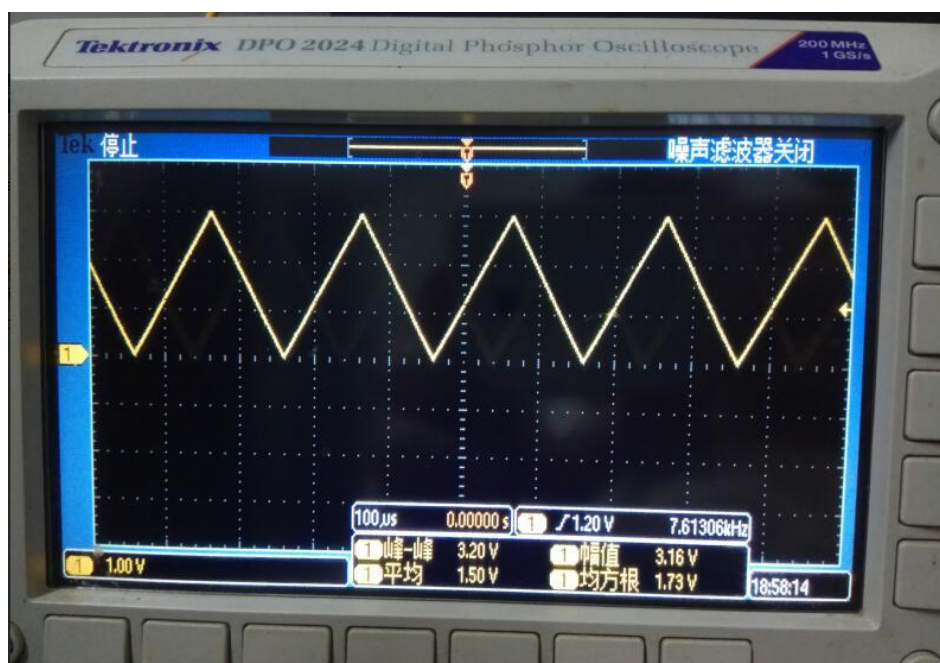
方波：



正弦波：



三角波：



12.5 总结

本章节为大家讲解的信号发生器还是比较实用的，建议实际动手操作下，有兴趣的话，还可以进一步优化升级。

第13章 RTX 操作系统版本二代示波器实现

本章教程为大家讲解 RTX 操作系统版本的二代示波器实现。主要讲解 RTOS 设计框架，即各个任务实现的功能，任务间的通信方案选择，任务栈，系统栈以及全局变量共享问题。同时，工程调试方法也专门做了说明。

- 13.1 注意事项（重要必读）
- 13.2 任务功能划分
- 13.3 用户任务优先级设置
- 13.4 全局变量分配，系统堆栈和任务堆栈
- 13.5 任务间通信和全局变量共享问题
- 13.6 RTX 配置向导
- 13.7 RTX 系统调试
- 13.8 MDK 优化等级
- 13.9 总结

13.1 注意事项（重要必读）

- ◆ 学习本章节前，务必保证已经学习完毕前面章节。另外，工程代码注释已经比较详细，了解了框架后，**直接看源码即可。**
- ◆ RTX 操作系统版本的限制使用 MDK4.74，其它 MDK 版本不支持。详情看我们 RTX 教程即可：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。
- ◆ 仅支持 800*480 分辨率显示屏，如果是电容屏，无需校准。如果是电阻屏，需要校准，按下按键 K1 即可进入校准界面。
- ◆ 由于按键不够用，在 MainTask.c 文件的 MainTask 函数里面对按键 K1 的消息处理做了一个条件编译，大家可以根据需要选择执行触摸校准功能还是截图功能。#if 1 表示执行触摸校准，#if 0 表示执行截图功能。

```
case KEY_1_DOWN:
    /*****由于按键不够用，将截图功能取消*****/
    #if 0
        hTouchWin = WM_CreateWindowAsChild(0,
            0,
            800,
            480,
            WM_HBKWIN,
            WM_CF_SHOW,
            _cbTouchCalibration,
```



```

                                0);

    WM_Exec();
    WM_SelectWindow(hTouchWin);

    /* 执行触摸校准 */
    TOUCH_Calibration();

    WM_SelectWindow(0);

    WM_DeleteWindow(hTouchWin);
    WM_Exec();

    /* 自动触发暂停状态 */
    if(g_Flag->hWinRunStop == 1)
    {
        g_Flag->ucWaveRefresh = 1;
    }

    /* 普通触发暂停状态 */
    if(TriggerFlag == 1)
    {
        TriggerFlag = 2;
    }
#else
    os_sem_send (&semaphore);
#endif
break;

```

- ◆ 文件系统是用的 RL-FlashFS，如果大家想学习 RL-FlashFS 的使用，学习 KEIL 给的手册即可：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=2988>。
- ◆ MDK 安装目录里面带的 emWin5.4x 版本的截图功能有 bug，详情看此贴：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=82445>。
当前用的 5.36 版本，也是来自 MDK。

13.2 任务功能划分

前面第三章已经将任务功能划分好：



根据这个功能划分，创建所需要的任务。另外，RTX 本身是不支持 CPU 利用率统计的，所以专门创建了一个任务实现 CPU 利用率统计。

13.2.1 主函数创建

在 main.c 文件实现：

```
/*
*****
*   函 数 名: main
*   功能说明: 标准 c 程序入口。
*   形    参: 无
*   返 回 值: 无
*****
*/
int main (void)
{
    /* 初始化外设 */
    bsp_Init();

    /* 创建启动任务 */
    os_sys_init_user (AppTaskStart,          /* 任务函数 */
                      6,                      /* 任务优先级 */
                      &AppTaskStartStk,      /* 任务栈 */
                      sizeof(AppTaskStartStk)); /* 任务栈大小, 单位字节数 */

    while(1);
}
```

硬件外设的初始化函数 bsp_Init 是在 bsp.c 文件实现：

```
/*
*****
*   函 数 名: bsp_Init
*   功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*   形    参: 无
*   返 回 值: 无
*****
*/
void bsp_Init(void)
{
    /*
        由于 ST 固件库的启动文件已经执行了 CPU 系统时钟的初始化，所以不必再次重复配置系统时钟。
        启动文件配置了 CPU 主时钟频率、内部 Flash 访问速度和可选的外部 SRAM FSMC 初始化。

        系统时钟缺省配置为 168MHz，如果需要更改，可以修改 system_stm32f4xx.c 文件
    */

    /* 优先级分组设置为 4，可配置 0-15 级抢占式优先级，0 级子优先级，即不存在子优先级。*/
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);

    SystemCoreClockUpdate(); /* 根据 PLL 配置更新系统时钟频率变量 SystemCoreClock */

    bsp_InitDWT();           /* 初始化 DWT */
    bsp_InitUart();           /* 初始化串口 */
    bsp_InitKey();           /* 初始化按键变量（必须在 bsp_InitTimer() 之前调用） */

    bsp_InitI2C();           /* 配置 I2C 总线 */

    bsp_InitExtSDRAM();
    bsp_DetectLcdType();     /* 检测触摸板和 LCD 面板型号，结果存在全局变量 g_TouchType, g_LcdType */
}
```



```
TOUCH_InitHard(); /* 初始化配置触摸芯片 */
LCD_ConfigLTDC(); /* 初始化配置 LTDC */

DSO_ConfigCtrlGPIO(); /* 初始化示波器模块的引脚配置 */

bsp_InitADC(); /* 初始化 ADC1, ADC2 和 ADC3 */
bsp_InitDAC1(); /* 初始化 DAC1 */

g_DAC1.ucDuty = 50; /* 初始化 DAC 配置, 用于信号发生器 */
g_DAC1.ucWaveType = 0;
g_DAC1.ulAMP = 4095;
g_DAC1.ulFreq = 10000;
dac1_SetSinWave(g_DAC1.ulAMP, g_DAC1.ulFreq);

MountSD(); /* 挂载 SD 卡 */

TIM8_MeasureTrigConfig(); /* 初始化 TIM8 用于记录一段波形 */
}
```

创建任务的主要功能是硬件外设初始化和启动任务的创建，相对比较简单。

13.2.2 启动任务（信号处理）

启动任务用于二代示波器的信号处理：

```
/*
*****
* 函数名: AppTaskStart
* 功能说明: 启动任务, 也是最高优先级任务, 用于信号处理。
* 形参: 无
* 返回值: 无
* 优先级: 6
*****
*/
__task void AppTaskStart(void)
{
    OS_RESULT xResult;

    /* 优先创建统计任务-----*/
    HandleTaskStat = os_tsk_create_user(AppTaskStatistic, /* 任务函数 */
                                        1, /* 任务优先级 */
                                        &AppTaskStatStk, /* 任务栈 */
                                        sizeof(AppTaskStatStk)); /* 任务栈大小, 单位字节数 */

    OSStatInit();

    /* 创建任务间通信机制和动态内存分配, 此函数要优先调用 */
    AppObjCreate();

    /* 创建任务 */
    AppTaskCreate();

    /* 实数序列 FFT 长度 */
    fftSize = 2048;
}
```

```
/* 正变换 */
ifftFlag = 0;

/* 初始化结构体 S 中的参数 */
arm_rfft_fast_init_f32(&S, fftSize);

HandleTaskStart = os_tsk_self();

while(1)
{
    if(os_evt_wait_or(StartTaskWaitFlag, 0xFFFF) == OS_R_EVT)
    {
        xResult = os_evt_get ();

        switch (xResult)
        {
            case DspFFT2048Pro_15:
                /* 读取的是 ADC3 的位置 */
                g_DS01->usCurPos = 10240 - DMA2_Stream1->NDTR;

                /* 读取的是 ADC1 的位置 */
                g_DS02->usCurPos = 10240 - DMA2_Stream0->NDTR;

                DS02_WaveTrig(g_DS02->usCurPos);
                DS01_WaveTrig(g_DS01->usCurPos);
                DS02_WaveProcess();
                DS01_WaveProcess();
                break;

            case DspMultiMeterPro_0:
                g_uiAdcAvgSample = ADC_GetSampleAvgN();
                break;

            /* 其它位暂未使用 */
            default:
                printf_taskdbg("xResult = %x\r\n", xResult);
                break;
        }
    }
}
```

除了信号处理，还有一个重要的功能要在启动任务里面优先实现，就是统计任务的创建和执行，用于统计 CPU 利用率，实现步骤如下：

1. 进入到启动任务后，其它任何任务都不要创建，先创建一个统计任务，不让执行。
2. 启动任务延迟 100ms，延迟的这 100ms 时间基本都是空闲任务在执行，在空闲任务里面做 32 位变量加 1 计算。我们就以这 100ms，变量计数的最大值作为 CPU 利用率的分母。
3. 然后开启统计任务的执行，每 100ms 执行一次，统计即可。空闲任务此时的计数值作为分子。通过这种方式就实现了 CPU 利用率的统计。

统计任务执行后就是任务间通信机制函数 AppObjCreate（动态内存分配也是在这个函数里面实现，在本章 13.4 小节有说明）和任务创建函数 AppTaskCreate，代码比较简单，我们这里就不贴出来了。

说完了前面这些，最重要的还是信号处理。根据不同的事件标志处理不同的功能，任务里面主要是分为了两类：

◆ 双通道波形数据处理

主要实现软件触发，计算 FFT，FIR，RMS，最大值，最小值，平均值和峰峰值。两个通道都进行了处理。具体实现方法已经在前面章节为大家做了讲解。

◆ 另一个是简单电压测量处理

这个功能比较简单，就是获取一组 ADC 数值，然后求平均。

13.2.3 统计任务

统计任务的实现代码如下：

```
/*
*****
*   函 数 名: AppTaskStatistic
*   功能说明: 统计任务，用于实现 CPU 利用率的统计。为了测试更加准确，可以开启注释调用的全局中断开关
*   形    参: 无
*   返 回 值: 无
*   优 先 级: 1 (数值越小优先级越低，这个跟 uCOS 相反)
*****
*/
void OSStatInit (void)
{
    OSStatRdy = FALSE;

    os_dly_wait(2u);          /* 时钟同步 */

    __disable_irq();
    OSIdleCtr = 0uL;          /* 清空闲计数 */
    __enable_irq();

    os_dly_wait(100);         /* 统计 100ms 内，最大空闲计数 */

    __disable_irq();
    OSIdleCtrMax = OSIdleCtr; /* 保存最大空闲计数 */
    OSStatRdy = TRUE;
    __enable_irq();
}

__task void AppTaskStatistic (void)
{
    while (OSStatRdy == FALSE)
    {
        os_dly_wait(200);     /* 等待统计任务就绪 */
    }

    OSIdleCtrMax /= 100uL;
    if (OSIdleCtrMax == 0uL)
    {
        OSCPUUsage = 0u;
    }
}
```

```
//__disable_irq();
OSIdleCtr = OSIdleCtrMax * 100uL; /* 设置初始 CPU 利用率 0% */
//__enable_irq();

for (;;)
{
    //__disable_irq();
    OSIdleCtrRun = OSIdleCtr; /* 获得 100ms 内空闲计数 */
    OSIdleCtr = 0uL; /* 复位空闲计数 */
    //__enable_irq(); /* 计算 100ms 内的 CPU 利用率 */
    OSCPUUsage = (100uL - (float)OSIdleCtrRun / OSIdleCtrMax);
    os_dly_wait(100); /* 每 100ms 统计一次 */
}
```

统计任务的实现思路就是前面 13.2.2 小节中介绍的方法。这个统计任务的实现思路是由 uCOS-II 修改而来的，如果大家研究过 uCOS-II 的源码，这里的代码还是比较好理解的。没有研究过也没有关系，直接根据 13.2.2 小节里面介绍的思路看统计任务的实现代码即可。

13.2.4 GUI 任务

emWin 任务的实现代码如下：

```
/*
*****
* 函数名: AppTaskGUI
* 功能说明: GUI 任务。
* 形参: 无
* 返回值: 无
* 优先级: 2
*****
*/
__task void AppTaskGUI(void)
{
    while(1)
    {
        MainTask();
    }
}
```

emWin 的代码都是在函数 MainTask 里面实现，这样做是方便在 main.c 文件里面统一管理任务。关于 GUI 部分最重要的界面优化，波形刷新优化，波形浏览等，在前面章节已经都做了讲解，我们这里不再赘述。更详细的实现，需要结合前面章节的讲解去看源码。

13.2.5 用户接口任务

这个任务暂时未执行任何功能，保留供以后升级使用。代码如下：

```
/*
*****
* 函数名: AppTaskUserIF
* 功能说明: 保留，暂未使用。
*****
*/
```

```
* 形 参: 无
* 返回值: 无
* 优先级: 3
*****
*/
__task void AppTaskUserIF(void)
{
    while(1)
    {
        os_dly_wait(2000);
    }
}
```

13.2.6 文件系统处理任务

当前文件系统处理任务主要用来做截图功能，将 GUI 界面以 BMP 格式存储到 SD 卡里面：

```
/*
*****
* 函 数 名: AppTaskFsPro
* 功能说明: 文件系统处理任务。
* 形 参: 无
* 返回值: 无
* 优先级: 4
*****
*/
__task void AppTaskFsPro(void)
{
    OS_RESULT xResult;
    const uint16_t usMaxBlockTime = 0xFFFF;
    uint8_t Pic_Name = 0;
    char buf[40];

    while(1)
    {
        xResult = os_sem_wait (&semaphore, usMaxBlockTime);

        switch (xResult)
        {
            /* 无需等待接受到信号量同步信号 */
            case OS_R_OK:

                /* 信号量不可用，usMaxBlockTime 等待时间内收到信号量同步信号 */
                case OS_R_SEM:
                    sprintf((char *)buf, "M0:\\PicSave\\%d.bmp", Pic_Name);

                    foutbmp = fopen (buf, "w");
                    if (foutbmp != NULL)
                    {
                        /* 向 SD 卡绘制 BMP 图片 */
                        GUI_BMP_Serialize(_WriteByte2File, foutbmp);

                        /* 关闭文件 */
                        fclose(foutbmp);
                    }
                    printf_taskdbg("截图完成\r\n");
                }
            }
}
```

```
Pic_Name++;  
break;  
  
/* 超时 */  
case OS_R_TM0:  
break;  
  
/* 其他值不处理 */  
default:  
break;  
}  
}  
}
```

后期这个任务将被升级，用于将波形数据以 CSV 文件格式存储到 SD 卡里面。

13.2.7 触摸和按键任务

触摸和按键任务实现的功能比较简单，主要是按键扫描和触摸扫描：

```
/*  
*****  
* 函数名: AppTaskMsgPro  
* 功能说明: 按键和触摸检测  
* 形参: 无  
* 返回值: 无  
* 优先级: 5  
*****  
*/  
__task void AppTaskMsgPro(void)  
{  
    uint8_t ucCount = 0;  
  
    while(1)  
    {  
        /* 1ms 一次触摸扫描，电阻触摸屏 */  
        if(g_tTP.Enable == 1)  
        {  
            TOUCH_Scan();  
  
            /* 按键扫描 */  
            ucCount++;  
            if(ucCount == 10)  
            {  
                ucCount = 0;  
                bsp_KeyScan();  
            }  
            os_dly_wait(1);  
        }  
  
        /* 10ms 一次触摸扫描，电容触摸屏 GT811 */  
        if(g_GT811.Enable == 1)  
        {  
            bsp_KeyScan();  
            GT811_OnePiontScan();  
        }  
    }  
}
```

```
        os_dly_wait(10);
    }

    /* 10ms 一次触摸扫描，电容触摸屏 FT5X06 */
    if(g_tFT5X06.Enable == 1)
    {
        bsp_KeyScan();
        FT5X06_OnePiontScan();
        os_dly_wait(10);
    }
}
```

- 知识点拓展

新版 emWin 教程第 4 章或者第 5 章，对触摸的实现做了详细讲解：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834> 。

13.3 用户任务优先级设置

当前任务的优先级安排如下（数值越小，优先级越低）：

- ◆ AppTaskStatistic 任务 ：优先级 1。

统计任务的优先级最低，这个毫无争议，因为它要统计 CPU 利用率。

- ◆ AppTaskGUI 任务 ：优先级 2。

emWin 任务是除了空闲任务，统计任务以外最低优先级的，因为 emWin 极其占用系统资源，而且时间长，如果这个任务设置为高优先级，会直接影响低优先级任务的执行。

- ◆ AppTaskUserIF 任务 ：优先级 3。

保留，未使用任务，暂且安排为这个优先级。

- ◆ AppTaskFsPro 任务 ：优先级 4。

AppTaskMsgPro 任务 ：优先级 5。

触摸和按键任务，以及文件系统任务的优先级谁高谁低都没有关系。

- ◆ AppTaskStart 任务 ：优先级 6。

DSP 任务一定要是优先级最高的，因为采集的数据要实时处理。

- 知识点拓展

关于任务优先级的安排，在我们 RTX 操作系统教程第 8 章的 8.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。

在我们 FreeRTOS 操作系统教程的第 13 章的 13.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

13.4 全局变量分配，系统堆栈和任务堆栈

◆ 全局变量分配

示波器的设计需要很多变量进行逻辑管理，从设计之初就需要将变量分类进行结构体封装，方便以后的维护升级。这一步至关重要，实际中差不多要定义上百个变量，如果不进行分类管理，以后的升级维护将非常麻烦。

这种方式还有一个好处是方便我们将 F429 的 CCM RAM 空间分配给这些变量使用。使用 CCM RAM 的好处是速度比通用 RAM 要快些，缺点是这部分空间不支持 DMA 操作。初次使用的用户比较容易在这个地方犯错误。所以在使用局部变量时，切勿将局部变量用于 DMA 传输。

当前需要频繁调用的变量已经通过动态内存管理分配给各个结构体变量，使用的 CCM RAM 空间。

```
uint64_t AppMallocCCM[40*1024/8] __attribute__((at(0x10000000 + 1024*24))); /* 数字信号处理 */

/*
*****
*   函 数 名: AppObjCreate
*   功能说明: 创建任务通信机制和动态内存分配
*   形    参: 无
*   返 回 值: 无
*****
*/
static void AppObjCreate (void)
{
    /* 创建信号量计数值是 0，用于任务同步 */
    os_sem_init (&semaphore, 0);

    /* 将内部 CCM SRAM 的 40KB 全部供动态内存使用 */
    os_init_mem(AppMallocCCM, 1024*40);

    /* 申请示波器通道 1 动态内存 */
    g_DS01 = (DSO_T *)os_alloc_mem(AppMallocCCM, sizeof(DSO_T));

    /* 申请示波器通道 2 动态内存 */
    g_DS02 = (DSO_T *)os_alloc_mem(AppMallocCCM, sizeof(DSO_T));

    /* 申请游标测量结构体变量动态内存 */
    g_Cursors = (CURSORS_T *)os_alloc_mem(AppMallocCCM, sizeof(CURSORS_T));

    /* 申请标志位结构体变量动态内存 */
    g_Flag = (FLAG_T *)os_alloc_mem(AppMallocCCM, sizeof(FLAG_T));

    /* 申请触发结构体变量动态内存 */
    g_TrigVol = (TRIVOLTAGE_T *)os_alloc_mem(AppMallocCCM, sizeof(TRIVOLTAGE_T));

    /* 申请 FFT 动态内存 */
    testInput_fft_2048 = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*2048);
    testOutput_fft_2048 = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*2048);

    /* 申请 RMS 动态内存 */
    g_RMSBUF = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*600);
}
```

```
/* 申请 FIR 动态内存 */
FirDataInput = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_LENGTH_SAMPLES);
FirDataOutput = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_LENGTH_SAMPLES);
firStateF32 = (float32_t *)os_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_StateBufSize);
}
```

◆ 任务栈分配

任务栈也是用的 CCM RAM 空间，具体分配如下：

```
/*
*****
                        任务栈和任务句柄
*****
*/
static uint64_t AppTaskStatStk[1024/8] __attribute__((at(0x10000000))); /* 任务栈 */
static uint64_t AppTaskGUIStk[4096/8] __attribute__((at(0x10000000+1024))); /* 任务栈 */
static uint64_t AppTaskUserIFStk[1024/8] __attribute__((at(0x10000000+1024*5))); /* 任务栈 */
static uint64_t AppTaskFsProStk[1024/8] __attribute__((at(0x10000000+1024*6))); /* 任务栈 */
static uint64_t AppTaskMsgProStk[1024/8] __attribute__((at(0x10000000+1024*7))); /* 任务栈 */
static uint64_t AppTaskStartStk[4096/8] __attribute__((at(0x10000000+1024*8))); /* 任务栈 */
```

将任务栈定义成 uint64_t 类型可以保证任务栈是 8 字节对齐的，8 字节对齐的含义就是数组的首地址对 8 求余等于 0。如果不做 8 字节对齐的话，部分 C 语言库函数、浮点运算和 uint64_t 类型数据运算会出问题。

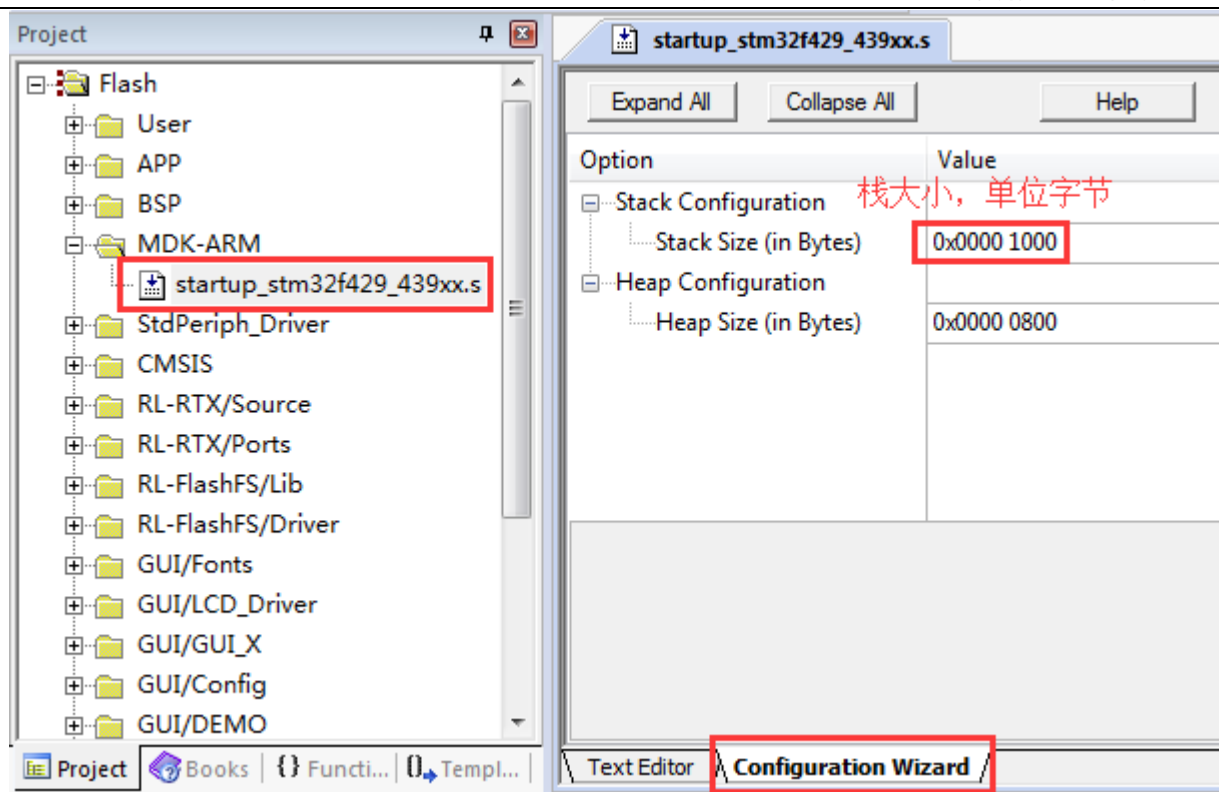
● 知识点拓展

关于任务栈大小应该分配多大的问题，可以看 FreeRTOS 教程第 11 章，对于 RTX 系统也是适用的。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

◆ 系统栈分配

系统栈分配的大小如下：



13.5 任务间通信和全局变量共享问题

二代示波器的双通道ADC通过DMA方式在实时的采集数据,每个通道的缓冲大小是1024*20字节,采集的数据经过信号处理后送给GUI任务进行波形显示和测量值显示。为了实现这个功能,专门测试了两种方案。

◆ 方案一

采用DMA双缓冲,一路缓冲采集波形的时候,另一路已经采集的波形数据发给数字信号处理任务,信号处理任务再将整理好的波形数据和测量值发给emWin任务做刷新。这种方式的优点是ADC采集的数据可以实时处理。缺点是F429处理不过来,比如我们一个通道的采样率是2Msps,缓冲大小设置为2048,将缓冲填满需要1ms左右的时间,而我们仅做一个2048点的实数FFT就需要0.862ms,其它的FIR, RMS等都还没有做,而且已经没有时间发消息给emWin任务做界面刷新了。如果我们降低FFT, FIR等信号处理的点数,也就失去了实时处理的意义。也许读者会说,加大缓冲不就好了,其实不然。如果我们加大了缓冲,我们要处理的数据也增加了,还是处理不过来,而且我们现在要处理的是双通道。

除了F429的性能问题,这种方式还有一个比较棘手的问题需要解决,就是用户操作界面的时候,GUI任务基本已经没有时间去处理数字信号处理任务发来的数据,为了解决这个问题,大大增加了软件设计的复杂度,特别是波形暂停和运行的切换,窗口的切换以及其它操作时,都要注意这个问题。

如果没有复杂的界面操作,而且采样率较低的话,方案一还是比较合适的。由于我们需要滑动操作波形,而且要实现双通道,每个通道最高采样率是2.8Msps,所以放弃这种方案。

◆ 方案二

与方案一恰恰相反,ADC 数据依然是通过 DMA 方式实时采集,而任务间的通信反过来进行,emWin 任务需要波形数据刷新时给数字信号处理任务发消息获取,这样就有效地解决了方案一中 F429 性能不够的问题,而且方案一中棘手的软件问题得到了很好的解决,随时都可以操作界面。

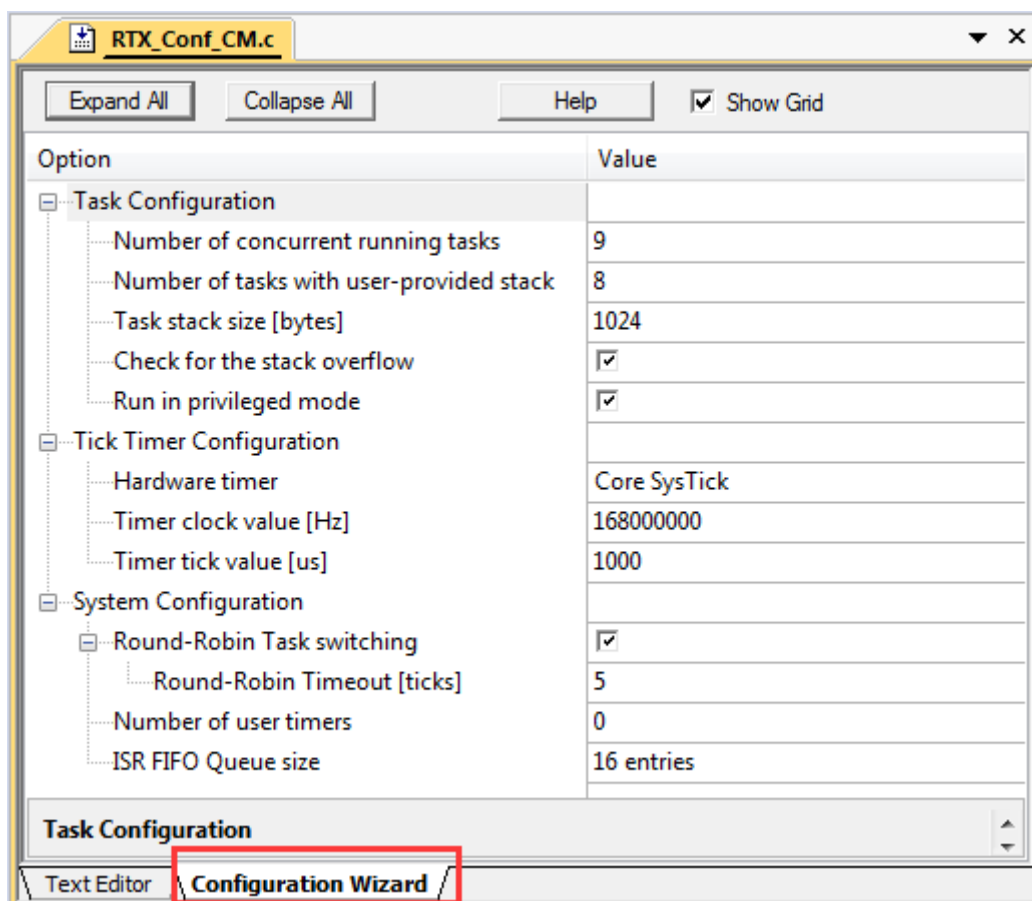
并且这种方式无形中解决了 emWin 任务和数字信号处理任务之间共同操作全局变量的问题,因为 emWin 是低优先级任务,而数字信号处理任务在 emWin 任务发消息后才会执行,这样就不存在抢占问题了,有效地解决了全局变量共享问题。

但是这种方式也有一个缺陷,无法实时刷新波形和测量值了,不过可以通过普通触发来解决了,普通触发方式实时采集了触发值前后各 1024 字节的数据,并且可以滑动浏览。不过工程中未对这种方式做 FFT 和 FIR 的支持。

总结,二代示波器中最终选择了方案二。

13.6 RTX 配置向导

RTX 配置向导详情如下:



◆ Task Configuration

- Number of concurrent running tasks

允许创建 9 个任务,实际创建了如下 6 个任务:

AppTaskStatistic 任务:统计任务,获取 CPU 利用率。

- AppTaskGUI 任务 : emWin 任务。
- AppTaskUserIF 任务 : 保留, 暂未使用。
- AppTaskFsPro 任务 : 文件系统任务。
- AppTaskMsgPro 任务 : 按键和触摸检测。
- AppTaskStart 任务 : 启动任务, 也是最高优先级任务, 用于信号处理。
- Number of tasks with user-provided stack
8 个任务可以采用自定义堆栈方式。
 - Run in privileged mode
设置任务运行在非特权级模式。

13.7 RTX 系统调试

MDK 自带 RTX 调试组件, 展示系统信息非常方便, 本工程的展示效果如下:

System and Thread Viewer

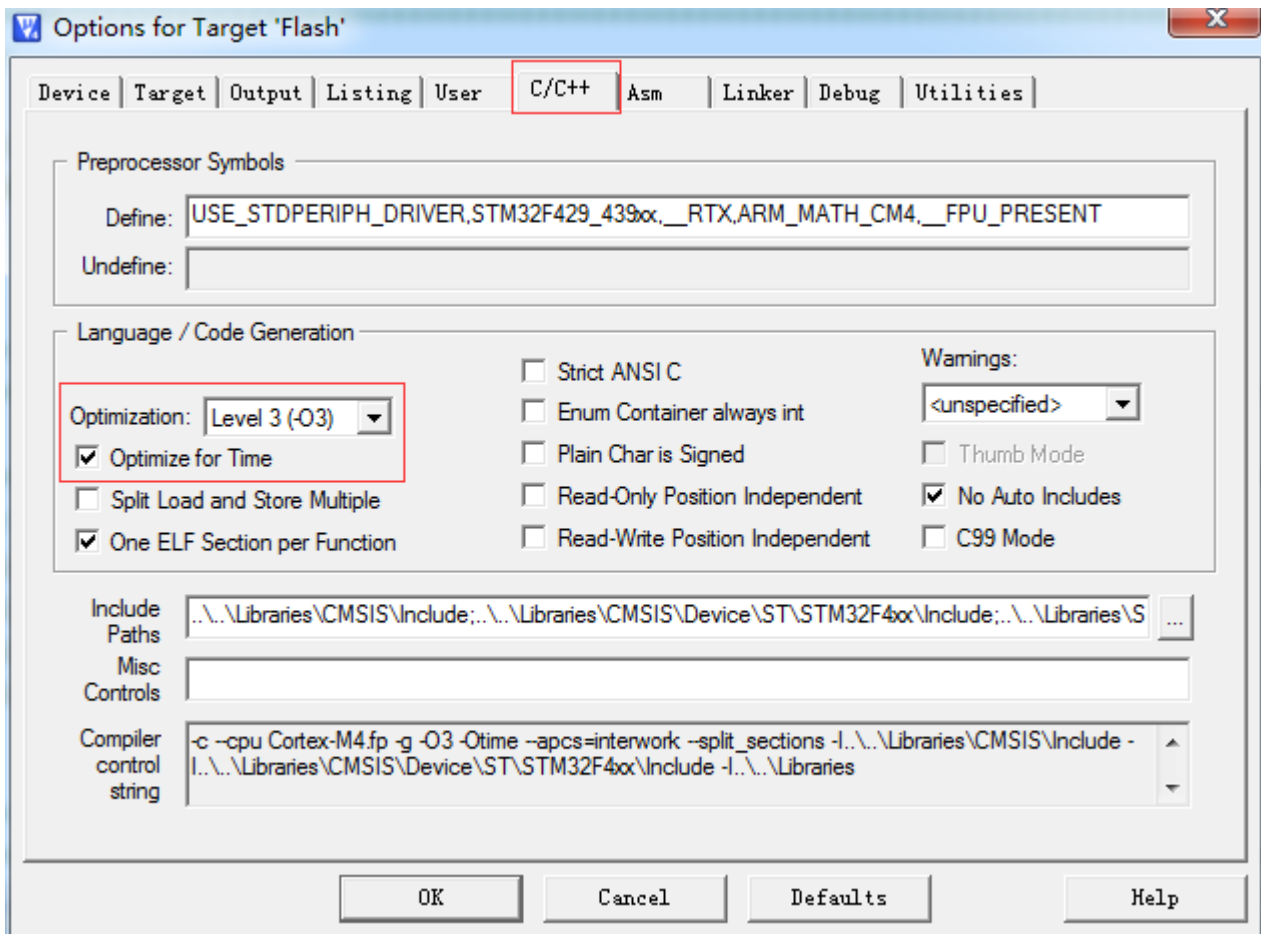
Property	Value							
System	Item	Value						
	Timer Number:	0						
	Tick Timer:	1.000 mSec						
	Round Robin Timeout:	5.000 mSec						
	Stack Size:	1024						
	Tasks with User-provided Stack:	8						
	Stack Overflow Check:	Yes						
	Task Usage:	Available: 9, Used: 6						
	User Timers:	Available: 0, Used: 0						
Tasks	ID	Name	Priority	State	Delay	Event Value	Event Mask	Stack Load
	255	os_idle_demon	0	Ready				6%
	6	AppTaskMsgPro	5	Wait_DLY	7			6%
	5	AppTaskFsPro	4	Wait_SEM				10%
	4	AppTaskUserIF	3	Wait_DLY	1758			6%
	3	AppTaskGUI	2	Running	3			6%
	2	AppTaskStatistic	1	Wait_DLY	7			20%
	1	AppTaskStart	6	Wait_OR		0x0000	0x8001	4%

调试组件的使用方法请看 F429 的 RTX 教程第 3 章 3.4 小节, 有详细说明:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。

13.8 MDK 优化等级

为了发挥 STM32F429 的最高性能，需要大家开启最高等级优化和时间优化，即下面两个选项：



- 知识点拓展

MDK 曾经做的专题：如何实现 MDK 编译器的代码最小优化和性能最佳优化。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=1794>。

13.9 总结

RTX 系统设计二代示波器的关键问题在本章节都做了阐释，建议大家学习完本章节后，直接看源码做实战演练，这样理解的更透彻，而且这时再做改进拓展也容易些。

第14章 uCOS-III 操作系统版本二代示波器实现

本章教程为大家讲解 uCOS-III 操作系统版本的二代示波器实现。主要讲解 RTOS 设计框架，即各个任务实现的功能，任务间的通信方案选择，任务栈，系统栈以及全局变量共享问题。同时，工程调试方法也专门做了说明。

- 14.1 注意事项（重要必读）
- 14.2 任务功能划分
- 14.3 用户任务优先级设置
- 14.4 全局变量分配，系统堆栈和任务堆栈
- 14.5 任务间通信和全局变量共享问题
- 14.6 uCOS-III 系统调试
- 14.7 MDK 优化等级
- 14.8 总结

14.1 注意事项（重要必读）

- ◆ 学习本章节前，务必保证已经学习完毕前面章节。另外，工程代码注释已经比较详细，了解了框架后，**直接看源码即可**。
- ◆ 仅支持 800*480 分辨率显示屏，如果是电容屏，无需校准。如果是电阻屏，需要校准，按下按键 K1 即可进入校准界面。
- ◆ 由于按键不够用，K1 按键的消息处理做了三个条件编译，详情见本章 14.6 小节。默认 K1 按键执行触摸校准，也可以选择执行截图或者串口打印任务执行情况。另外，不管当前处于任何界面都可以进行触摸校准，仅电阻屏需要校准，电容屏无需校准。
- ◆ STemWin5.40 版本的截图功能有 bug，详情看此贴：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=82445>。
当前用的 5.32 版本，也是来自 STemWin 软件包。
- ◆ Micrium 官方曾经发布过一个非常棒的文档，如何发挥 uCOS-III 最高性能之重要提示和项目应用建议，推荐大家看看：<http://forum.armfly.com/forum.php?mod=viewthread&tid=31634>。
- ◆ uCOS-III 工程的文件系统是采用的 FatFS，当前开启了 MDK 最高等级优化和时间优化。如果大家要使用 FatFS 功能，请务必关闭时间优化，因为 FatFS 在时间优化下会工作异常。详情见本章 14.7 小节。

- ◆ 工程编译支持 MDK4.7X 和 MDK5。另外不支持 MDK 发布的 MDK5.24 及其以上版本，因为这个版本不支持 MDK4 创建的工程转换为 MDK5 了，所以要使用这个最新的版本，需要给 MDK5 安装 MDK4 的兼容包。

14.2 任务功能划分

前面第三章已经将任务功能划分好：



根据这个功能划分，创建所需要的任务。

14.2.1 主函数创建

在 main.c 文件实现：

```

/*
*****
*   函 数 名: main
*   功能说明: 标准 c 程序入口。
*   形    参: 无
*   返 回 值: 无
*****
*/
int main(void)
{
    OS_ERR  err;

    /* 动态内存分配 */
    MallocInit();

    /* 初始化 uC/OS-III 内核 */
    OSInit(&err);

    /* 创建一个启动任务（也就是主任务）。启动任务会创建所有的应用程序任务 */
    OSTaskCreate((OS_TCB *)AppTaskStartTCB, /* 任务控制块地址 */
                 (CPU_CHAR *)"App Task Start", /* 任务名 */
                 (OS_TASK_PTR) AppTaskStart, /* 启动任务函数地址 */
                 (void *)0, /* 传递给任务的参数 */
                 (OS_PRIO) APP_CFG_TASK_START_PRIO, /* 任务优先级 */
                 (CPU_STK) &AppTaskStartStk[0], /* 堆栈基地址 */
                 0);
}

```

```
(CPU_STK_SIZE ) APP_CFG_TASK_START_STK_SIZE / 10, /* 堆栈监测区, 这里表示后 10%作为监测区 */
(CPU_STK_SIZE ) APP_CFG_TASK_START_STK_SIZE, /* 堆栈空间大小 */
(OS_MSG_QTY ) 0, /* 本任务支持接受的最大消息数 */
(OS_TICK ) 0, /* 设置时间片 */
(void ) 0, /* 堆栈空间大小 */
(OS_OPT ) (OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),

/* 定义如下:
OS_OPT_TASK_STK_CHK 使能检测任务栈, 统计任务栈已用的和未用的
OS_OPT_TASK_STK_CLR 在创建任务时, 清零任务栈
OS_OPT_TASK_SAVE_FP 如果 CPU 有浮点寄存器, 则在任务切换时保存浮点寄存器的内容
*/
(OS_ERR ) (&err);

/* 启动多任务系统, 控制权交给 uC/OS-III */
OSStart(&err);

(void)&err;

return (0);
}
```

在主函数中, 首先是初始化任务所需的栈空间以及部分全局变量所需的空间, 通过函数 MallocInit 实现(详情看本章节 14.4 小节)。然后就是创建一个启动任务, 其它外设初始化, 任务创建, 任务消息创建等, 都是在启动任务里面实现。

14.2.2 启动任务 (触摸和按键)

启动任务实现的功能比较简单, 主要是按键扫描和触摸扫描:

```
/*
*****
* 函数名: AppTaskStart
* 功能说明: 这是一个启动任务, 在多任务系统启动后, 必须初始化滴答计数器。本任务主要实现按键和触摸检测。
* 形参: p_arg 是在创建该任务时传递的形参
* 返回值: 无
* 优先级: 3
*****
*/
static void AppTaskStart (void *p_arg)
{
    OS_ERR      err;
    uint8_t ucCount = 0;

    /* 仅用于避免编译器告警, 编译器不会产生任何目标代码 */
    (void)p_arg;

    /* BSP 初始化。BSP = Board Support Package 板级支持包, 可以理解为底层驱动。*/
    CPU_Init(); /* 此函数要优先调用, 因为外设驱动中使用的 us 和 ms 延迟是基于此函数的 */
    bsp_Init();
    BSP_Tick_Init();

    #if OS_CFG_STAT_TASK_EN > 0u
        OSStatTaskCPUUsageInit(&err);
    #endif
}
```

```
#ifdef CPU_CFG_INT_DIS_MEAS_EN
    CPU_IntDisMeasMaxCurReset();
#endif

/* 创建任务通信 */
AppObjCreate();

/* 创建应用程序的任务 */
AppTaskCreate();

while(1)
{
    /* 1ms 一次触摸扫描，电阻触摸屏 */
    if(g_tTP.Enable == 1)
    {
        TOUCH_Scan();

        /* 按键扫描 */
        ucCount++;
        if(ucCount == 10)
        {
            ucCount = 0;
            bsp_KeyScan();
        }
        OSTimeDly(1, OS_OPT_TIME_DLY, &err);
    }

    /* 10ms 一次触摸扫描，电容触摸屏 GT811 */
    if(g_GT811.Enable == 1)
    {
        bsp_KeyScan();
        GT811_OnePiontScan();
        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }

    /* 10ms 一次触摸扫描，电容触摸屏 FT5X06 */
    if(g_tFT5X06.Enable == 1)
    {
        bsp_KeyScan();
        FT5X06_OnePiontScan();
        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}
```

在启动任务里，首先做了硬件外设初始化，任务间通信创建和任务创建。然后是触摸和按键扫描。

- 知识点拓展

新版 emWin 教程第 4 章或者第 5 章，对触摸的实现做了详细讲解：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834> 。

硬件外设的初始化函数 bsp_Init 是在 bsp.c 文件实现：

```
/*
*****
* 函数名: bsp_Init
```

```
* 功能说明：初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
* 形 参：无
* 返 回 值：无
*****
*/
void bsp_Init(void)
{
    /*
    由于 ST 固件库的启动文件已经执行了 CPU 系统时钟的初始化，所以不必再次重复配置系统时钟。
    启动文件配置了 CPU 主时钟频率、内部 Flash 访问速度和可选的外部 SRAM FSMC 初始化。

    系统时钟缺省配置为 168MHz，如果需要更改，可以修改 system_stm32f4xx.c 文件
    */
    /* 使能 CRC 因为使用 STemWin 前必须要使能 */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_CRC, ENABLE);

    /* 优先级分组设置为 4，可配置 0-15 级抢占式优先级，0 级子优先级，即不存在子优先级。*/
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);

    SystemCoreClockUpdate(); /* 根据 PLL 配置更新系统时钟频率变量 SystemCoreClock */

    bsp_InitUart(); /* 初始化串口 */
    bsp_InitKey(); /* 初始化按键变量（必须在 bsp_InitTimer() 之前调用） */

    bsp_InitI2C(); /* 配置 I2C 总线 */

    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
    bsp_DetectLcdType(); /* 检测触摸板和 LCD 面板型号，结果存在全局变量 g_TouchType, g_LcdType */

    TOUCH_InitHard(); /* 初始化配置触摸芯片 */
    LCD_ConfigLTDC(); /* 初始化配置 LTDC */

    DSO_ConfigCtrlGPIO(); /* 初始化示波器模块的引脚配置 */

    bsp_InitADC(); /* 初始化 ADC1, ADC2 和 ADC3 */
    bsp_InitDAC1(); /* 初始化 DAC1 */

    g_DAC1.ucDuty = 50; /* 初始化 DAC 配置，用于信号发生器 */
    g_DAC1.ucWaveType = 0;
    g_DAC1.ulAMP = 4095;
    g_DAC1.ulFreq = 10000;
    dac1_SetSinWave(g_DAC1.ulAMP, g_DAC1.ulFreq);

    MountSD(); /* 挂载 SD 卡 */

    TIM8_MeasureTrigConfig(); /* 初始化 TIM8 用于记录一段波形 */
}
```

任务间的通信创建如下：

```
/*
*****
* 函 数 名：AppObjCreate
* 功能说明：创建任务通讯
* 形 参：p_arg 是在创建该任务时传递的形参
* 返 回 值：无
*****
*/
static void AppObjCreate (void)
```

```
{
    OS_ERR      err;

    /*
     * 创建信号量数值为 1 的时候可以实现互斥功能，也就是只有一个资源可以使用
     * 本例程是将串口 1 的打印函数作为保护的资源。防止串口打印的时候被其它任务抢占
     * 造成串口打印错乱。
     */
    OSSemCreate((OS_SEM      *)&AppPrintfSemp,
                (CPU_CHAR    *)"AppPrintfSemp",
                (OS_SEM_CTR  )1,
                (OS_ERR      *)&err);

    /* 创建计数值为 0，用于实现任务同步功能 */
    OSSemCreate((OS_SEM      *)&SEM_SYNCH,
                (CPU_CHAR    *)"SEM_SYNCH",
                (OS_SEM_CTR  )0,
                (OS_ERR      *)&err);
}
```

任务创建如下：

```
/*
*****
* 函数名: AppTaskCreate
* 功能说明: 创建应用任务
* 形参: p_arg 是在创建该任务时传递的形参
* 返回值: 无
*****
*/
static void AppTaskCreate (void)
{
    OS_ERR      err;

    /******创建 MsgPro 任务******/
    OSTaskCreate((OS_TCB      *)&AppTaskMsgProTCB,
                (CPU_CHAR      *)"App Task MsgPro",
                (OS_TASK_PTR   )AppTaskMsgPro,
                (void          *)0,
                (OS_PRIO       )APP_CFG_TASK_MsgPro_PRIO,
                (CPU_STK        *)&AppTaskMsgProStk[0],
                (CPU_STK_SIZE   )APP_CFG_TASK_MsgPro_STK_SIZE / 10,
                (CPU_STK_SIZE   )APP_CFG_TASK_MsgPro_STK_SIZE,
                (OS_MSG_QTY     )0,
                (OS_TICK        )0,
                (void          *)0,
                (OS_OPT         )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR | OS_OPT_TASK_SAVE_FP),
                (OS_ERR         *)&err);

    /******创建 USER IF 任务******/
    OSTaskCreate((OS_TCB      *)&AppTaskUserIFTCB,
                (CPU_CHAR      *)"App Task UserIF",
                (OS_TASK_PTR   )AppTaskUserIF,
                (void          *)0,
                (OS_PRIO       )APP_CFG_TASK_USER_IF_PRIO,
                (CPU_STK        *)&AppTaskUserIFStk[0],
                (CPU_STK_SIZE   )APP_CFG_TASK_USER_IF_STK_SIZE / 10,
                (CPU_STK_SIZE   )APP_CFG_TASK_USER_IF_STK_SIZE,
                (OS_MSG_QTY     )0,
```

```

(OS_TICK      )0,
(void         *)0,
(OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
(OS_ERR       *)&err);

/*****创建 GUI 任务*****/
OSTaskCreate((OS_TCB      *)AppTaskGUITCB,
              (CPU_CHAR    *)"App Task GUI",
              (OS_TASK_PTR )AppTaskGUI,
              (void        *)0,
              (OS_PRIO     )APP_CFG_TASK_GUI_PRIO,
              (CPU_STK     *)&AppTaskGUIStk[0],
              (CPU_STK_SIZE)APP_CFG_TASK_GUI_STK_SIZE / 10,
              (CPU_STK_SIZE)APP_CFG_TASK_GUI_STK_SIZE,
              (OS_MSG_QTY  )0,
              (OS_TICK     )0,
              (void        *)0,
              (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR | OS_OPT_TASK_SAVE_FP),
              (OS_ERR       *)&err);

/*****创建 DSO 任务*****/
OSTaskCreate((OS_TCB      *)AppTaskDsoTCB,
              (CPU_CHAR    *)"App Task DSO",
              (OS_TASK_PTR )AppTaskDSO,
              (void        *)0,
              (OS_PRIO     )APP_CFG_TASK_DSO_PRIO,
              (CPU_STK     *)&AppTaskDsoStk[0],
              (CPU_STK_SIZE)APP_CFG_TASK_DSO_STK_SIZE / 10,
              (CPU_STK_SIZE)APP_CFG_TASK_DSO_STK_SIZE,
              (OS_MSG_QTY  )5,
              (OS_TICK     )0,
              (void        *)0,
              (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR | OS_OPT_TASK_SAVE_FP),
              (OS_ERR       *)&err);
}

```

14.2.3 信号处理任务

信号处理任务的实现如下：

```

/*
*****
* 函数名: AppTaskDSO
* 功能说明: 双通道示波器数据处理任务。
* 形参: 无
* 返回值: 无
* 优先级: 2
*****
*/
static void AppTaskDSO (void *p_arg)
{
    OS_ERR      err;
    CPU_TS      ts;
    void        *p_msg;
    OS_MSG_SIZE  msg_size;
    uint32_t    *ucReceive,

```

```
/* 实数序列 FFT 长度 */
fftSize = 2048;

/* 正变换 */
ifftFlag = 0;

/* 初始化结构体 S 中的参数 */
arm_rfft_fast_init_f32(&S, fftSize);

while(1)
{
    /* 接受数据 */
    p_msg = OSTaskQPend(0,
                        OS_OPT_PEND_BLOCKING,
                        &msg_size, /* 此参数是接收到数据个数 */
                        &ts,
                        &err);

    if(err == OS_ERR_NONE)
    {
        ucReceive = (uint32_t *)p_msg;

        switch (*ucReceive)
        {
            /* 双通道波形处理 */
            case DspFFT2048Pro_15:
                /* 读取的是 ADC3 的位置 */
                g_DS01->usCurPos = 10240 - DMA2_Stream1->NDTR;

                /* 读取的是 ADC1 的位置 */
                g_DS02->usCurPos = 10240 - DMA2_Stream0->NDTR;

                DS02_WaveTrig(g_DS02->usCurPos);
                DS01_WaveTrig(g_DS01->usCurPos);
                DS02_WaveProcess();
                DS01_WaveProcess();
                break;

            /* 用于简单的 ADC 数据采集 */
            case DspMultiMeterPro_14:
                g_uiAdcAvgSample = ADC_GetSampleAvgN();
                break;

            /* 仅用于调试目的，打印任务的执行情况，默认不使用 */
            case DspTaskInfo_13:
                DispTaskInfo();
                break;

            /* 其它位暂未使用 */
            default:
                App_Printf("ucReceive = %x\r\n", *ucReceive);
                break;
        }
    }
}
```


根据接收到的不同任务消息来处理不同的功能，要处理的消息分为三类：

◆ 双通道波形数据处理

主要实现软件触发，计算 FFT，FIR，RMS，最大值，最小值，平均值和峰峰值。两个通道都进行了处理。具体实现方法已经在前面章节为大家做了讲解。

◆ 简单电压测量处理

这个功能比较简单，就是获取一组 ADC 数值，然后求平均。

◆ 打印任务执行情况

通过串口打印任务栈的使用情况和各个任务的 CPU 利用率。

14.2.4 GUI 任务

emWin 任务的实现代码如下：

```
/*
*****
*   函数名: AppTaskGUI
*   功能说明: GUI 任务，最低优先级
*   形参: p_arg 是在创建该任务时传递的形参
*   返回值: 无
*   优先级: OS_CFG_PRIO_MAX - 4u
*****
*/
static void AppTaskGUI(void *p_arg)
{
    (void)p_arg;      /* 避免编译器告警 */

    while (1)
    {
        MainTask();
    }
}
```

emWin 的代码都是在函数 MainTask 里面实现，这样做是方便在 main.c 文件里面统一管理任务。关于 GUI 部分最重要的界面优化，波形刷新优化，波形浏览等，在前面章节已经都做了讲解，我们这里不再赘述。更详细的实现，需要结合前面章节的讲解去看源码。

14.2.5 用户接口任务

这个任务暂时未执行任何功能，保留供以后升级使用。代码如下：

```
/*
*****
*   函数名: AppTaskUserIF
*   功能说明: 保留，暂未使用。
*   形参: p_arg 是在创建该任务时传递的形参
*   返回值: 无
*   优先级: 5
*****
*/
```

```
*/
static void AppTaskUserIF(void *p_arg)
{
    OS_ERR      err;

    (void)p_arg;          /* 避免编译器报警 */

    while (1)
    {
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
    }
}
```

14.2.6 文件系统处理任务

当前文件系统处理任务主要用来做截图功能，将 GUI 界面以 BMP 格式存储到 SD 卡里面：

```
/*
*****
*   函 数 名: AppTaskMsgPro
*   功能说明: 实现截图功能，将图片以 BMP 格式保存到 SD 卡中
*   形    参: p_arg 是在创建该任务时传递的形参
*   返 回 值: 无
*   优 先 级: 4
*****
*/
static void AppTaskMsgPro(void *p_arg)
{
    uint32_t ulStart, ulEnd;
    OS_ERR      err;
    uint8_t      Pic_Name = 0;
    char buf[20];

    (void)p_arg;

    while(1)
    {
        /* 等待获取信号量同步消息，接收到后实现截图功能，将图片以 BMP 格式保存到 SD 卡中 */
        OSSemPend((OS_SEM *)&SEM_SYNCH,
                  (OS_TICK )0,
                  (OS_OPT )OS_OPT_PEND_BLOCKING,
                  (CPU_TS  )0,
                  (OS_ERR *)&err);

        if(err == OS_ERR_NONE)
        {
            sprintf(buf, "0:/PicSave/%d.bmp", Pic_Name);

            /* 记录截图前起始时间 */
            ulStart = OSTimeGet(&err);

            /* 开启调度锁 */
            //OSSchedLock(&err);

            /* 如果 SD 卡中没有 PicSave 文件，会进行创建 */

```

```
result = f_mkdir("0:/PicSave");

/* 创建截图 */
result = f_open(&file, buf, FA_WRITE|FA_CREATE_ALWAYS);

/* 向 SD 卡绘制 BMP 图片 */
GUI_BMP_Serialize(_WriteByte2File, &file);

/* 创建完成后关闭 file */
result = f_close(&file);

/* 开启调度锁 */
//OSSchedUnlock(&err);

/* 记录截图后时间并获取截图过程耗时 */
ulEnd = OSTimeGet(&err);
ulEnd -= ulStart;

App_Printf("截图完成, 耗时 = %dms\r\n", ulEnd);
Pic_Name++;
}
}
}
```

后期这个任务将被升级，用于将波形数据以 CSV 文件格式存储到 SD 卡里面。

14.3 用户任务优先级设置

当前任务的优先级安排如下（数值越小，优先级越高）：

- ◆ App Task DSO 任务：优先级 2。

DSP 任务一定要是优先级最高的，因为采集的数据要实时处理。

- ◆ App Task Start 任务：优先级 3。

App Task MsgPro 任务：优先级 4。

启动任务（触摸和按键扫描）以及 MsgPro（文件系统处理）任务的优先级谁高谁低都没有关系。

- ◆ App Task UserIF 任务：优先级 5。

保留，未使用任务，暂且安排为这个优先级。

- ◆ App Task GUI 任务：优先级 $OS_CFG_PRIO_MAX - 4u$ ，即 $32 - 4 = 28$

emWin 任务是除了空闲任务，统计任务以外最低优先级的，因为 emWin 极其占用系统资源，而且时间长，如果这个任务设置为高优先级，会直接影响低优先级任务的执行。

- 知识点拓展

关于任务优先级的安排，在我们 RTX 操作系统教程第 8 章的 8.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。

在我们 FreeRTOS 操作系统教程的第 13 章的 13.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

14.4 全局变量分配，系统堆栈和任务堆栈

◆ 全局变量分配

示波器的设计需要很多变量进行逻辑管理，从设计之初就需要将变量分类进行结构体封装，方便以后的维护升级。这一步至关重要，实际中差不多要定义上百个变量，如果不进行分类管理，以后的升级维护将非常麻烦。

这种方式还有一个好处是方便我们将 F429 的 CCM RAM 空间分配给这些变量使用。使用 CCM RAM 的好处是速度比通用 RAM 要快些，缺点是这部分空间不支持 DMA 操作。初次使用的用户比较容易在这个地方犯错误。所以在使用局部变量时，切勿将局部变量用于 DMA 传输。

当前需要频繁调用的变量已经通过动态内存管理分配给各个结构体变量，使用的 CCM RAM 空间。

```
uint64_t AppMallocCCM[64*1024/8] __attribute__((at(0x10000000))); /* CCM RAM 动态内存池 */

/*
*****
* 函数名: MallocInit
* 功能说明: 用户任务栈，用户 TCB(任务控制块)和示波器结构体变量使用 CCM RAM
* 形参: 无
* 返回值: 无
*****
*/
static void MallocInit(void)
{
    /* 将内部 CCM SRAM 的 64KB 全部供动态内存使用 */
    rt_init_mem(AppMallocCCM, 1024*64);

    /* 任务栈和任务控制块 TCB 的初始化，省略未写 */

    /* 申请示波器通道 1 动态内存 */
    g_DS01 = (DSO_T *)rt_alloc_mem(AppMallocCCM, sizeof(DSO_T));

    /* 申请示波器通道 2 动态内存 */
    g_DS02 = (DSO_T *)rt_alloc_mem(AppMallocCCM, sizeof(DSO_T));

    /* 申请光标测量结构体变量动态内存 */
    g_Cursors = (CURSORS_T *)rt_alloc_mem(AppMallocCCM, sizeof(CURSORS_T));

    /* 申请标志位结构体变量动态内存 */
    g_Flag = (FLAG_T *)rt_alloc_mem(AppMallocCCM, sizeof(FLAG_T));

    /* 申请触发结构体变量动态内存 */
    g_TrigVol = (TRIVOLTAGE_T *)rt_alloc_mem(AppMallocCCM, sizeof(TRIVOLTAGE_T));

    /* 申请 FFT 动态内存 */
    testInput_fft_2048 = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*2048);
    testOutput_fft_2048 = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*2048);

    /* 申请 RMS 动态内存 */
}
```

```
g_RMSBUF = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*600);

/* 申请 FIR 动态内存 */
FirDataInput = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_LENGTH_SAMPLES);
FirDataOutput = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_LENGTH_SAMPLES);
firStateF32 = (float32_t *)rt_alloc_mem(AppMallocCCM, sizeof(float32_t)*FIR_StateBufSize);
}
```

◆ 任务栈和任务控制块 TCB

任务栈和任务控制块 TCB 也是用的 CCM RAM 空间，具体分配如下：

```
uint64_t AppMallocCCM[64*1024/8] __attribute__((at(0x10000000))); /* CCM RAM 动态内存池 */

/*
*****
* 函数名: MallocInit
* 功能说明: 用户任务栈，用户 TCB(任务控制块)和示波器结构体变量使用 CCM RAM
* 形参: 无
* 返回值: 无
*****
*/
static void MallocInit(void)
{
    /* 将内部 CCM SRAM 的 64KB 全部供动态内存使用 */
    rt_init_mem(AppMallocCCM, 1024*64);

    /* 任务堆栈和任务控制块是使用 CCM RAM */
    AppTaskDsoTCB = (OS_TCB *)rt_alloc_mem(AppMallocCCM, sizeof(OS_TCB));
    AppTaskDsoStk = (CPU_STK *)rt_alloc_mem(AppMallocCCM,
                                             sizeof(CPU_STK)*APP_CFG_TASK_DSO_STK_SIZE);

    AppTaskStartTCB = (OS_TCB *)rt_alloc_mem(AppMallocCCM, sizeof(OS_TCB));
    AppTaskStartStk = (CPU_STK *)rt_alloc_mem(AppMallocCCM,
                                             sizeof(CPU_STK)*APP_CFG_TASK_START_STK_SIZE);

    AppTaskMsgProTCB = (OS_TCB *)rt_alloc_mem(AppMallocCCM, sizeof(OS_TCB));
    AppTaskMsgProStk = (CPU_STK *)rt_alloc_mem(AppMallocCCM,
                                             sizeof(CPU_STK)*APP_CFG_TASK_MsgPro_STK_SIZE);

    AppTaskUserIFTCB = (OS_TCB *)rt_alloc_mem(AppMallocCCM, sizeof(OS_TCB));
    AppTaskUserIFStk = (CPU_STK *)rt_alloc_mem(AppMallocCCM,
                                             sizeof(CPU_STK)*APP_CFG_TASK_USER_IF_STK_SIZE);

    AppTaskGUITCB = (OS_TCB *)rt_alloc_mem(AppMallocCCM, sizeof(OS_TCB));
    AppTaskGUIStk = (CPU_STK *)rt_alloc_mem(AppMallocCCM,
                                             sizeof(CPU_STK)*APP_CFG_TASK_GUI_STK_SIZE);

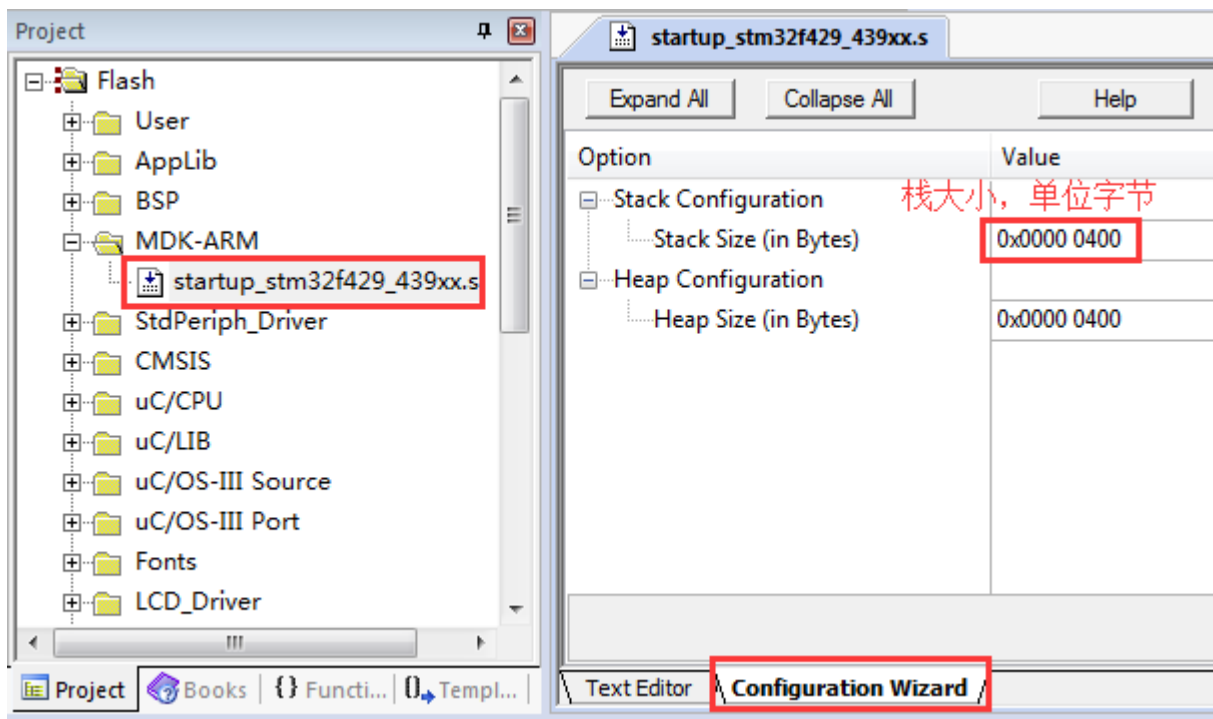
    /* 部分全局变量的初始化，省略未写 */
}
```

● 知识点拓展

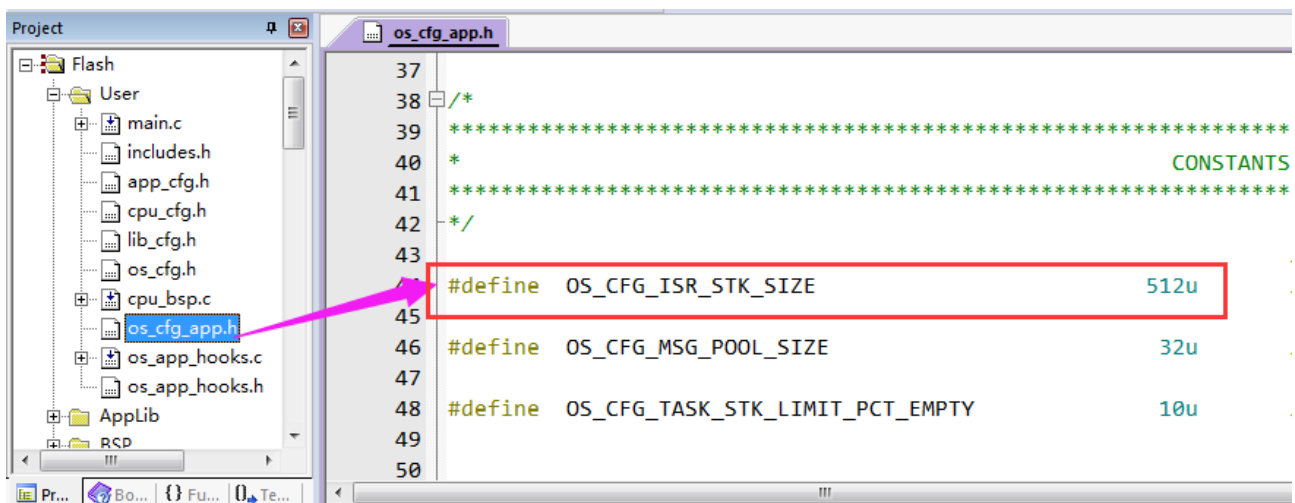
关于任务栈大小应该分配多大的问题，可以看 FreeRTOS 教程第 11 章，对于 uCOS-III 系统也是适用的：<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

◆ 系统栈分配

系统栈的大小不是在启动文件里面配置，因为系统启动过程中做了重新配置，所以启动文件里面配置的系统栈只在 uCOS-III 开启多任务之前使用：



重新配置的系统栈需要在如下所示位置设置，单位 4 字节，比如这里是配置的 512u，也就是 2048 字节。



14.5 任务间通信和全局变量共享问题

二代示波器的双通道 ADC 通过 DMA 方式在实时的采集数据，每个通道的缓冲大小是 1024*20 字节，采集的数据经过信号处理后送给 GUI 任务进行波形显示和测量值显示。为了实现这个功能，专门测试了两种方案。

◆ 方案一

采用 DMA 双缓冲，一路缓冲采集波形的时候，另一路已经采集的波形数据发给数字信号处理任务，信号处理任务再将整理好的波形数据和测量值发给 emWin 任务做刷新。这种方式的优点是 ADC 采集的数据可以实时处理。缺点是 F429 处理不过来，比如我们一个通道的采样率是 2Msps，缓冲大小设置为 2048，将缓冲填满需要 1ms 左右的时间，而我们仅做一个 2048 点的实数 FFT 就需要 0.862ms，其它的 FIR，RMS 等都还没有做，而且已经没有时间发消息给 emWin 任务做界面刷新了。如果我们降低 FFT，FIR 等信号处理的点数，也就失去了实时处理的意义。也许读者会说，加大缓冲不就好了，其实不然。如果我们加大了缓冲，我们要处理的数据也增加了，还是处理不过来，而且我们现在要处理的是双通道。

除了 F429 的性能问题，这种方式还有一个比较棘手的问题需要解决，就是用户操作界面的时候，GUI 任务基本已经没有时间去处理数字信号处理任务发来的数据，为了解决这个问题，大大增加了软件设计的复杂度，特别是波形暂停和运行的切换，窗口的切换以及其它操作时，都要注意这个问题。

如果没有复杂的界面操作，而且采样率较低的话，方案一还是比较合适的。由于我们需要滑动操作波形，而且要实现双通道，每个通道最高采样率是 2.8Msps，所以放弃这种方案。

◆ 方案二

与方案一恰恰相反，ADC 数据依然是通过 DMA 方式实时采集，而任务间的通信反过来进行，emWin 任务需要波形数据刷新时给数字信号处理任务发消息获取，这样就有效地解决了方案一中 F429 性能不够的问题，而且方案一中棘手的软件问题得到了很好的解决，随时都可以操作界面。

并且这种方式无形中解决了 emWin 任务和数字信号处理任务之间共同操作全局变量的问题，因为 emWin 是低优先级任务，而数字信号处理任务在 emWin 任务发消息后才会执行，这样就不存在抢占问题了，有效地解决了全局变量共享问题。

但是这种方式也有一个缺陷，无法实时刷新波形和测量值了，不过可以通过普通触发来解决了，普通触发方式实时采集了触发值前后各 1024 字节的数据，并且可以滑动浏览。不过工程中未对这种方式做 FFT 和 FIR 的支持。

总结，二代示波器中最终选择了方案二。

14.6 uCOS-III 系统调试

调试 uCOS-III 有两种方法，一种是 uC/Probe，还有一种是串口打印。

◆ uC/Probe 调试

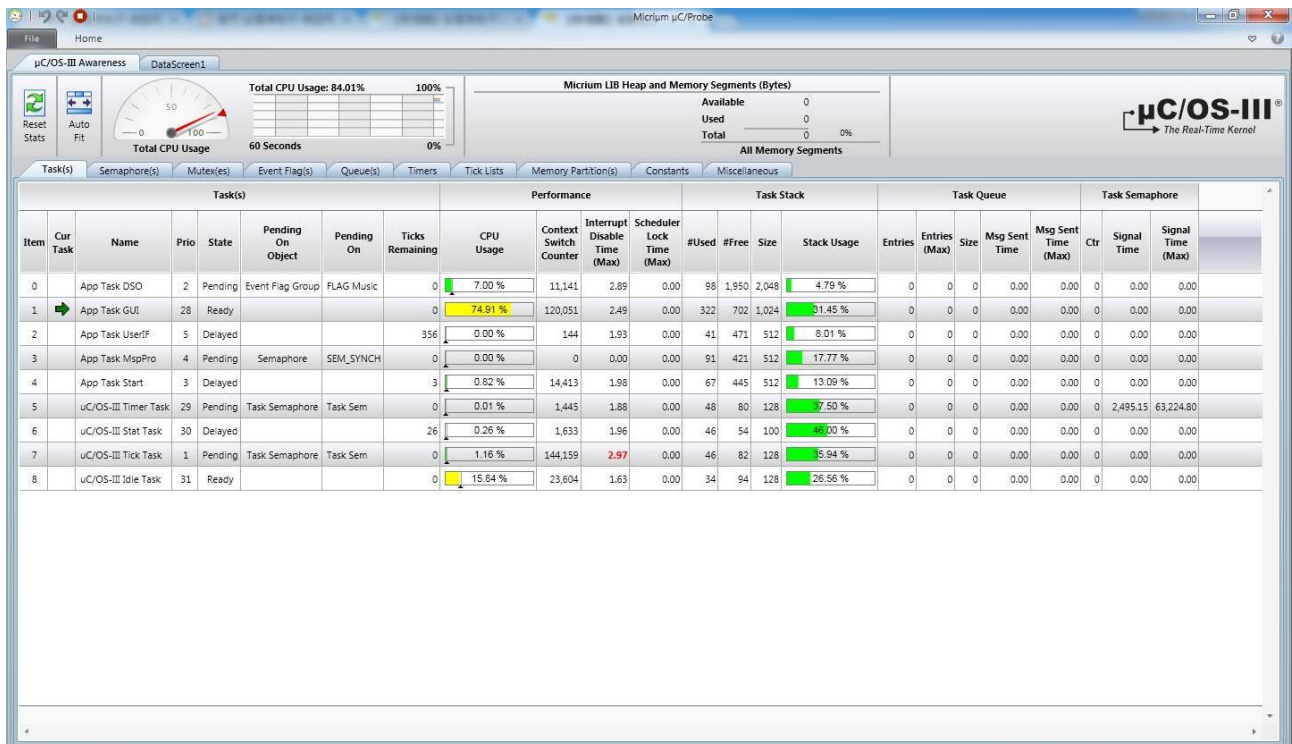
uCOS-III 的调试推荐使用 uC/Probe，性能强劲。针对 uC/Probe 的调试，我们专门做过一个专题教程，如果没有用过的话，务必先学习教程。

● 知识点拓展

【专题教程第 2 期】uC/Probe 简易使用说明，含 MDK 和 IAR，支持 F103，F407 和 F429 开发板

<http://forum.armfly.com/forum.php?mod=viewthread&tid=50280>。

另外特别注意一个问题，二代示波器的 uCOS-III 做了最高等级的优化和时间优化，使用 uC/Probe 调试需要关闭所有优化，否则无法使用，因为调试信息都被优化掉了。



◆ 串口打印调试

串口打印比较简单，按下按键 K1 即可。不过由于按键不够用，在 MainTask.c 文件的 MainTask 函数里面对按键 K1 的消息处理做了三个条件编译，大家可以根据需要选择执行触摸校准功能，截图功能还是串口打印功能。#if 0 取消执行，#if 1 表示执行。

```
case KEY_1_DOWN:
    #if 1
        hTouchWin = WM_CreateWindowAsChild(0,
            0,
            800,
            480,
            WM_HBKWIN,
            WM_CF_SHOW,
            _cbTouchCalibration,
            0);

        WM_Exec();
        WM_SelectWindow(hTouchWin);

        /* 执行触摸校准 */
        TOUCH_Calibration();

        WM_SelectWindow(0);

        WM_DeleteWindow(hTouchWin);
        WM_Exec();

        /* 自动触发暂停状态 */
    #endif
    #if 0
        // ... other code ...
    #endif
    #if 0
        // ... other code ...
    #endif
}
```

```

if(g_Flag->hWinRunStop == 1)
{
    g_Flag->ucWaveRefresh = 1;
}

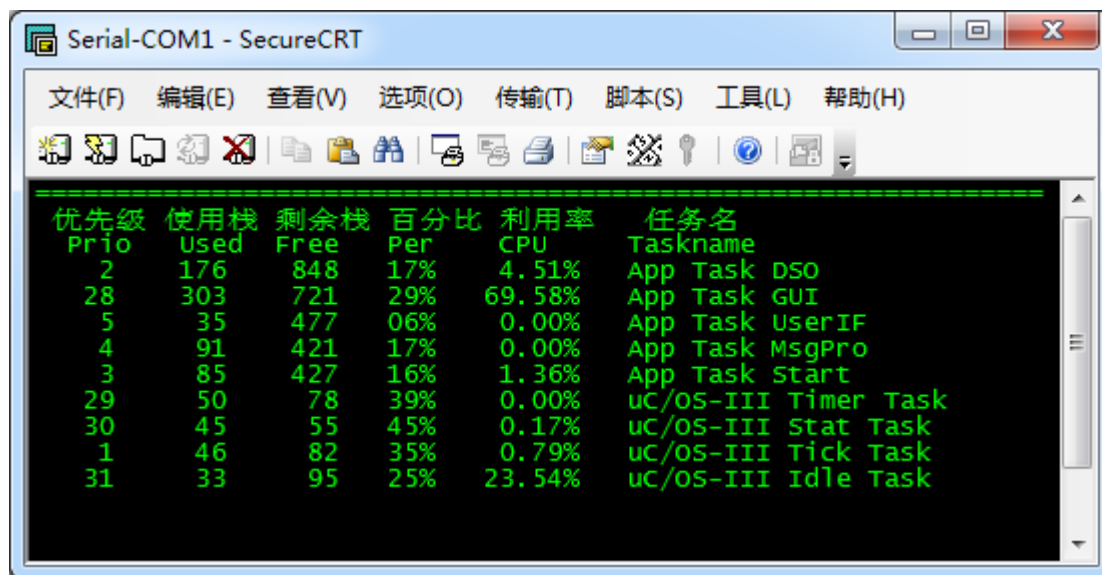
/* 普通触发暂停状态 */
if(TriggerFlag == 1)
{
    TriggerFlag = 2;
}
#endif

/* 用于截图 */
#if 0
    OSSemPost((OS_SEM *)&SEM_SYNCH,
              (OS_OPT )OS_OPT_POST_1,
              (OS_ERR *)&err);
#endif

/* 打印任务的执行情况 */
#if 0
    g_Flag->ucDsoMsg = DspTaskInfo_13;
    OSTaskQPost(AppTaskDsoTCB,
                (void *)&g_Flag->ucDsoMsg,
                sizeof(g_Flag->ucDsoMsg),
                OS_OPT_POST_FIFO,
                &err);
#endif
break;

```

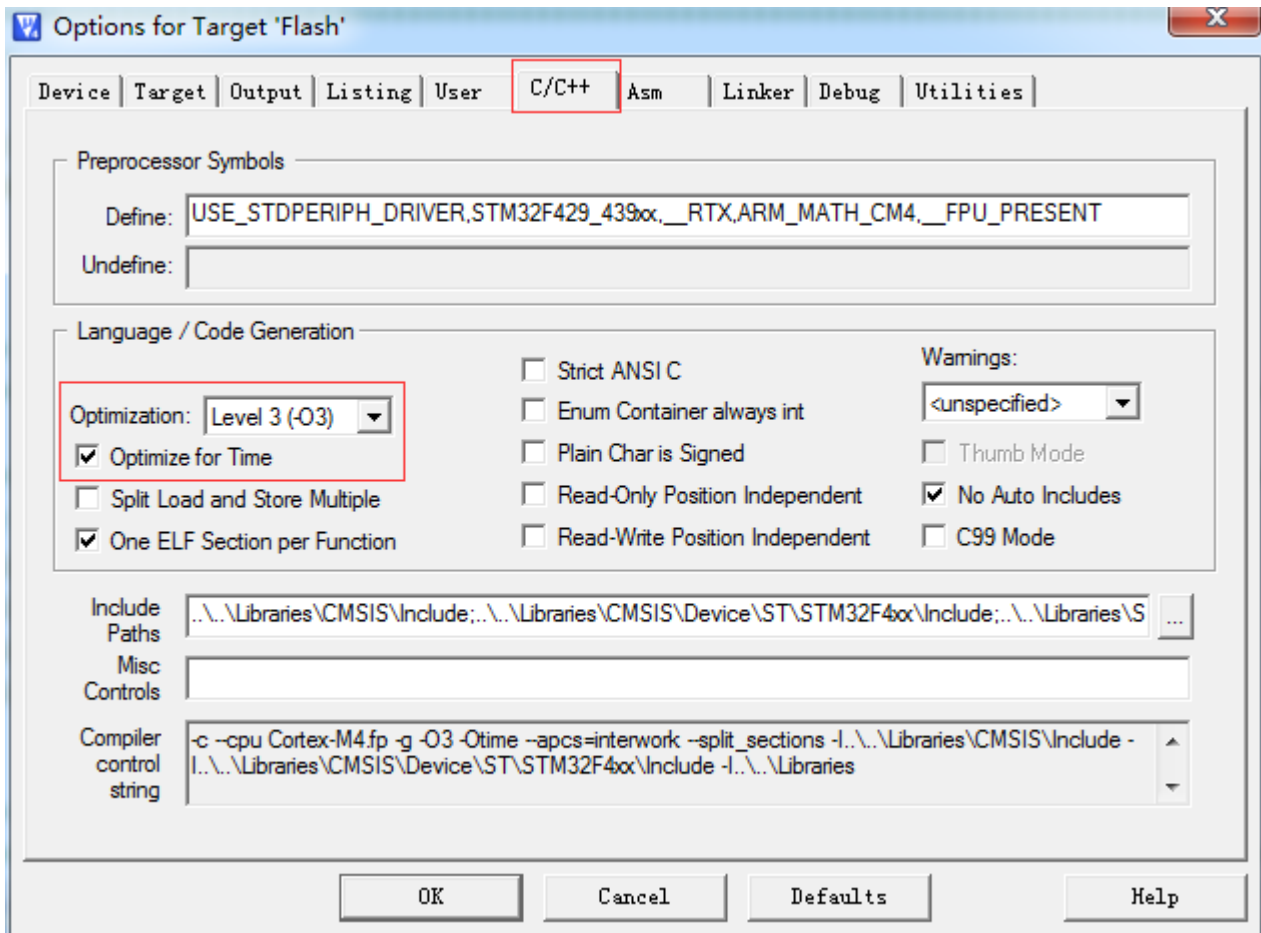
如果使能了串口打印的条件编译，串口打印任务执行情况如下：



优先级 Prio	使用栈 Used	剩余栈 Free	百分比 Per	利用率 CPU	任务名 Taskname
2	176	848	17%	4.51%	App Task DSO
28	303	721	29%	69.58%	App Task GUI
5	35	477	06%	0.00%	App Task UserIF
4	91	421	17%	0.00%	App Task MsgPro
3	85	427	16%	1.36%	App Task Start
29	50	78	39%	0.00%	uC/OS-III Timer Task
30	45	55	45%	0.17%	uC/OS-III Stat Task
1	46	82	35%	0.79%	uC/OS-III Tick Task
31	33	95	25%	23.54%	uC/OS-III Idle Task

14.7 MDK 优化等级

为了发挥 STM32F429 的最高性能，需要大家开启最高等级优化和时间优化，即下面两个选项：



uCOS-III 工程的文件系统是采用的 FatFS，当前开启了最高等级的三级优化和时间优化。如果大家要使用 FatFS 功能，请务必关闭时间优化，即 Optimize for Time，取消勾选即可。因为 FatFS 在时间优化下会工作异常。

- 知识点拓展

MDK 曾经做的专题：如何做 MDK 编译器的代码最小优化和性能最佳优化。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=1794>。

14.8 总结

uCOS-III 系统设计二代示波器的关键问题在本章节都做了阐释，建议大家学习完本章节后，直接看源码做实战演练，这样理解的更透彻，而且这时再做改进拓展也容易些。

第15章 FreeRTOS 操作系统版本二代示波器实现

本章教程为大家讲解 FreeRTOS 操作系统版本的二代示波器实现。主要讲解 RTOS 设计框架，即各个任务实现的功能，任务间的通信方案选择，任务栈，系统栈以及全局变量共享问题。同时，工程调试方法也专门做了说明。

- 15.1 注意事项（重要必读）
- 15.2 任务功能划分
- 15.3 用户任务优先级设置
- 15.4 全局变量分配，系统堆栈和任务堆栈
- 15.5 任务间通信和全局变量共享问题
- 15.6 FreeRTOS 系统调试
- 15.7 MDK 优化等级
- 15.8 总结

15.1 注意事项（重要必读）

- ◆ 学习本章节前，务必保证已经学习完毕前面章节。另外，工程代码注释已经比较详细，了解了框架后，**直接看源码即可**。
- ◆ 仅支持 800*480 分辨率显示屏，如果是电容屏，无需校准。如果是电阻屏，需要校准，按下按键 K1 即可进入校准界面。
- ◆ 由于按键不够用，K1 按键的消息处理做了三个条件编译，详情见本章 15.6 小节。默认 K1 按键执行触摸校准，也可以选择执行截图或者串口打印任务执行情况。另外，不管当前处于任何界面都可以进行触摸校准，仅电阻屏需要校准，电容屏无需校准。
- ◆ STemWin5.40 版本的截图功能有 bug，详情看此贴：
<http://forum.armfly.com/forum.php?mod=viewthread&tid=82445>。
当前用的 5.32 版本，也是来自 STemWin 软件包。
- ◆ FreeRTOS 工程的文件系统是采用的 FatFS，当前开启了 MDK 最高等级优化和时间优化。如果大家要使用 FatFS 功能，请务必关闭时间优化，因为 FatFS 在时间优化下会工作异常。详情见本章 15.7 小节。
- ◆ 工程编译支持 MDK4.7X 和 MDK5。另外不支持 MDK 发布的 MDK5.24 及其以上版本，因为这个版本不支持 MDK4 创建的工程转换为 MDK5 了，所以要使用这个最新的版本，需要给 MDK5 安装 MDK4 的兼容包。

15.2 任务功能划分

前面第三章已经将任务功能划分好：

GUI 任务

波形刷新
测量值显示
波形调节响应
波形浏览等

DSP 任务

计算 FFT, FIR,
RMS, 最大值,
最小值, 平均值,
峰峰值等

触摸和按键任务

电阻屏和电容屏
的触摸扫描,
按键扫描

文件系统任务

截图功能
数据记录

根据这个功能划分，创建所需要的任务。

15.2.1 主函数创建

在 main.c 文件实现：

```
/*
*****
* 函数名: main
* 功能说明: 标准 c 程序入口。
* 形参: 无
* 返回值: 无
*****
*/
int main(void)
{
    /*
    在启动调度前，为了防止初始化 STM32 外设时有中断服务程序执行，这里禁止全局中断(除了 NMI 和 HardFault)。
    这样做的好处是：
    1. 防止执行的中断服务程序中有 FreeRTOS 的 API 函数。
    2. 保证系统正常启动，不受别的中断影响。
    3. 关于是否关闭全局中断，大家根据自己的实际情况设置即可。
    在移植文件 port.c 中的函数 prvStartFirstTask 中会重新开启全局中断。通过指令 cpsie i 开启，__set_PRIMASK(1)
    和 cpsie i 是等效的。
    */
    __set_PRIMASK(1);

    /* 硬件初始化 */
    bsp_Init();

    /* 1. 初始化一个定时器中断，精度高于滴答定时器中断，这样才可以获得准确的系统信息 仅供调试目的，实际项目
    中不要使用，因为这个功能比较影响系统实时性。
    2. 为了正确获取 FreeRTOS 的调试信息，可以考虑将上面的关闭中断指令 __set_PRIMASK(1); 注释掉。
    */
    vSetupSysInfoTest();

    /* 创建任务通信机制 */
}
```

```
AppObjCreate();

/* 创建任务 */
AppTaskCreate();

/* 启动调度，开始执行任务 */
vTaskStartScheduler();

/*
   如果系统正常启动是不会运行到这里的，运行到这里极有可能是用于定时器任务或者空闲任务的
   heap 空间不足造成创建失败，此要加大 FreeRTOSConfig.h 文件中定义的 heap 大小：
   #define configTOTAL_HEAP_SIZE          ( ( size_t ) ( 17 * 1024 ) )
*/
while(1);
}
```

主函数中做的工作比较多，下面依次为大家做个说明：

◆ 调用函数 bsp_Init() 做硬件初始化

硬件外设的初始化函数 bsp_Init 是在 bsp.c 文件实现：

```
/*
*****
*   函 数 名: bsp_Init
*   功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*   形 参: 无
*   返 回 值: 无
*****
*/
void bsp_Init(void)
{
    /*
       由于 ST 固件库的启动文件已经执行了 CPU 系统时钟的初始化，所以不必再次重复配置系统时钟。
       启动文件配置了 CPU 主时钟频率、内部 Flash 访问速度和可选的外部 SRAM FSMC 初始化。

       系统时钟缺省配置为 168MHz，如果需要更改，可以修改 system_stm32f4xx.c 文件
    */
    /* 使能 CRC 因为使用 STemWin 前必须要使能 */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_CRC, ENABLE);

    /* 优先级分组设置为 4，可配置 0-15 级抢占式优先级，0 级子优先级，即不存在子优先级。*/
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);

    SystemCoreClockUpdate();    /* 根据 PLL 配置更新系统时钟频率变量 SystemCoreClock */

    bsp_InitDWT();    /* 初始 DWT */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitKey();    /* 初始化按键变量（必须在 bsp_InitTimer() 之前调用） */

    bsp_InitI2C();    /* 配置 I2C 总线 */

    bsp_InitExtSDRAM();
    bsp_DetectLcdType(); /* 检测触摸板和 LCD 面板型号，结果存在全局变量 g_TouchType, g_LcdType */

    TOUCH_InitHard(); /* 初始化配置触摸芯片 */
    LCD_ConfigLTDC(); /* 初始化配置 LTDC */

    DSO_ConfigCtrlGPIO(); /* 初始化示波器模块的引脚配置 */
}
```

```
bsp_InitADC();          /* 初始化 ADC1, ADC2 和 ADC3 */
bsp_InitDAC1();         /* 初始化 DAC1 */

g_DAC1.ucDuty = 50;      /* 初始化 DAC 配置, 用于信号发生器 */
g_DAC1.ucWaveType = 0;
g_DAC1.ulAMP = 4095;
g_DAC1.ulFreq = 10000;
dac1_SetSinWave(g_DAC1.ulAMP, g_DAC1.ulFreq);

MountSD();              /* 挂载 SD 卡 */

TIM8_MeasureTrigConfig(); /* 初始化 TIM8 用于记录一段波形 */
}
```

◆ 调用函数 vSetupSysInfoTest()初始化系统信息调试功能

这个函数涉及到的内容比较多, 需要大家专门看我们 FreeRTOS 教程的第 8 章, 有详细讲解:

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

◆ 调用函数 AppObjCreate()创建任务通信机制

实现代码如下, 其中事件标志的创建比较重要, emWin 任务和数字信号处理任务之间通信要使用。

```
/*
*****
* 函数名: AppObjCreate
* 功能说明: 创建任务通信机制
* 形参: 无
* 返回值: 无
*****
*/
static void AppObjCreate (void)
{
    /* 创建互斥信号量 */
    xMutex = xSemaphoreCreateMutex();

    if(xMutex == NULL)
    {
        /* 没有创建成功, 用户可以在这里加入创建失败的处理机制 */
    }

    /* 创建事件标志组 */
    xCreatedEventGroup = xEventGroupCreate();

    if(xCreatedEventGroup == NULL)
    {
        /* 没有创建成功, 用户可以在这里加入创建失败的处理机制 */
    }

    /* 动态内存申请部分, 省略未贴 */
}
```

◆ 调用函数 AppTaskCreate()创建任务

创建了如下五个任务:

```
/*
*****
* 函数名: AppTaskCreate
* 功能说明: 创建应用任务
* 形参: 无
*****
*/
```



```
* 返回值: 无
*****
*/
static void AppTaskCreate (void)
{
    xTaskCreate( vTaskGUI,          /* 任务函数 */
                "vTaskGUI",        /* 任务名 */
                1024,              /* 任务栈大小, 单位 word, 也就是 4 字节 */
                NULL,              /* 任务参数 */
                1,                 /* 任务优先级*/
                xHandleTaskGUI );  /* 任务句柄 */

    xTaskCreate( vTaskTaskUserIF,   /* 任务函数 */
                "vTaskUserIF",      /* 任务名 */
                512,                /* 任务栈大小, 单位 word, 也就是 4 字节 */
                NULL,               /* 任务参数 */
                2,                 /* 任务优先级*/
                &xHandleTaskUserIF ); /* 任务句柄 */

    xTaskCreate( vTaskMsgPro,        /* 任务函数 */
                "vTaskMsgPro",      /* 任务名 */
                512,                /* 任务栈大小, 单位 word, 也就是 4 字节 */
                NULL,               /* 任务参数 */
                3,                 /* 任务优先级*/
                &xHandleTaskMsgPro ); /* 任务句柄 */

    xTaskCreate( vTaskStart,         /* 任务函数 */
                "vTaskStart",       /* 任务名 */
                512,                /* 任务栈大小, 单位 word, 也就是 4 字节 */
                NULL,               /* 任务参数 */
                4,                 /* 任务优先级*/
                &xHandleTaskStart ); /* 任务句柄 */

    xTaskCreate( vTaskDS0,           /* 任务函数 */
                "vTaskDS0",         /* 任务名 */
                1024,               /* 任务栈大小, 单位 word, 也就是 4 字节 */
                NULL,               /* 任务参数 */
                5,                 /* 任务优先级*/
                &xHandleTaskStart ); /* 任务句柄 */
}
```

15.2.2 启动任务 (触摸和按键)

启动任务实现的功能比较简单，主要是按键扫描和触摸扫描：

```
/*
*****
* 函数名: vTaskStart
* 功能说明: 启动任务，主要实现按键检测和触摸检测。
* 形参: pvParameters 是在创建该任务时传递的形参
* 返回值: 无
* 优先级: 5
*****
*/
static void vTaskStart(void *pvParameters)
{
```

```
uint8_t ucCount = 0;

while(1)
{
    /* 1ms 一次触摸扫描，电阻触摸屏 */
    if(g_tTP.Enable == 1)
    {
        TOUCH_Scan();

        /* 按键扫描 */
        ucCount++;
        if(ucCount == 10)
        {
            ucCount = 0;
            bsp_KeyScan();
        }
        vTaskDelay(1);
    }

    /* 10ms 一次触摸扫描，电容触摸屏 GT811 */
    if(g_GT811.Enable == 1)
    {
        bsp_KeyScan();
        GT811_OnePiontScan();
        vTaskDelay(10);
    }

    /* 10ms 一次触摸扫描，电容触摸屏 FT5X06 */
    if(g_tFT5X06.Enable == 1)
    {
        bsp_KeyScan();
        FT5X06_OnePiontScan();
        vTaskDelay(10);
    }
}
```

● 知识点拓展

新版 emWin 教程第 4 章或者第 5 章，对触摸的实现做了详细讲解：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834> 。

15.2.3 信号处理任务

信号处理任务的实现如下：

```
/*
*****
* 函数名：AppTaskDS0
* 功能说明：双通道示波器数据处理任务。
* 形参：pvParameters 是在创建该任务时传递的形参
* 返回值：无
*****
*/
```

```
* 优先级: 6
*****
*/
static void vTaskDS0(void *pvParameters)
{
    EventBits_t uxBits;
    uint8_t pcWriteBuffer[512];

    /* 实数序列 FFT 长度 */
    fftSize = 2048;

    /* 正变换 */
    ifftFlag = 0;

    /* 初始化结构体 S 中的参数 */
    arm_rfft_fast_init_f32(&S, fftSize);

    while(1)
    {
        /* 等待所有任务发来事件标志 */
        uxBits = xEventGroupWaitBits(xCreatedEventGroup, /* 事件标志组句柄 */
                                     0xFFFF, /* 等待 0xFFFF 某一位被设置 */
                                     pdTRUE, /* 退出前 0xFFFF 位被清除, 这里是任意 0xFFFF 位被设置就“退出” */
                                     pdFALSE, /* 设置为 pdTRUE 表示等待 0xFFFF 任意位被设置 */
                                     portMAX_DELAY); /* 等待延迟时间 */

        switch (uxBits)
        {
            /* 双通道波形处理 */
            case DspFFT2048Pro_15:
                /* 读取的是 ADC3 的位置 */
                g_DS01->usCurPos = 10240 - DMA2_Stream1->NDTR;

                /* 读取的是 ADC1 的位置 */
                g_DS02->usCurPos = 10240 - DMA2_Stream0->NDTR;

                DS02_WaveTrig(g_DS02->usCurPos);
                DS01_WaveTrig(g_DS01->usCurPos);
                DS02_WaveProcess();
                DS01_WaveProcess();
                break;

            /* 用于简单的 ADC 数据采集 */
            case DspMultiMeterPro_14:
                g_uiAdcAvgSample = ADC_GetSampleAvgN();
                break;

            /* 仅用于调试目的, 打印任务的执行情况, 默认不使用 */
            case DspTaskInfo_13:
                App_Printf("=====\r\n");
                App_Printf("任务名      任务状态  优先级   剩余栈  任务序号\r\n");
                vTaskList((char *)&pcWriteBuffer);
                App_Printf("%s\r\n", pcWriteBuffer);

                App_Printf("\r\n任务名      运行计数      使用率\r\n");
                vTaskGetRunTimeStats((char *)&pcWriteBuffer);
                App_Printf("%s\r\n", pcWriteBuffer);
                App_Printf("当前动态内存剩余大小 = %d 字节\r\n", xPortGetFreeHeapSize());
```

```
break;

/* 其它位暂未使用 */
default:
    App_Printf("ucReceive = %x\r\n", uxBits);
    break;
}
}
```

根据接收到的不同任务消息来处理不同的功能，要处理的消息分为三类：

◆ 双通道波形数据处理

主要实现软件触发，计算 FFT，FIR，RMS，最大值，最小值，平均值和峰峰值。两个通道都进行了处理。具体实现方法已经在前面章节为大家做了讲解。

◆ 简单电压测量处理

这个功能比较简单，就是获取一组 ADC 数值，然后求平均。

◆ 打印任务执行情况

通过串口打印任务栈的使用情况和各个任务的 CPU 利用率。

15.2.4 GUI 任务

emWin 任务的实现代码如下：

```
/*
*****
*   函数名: vTaskGUI
*   功能说明: emWin 任务
*   形参: pvParameters 是在创建该任务时传递的形参
*   返回值: 无
*   优先级: 1 (数值越小优先级越低, 这个跟 uCOS 相反)
*****
*/
static void vTaskGUI(void *pvParameters)
{
    while (1)
    {
        MainTask();
    }
}
```

emWin 的代码都是在函数 MainTask 里面实现，这样做是方便在 main.c 文件里面统一管理任务。关于 GUI 部分最重要的界面优化，波形刷新优化，波形浏览等，在前面章节已经都做了讲解，我们这里不再赘述。更详细的实现，需要结合前面章节的讲解去看源码。

15.2.5 用户接口任务

这个任务暂时未执行任何功能，保留供以后升级使用。代码如下：

```
/*
*****
*   函数名: vTaskTaskUserIF
*   功能说明: 保留, 暂时未用到。
*   形参: pvParameters 是在创建该任务时传递的形参
*   返回值: 无
*   优先级: 2
*****
*/
static void vTaskTaskUserIF(void *pvParameters)
{
    while(1)
    {
        vTaskDelay(1000);
    }
}
```

15.2.6 文件系统处理任务

当前文件系统处理任务主要用来做截图功能, 将 GUI 界面以 BMP 格式存储到 SD 卡里面:

```
/*
*****
*   函数名: vTaskMsgPro
*   功能说明: 实现截图功能, 将图片以 BMP 格式保存到 SD 卡中
*   形参: pvParameters 是在创建该任务时传递的形参
*   返回值: 无
*   优先级: 4
*****
*/
static void vTaskMsgPro(void *pvParameters)
{
    uint8_t Pic_Name = 0;
    uint32_t ulStart, ulEnd;
    char buf[20];

    while(1)
    {
        ulTaskNotifyTake( pdTRUE, /* 此参数设置为 pdTRUE, 接收到的 notification value 清零 */
                        portMAX_DELAY ); /* 无限等待 */

        sprintf(buf, "0:/PicSave/%d.bmp", Pic_Name);

        /* 记录截图前起始时间 */
        ulStart = xTaskGetTickCount();

        /* 开启调度锁 */
        //vTaskSuspendAll();

        /* 如果 SD 卡中没有 PicSave 文件, 会进行创建 */
        result = f_mkdir("0:/PicSave");
        /* 创建截图 */
        result = f_open(&file, buf, FA_WRITE|FA_CREATE_ALWAYS);
        /* 向 SD 卡绘制 BMP 图片 */
        GUI_BMP_Serialize(_WriteByte2File, &file);
    }
}
```

```
/* 创建完成后关闭 file */
result = f_close(&file);

/* 关闭调度锁 */
//xTaskResumeAll ();

/* 记录截图后时间并获取截图过程耗时 */
ulEnd = xTaskGetTickCount();
ulEnd -= ulStart;

App_Printf("截图完成, 耗时 = %dms\r\n", ulEnd);
Pic_Name++;
}
}
```

后期这个任务将被升级，用于将波形数据以 CSV 文件格式存储到 SD 卡里面。

15.3 用户任务优先级设置

当前任务的优先级安排如下（数值越小，优先级越低）：

- ◆ vTaskDSO 任务：优先级 5。

DSP 任务一定要是优先级最高的，因为采集的数据要实时处理。

- ◆ vTaskStart 任务：优先级 4。

- ◆ vTaskMsgPro 任务：优先级 3。

启动任务（触摸和按键扫描）以及 MsgPro（文件系统处理）任务的优先级谁高谁低都没有关系。

- ◆ vTaskUserIF 任务：优先级 2。

保留，未使用任务，暂且安排为这个优先级。

- ◆ vTaskGUI 任务：优先级 1。

emWin 任务是除了空闲任务，统计任务以外最低优先级的，因为 emWin 极其占用系统资源，而且时间长，如果这个任务设置为高优先级，会直接影响低优先级任务的执行。

- 知识点拓展

关于任务优先级的安排，在我们 RTX 操作系统教程第 8 章的 8.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837>。

在我们 FreeRTOS 操作系统教程的第 13 章的 13.2 小节有些拓展：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658>。

15.4 全局变量分配，系统堆栈和任务堆栈

示波器的设计需要很多变量进行逻辑管理，从设计之初就需要将变量分类进行结构体封装，方便以后的维护升级。这一步至关重要，实际中差不多要定义上百个变量，如果不进行分类管理，以后的升级维护将非常麻烦。

这种方式还有一个好处是方便我们将 F429 的 CCM RAM 空间分配给这些变量使用。使用 CCM RAM 的好处是速度比通用 RAM 要快些，缺点是这部分空间不支持 DMA 操作。初次使用的用户比较容易在这个地方犯错误。所以在使用局部变量时，切勿将局部变量用于 DMA 传输。

◆ 任务栈

因为直接将 FreeRTOS 的动态内存管理文件 heap_4.c 中的数组重定向到 CCM RAM 空间了，那么任务栈以及所有组件需要的内存空间都是来自 CCM RAM。

```
/*
*****
FreeRTOS 的任务栈空间使用 CCM RAM 区
*****
*/
#ifdef (__CC_ARM) /* MDK 编译器 */
    uint8_t ucHeap[64*1024] __attribute__((at(0x10000000)));
#elif defined (__ICCARM__) /* IAR 编译器 */
    #pragma location=0x10000000
    uint8_t ucHeap[64*1024];
#endif
```

● 知识点拓展 1

关于任务栈大小应该分配多大的问题，可以看 FreeRTOS 教程第 11 章：

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658> 。

● 知识点拓展 2

FreeRTOS 教程第 28 章：动态内存管理。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658> 。

◆ 全局变量分配

当前需要频繁调用的变量也通过动态内存管理分配给各个结构体变量，使用的 CCM RAM 空间。

```
/*
*****
* 函数名: AppObjCreate
* 功能说明: 创建任务通信机制
* 形参: 无
* 返回值: 无
*****
*/
static void AppObjCreate (void)
{

    /* 任务消息机制的创建，省略未写 */

    /* 申请示波器通道 1 动态内存 */
    g_DS01 = (DS0_T *)pvPortMalloc(sizeof(DS0_T));

    /* 申请示波器通道 2 动态内存 */
    g_DS02 = (DS0_T *)pvPortMalloc(sizeof(DS0_T));
```



```
/* 申请游标测量结构体变量动态内存 */
g_Cursors = (CURSORS_T *)pvPortMalloc(sizeof(CURSORS_T));

/* 申请标志位结构体变量动态内存 */
g_Flag = (FLAG_T *)pvPortMalloc(sizeof(FLAG_T));

/* 申请触发结构体变量动态内存 */
g_TrigVol = (TRIVOLTAGE_T *)pvPortMalloc(sizeof(TRIVOLTAGE_T));

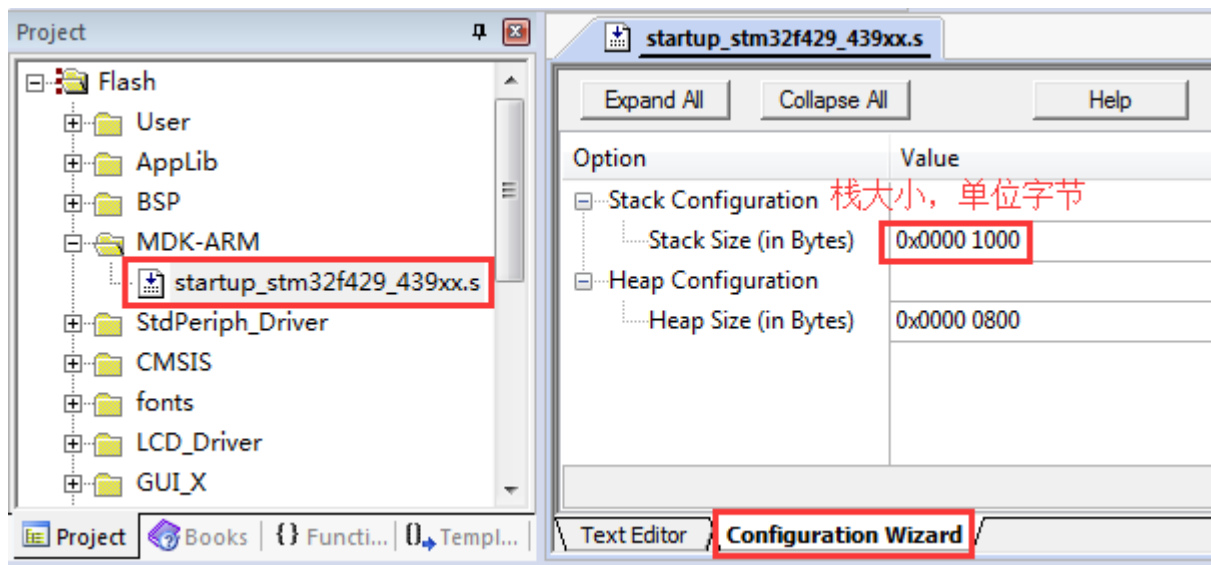
/* 申请 FFT 动态内存 */
testInput_fft_2048 = (float32_t *)pvPortMalloc(sizeof(float32_t)*2048);
testOutput_fft_2048 = (float32_t *)pvPortMalloc(sizeof(float32_t)*2048);

/* 申请 RMS 动态内存 */
g_RMSBUF = (float32_t *)pvPortMalloc(sizeof(float32_t)*600);

/* 申请 FIR 动态内存 */
FirDataInput = (float32_t *)pvPortMalloc(sizeof(float32_t)*FIR_LENGTH_SAMPLES);
FirDataOutput = (float32_t *)pvPortMalloc(sizeof(float32_t)*FIR_LENGTH_SAMPLES);
firStateF32 = (float32_t *)pvPortMalloc(sizeof(float32_t)*FIR_StateBufSize);
}
```

◆ 系统栈分配

系统栈分配的大小如下：



15.5 任务间通信机制和全局变量共享问题

二代示波器的双通道ADC通过DMA方式在实时的采集数据，每个通道的缓冲大小是1024*20字节，采集的数据经过信号处理后送给GUI任务进行波形显示和测量值显示。为了实现这个功能，专门测试了两种方案。

◆ 方案一

采用DMA双缓冲，一路缓冲采集波形的时候，另一路已经采集的波形数据发给数字信号处理任务，信号处理任务再将整理好的波形数据和测量值发给emWin任务做刷新。这种方式的优点是ADC采集的

数据可以实时处理。缺点是 F429 处理不过来，比如我们一个通道的采样率是 2Msps，缓冲大小设置为 2048，将缓冲填满需要 1ms 左右的时间，而我们仅做一个 2048 点的实数 FFT 就需要 0.862ms，其它的 FIR，RMS 等都没有做，而且已经没有时间发消息给 emWin 任务做界面刷新了。如果我们降低 FFT，FIR 等信号处理的点数，也就失去了实时处理的意义。也许读者会说，加大缓冲不就好了，其实不然。如果我们加大了缓冲，我们要处理的数据也增加了，还是处理不过来，而且我们现在要处理的是双通道。

除了 F429 的性能问题，这种方式还有一个比较棘手的问题需要解决，就是用户操作界面的时候，GUI 任务基本已经没有时间去处理数字信号处理任务发来的数据，为了解决这个问题，大大增加了软件设计的复杂度，特别是波形暂停和运行的切换，窗口的切换以及其它操作时，都要注意这个问题。

如果没有复杂的界面操作，而且采样率较低的话，方案一还是比较合适的。由于我们需要滑动操作波形，而且要实现双通道，每个通道最高采样率是 2.8Msps，所以放弃这种方案。

◆ 方案二

与方案一恰恰相反，ADC 数据依然是通过 DMA 方式实时采集，而任务间的通信反过来进行，emWin 任务需要波形数据刷新时给数字信号处理任务发消息获取，这样就有效地解决了方案一中 F429 性能不够的问题，而且方案一中棘手的软件问题得到了很好的解决，随时都可以操作界面。

并且这种方式无形中解决了 emWin 任务和数字信号处理任务之间共同操作全局变量的问题，因为 emWin 是低优先级任务，而数字信号处理任务在 emWin 任务发消息后才会执行，这样就不存在抢占问题了，有效地解决了全局变量共享问题。

但是这种方式也有一个缺陷，无法实时刷新波形和测量值了，不过可以通过普通触发来解决了，普通触发方式实时采集了触发值前后各 1024 字节的数据，并且可以滑动浏览。不过工程中未对这种方式做 FFT 和 FIR 的支持。

总结，二代示波器中最终选择了方案二。

15.6 FreeRTOS 系统调试

FreeRTOS 的调试比较简单，采用串口打印，按下按键 K1 即可。不过由于按键不够用，在 MainTask.c 文件的 MainTask 函数里面对按键 K1 的消息处理做了三个条件编译，大家可以根据需要选择执行触摸校准功能，截图功能还是串口打印功能。#if 0 取消执行，#if 1 表示执行。

```
case KEY_1_DOWN:
    #if 1
        hTouchWin = WM_CreateWindowAsChild(0,
                                            0,
                                            800,
                                            480,
                                            WM_HBKWIN,
                                            WM_CF_SHOW,
                                            _cbTouchCalibration,
                                            0);

        WM_Exec();
        WM_SelectWindow(hTouchWin);
    #endif
    #if 0
        // 串口打印功能
    #endif
    #if 0
        // 截图功能
    #endif
```

```

/* 执行触摸校准 */
TOUCH_Calibration();

WM_SelectWindow(0);

WM_DeleteWindow(hTouchWin);
WM_Exec();

/* 自动触发暂停状态 */
if(g_Flag->hWinRunStop == 1)
{
    g_Flag->ucWaveRefresh = 1;
}

/* 普通触发暂停状态 */
if(TriggerFlag == 1)
{
    TriggerFlag = 2;
}
#endif

/* 用于截图 */
#if 0
    xTaskNotifyGive(xHandleTaskMsgPro);
#endif

/* 打印任务的执行情况 */
#if 0
    g_Flag->ucDsoMsg = DspTaskInfo_13;
    xEventGroupSetBits(xCreatedEventGroup, g_Flag->ucDsoMsg);
#endif

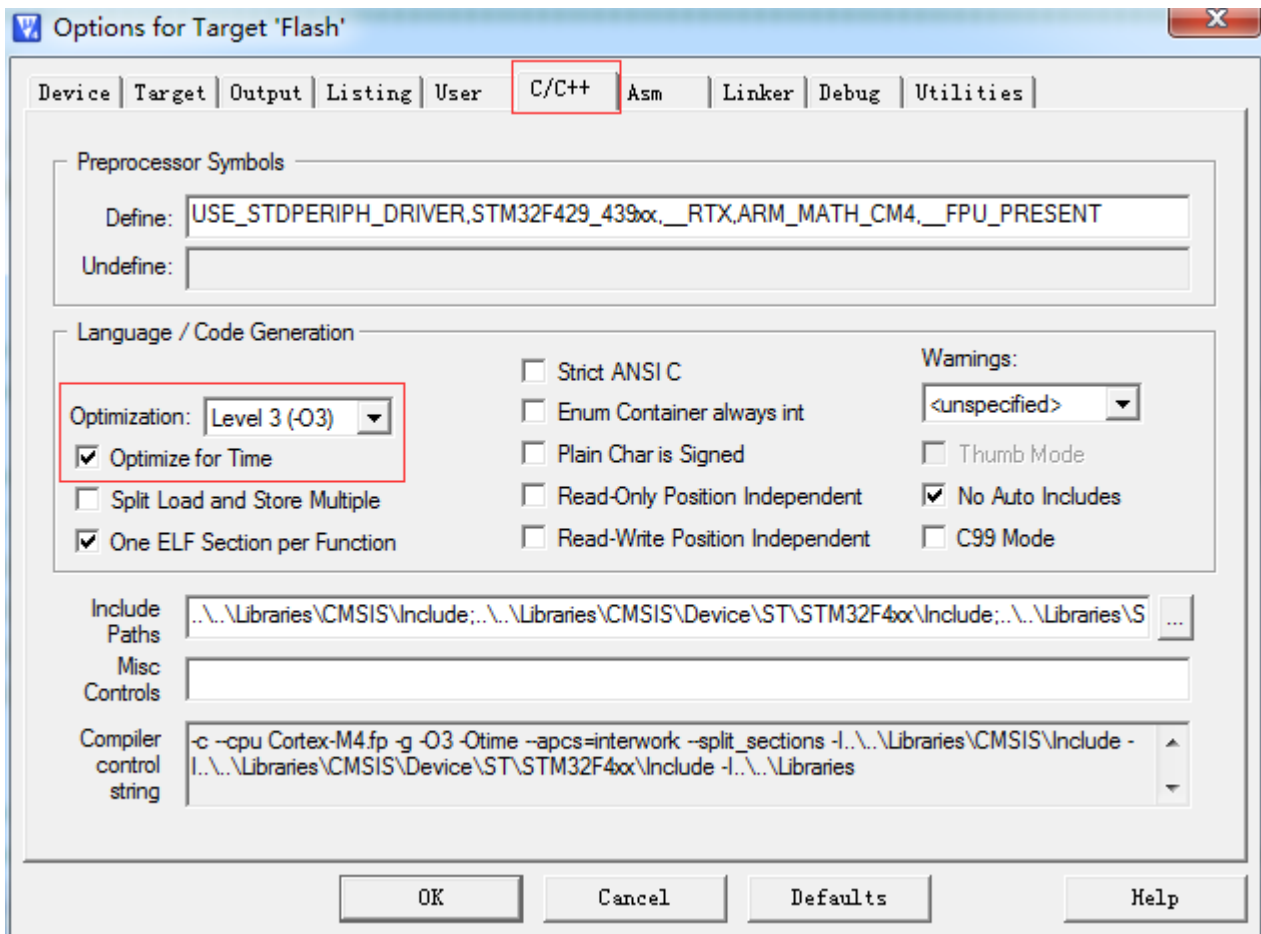
```

如果使能了串口打印的条件编译，串口打印任务执行情况如下：



15.7 MDK 优化等级

为了发挥 STM32F429 的最高性能，需要大家开启最高等级优化和时间优化，即下面两个选项：



FreeRTOS 工程的文件系统是采用的 FatFS，当前开启了最高等级的三级优化和时间优化。如果大家要使用 FatFS 功能，请务必关闭时间优化，即 Optimize for Time，取消勾选即可。因为 FatFS 在时间优化下会工作异常。

- 知识点拓展

MDK 曾经做的专题：如何做 MDK 编译器的代码最小优化和性能最佳优化。

<http://forum.armfly.com/forum.php?mod=viewthread&tid=1794>。

15.8 总结

FreeRTOS 系统设计二代示波器的关键问题在本章节都做了阐释，建议大家学习完本章节后，直接看源码做实战演练，这样理解的更透彻，而且这时再做改进拓展也容易些。

第16章 附件 A---电阻屏触摸校准

二代示波器的触摸校准比较简单，随时随地都可以做触摸校准，按下 K1 按键即可校准。有时候我们做触摸校准界面，需要在特定的界面才可以进入触摸校准状态，非常繁琐，这里采用的方法就非常省事了，也推荐大家在项目中使用。

不管当前示波器打开的是哪个界面，都可以按下 K1 按键做触摸校准，实现方法如下：

```
hTouchWin = WM_CreateWindowAsChild(0,
                                     0,
                                     800,
                                     480,
                                     WM_HBKWIN,
                                     WM_CF_SHOW,
                                     _cbTouchCalibration,
                                     0);

WM_Exec();
WM_SelectWindow(hTouchWin);

/* 执行触摸校准 */
TOUCH_Calibration();

WM_SelectWindow(0);

WM_DeleteWindow(hTouchWin);
WM_Exec();
```

=====

关于这个触摸校准，有以下两点要注意：

- ◆ 电阻屏四点触摸校准，校准后参数将保存到板载的 EEPROM 里面，下次上电会自动从 EEPROM 里面加载。
- ◆ 电阻屏需要校准，电容屏无需校准。

按下 K1 按键后，依次点击四个小圆圈进行 4 点触摸校准，最好用触摸笔去点击，这样校准的结果比较准确，校准完毕后，自动返回示波器界面，在示波器界面里面点击控件测试是否准确即可：

- 知识点拓展

如果大家需要更好的测试电阻触摸屏的校准效果，学习新版 emWin 教程的第 66 章：附件 A - 电阻触摸屏校准说明：<http://forum.armfly.com/forum.php?mod=viewthread&tid=19834>。



☐ Touch Calibration

Touch Calibration





Touch Calibration



Touch Calibration



第17章 附件 B---功能扩展和改进方向

虽然二代示波器实现了很多功能，但是当前依然有很多功能可以扩展和改进，下面为大家提供一些方向。

- ◆ 外接 ADC，DAC 模块进行支持，比如 AD7606，AD7705 等。
- ◆ 当前的波形放缩是调节不同采样率实现的，大家可以尝试采用软件方式做波形放缩。
- ◆ 当前所有功能对通道 1 进行了支持，通道 2 的部分功能未做支持，比如 FFT，FIR，普通触发等。大家可以对通道 2 也做所有功能的支持。
- ◆ 支持网络版示波器。
- ◆ 支持 USB 虚拟示波器。

当前就为大家提供这五个方向。

第18章 附件 C---波形拟合

emWin5.44 中新增的样条函数 Spline 可以实现波形拟合，即波形插补。不过当前可用于芯片的 emWin 库还没有升级到 5.44，等升级到 5.44 后，大家可以做研究。

下面知识点来自百度百科：

在数学学科数值分析中，样条是一种特殊的函数，由多项式分段定义。样条的英语单词 spline 来源于可变形的样条工具，那是一种在造船和工程制图时用来画出光滑形状的工具。在中国大陆，早期曾经被称做“齿函数”。后来因为工程学术语中“放样”一词而得名。

在插值问题中，样条插值通常比多项式插值好用。用低阶的样条插值能产生和高阶的多项式插值类似的效果，并且可以避免被称为龙格现象的数值不稳定的出现。并且低阶的样条插值还具有“保凸”的重要性质。

在计算机科学的计算机辅助设计和计算机图形学中，样条通常是指分段定义的多项式参数曲线。由于样条构造简单，使用方便，拟合准确，并能近似曲线拟合和交互式曲线设计中复杂的形状，样条是这些领域中曲线的常用表示方法。

- About this document
- Table of Contents
- Introduction to emWin
- Getting Started
- Simulation
- Viewer
- emWinSPY
- Displaying Text
- Displaying Values
- 2-D Graphic Library
 - 8.1 Graphic API
 - 8.1.1 Drawing related functions
 - 8.1.2 Basic drawing routines
 - 8.1.3 Alpha blending
 - 8.1.4 Drawing bitmaps
 - 8.1.5 Drawing streamed bitmaps
 - 8.1.6 Drawing lines
 - 8.1.7 Drawing polygons
 - 8.1.8 Drawing circles
 - 8.1.9 Drawing ellipses
 - 8.1.10 Drawing arcs
 - 8.1.11 Drawing a graph
 - 8.1.12 Drawing pie charts
 - 8.1.13 Drawing Splines
 - 8.1.14 Drawing QR-Codes
 - 8.1.15 Saving and restoring the GUI-context
 - 8.1.16 Info about screen changes
 - 8.1.17 Cache with color reducer
 - 8.1.18 Setting hook functions
- Displaying bitmap files
- Bitmap Converter
- Fonts
 - 11.1 Introduction
 - 11.2 Font types
 - 11.3 Font formats

8.1.13 Drawing Splines



GUI_SPLINE_Create()

Description
This function creates a spline handle with the given parameters.

Prototype
GUI_HMEM GUI_SPLINE_Create(const int * px, const int * py, unsigned NumPoints)

Parameter	Description
px	Pointer to an array of values for x coordinates.
py	Pointer to an array of values for y coordinates.
NumPoints	Number of points to be used.

Table 8.70: GUI_SPLINE_Create() parameter list

Return value
A handle of a spline. If zero, creation failed.

Additional information
The amount of points for x and y coordinates have to be the same. Please note that the values for the x coordinates have to be in a increasing order. It is not possible to create a spline where $px[0] > px[1]$.

GUI_SPLINE_Delete()

Description

第19章 附件 E---参考资料

- ◆ DSP 教程
<http://forum.armfly.com/forum.php?mod=viewthread&tid=3886> 。
- ◆ FreeRTOS 教程
<http://forum.armfly.com/forum.php?mod=viewthread&tid=17658> 。
- ◆ RTX 教程
<http://forum.armfly.com/forum.php?mod=viewthread&tid=14837> 。
- ◆ uCOS-III 教程
<http://forum.armfly.com/forum.php?mod=viewthread&tid=1788> 。
- ◆ uCOS-III 官方的用户手册，参考手册和 API 手册
<http://forum.armfly.com/forum.php?mod=viewthread&tid=32946> 。
- ◆ Micrium 官方曾经发布过一个非常棒的文档，如何发挥 uCOS-III 最高性能之重要提示和项目应用建议，推荐大家看看：<http://forum.armfly.com/forum.php?mod=viewthread&tid=31634> 。
- ◆ 论坛开源示波器专栏搜集的文档，PPT 和开源项目
<http://forum.armfly.com/forum.php?mod=forumdisplay&fid=36> 。



第20章 文档更新记录

版本	更改说明	作者	发布日期
V1.0	初版，20章。 从2018-01-20开始编写，2018-02-08完成，耗时20天完成。	白永斌 (Eric2013)	2018-02-08 (189页)